

Manual

MES Development Suite AIS: Server MDS-AIS 8.1

Version 1.4.23049

Last changed on: 01.09.2020

Copyright

©Copyright 2020 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	Overview – Server.....	13
1.1	Features.....	13
2	Customer-specific Database Contents.....	14
2.1	HYDRA SQL syntax	14
2.2	HYDRA SQL Interpreter hysql.....	16
2.3	Namespaces for customer-specific database objects	16
2.4	Conventions for names in the DB (tables, columns, ...)	17
2.5	Supported data types	17
2.5.1	Overview	17
2.5.2	Data type SERIAL	18
2.6	Creating functions and triggers.....	19
2.7	Example	20
2.7.1	Creating database objects	20
2.7.2	Inserting data in a table	22
2.7.3	Changing data in a table.....	23
2.7.4	Selecting data from a table	24
2.8	HYDRA SQL syntax reference	24
2.8.1	Maximum length of SQL statements.....	24
2.8.2	Name of database objects	25
2.8.3	Strings.....	28
2.8.4	Transactions.....	29
2.8.5	Current date, current time.....	30
2.8.6	Date format	30
2.8.7	Date functions	31
2.8.8	Other functions.....	31
2.8.9	Query of NULL values	32
2.8.10	Sort by NULL values.....	32
2.8.11	Sorting with union select.....	32
2.8.12	Group by calculated expressions.....	33
2.8.13	Outer Join.....	33
2.8.14	Temporary tables	33
2.8.15	unique / distinct	34

2.8.16	like / matches	35
2.8.17	Loading and unloading data	35
2.8.18	create table as select.....	36
2.8.19	CASE in the select clause	36
1.24	Integer division	36
2.8.20	Changing tables	37
2.8.21	Reserved keyword "key"	37
2.8.22	Default values in the database schema	37
2.8.23	Process "clustered index"	39
2.8.24	Optimizing "update statistics" under ORACLE	39
2.9	Notes on the performance	40
2.9.1	Union versus union all	40
2.9.2	Substrings in the WHERE clause	40
2.9.3	truncate table.....	41
2.10	Access to several databases.....	41
2.10.1	Syntax	41
2.10.2	Restrictions	42
3	Server Scripting.....	43
3.1	General	43
3.2	Naming conventions.....	43
3.2.1	Script files.....	43
3.2.2	Identifiers in the script.....	48
3.3	Structure of a server script	49
3.3.1	Overview	49
3.3.2	The header	49
3.3.3	Global data definitions	50
3.3.4	Definition of functions	50
3.4	Programming aids	50
3.4.1	Adding comments.....	50
3.4.2	Include files	51
3.4.3	Identifying the version of the script interpreter	51
3.5	Declaration of data	52
3.5.1	Supported data types	52
3.5.2	Global variables.....	53
3.5.3	Local variables in functions.....	54

3.5.4	Function parameters.....	55
3.5.5	Constants	56
3.5.6	Implicit type conversions	56
3.6	Exchanging data with the MPDV software that calls the script.....	58
3.6.1	Import variables.....	58
3.6.2	Export variables.....	58
3.7	Definition of functions	59
3.7.1	The function "main"	60
3.8	Statements.....	60
3.8.1	if / else (control structure)	61
3.8.2	for (control structure)	62
3.8.3	while (control structure)	63
3.8.4	= (assignment)	63
3.8.5	pprint (output to log file for test purposes).....	64
3.8.6	print (screen output)	65
3.8.7	dprint (screen output for test purposes)	65
3.8.8	eprint (output in error log)	65
3.8.9	system (system calls)	66
3.8.10	sleep (waiting, execution pause).....	67
3.8.11	sqlexec (executing SQL command)	67
3.8.12	into (transferring data from SQL command to variables).....	68
3.9	Built-in functions.....	69
3.9.1	today (current date)	69
3.9.2	now (current time)	69
3.9.3	month (month from a date)	69
3.9.4	day (day of a date).....	69
3.9.5	year (year of a date)	70
3.9.6	weekday (weekday of a date)	70
3.9.7	yearweek (calendar week of a date)	70
3.9.8	mdy (date from values for month, day and year).....	70
3.9.9	add_bapi_val (adding ID with value to BAPI string)	70
3.9.10	set_bapi_val (replacing/adding ID with value in BAPI string)	71
3.9.11	get_bapi_val (identifying value of BAPI string via ID or position).....	72
3.9.12	test_bapi_val (checking whether ID is available in BAPI string)	73
3.9.13	del_bapi_val (removing ID with value from BAPI string)	73
3.9.14	hy_change_sep (changing separators in strings).....	73

3.9.15	sysresult (return value of a called program)	74
3.9.16	bv (embedding bind variable in SQL command)	74
3.9.17	bvmnr (embedding machine number as bind variable in SQL command)	75
3.9.18	sqlcode (SQL error code)	75
3.9.19	sqlerrormessage (error text of the database)	76
3.9.20	SqlGetColNbr (number of columns in SQL result)	76
3.9.21	SqlColumn (transferring data from SQL command to variables)	77
3.9.22	sqlnumrows (number of changed data records)	77
3.9.23	sqlstatement (last SQL command)	78
3.9.24	sqlserial (data record number)	78
3.9.25	sqleroffset (position of an SQL error)	79
3.9.26	posc (searching for substring in string, case sensitive)	79
3.9.27	pos (searching for substring in string, not case sensitive)	80
3.9.28	strlen (string length)	80
3.9.29	strsize (identifying the size of a char variable)	81
3.9.30	strlwr (string to lower case letters)	82
3.9.31	strupr (string to upper case letters)	82
3.9.32	pow (exponentiation)	82
3.9.33	fopen (open file)	83
3.9.34	fileresult (file operations result code)	84
3.9.35	fprint / fprintf (output of a line into a file)	84
3.9.36	fgetline (read line of file)	85
3.9.37	fflush (empty file write buffer)	86
3.9.38	fclose (closing file)	86
3.9.39	hyfilepath (HYDRA path with multi-system installation)	86
3.9.40	fsize (identifying file size)	87
3.9.41	rename (renaming file)	87
3.9.42	unlink (deleting file)	88
3.9.43	errno (system error number)	88
3.9.44	file_get_first(), file_get_next(), fileresult(), fclose()	90
3.9.45	get_list_column (identifying value of a column from the data row)	91
3.9.46	set_list_column (setting value in a column in the data row)	91
3.9.47	char2long (converting char(n) to long or long64)	92
3.9.48	char2long64 (conversion of char(n) to long64)	93
3.9.49	char2double (converting char(n) to double)	93

3.9.50	char2date (converting char(n) to date).....	94
3.9.51	char2datetime (converting char(n) to datetime)	94
3.9.52	get_date (date from datetime)	95
3.9.53	get_time (time from datetime).....	95
3.9.54	date_time (datetime from date and time)	96
3.9.55	hygetenv (access to environment variables and registry)	96
3.9.56	hsysinfo (system information on the server, database and software)	96
3.9.57	hy_read_ini_data (reading INI configuration)	98
3.9.58	push_env_sql_sys(), pop_env_sql_sys ()	98
3.9.59	set_dec_sep (setting decimal separator for „using“)	99
3.10	Operators	100
3.10.1	ascii (output of any ASCII characters).....	100
3.10.2	ordinal (identifying ordinal value)	101
3.10.3	clipped and stripped (suppressing blanks).....	101
3.10.4	Substrings ([and])	101
3.10.5	Arithmetic operators	102
3.10.6	Logical comparison operators.....	104
3.10.7	Logical operators.....	104
3.10.8	using (formatting of dates to strings).....	104
3.11	The Callback function	108
3.11.1	Built-in Callback functions	108
3.12.2	Examples	113
3.13	Interpreter hydscr	114
4	Server Scripting – Generic User Exits.....	116
4.1	Overview	116
4.2	Return values of the multi script functions executed	116
4.3	Generic user exit for editing functions(BAPI)	117
4.3.1	Function "long main()"	121
4.3.2	Function "long bapi_check_before()"	121
4.3.3	Function "long bapi_check_after()"	122
4.3.4	Function "long bapi_action_before()"	122
4.3.5	Function "long bapi_end()"	123
4.3.6	Function "long bapi_action_after()"	124
4.3.7	Function "modify_list_file_line()" und "append_list_file()"	125

4.3.8	Example	125
4.4	Generic user exit for collection dialogs (DDI).....	129
4.4.1	Function "long main()"	134
4.4.2	Function "long dlg_init_before()"	134
4.4.3	Function "long dlg_init_after()"	134
4.4.4	Function "long dlg_check_before()"	135
4.4.5	Function "long dlg_check_after()"	135
4.4.6	Function "long dlg_action_before()"	136
4.4.7	Function "long dlg_action_after()"	136
4.4.8	Function "long dlg_end()"	137
4.4.9	Function "modify_list_file_line()" und "append_list_file()"	138
4.4.10	Example	138
5	User Exit Reference	141
5.1	Overview	141
5.2	Objectives and guidelines for the use of script files	141
5.3	HYDRA script language	141
5.4	Server user exits: Kernel	141
5.4.1	Modify dialog data	141
5.4.2	Logging of dialog data	143
5.4.3	Modifying batch call data	145
5.4.4	Modify event data	146
5.4.5	Reload manager: Reload plug-ins	147
5.5	Server user exits: ADE	149
5.5.1	Modification of the order list.....	149
5.5.2	Extending the machine list.....	154
5.5.3	Extending the ANR Bapi.....	155
5.5.4	Dialog processing HYMW.....	156
5.5.5	Extending the machine status list - Defining additional columns	166
5.5.6	Extending the data cursor for HYASPROT	168
5.5.7	Extending the data cursor for HYPSPROT	173
5.5.8	Overriding the HYDRA basic settings with machine configuration	179
5.5.9	Extension of the BDE archiver	180
5.5.10	3.9 Extension of the function ade_auto_verarb_insert	182
5.5.11	Extension of the data cursor for the maintenance of postings (DQADEPRO)	183

5.5.12	Modification of event maintenance data (HYEEDIT)	186
5.5.13	Setup change	188
5.5.14	PZE (IN/OUT) controls BDE / waiting period processing	189
5.6	Server user exits - LLE - incentive wages.....	193
5.6.1	Identifying the wage type of a time ticket	193
5.6.2	Identifying the time type of a time ticket.....	194
5.6.3	Recalculating time tickets	194
5.6.4	Group allocation Step 1: Distribution of data in premium accounts	195
5.6.5	Group allocation Step 2: Calculation of group results	196
5.6.6	Group allocation: Assigning group results to individual time tickets	197
5.6.7	Time period results for persons and premium groups	198
5.6.8	LLE info function on PZE terminal	199
5.6.9	4.7 LLE interface – data collection.....	200
5.6.10	4.7 LLE interface – data output.....	200
5.6.11	Active PZE/ADE comparison - RPA distribution and changing data.....	201
5.6.12	Active ADE/PZE comparison - after daily personal results.....	205
5.6.13	Active ADE/PZE comparison – Where clause.....	206
5.6.14	Labor time comparison (list)	208
5.7	Server user exits - MLE	209
5.7.1	5.1 Extension of data transfer (MLE72IMP)	209
5.7.2	Extension of the upload/confirmation (MYERPRCK/MPLRFRCK)	217
5.7.3	Extension of CAQ confirmation/upload (CAQRCK).....	223
5.7.4	Extension of the file port (hyalesrv).....	225
5.7.5	Extension of the upload client (hysapaupl.exe/out)	229
5.7.6	Extension of the QM upload client (hysapqmc.exe/out)	231
5.7.7	HR-PDC uploads and downloads	234
5.7.8	Extension of the program hysap_dp (SAP Dispatcher)	240
5.7.9	Sort sequence of the data cursor.....	240
5.8	Server user exits: MPL	242
5.8.1	Setting batch status (STKOMBI).....	242
5.8.2	Extension of ZWAU/ZWEI goods movement confirmation/upload (MPLRFRCK)	243
5.8.3	Material movement (C_MBEW)	244
5.8.4	Setting the retrograde consumption type (backflush)	245
5.8.5	Processing surplus consumption quantities when logging off batches.....	246

5.8.6	Changing data while generating goods movements.....	247
5.8.7	Itemizing the generated batch number	251
5.8.8	Processing consumption quantities of input batches.....	251
5.9	Server user exits – PZW Personnel TimeManagement	252
5.9.1	General import/export parameters	252
5.9.2	Data output: interface to payroll accounting	254
5.9.3	Work day evaluation, pre-calculation and post-calculation.....	258
5.9.4	Wage type posting, pre-calculation and post-calculation	259
5.9.5	5.1 Month evaluation, pre- and post-allocation	262
5.9.6	Monthly evaluation, processing of account limits	264
5.9.7	Information display at the terminal	268
5.9.8	Display online balances during clocking	270
5.9.9	Planning data source.....	271
5.9.10	Data source of account planning	272
5.9.11	Attendance/absence overview.....	273
5.9.12	Labor time statistics.....	274
5.9.13	Time sheet	275
5.9.14	HR master data download from SAP	280
5.9.15	Uploading time events to SAP	280
5.10	Server user exits: CAQ.....	281
5.10.1	User exits in the context of operation and order events	281
5.10.2	User exit for inspection requirements and inspection steps	293
5.10.3	User exit for entries in the CAQ number pool.....	297
5.10.4	User exits calculating measured values for characteristics	298
5.10.5	Server user exits for MDI measurement recording.....	299
5.10.6	CAQ list extensions	304
5.11	Server user exits: ESK	307
5.11.1	Overriding KEY fields in the escalation configuration.....	308
5.11.2	Extension by cyclic requests (escalations).....	309
5.11.3	Modify escalation data before processing.....	312
5.11.4	Changing e-mail address(es).....	313
5.11.5	Save escalation message.....	315
5.12	Server user exits: HYD-SIG.....	317
5.12.1	Additional information for signature check/collection.....	317
5.13	Server user exits: PDV	318
5.13.1	Specification list search	318

5.13.2	Writing interface file for data collection	319
5.14	Server user exits: WRM	320
5.14.1	Control mounting of resources (RES_EIN)	320
5.14.2	Control demounting of resources (RES_AUS)	322
5.15	Server user exits: MES-Cockpit.....	323
5.15.1	Exporter – extension of existing objects.....	323
5.15.2	Exporter: export separate objects	327
5.16	Server user exits: MDE	327
5.16.1	Machine status depends on parallel status	327
5.16.2	User exit after INSERT of MDE log record.....	328
5.16.3	Extended status evaluation (hym_stat72)	330
5.17	Server user exits: HLS	331
5.17.1	Configuration of planning component	331
5.17.2	Saving planned data.....	332
5.17.3	Saving changed planned data after planning/scheduling on the server	333
5.18	Server user exits: ZKS - Access Control System.....	334
5.18.1	8 List of access authorizations.....	334
5.18.2	Access log	335
5.19	Server user exits: ETD	337
5.19.1	Log additional information for reprinting.....	337
5.19.2	Additional activities after creating the reprint file	337
5.19.3	Extended reprint list file (terminal)	338
6	Creating PDM-BAPIs using HYDRA Script.....	340
6.1	Overview	340
6.2	Using the Server Scripting.....	341
6.3	Basic structure	341
6.3.1	Parameters of the callback function AddElement.....	342
6.4	Extended options	344
6.4.1	Definition of a dialog list in the basic structure	344
6.4.2	Import and export variables	345
6.4.3	Error handling.....	346
6.4.4	Additional joins with SELECT and LIST	347
6.4.5	Sorting with LIST	348
6.4.6	Additional where clauses	348

6.4.7	Available BAPI standard functions in the script.....	348
6.4.8	Functions that can be overwritten	354
6.5	Tips and tricks	363
6.5.1	Set default values of fields.....	363
6.5.2	Versioned master data	363
6.6	Tutorial	364
6.6.1	Task	364
6.6.2	Simple version.....	364
6.6.3	Version including display of personnel data.....	366
6.6.4	Version including date-dependent assignment	368
6.6.5	Extended check "Person available"	369
6.6.6	When delete, only mark as deleted.....	371
6.6.7	Additional BAPI for authorization	374
7	Creating PDM lists using HYDRA script.....	379
7.1	Overview	379
7.2	Using the Server Scripting.....	380
7.2.1	Import and export variables	381
7.2.2	Script function long main()	381
7.2.3	Callback function long "SetTables"	382
7.2.4	Callback function long "AddColumn".....	382
7.2.5	Callback function long "SetClauses"	384
7.2.6	Callback function long "MakeList"	384
7.2.7	Callback function long "WriteLn"	385
7.2.8	Error handling	385
7.3	Creating the list	388
7.4	Introduction: Basic structure	388
7.4.1	Example 1: Simple list of persons.....	388
7.4.2	Example 2: List of persons with additional info and selection.....	390
7.5	Introduction: Extended options	392
7.5.1	Example 3: List of persons	392

1 Overview – Server

1.1 Features

You can use the MES Development Suite (MDS) to change and extend the server functions of the data collection. You can also use user exits to intervene in the processing of the standard at other predefined points.

The document **MDS-AIS_81_Server** describes the functions that the MES Development Suite Business Applications & Services provides to change and extend the processing in the server according to your requirements.

To make changes in the server, the performant script language "HYDRA script" is available. HYDRA script is easy to learn and is optimized to match the functions required in the system environment of the server. To access the database, you use the query language SQL.

- You can extend the standard processing and add additional processing steps, e.g. additional validation checks or data.
- You can extend the HYDRA database and include own objects.
- You can create own server commands (PDM dialogs) to record and change data (so-called BAPIs).
- You can create your own lists that you can use to display data on the shop floor terminal or as selection list.
- You can not only change the data collection using the MES Development Suite Business Applications & Services, but you can also use the specified user exits in other parts of the server software. Examples: Customization of the labor time calculation of the PZW or generation of manual editing options for order-related BDE postings on the MOC.

The document **MDS-AIS_81_Server** is the reference manual of the functions provided. To learn all about the MES Development Suite Business Applications & Services, MPDV offers specific trainings. MPDV recommends to attend this training to be able to successfully use this product.

2 Customer-specific Database Contents

2.1 HYDRA SQL syntax

The system supports different database systems: Microsoft SQL Server and Oracle. These two databases do not always use the same SQL syntax and the data types required are partly different.

MPDV software therefore uses a special SQL syntax. The internal database interfaces convert the MPDV SQL syntax dynamically into the required syntax depending on the actually used database system.

With customer-specific solutions, the HYDRA SQL syntax is less important when you perform SQL queries and statements for Data Manipulation (DML) because these statements only have to work on the customer's database system. With customer-specific developments, you can therefore use the native SQL syntax of this database system.

But you must use the HYDRA SQL syntax for Data Definition (DDL) statements so that the objects created for the customer are compatible with all areas and tools of the MPDV software.

If you create objects like tables, columns, views, triggers, indices and functions, you must use the HYDRA SQL syntax. For this purpose, use an SQL client which converts the statements into the required SQL dialects.

For further information on the HYDRA SQL syntax, refer to one of the following sections.

Only the following command line tools are authorized SQL clients for the Data Definition Language (DDL) used to create and modify data objects:

- HYDRA SQL Interpreter `hysql.exe` (Windows) or `hysql.out` (Linux)
- HYDRA Script Interpreter `hydscr.exe` (Windows) or `hydscr.out` (Linux)

HYDRA SQL Interpreter can process SQL statements including DDL in an interactive prompt or it processes text files including SQL statements one after the other. The procedure is described in detail in the following sections.

MPDV uses HYDRA Script Interpreter to execute database patches, e.g. for product upgrades.

Use the defined SQL clients and respect the conventions mentioned in the following to ensure that the customer-specific data objects are compatible with all MPDV tools.



If you do not respect these rules, serious problems might be the result when you process data objects using MPDV tools, e.g. data inconsistency, loss of DB objects like views, triggers or indices, or even loss of data. This applies in particular with upcoming release upgrades, data transfers between different systems and required database reorganizations.

2.2 HYDRA SQL Interpreter hysql

The HYDRA SQL interpreter is a command line tool on the server. To launch it, you have to start the command line for the selected system number on a Windows server. On a Linux server you have to change to the required system number with hsys.scr.

Start as follows, if you want to execute text files including SQL statements:

Windows: `hysql -r statements.sql`

Linux: `hysql.out -r statements.sql`

Start as interactive SQL command line interpreter:

Windows: `hysql -r -`

Linux: `hysql.out -r -`

The parameter "-r" returns a result row after each SQL statement. This result row contains the SQL code, the number of affected data records and in case of SELECT statements the first selected row with columns separated by the pipe character "|".

The single minus sign as last parameter starts the interactive mode.

Find examples in the sections that follow.

2.3 Namespaces for customer-specific database objects

We have defined a separate namespace for customer-specific objects in order to avoid conflicts between MPDV standard objects (e.g. new features) and customer-specific objects. MPDV standard objects do not use the customer-specific namespace.

- Prefix all customer-specific objects in the database (tables, columns, views, indices,...) with "" .
- Columns in a customer-specific table need not start with "" , as the table itself is located in the customer's namespace.
- Customer-specific columns are not allowed for the standard table. Better create a customer-specific table. Maintain this table in parallel to the standard table and join it for SELECT statements. If you cannot avoid to insert customer-specific columns in standard tables, the columns must have the prefix "" .
- If necessary, MPDV also uses the customer-specific namespace for customizations to customer-specific tables. These tables are listed in the customer documentation.

2.4 Conventions for names in the DB (tables, columns, ...)

- Names for tables, columns, etc. can be made up of the Latin letters "a" to "z", the underscore "_" and the numbers "0" to "9". The name must start with a letter. Using special characters, umlauts or characters from other character sets is not allowed.
- In the HYDRA SQL syntax, write all identifiers (table name, column name, indices, views,...) in **lower case letters**.
- The identifiers' length should not exceed 30 characters so all MPDV tools can process the objects properly.**
- Do not use other data types** than the data types specified. If you use data types that are not implemented in the MPDV SQL clients, the system will probably react with undefined and incorrect behavior.
- Use the **customer's namespace** ("u_").

2.5 Supported data types

2.5.1 Overview

The supported data types are in some kind the "lowest common denominator". These data types are accepted in all supported database systems and implemented in all MPDV database clients.

The following table shows the data types used in HYDRA SQL and how these data types are implemented in the databases:

HYDRA SQL	Comment	Oracle	Sql Server
smallint	Integer ranging between -32768 and 32767. The value -32768 stands for an empty column (null).	NUMBER(37)	SMALLINT
integer	Integer ranging between -2147483648 und 2147483648. If the value is -2147483647, the column is empty (null).	NUMBER(22)	INTEGER
serial	See below	NUMBER(36)	INTEGER IDENTITY
decimal(m,n)	Decimal(18,6) is used by default.	NUMBER(m,n)	DECIMAL(m,n)
smallfloat	This data type is rarely used and should be avoided.	FLOAT(125)	smallfloat (→ REAL, see below)
float		FLOAT	FLOAT
float(n)	This data type is not used in the standard. Avoid this data type.	FLOAT(n)	FLOAT(n)

char or char(1)		CHAR (1)	CHAR (1)
char(n)		NVARCHAR2(n) with n > 4000 LONG	NVARCHAR(n) with n > 4000 TEXT
text	Storing large text fields. This data type is not used in the standard. Avoid this data type.	NCLOB	TEXT
date	Date without time	DATE	hydate (→ DATETIME, see below)
datetime	Timestamp including date and time.	TIMESTAMP(3)	DATETIME
image	Storing binary objects	BLOB	IMAGE

With a Microsoft SQL server, the following data types are created as user-defined data types in order to distinguish them:



hydate: EXEC sp_addtype date, datetime, 'NULL'

smallfloat: EXEC sp_addtype smallfloat, real, 'NULL'

The system automatically creates the user-defined data types when you create the database during the default installation process of the system.

2.5.2 Data type SERIAL

The data type SERIAL contains a numeric key that the database automatically assigns in ascending order. You use this data type, if a table does not contain a unique key.

ORACLE implements this data type by creating a sequence named *S_<tablename>*. This sequence provides the unique values. In addition, a trigger named *T_<tablename>* is created. If a data record is inserted into the table, this trigger reads the next value from the sequence and adds the value to the relevant column. The same trigger also guarantees that the value of the SERIAL column cannot be changed.



When you create a table including a SERIAL column, you must create a UNIQUE INDEX for this column.

2.6 Creating functions and triggers

To create functions and triggers, you also use an own syntax in HYDRA SQL for Oracle and Microsoft SQL Server because the native syntax can be quite different. MPDV therefore always lists the two statements for both SQL Server and Oracle in the corresponding SQL or HYDRA script files for functions and triggers of the standard. The clients HYDRA SQL Interpreter or HYDRA Script Interpreter execute only the relevant statement depending on the actually used database system.

You must query functions as qualified identifier preceded by database user "mipadm" (or "hydadm" preceding MW 4.0 pe) .

See also the following example:

2.7 Example

In the following example, a customer-specific table is created that includes columns with the most important data types. An index is created for the table. In addition, a function, a trigger and a view are created.

Further examples show how to write SQL statements for hysql to insert or change data in a table. The example shows how to write values for columns including data types like date, datetime and float or decimal in the statements.

2.7.1 Creating database objects

An SQL file is created in the subdirectory "db_sql" on the server:

```
create table u_machine_detail
(
  machine_nbr char(20),
  detail_text char(100),
  room_nbr integer,
  purchase_price float,
  last_maintenance_date date,
  last_maintenance_time integer,
  last_maintenance_ts datetime,
  modified_by char(10),
  modified_ts datetime,
  internal_id serial not null
);
revoke all on u_machine_detail from "public";

create unique index u_mdet_m on u_machine_detail (machine_nbr);
create unique index u_mdet_id on u_machine_detail (internal_id);

define function u_get_timestamp for oracle as
create function u_get_timestamp( p_d in date, p_n in number) return timestamp as
begin
  return cast((cast(p_d as timestamp) +
    case when (MONTHS_BETWEEN(cast(p_d as timestamp),sysdate)/12)>2000 and p_n=86400 then 0
      else p_n/86400 end) as timestamp) <EOS>
end;

define function u_get_timestamp for sqlserver as
CREATE FUNCTION u_get_timestamp( @p_d hydate, @p_n int ) RETURNS datetime AS
BEGIN
  RETURN @p_d + case when @p_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0) end <EOS>
END;

define trigger u_t_machine_detail_mt_ts for oracle as
CREATE TRIGGER u_t_machine_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW
BEGIN
  :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.last_maintenance_time)<EOS>
END;

define trigger u_t_machine_detail_mt_ts for sqlserver as
CREATE TRIGGER u_t_machine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0) RETURN
BEGIN
  SET NOCOUNT ON
  UPDATE u_machine_detail
  SET last_maintenance_ts =
    u_machine_detail.last_maintenance_date + ((u_machine_detail.last_maintenance_time + 0.001)/86400.0)
  FROM inserted
  WHERE u_machine_detail.internal_id = inserted.internal_id
  SET NOCOUNT OFF
END;

create view u_v_machine_detail_only_ts as
select machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts,
  modified_by,
  modified_ts,
  internal_id
  from u_machine_detail;
```

The SQL file is then started from the command line for the relevant system using the HYDRA SQL Interpreter hysql. The following example shows the query from a Windows server. The grayed out sections are keyboard inputs. With a Linux server, the command is "hysql.out -r db_sql/u_table_example.sql".

```
hydadm:3:F:\hydra3>hysql -r db_sql\u_table_example.sql

01.03.2017 08:56:39 PROCESSING db_sql\u_table_example.sql...

create table u_machine_detail ( machine_nbr char(20), detail_text char(100),
    room_nbr integer, purchase_price float, last_maintenance_date date, las
t_maintenance_time integer, last_maintenance_ts datetime, modified_by char(1
0), modified_ts datetime, internal_id serial not null );
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

revoke all on u_machine_detail from "public";
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_m on u_machine_detail (machine_nbr);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_id on u_machine_detail (internal_id);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

define function u_get_timestamp for oracle as create function u_get_timestamp( p
_d in date, p_n in number) return timestamp as begin return cast((cast(
p_d as timestamp) + case when (MONTHS_BETWEEN(cast(p_d as timestamp),s
ysdate)/12)>2000 and p_n=86400 then 0 else p_n/86400 end) as time
stamp) <EOS> end;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define function u_get_timestamp for sqlserver as CREATE FUNCTION u_get_timestamp
( @p_d hydate, @p_n int ) RETURNS datetime AS BEGIN RETURN @p_d + case when @p
_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0
) end <EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for oracle as CREATE TRIGGER u_t_machine
_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW BEGI
N :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.l
ast_maintenance_time)<EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for sqlserver as CREATE TRIGGER u_t_mach
ine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0)
RETURN BEGIN SET NOCOUNT ON UPDATE u_machine_detail SET las
t_maintenance_ts = u_machine_detail.last_maintenance_date + ((u_machin
e_detail.last_maintenance_time + 0.001)/86400.0) FROM inserted WHERE u
_machine_detail.internal_id = inserted.internal_id SET NOCOUNT OFF END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|
```

```
create view u_v_machine_detail_only_ts as select machine_nbr, detail_text, room_nbr, purchase_price, hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts, modified_by, modified_ts, internal_id from u_machine_detail;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|
hydadm:3:F:\hydra3>
```

2.7.2 Inserting data in a table

You can also start the HYDRA SQL Interpreter in the interactive input mode. This is useful with small SQL statements or with statements inserted via clipboard. The example also shows the output of result data with SELECT statements. Start the interactive mode using the command "hysql -r -" (Windows) or "hysql.out -r -" (Linux). (Note: the minus sign at the end!) End the interactive mode using the command "exit;".

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -
02.03.2017 08:40:23 PROCESSING STDIN...
SQL> insert into u_machine_detail
(
  machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  last_maintenance_date,
  last_maintenance_time,
  modified_by,
  modified_ts
)
values
(
  '00000100',
  'Detail text for machine 00000100',
  1020,
  125000.000,
  '12/31/2016',
  14.5*3600,
  '12345',
  '12/31/2016 14:25:17.123'
);
insert into u_machine_detail ( machine_nbr, detail_text, room_nbr, purchase_price, last_maintenance_date, last_maintenance_time, modified_by, modified_ts ) values ( '00000100', 'Detail text for machine 00000100', 1020, 125000.000, '12/31/2016', 14.5*3600, '12345', '12/31/2016 14:25:17.123' );
OK. NR OF ROWS 1.
RESULT:
|0|5|1|0|
SQL>
```

The example shows:

- You can write strings in single or in double quotes.
- The dot is the decimal separator for float and decimal.
- You store time of the type *Integer* on the database tables. The content refers to "seconds since midnight". You must therefore multiply time by 3600. In the example above, the time is 14:30. Instead of entering "14.5*3600", you can directly enter 52200.
- Dates are in the 'MM/DD/YYYY' format.
- Timestamps for columns of the type datetime are in the 'MM/DD/YYYY hh:mm:ss.ccc' format. 'ccc' are milliseconds.
- A continuous number is automatically assigned to columns of the type serial, if data is inserted. The columns are not indicated in the statement.
- The created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

2.7.3 Changing data in a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>mysql -r -

02.03.2017 08:40:23 PROCESSING STDIN...

SQL> update u_machine_detail set
  detail_text = 'Test machine for training',
  room_nbr = 1021,
  purchase_price = 27235.50,
  last_maintenance_date = '02/28/2017',
  last_maintenance_time = 21600,
  modified_by = 'trainee',
  modified_ts = '03/02/2017 08:54:30.468'
where machine_nbr = '00000100';
update u_machine_detail set detail_text = 'Test machine for training', room_
nbr = 1021, purchase_price = 27235.50, last_maintenance_date = '02/28/2017',
last_maintenance_time = 21600, modified_by = 'trainee', modified_ts = '03
/02/2017 08:54:30.468' where machine_nbr = '00000100';
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|

SQL>
```

Also on changing data, the created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

You cannot change columns of the type serial. The columns can only be used as key in the WHERE clause.

2.7.4 Selecting data from a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:04:07 PROCESSING STDIN...

SQL> select personalnummer, name, kostenstelle, eintritt from personen where personalnummer =
40256;
select personalnummer, name, kostenstelle, eintritt from personen where personal
nummer = 40256;
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|2|4|40256|0|83|Pernikova, Lisa|0|10|105|7|10|10/09/1993|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

Example for an SQL command including an error:

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:11:53 PROCESSING STDIN...

SQL> select error from error where error = 'TRUE';
select error from error where error = 'TRUE';
^
ERROR -208, CISAM 0, OFFSET 0: [42S02][208][Microsoft][ODBC SQL Server Dri
ver][SQL Server]Unknown object name 'error'.
RESULT:
|-208|0|0|0|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

The database system provides the error message text (e.g. MS SQL server or Oracle) and is not within the control of MPDV.

2.8 HYDRA SQL syntax reference

2.8.1 Maximum length of SQL statements

The maximum length of SQL statements is 32000 characters. Longer statements are automatically cut.
Note: necessary implementations for individual databases can increase the length.

2.8.2 Name of database objects

Do not use the following names for tables because these names have a special meaning under ORACLE: evt_*, sm*_s, sm*_x, sm*links, smc*, smp_*. In addition, the following names are not allowed for views: sm*_v, smp_*

2.8.2.1 Data type IMAGE

The system support the data type IMAGE (SQL server: IMAGE, Oracle: BLOB) for the following application cases:

- Creation of customer-specific tables via hysql that include data type BLOB
- Export and import of tables including BLOB data

Requirements:

- Oracle: as of MW 3.0
- SQL Server: as of MW 3.0, hyaccsql.dll version 8.1.2.10

Create tables with column type IMAGE

Requirements

Tables including one or more IMAGE columns must have a SERIAL column named **VERWEIS** (= "reference") (because of Oracle database).

Implementation

Use the tool hysql.out|exe if you want to create tables including the HYDRA data type IMAGE. The IMAGE column has data type IMAGE without any indication of size.

Example:

```
create table u_document
(
    verweis serial not null,
    ...
    opticalfingerprint image,
    ...
);
revoke all on u_document from "public";

create unique index u_document_vw on u_document (verweis);
```

Export

Store the schema of tables including IMAGE columns in the export SQL file as described in the example of section 3.1.

For each IMAGE field (per row/per column), an own IMAGE file is stored in the subdirectory <table name>, if the database field contains an IMAGE information (size > 0). The reference to the relevant file is stored in the UNLOAD file <TABLE NAME>.UNL.

All IMAGE files of a table are stored in the sub directory <TABLE NAME>.

Use the following rule to name IMAGE files:

<TABLE NAME>/<COLUMN NAME>_<current number per table>.**IMG**

Notes:

- Separator with WINDOWS systems is the backslash "\".
- The "current number per table" starts at 1 and is issued with 10 digits and leading zeros.
- The files have the extension: **IMG**
- All database fields in the relevant UNL file are populated with the references to the possible IMAGE files.
- Only IMAGE files that contain IMAGE information are exported to IMAGE fields.

Export example (hyexport.exe blob):

```
File blob.sql:

...
create table u_document
(
    verweis serial not null,
    charge_id char(20),
    documentorderid char(20),
    run char(12),
    sex char(1),
    countrycode char(3),
    nationalitycode char(3),
    documentserial char(9),
    dateofbirth datetime,
    dateofissue datetime,
    dateofexpiry datetime,
    faceimage image,
    signatureimage image,
    destinationoffice char(3),
    surname char(40),
    secondsurname char(40),
    givennames char(35),
    countrynationality char(12),
    nationality char(22),
    placeofbirth char(22),
    opticalfingerprint image,
    applicationnumber integer,
    civilbirthregistry char(30),
    diasbilityreg char(1),
    serialnumber char(10),
    residencetype char(10),
    visa char(20),
    profession char(30),
    mainfingerprtmin image,
    secfingerprtminu image,
    facialrecpattern image,
    authority char(50)
);
revoke all on u_document from "public";
```

```
create index u_document_cid on u_document (charge_id);
create unique index u_document_vw on u_document (verweis);
...
```

File `u_document.unl`

```
$COLUMN$VERWEIS|...| FACEIMAGE| SIGNATUREIMAGE|...|
24055|...| faceimage_0000000001.img| signatureimage_0000000002.img|...|
24056|...| faceimage_0000000003.img| signatureimage_0000000004.img|...|
.....
```

Storage of the **IMAGE** files

```
<HYDRADIR>
  → blob.exp [directory]
    → u_document [directory]
      faceimage 0000000001.img
      faceimage 0000000003.img
      signatureimage_0000000002.img
      ...
```

Import

If you import tables including IMAGE columns, create the tables as described in section 3.1. Perform the internal processing as described in the following:

Oracle:

- On reading the UNL files, all columns are read that are not of database type IMAGE and inserted via "INSERT INTO <TABLE>".
- The IMAGE columns are initialized using the ORACLE function **EMPTY_BLOB()**.
- Each IMAGE file (if existing) is read and inserted into the respective column.

SQL Server:

- Each IMAGE file is read before the "INSERT INTO <TABLE>" and is directly bound to the SQL statement.

2.8.2.2 Data type TEXT

The columns of HYDRA data type **char** and a length of more than 1999 characters are stored under ORACLE as data type NCLOB. The HYDRA data type is **TEXT**. You can store data up to 4Gbytes in this data type.

Restrictions / Conditions

This data type does not support the following SQL operations:

SQL Operations Not Supported	Example of unsupported usage
SELECT DISTINCT	SELECT DISTINCT clobCol from...
SELECT clause ORDER BY	SELECT... ORDER BY clobCol
SELECT clause GROUP BY	SELECT avg(num) FROM... GROUP BY clobCol
UNION, INTERSECT, MINUS (Note that UNION ALL works for LOBs.)	SELECT clobCol1 from tab1 UNION SELECT clobCol2 from tab2;
Join queries	SELECT... FROM... WHERE tab1.clobCol = tab2.clobCol
Index columns	CREATE INDEX clobIndx ON tab(clobCol)...

2.8.3 Strings

2.8.3.1 Constant strings

Use single or double quotes to limit constant strings in HYDRA SQL. ORACLE and SQL Server automatically replace double quotes by single quotes.

2.8.3.2 Substrings

Substrings with SELECT

Following the example of the previously available database INFORMIX, you can select string parts with HYDRA SQL using square brackets. Separate the first and the last digit by comma within the square brackets. ORACLE and SQL Server use the functions SUBSTR or SUBSTRING to which the first digit and the length of the result string are transferred.

HYDRA SQL: `select column_name[3,4] from table_name;`
 ORACLE: `select substr( column_name,3,2) from table_name;`
 SQL Server: `select substring(column_name,3,2) from table_name;`

Substrings with UPDATE

Also with UPDATE, you can access string parts with HYDRA SQL. The automatic implementation with ORACLE and SQL Server is as follows:

HYDRA SQL: `update table_name set column_name[3,4] = "ab";`
 ORACLE: `update table_name set column_name =
 substr(column_name, 1, 2) || 'ab' || substr(column_name, 5);`
 SQL Server: `update table_name set column_name =
 substring(column_name, 1, 2) + 'ab' + substring(column_name, 5, 2000);`

With SQL Server, you must always specify a length with the function SUBSTRING. But as on implementing the statement the field length is not known, a length of 2000 is assumed to add the rest of the field.

2.8.3.3 Concatenation of strings

You concatenate strings in HYDRA SQL using the operator '||'. With the SQL Server, this operator is automatically replaced with '+':

HYDRA SQL: `select "string" || column || "string" from tablename;`

SQL Server: `select 'string' + column + 'string' from tablename;`

2.8.4 Transactions

Transactions are explicitly started with HYDRA SQL. ORACLE automatically starts a new transaction once a transaction has been finished using *commit*. If there is no active transaction, ORACLE automatically performs a *commit* after each data change:

HYDRA SQL: `begin work;`

SQL Server: `begin transaction;`

and

HYDRA SQL: `commit [work];`

SQL Server: `commit transaction;`

and

HYDRA SQL: `rollback [work];`

SQL Server: `rollback transaction;`

In case of an SQL server with default settings, a session processes a data record in a transaction and another session must wait until the transaction is finished before it has **read** access. And vice versa, **also the read access (open cursor) locks the data and the data cannot be updated by another user.**

If you use the command "**set transaction isolation level read uncommitted**" with SQL server, the second session does not wait but reads the changed (possibly not consistent) data that has not yet been "committed". This is different with ORACLE. Here, the (consistent) data is issued before the beginning of the transaction. **You nevertheless use this isolation level with SQL Server as otherwise there might be time delays and deadlocks.**

SQL Server processes nested transactions. As this is not possible with ORACLE, HYDRA SQL declines a *begin work* in a current transaction and creates an SQL error.

2.8.5 Current date, current time

HYDRA SQL uses *today* and *current* to query the current date and the current time. SQL Server offers the function *getdate()*, which returns date and time just like *sysdate* with ORACLE. When implementing *today*, the time must be cut (set to midnight) as otherwise in case of queries similar to "... where datum between today and today + 1" the current day is not included in the selection.

HYDRA SQL	ORACLE	SQL SERVER
today	trunc(sysdate,'DD')	cast(convert(char,getdate(),101) as datetime)
current	systimestamp	getdate()

Note:

Both functions always provide date and time in relation to the time of the operating system.

If you use the HYTIMEZONE functionality, the values are therefore not correct. For this reason, you may not use these two functions in the standard.

2.8.6 Date format

HYDRA SQL uses the date format "MM/TT/YYYY".

2.8.7 Date functions

The following table includes the available date functions and their implementation for ORACLE and SQL Server.

HYDRA SQL	ORACLE	SQL SERVER
year(...)	to_number(to_char(to_date(...),'YYYY'))	year(...)
month(...)	to_number(to_char(to_date(...),'MM'))	month(...)
day(...)	to_number(to_char(to_date(...),'DD'))	day(...)
weekday(...)	to_number(to_char(to_date(...),'D'))	datepart(dw,...) – 1
date(...)	to_date(...)	cast(... as datetime)
get_date(...)	trunc(...,'DD')	cast(convert(char,..., 101) as datetime)
get_time(...)	to_number(to_char(...,'SSSS'))	(datepart(Hh,...) * 60 + datepart(Mi,...) * 60 + datepart(Ss,...)

The function get_date(...) returns the column date of type *DATETIME*. The function get_time(...) returns the column time in seconds since midnight of type *DATETIME*. Other functions

2.8.8 Other functions

Implement other functions for ORACLE and SQL Server as follows:

HYDRA SQL	ORACLE	SQL SERVER
length(...)	(no implementation)	len(...)
nvl(...)	(no implementation)	isnull(...)
trim(...)	rtrim(ltrim(...))	rtrim(ltrim(...))
rtrim(...)	(no implementation)	(no implementation)
ltrim(...)	(no implementation)	(no implementation)
string(...)	to_char(...)	ltrim(str(...))
value(...)	to_number(...)	cast(... as integer)

The functions **trim(...)**, **rtrim(...)** and **ltrim(...)** currently only work independently of databases if a space character is used. The transfer of another character, which should be cut off on the left and/or right of a string, is not supported.

When sorting is concerned, the function **nvl(...)** has a special meaning in HYDRA SQL (see chapter "Sorting of NULL values").

2.8.9 Query of NULL values

SQL Server makes a difference between character strings including an empty string and the ones with a NULL value. At the same time, SQL Server saves the NULL values as empty string after export and subsequent reimport. Therefore, you must always combine these two queries:

HYDRA SQL: ... where (*column* is null **or** *column* = "")

With fields of type *char(1)*, you must additionally include a space character in the query:

HYDRA SQL: ... where (*column* is null **or** *column* = "" **or** *column* = " ")

The same applies, if you query not NULL:

HYDRA SQL: ... where (*column* is not null **and** *column* != "")

or

HYDRA SQL: ... where (*column* is not null **and** *column* != "" **and** *column* != " ")

2.8.10 Sort by NULL values

Contrary to SQL Server where NULL values are returned on top of the column on sorting, ORACLE sorts the NULL values at the bottom. Use the function *nvl()* to change this:

HYDRA SQL: ... order by **nvl(char_column, " ")**, **nvl(number_column, -1)**;

To enable this change with ORACLE, enter a space character (not an empty string!) as replacement value for columns of type *char*. The example above states -1 with numeric columns. If necessary, you must select a lower value. With SQL Server, this function is therefore deleted in the statement (only in *order by*).

2.8.11 Sorting with union select

If you use *union select* in a statement for sorting, you can use either the column number, the column name or the alias in the *order by* clause.

We recommend to create and use an alias with columns of a *union* that you want to sort.

2.8.12 Group by calculated expressions

If you want to group by a calculated expression, ORACLE and SQL Server expect the calculated expression in the *group by* clause. To integrate this conveniently in HYDRA SQL and to be independent of databases in the future, HYDRA SQL uses the alias to state calculated expressions in *group by*:

HYDRA SQL: `select column + 2 alias from table group by alias;` ORACLE +
SQL Server: `select column + 2 alias from table group by column + 2;`

2.8.13 Outer Join

Use the ANSI syntax for outer joins. All databases can process this syntax and for this reason it need not be implemented:

HYDRA SQL: `... from table1 a left outer join table2 b on a.column = b.column`

If you use an outer join to read in another table the name of a value of the first table, you can use a so-called lookup:

`select column1, (select column2 from table2 b where a.column1 = b.column1) from table1 a ...`

A lookup is a subselect, which replaces a column. You **must** ensure with this subselect that at least 1 data record is found, otherwise an SQL error is created. If no data record matches the condition, a NULL value is returned (similar to an outer join).

To provide compatibility, an outdated independent HYDRA SQL syntax is supported, which combines the outdated syntaxes of INFORMIX and older Oracle versions:

HYDRA SQL: `... from table1 a, outer table2 b where a.column = b.column (+)`
ORACLE: `... from table1 a, table2 b where a.column = b.column (+)`
SQL Server: `... from table1 a, table2 b where a.column *= b.column`
[INFORMIX: `... from table1 a, outer table2 b where a.column = b.column`]

2.8.14 Temporary tables

2.8.14.1 Overview

Temporary tables are tables that are only valid for the current database connection and include an intermediate result. The tables are automatically deleted when the program is finished, if not yet done. The following two sections show 2 possibilities how to create a temporary table.

2.8.14.2 Restrictions

For technical reasons, the number of temporary tables is limited to 12 for one database connection.

Also programs running as services or deamons have this limit; for example the "hymw" services of the data collection. This means: When an input dialog is processed, a maximum of 12 temporary tables can be created and used at a time. If you do not explicitly drop these temporary tables, these tables decrease the number of temporary tables available in the program until the service or daemon is stopped. In this case, for example in user exits of the data collection, it is absolutely necessary to drop the temporary tables when they are not required any more to avoid problems with other software parts that also require temporary tables.

If more than 12 (or almost 12) temporary tables are required at the same time, it is more secure to use normal permanent tables. If you use permanent tables, the tables must either be dynamic tables with table names that are unique in the entire system or the data records included must be clearly linked to the database connection. To create a unique table name, you can add the terminal number to the table name or add it as additional column in the data records, for example.

2.8.14.3 create temp table

Create a temporary table using the command *create temp table*:

```
HYDRA SQL:  create temp table tablename (...);
ORACLE:     create global temporary table tablename_<pid> (...)
              on commit preserve rows;
SQL Server:  create table #tablename (...);
```

With ORACLE, the *global temporary tables* are available for all users. Add the PID (process ID) to the table name separated by an underscore so that the relevant process can distinctly identify the table.

2.8.14.4 select into temp

The second possibility to create a temporary table is a *select into temp*:

```
HYDRA SQL:  select ... from ... where ... into temp tablename;
ORACLE:     create global temporary table tablename_<pid>
              on commit preserve rows as select ... from ... where ...;
SQL Server:  select ... into #tablename from ... where ...;
```

2.8.15 unique / distinct

„*select unique*“ and „*select distinct*“ both work with HYDRA SQL. With SQL Server, *unique* is replaced in the statement by *distinct*.

2.8.16 like / matches

You can use *matches* as synonym of *like* in HYDRA SQL. With both comparison operators, '*' and '%' are processed as wildcard for any number of characters. '?' and '_' substitute any other character.

2.8.17 Loading and unloading data

Use the command *unload* to unload data from a database into a file. The columns of a data record are written in a row of the file separated by '|'. The command *xunload* additionally writes the row names in the first row. Use this command, if the number or the order of the columns do not coincide with those in the table into which you want to load the data. Example:

HYDRA SQL: **xunload to** *filename* select *columns* from *table*;

Use the command *load* to load such data back into the database. Use the command *fload* to load a great number of data records faster with ORACLE if the table includes a column of data type *serial*. To this end, delete the trigger and the sequence to create distinct values and recreate them after having loaded the data. Also use the command *fload* if the data to be loaded include very high values in the *serial* column because if you insert these high values, the sequence must be counted up to this value.

HYDRA SQL: **load from** *filename* insert into *table*;

Unload files may include comments with additional information. This way, you might store the table schema and the indices into the unload file. If you want to store comments, the following rules apply:

- You can identify the beginning of a comment because the row begins and ends with the character '\$'. Comment rows are therefore different to data rows which always end with the character '|'.

Each comment ends with "\$END\$" in a the separate row. Comment that only include one row are not possible.

The comments may include any information.

All comments must be at the beginning of the file and precede the row "\$COLUMNS\$".

Example:

```
$SCHEMA$
create table tablename ( column integer );
create unique index indexname on tablennane ( column );
$END$
$VERSIONS$
hymw 8.1.1.417
$END$
$DBPATCHES$
dbp mw30 18.12.2010
$END$
$COLUMNS$column|
0|
1|
```

2.8.18 create table as select

With ORACLE and SQL Server, you can create a table from a query:

HYDRA SQL: **create table** *tablename* **as** select ... from ...;

SQL Server: select ... **into** *tablename* from ...;

2.8.19 CASE in the select clause

In the SQL-92 standard, the CASE function is implemented in SQL. Using the CASE function, you can make decisions on result set level.

Example:

In MDE log records ("ereignis" table), yield and scrap quantities may only be taken from the end-of-shift records. However, the duration of statuses has to be determined from the "N" and "P" records. Therefore, our previous programs have cumulated the quantities in a separate UNION. This problem can be solved by an SQL statement using CASE syntax. This provides fundamental performance benefits.

```
select masch_nr,
       sum(dauer),
       sum(case when (satzart = 'N') then zaehler1 else 0 end) gut,
       sum(case when (satzart = 'N') then zaehler3 else 0 end) aus
from ereignis
where masch_nr like '%'
      and bmktonr between 1 and 11
      and satzart in ('P', 'N')
group by masch_nr;
```

The following example returns name and first name separated by comma but only if a first name exists in the HR master.

```
select case when (person_vorname is null) or (person_vorname = '')
           then person_name
           else person_name || ', ' || person_vorname
           end
from personalstamm;
```

1.24 Integer division

ORACLE returns a float if you perform a division of two columns including integer data types. With SQL Server, the result and the two operands are integers:

HYDRA SQL: select 215 / 10 from setup;

ORACLE: Result = 21.5

SQL Server: Result = 21

You can avoid this difference by replacing one of the two numbers by a decimal value:

HYDRA SQL: `select 215 / 10.0 from setup;`

2.8.20 Changing tables

2.8.20.1 Adding columns

The different databases use different syntaxes to add columns:

HYDRA SQL: `alter table tablename add ( column1 integer, column2 char(1) );`

SQL Server: `alter table tablename add column1 integer, column2 char(1);`

2.8.20.2 Changing columns

The different databases use different syntaxes to change columns:

HYDRA SQL: `alter table tablename modify ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename alter column column1 varchar(20);`
`alter table tablename alter column column2 varchar(40);`

2.8.20.3 Deleting columns

The different databases use different syntaxes to drop/delete columns:

HYDRA SQL: `alter table tablename drop ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename drop column column1 varchar(20);`
`alter table tablename drop column column2 varchar(40);`

2.8.21 Reserved keyword "key"

The keyword "key" is reserved with SQL Server and DB2. If you select a column named "key", "key" must be set in double quotes.

HYDRA SQL: `select key from tabelle;`

SQL Server: `select "key" from tabelle;`

2.8.22 Default values in the database schema

If you create a table, you can define a default value for the columns.

HYDRA SQL: `create table tabelle ( column char(1) default "N" not null );`

SQL Server: `create table tabelle ( column char(1)`
`constraint df_tabelle_column default "N" not null );`

Using a default value can help to avoid unnecessary *or* statements that might create errors. In addition, you can avoid NULL values in the relevant column if you add *not null*.

If you add a column with a default value to a table, this column is automatically populated in the existing data records.

HYDRA SQL: alter table *tabelle* add (*column* char(1) **default** "N" not null);
SQL Server: alter table *tabelle* add *column* char(1)
 constraint df_*tabelle_***column default** "N" **with values** not null);

If you subsequently add a default value to an existing column, the statement must include (because of former DB Informix) the current data type. You cannot change the data type and the default value at the same time (because of SQL Server).

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default** "N" [not null]);
SQL Server: alter table *tabelle* **add constraint df_***tabelle_***column default** "N"
 for *column*;

Note:

With ORACLE, if you change a column that already is "not null", you must not state "not null" because this would create SQL error 1442.

You can delete the default value of a column with the following statement. Note that you must first reset the attribute *not null* for the column:

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default** null);
SQL Server: alter table *tabelle* **drop constraint df_***tabelle_***column**;

Notes:

You may only include one column per statement if you add and delete default values.

If you want to assign a default value to several columns of a table, you must use several individual SQL statements.

If you want to change the default value of a column, you must first delete the default value and then add the new default value. Here, you need not reset the attribute *not null*.

If you want to delete a column including a default value, you must first delete the default value and then drop the column (with SQL Server).

2.8.23 Process "clustered index"

With SQL Server, you can create a "clustered index" per table. With ORACLE, the keyword "clustered" is deleted from the statement:

HYDRA SQL: create [unique] **clustered** index *indexname* on ...;

ORACLE: create [unique] index *indexname* on ...;

2.8.24 Optimizing "update statistics" under ORACLE

Function

As of the below mentioned program version of the ORACLE backend, you can control the functioning of the command "update statistics" via environment variables. Using these variables, you can change from COMPUTE to ESTIMATE processing and vice versa. COMPUTE uses all data records of a table or an index to generate the statistics. ESTIMATE only uses parts of the data records (depending on the configuration).

Configuration

You control the activation of the extended functionality via environment variables.

UPD_STAT_NUM_ROWS ... Once this number of entries in a table is reached, it is changed to ESTIMATE.

UPD_STAT_ESTIMATE_PERCENT ... Percentage for ESTIMATE (value range between 1 and 100)

Example for Windows

```
set UPD_STAT_NUM_ROWS=10
set UPD_STAT_ESTIMATE_PERCENT=20
```

Example for Unix

```
export UPD_STAT_NUM_ROWS=10
export UPD_STAT_ESTIMATE_PERCENT=20
```

You can set the environment variables as described above in the script hy_env.scr (UNIX) or under Windows as system variable or entry in the registry.

Activation and default values

UPD_STAT_NUM_ROWS	UPD_STAT_ESTIMATE_PERCENT	Executed syntax
not set	not set	Old syntax (corresponds to COMPUTE)
Value greater than or equal to 0	not set → Default: 10 (%)	New syntax (corresponds to ESTIMATE with 10 % for tables with more than UPD_STAT_NUM_ROWS data records)

not set → Default: 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for all tables)
Value greater than or equal to 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for tables with more than UPD_STAT_NUM_ROWS data records)

Note

If the number of data records in the relevant tables is smaller than **UPD_STAT_NUM_ROWS**, the "new syntax" with estimate_percent with **NULL** (→ corresponds to COMPUTE) is used.

2.9 Notes on the performance

2.9.1 Union versus union all

Use *union* to summarize data of several tables to a result set. This *union* also removes duplicate data records. To do so, the database sorts the result set by all columns. If you do not want to remove the duplicate data records with *union* or if there cannot be duplicate data records, use the syntax *union all* instead of *union*. You save a lot of time because *union all* does not sort and remove the duplicate data records.

2.9.2 Substrings in the WHERE clause

If you use substrings in the WHERE clause, you must bear in mind that ORACLE does not use an existing index for the relevant column.

ORACLE: select ... from auftrags_bestand **where** auftrag_nr[1,8] = „...“;

In this example, ORACLE does not use the index for the column auftrag_nr and performs a sequential scan in worst-case.

A possible change of the statement is:

ORACLE: select ... from auftrags_bestand
 where auftrag_nr like „...%“
 and auftrag_nr[1,8] = „...“;

2.9.3 truncate table

If you want to delete all data records of a table, use *truncate table* instead of "delete from..." with HYDRA SQL. Using this command with ORACLE, the data records to be deleted are not stored in log files and processing is accelerated.

HYDRA SQL: **truncate table** *tablename*;

2.10 Access to several databases

2.10.1 Syntax

Currently, the access to several databases is only realized for ORACLE under Linux. Open a connection to **one** additional database using the statement:

connect <connectstring> [user <username>] [password <password>];

Example: connect linux1ora user hydadm password mpdv;

Here, the parameters *user* and *password* are optional. The default value for both parameters is "hydadm". Following the example of the local database, you can override the default values using environment variables. You can use the environment variables HYDBUSER and HYDBPW to define user and password of the local database. But you must add the connect string in capital letters separated by an underscore to add an additional database (example: HYDBUSER_DEC1ORA).

Notes:

You may only open the additional database using "connect..." once the default database has been opened.

You may not use bind variables to pass <connectstring>, <username> and <password>.

To perform statements on the second database precede the statement by

at <connectstring> ...

Example: at linux1ora select projekt from setup;

Close the additional database using the statement

close database <connectstring>;

Example: close database linux1ora;

Note:

You must close the connection to the additional database before closing the default database.

2.10.2 Restrictions

The following restrictions apply if you want to access the additional database:

- You must not create, change and delete tables (mainly because of the SERIAL columns). This also applies for the use of temporary tables.
- The command fload (fast load) is not allowed.
- You may only perform *update statistics* on the local database.
- Currently, this extension is only available under Linux (under Windows you still use HOLD_CURSOR=YES).

3 Server Scripting

3.1 General

HYDRA Script is a tool to create program parts that need not be edited by the MPDV software development, but that can be created and/or changed by trained MPDV employees, partners or customers.

Such a program part is also called "user exit".

The program parts are integrated in the MPDV software. Using these integrated program parts, the standard is changed or overwritten so that calculations and processing are changed according to the customer's requirements.

The calling MPDV software passes defined values to the script. The script is executed and defined values are returned.

The calling MPDV software can also provide a callback function. The callback function can be called with parameters from the script (also several times). It can be used to request data, which is dynamically identified, or to trigger particular actions. The callback function provides only the necessary actions, so as to avoid misuse of the function by the script.

The script language has a syntax which is similar to the C programming language. Certain exceptions are based on the close interaction with the databases used and the required application.



The script language does not differentiate between upper and lower case. "dprint" and "dPrint" are therefore synonymous!

3.2 Naming conventions

3.2.1 Script files



Script file names must be in lower case letters. With Linux operating systems, the user exits with file names in upper case letters are not loaded!

With multi scripts, also the appendix must be in lower case letters.

The following applies for script names: customer-specific scripts must start with "u_" unless the name is otherwise specified by its intended use. This ensures that customer-specific scripts will not be overwritten by MPDV updates at a later point in time.

Optionally, you can extend script files via project abbreviation/customer numbers, multi script appendices and scopes. The maximum possible structure is as follows. The different elements of the name are described in the sections that follow.

```

1\custom\userexit\d_a_an#pdv72#_mercedes@local.hsc
Path          +-----+ | | | | | | | | |
Base name (Name of userexit) +---+ | | | | | | |
[Appendix (Multiscript)]      +---+ | | | | | | |
[Customer apprev.]           +-----+ | | | | | | |
[Scope]                      +---+ | | | | |
File extension „.hsc“                +--+

```

1.2.1.1 Priority via project abbreviation, customer number and scopes

Script files can be customized for customers and include file name extensions with the customer number or project abbreviation. The file name extensions ensure that the user exits are not active with other customers if installed by accident. And you can identify for each script file to which customer it belongs.

The following file name extensions using customer numbers or project abbreviation are possible:

1. **No** extension of the script name using project abbreviation or customer number
2. Script name extension via **project abbreviation**; an **underscore** separates project abbreviation and script name. The script name is specified in lower case letters without leading or trailing blanks. Possible blanks in the project abbreviation must be replaced with underscores in the file name.
3. Old: Script name extension with **customer number**; the customer number is directly added to the script name.
4. Old: Script name extension with **customer number**; an **underscore** separates customer number and script name. The customer number is added to the script name without leading zeros or blanks.



The extension of script names via customer numbers is outdated. It is common to use the project abbreviation.

The system ignores script files with file name extensions that include invalid customer numbers and project abbreviation or that are not compatible with the system.

The scripts can be available in different scopes. The valid scopes are the following:

1. **<none>** no file extension via scope. This is the standard for scripts supplied by MPDV.
2. **@var** „Value Added Reseller“: e.g. scripts developed by partners for specific use cases, which replace the script files supplied by MPDV without specification of scope.
3. **@local** For scripts that are valid in this customer server and that replace the scripts of all other scopes. If customers make developments, these developments are also included in the "@local" scope.

The system ignores script files with file name extensions that include invalid scopes.

The file name extensions with project abbreviation/customer number and scope result in a priority. The script file with the highest priority is executed and completely replaces the script files of lower priority.

Priority	Scope	Extension customer number/project abbreviation	Example, directory <sysnr>/custom/userexit
1 (highest)	@local	Old: Customer number separated by underscore from the script name.	d_a_un_61123@local.hsc
2	@local	Old: Customer number added to the script name.	d_a_un61123@local.hsc
3	@local	An underscore separates project abbreviation and script name.	d_a_un_kunde@local.hsc
4	@local	None	d_a_un@local.hsc
5	@var	Old: Customer number separated by underscore from the script name.	d_a_un_61123@var.hsc
6	@var	Old: Customer number added to the script name.	d_a_un61123@var.hsc
7	@var	An underscore separates project abbreviation and script name.	d_a_un_kunde@var.hsc
8	@var	None	d_a_un@var.hsc
9		Old: Customer number separated by underscore from the script name.	d_a_un_61123.hsc
10		Old: Customer number added to the script name.	d_a_un61123.hsc
11		An underscore separates project abbreviation and script name.	d_a_un_kunde.hsc
12 (lowest)		None	d_a_un.hsc

If you use the *multi scripts* described in the following section, only the separate *components* of the multi scripts have a specific priority. The priorities of other components do not influence a component.



The extension of script names via customer numbers is outdated. It is common to use the project abbreviation.

1.2.1.2 Multi Scripts

You use *multi scripts* to execute several script files in a specified order using one script user exit. A *multi script* includes several *components*. The components are specified using the basic name of the script file and an additional appendix. The appendix identifies each *component*. Also the standard user exit without extension via appendix is a *component of the multi script*.

The appendix has the following structure:

d_a_an#<-><appendix>#.hsc

- # mandatory character to start the appendix
- <-> optional minus character to sort the call sequence
before the standard (default: sorting after the standard)
- <appendix> name extension, e.g. "pdv72".
- # mandatory character to close the appendix.

The appendix also specifies the call sequence of the components in a multi script.

Example of a BAPI user exit d_a_an.hsc. All user exit files listed are executed in the specified order:

Position	Description	Example
1	Component with minus sign in the appendix. If several components are available, the components are sorted in alphanumeric order of the appendix.	./custom/userexit/d_a_an#-qmidi#.hsc ./custom/userexit/d_a_an#-zksxy#.hsc
2	Component without name extension	./custom/userexit/d_a_an.hsc
3	Component without minus sign in the appendix. If several components are available, the components are sorted in alphanumeric order of the appendix.	./custom/userexit/d_a_an#qmidi#.hsc ./custom/userexit/d_a_an#zksxy#.hsc

The priorities resulting from project abbreviation/customer number and scopes as described in the previous section, are only valid within the different *components* of the multi script. The priorities of other components do not influence a component.

In case of include files, the script library does not check if an appendix is available. This appendix must be specified in the include directive in the higher-level file, if required. Existing include files with appendix are ignored, if the files are not explicitly included with the appendix in the name.

Return values of the multi script functions executed

If a function has a return value that is evaluated by the calling program, the function in a script must know the return value of a function with the same name in a different multi script file that might have been executed previously. Only then, the function can react to this return value or forward the value and does not overwrite this value.

This is especially important with user exits for plausibility checks of BAPIs and dialogs.

Example:

A customer-specific user exit `b_anr_kunde.hsc` sets a customer-specific plausibility error 424 "Invalid XXX" in the function `bapi_check_before()`. If a multi script user exit `b_anr#pdv72#.hsc` also includes the function `bapi_check_before()`, then the function is executed after that. If the function just returns the value 0 and does not integrate the return value of the previous function, the previously identified customer-specific plausibility error is overwritten.

In a script, you can access the return value of the previous function using the **import variable ERRORCODE**. If no other multi script function has been run before, the variable is initialized with the value 0.

```
...
import ERRORCODE          long;
...

long dlg_check_before()
{
    ret          long;

    ret = ERRORCODE;

    // perform further plausibility checks if no error has yet occurred
    if( ret = 0 )
    {
        if( XXX = YYY )
        {
            ret = 1023; // P_DARF_A_NICHT_UNTERBRECHEN = Person is not allowed to interrupt
        }
    }

    ...

    return ret;
}
```

Multi script behavior with errors (syntax and run time)

- **Syntax error** can be detected prior to execution when reading the components of a multi script. If a syntax error occurs in a component of a multi script, the user exit is regarded as incorrect and none of the components of the multi script are executed.
- Runtime errors can only be detected during execution. The first runtime error in a component (for example, division by 0) causes the system to terminate. The components before the incorrect component are executed and also export to the host variables. All components after the run time error are not executed.

Example for multi scripts

There is a user exit for the logon of operations "d_a_an.hsc", which MPDV provides for different customers with different contents, e.g. "d_a_an_mercedes.hsc". A standard extension for the PDV version 7.2 is also integrated in this user exit. This extension must always be executed. It does not matter if a customization already exists in this user exit or not. Without multi scripts, customization with project abbreviations as file name extensions would overwrite standard PDV processing, since PDV cannot be delivered with project abbreviations.

Solution:

The standard extension of the PDV version 7.2 is created as a component of a multi script with appendix #pdv72#, i.e. with file name "d_a_an#pdv72#.hsc". If a customer does not have any customizations, only "d_a_an#pdv72#.hsc" is executed. With the customer of the example with customization "d_a_an_mercedes.hsc", the system first executes "d_a_an_mercedes.hsc", then "d_a_an#pdv72#.hsc".

3.2.2 Identifiers in the script

It is common practice to write import and export variables in upper case letters. With other variable names, you usually write the name in lower case letters.

With names of variables, functions and other names with a general content: Prefix the names with customer initials or "u_". This ensures that the customer-specific script can still be executed if a function or another identifier with the same name is added to the script language.

Example:

In a customer script the function `get_date()` was defined. Also the script language was later extended and the function `get_date()` was added with the new data type `datetime`. The customer script could no longer be executed. A problem like this is easy to avoid. The function in the customer script must only be included in the customer name space. To do so, you must prefix the function name with a "u_" or integrate the customer initials in the function name. Correct function names would have been, for example for customer "MERCEDES":

```
u_get_date();
mercedes_get_date();
get_mercedes_date();
```

3.3 Structure of a server script

3.3.1 Overview

A script file includes the following sections:

Header	This section includes a single statement and specifies the HYDRA script language. At the moment the only script language available is "hydra basic".
Global data definitions	In this section, global data can be defined which is valid for the entire script.
Definition of functions	Functions are defined in this section. At least one function must be defined, i.e. the "main" function. You can define any number of additional functions.

A script must contain the header and function definition sections.

Example of a minimum script:

```
hydra basic;

long main()
{
    dprint( "Hello world!" );
    return 0;
}
```

3.3.2 The header

In HYDRA scripts, the header section is always composed of the keywords "hydra basic;".

```
hydra basic;
```

3.3.3 Global data definitions

In this section, global data can be defined which is valid for the entire script.

This data represents variables. There are three possible types of variables:

- 1) Variables, which are predefined by the calling MPDV software (import variables).
- 2) Variables, which are queried by the calling MPDV software during or after execution of the script (export variables). These export variables can also be predefined by the calling the HYDRA software.
- 3) Simple script variables that are neither predefined nor queried by the calling MPDV software.

```
import    input_value    double;
export    output_value    double;
variable  internal_value double;
```

The exact procedure for the declaration of data is described in a section below.

3.3.4 Definition of functions

This section defines functions:

```
long my_hello_world( par1 char(20), par2 long )
{
    dprint( "My Hello world!" Parameter: " || par1 || ", " || par2 );
    return 1;
}

long main()
{
    retval long;

    retval = 1;

    retval = my_hello_world( "Called by \"main\"", retval );

    return retval;
}
```

The exact procedure for the declaration of functions is described in a section below.

3.4 Programming aids

3.4.1 Adding comments

You can add comments to any part of the script. You can use comments consisting of a single line or multiple lines.

A multi-line comment is put between `/*` and `*/`. A single-line comment begins with `//` and ends at the end of the line.

Nested comments are not allowed.

```
/*  
  This is a multi line  
  comment.  
*/  
  
// This is a single line comment.  
  
x = x + y; // This is a single line comment, too.
```

3.4.2 Include files

You can add further files via the directive **#include "<FileName>":**

```
#include "util_abc.hsc"  
...  
#include "util_xyz.hsc"
```

With the file names and the specification of the directory, you use the same storage and naming conventions as for the main script files, but include files cannot have a name extension for multi scripts (appendix). If an include file has an appendix, this file is ignored.

Only use the include files, if an organization of the scripts has considerable advantages. If you do not use includes with simple scripts, you improve transparency and clarity and avoid dependencies.

You can include an include file at any place of a script.



At the moment, if a run time error occurs in an included file, the system does not issue the file name of the include file for technical reasons, but the file name of the higher-level file.

3.4.3 Identifying the version of the script interpreter

Some features of the HYDRA script language have been added later on. These features are only available as of a specific version of the interpreter. This information is documented at the relevant places.

Because the script interpreter is directly integrated in the different programs on the server, different versions of the script interpreter can be active in a system according to the update status of the different programs. There are two methods to identify the version that is available in a program or user exit:

- 1) In a HYDRA script using the integrated function `hysysinfo()`.
- 2) Via activation of the logging of the server program.

Explanation

Integrated function `hysysinfo()`

You can identify the interpreter version in a HYDRA script using the integrated function `hysysinfo()` with the ID `HYDSCR.VER`:

```
hydra basic;
```

```
long main()
{
    dprint( "Version: " || get_bapi_val( hsysinfo(), "HYDSCR.VER" ) );
    return 0;
}
```

Output (example):

Version: 84166

Logging of server programs

Usually, this method is only used by MPDV employees. If the logging is active with a server program, the system outputs the interpreter version when the script is loaded:

```
...
..dscr_lib.c(18060):    0 > script prepare from file - Version 84166 -----
..dscr_lib.c(18061):    0   File: ".\test_example.hsc"
...
```

3.5 Declaration of data

3.5.1 Supported data types

The following data types are available:

long

Variables of type long are integers. These data types are also used for times and dates and they then contain the time in seconds.

The permitted value range is between –2147483647 and 2147483647.



For the data type long, the value -2147483648 is reserved for the zero value of the database.

long64

Variables of type long64 are integers with a value range that is greater than the value range of data type long. Variables of this data type are mainly used for internal 64-bit IDs of data records in the relational database (columns of type "bigserial")

The permitted value range is between –9223372036854775807 and 9223372036854775807.



For data type long64, the value -9223372036854775808 is reserved for the zero value of the database.



The data type *long64* is available as of HYDRA script version 87196. See section "3.4.3 Identifying the version of the script interpreter". This version is available for most server programs as of service pack 14 (approx. 2019). In specific cases, the version is available even

earlier for specific user exits after consulting with MPDV and after check of the program versions.



To exchange data of data type long64 with the database, the database interface installed must support 64-bit integers. The version of the initial installation of the system specifies if this is possible. You can identify if the database interface, which is active in your system, supports a data exchange of 64-bit integers. Use the integrated function `hysysinfo()` with the ID `DB.ITF64BIT` at run time of the script to this end.

double

Variables of type *double* are floating point values. The precision of the values can include up to 16 places after the decimal point, but sometimes, as a result of implicit type conversions into character strings, a precision of only up to 6 places after the decimal point can be guaranteed.

char(int1)

Variables of type *char(int1)* are character strings. The integer *int1* defines the maximum length of the character string. The greatest length that is available in HYDRA scripts is 32767 characters.

date

Variables of type *date* contain date values. Date values are used for calculations. You can subtract the values or you can increase or reduce the values by integer values.

datetime

Variables of type *datetime* include a date value and a time, i.e. time stamp. The time is precise to the millisecond. The text output format is "mm/dd/yyyy hh:mm:ss.ccc". To calculate with variables of this data type, specific restrictions apply. See the description of the arithmetic operators.

3.5.2 Global variables

Global variables are declared at the beginning of the script.



Functions may not have the same name as global variables or constants.

Local variables may not have the same name as previously defined functions.

Syntax

<code><type_of_variable> <name> <data_type></code>
--

Meaning

<type_of_variable>

The variable type specifies if the calling MPDV software can predefine or query a variable:

Variable type *import*: The MPDV software can predefine the variable.

Variable type *export*: The MPDV software can predefine and query the variable.

Variable type *variable*: The MPDV software cannot predefine or query the variable.

You can only use the variable in the script itself. You can drop the key word *variable*.

<name>

The name of the variable. The name must begin with a letter, and is permitted to contain the letters from a – z, the digits from 0 – 9 and the “_” underscore symbol. Other special characters or umlauts are not allowed. Please also see the naming conventions in section 1.2.

It is common practice to write import and export variables in upper case letters. With other variable names, you usually write the name in lower case letters.

<data_type>

Specifies the data type of the variable. The supported data types are listed above.

Example

```
import    INPUT_VALUE    double;
export    OUTPUT_VALUE    double;
variable  some_value      double;
          another_value    double;
```

3.5.3 Local variables in functions

You declare local variables in functions in a function definition like the global variables. Only variables of type *variable* are permitted.



Functions may not have the same name as global variables or constants.



Local variables may not have the same name as previously defined functions.



Local variables may have the same name as global variables, but in this case, the “variable” keyword must be used in the declaration of the variable.

See the naming conventions in section Naming conventions.

Example

```
...
variable text char(20);
...

long main()
{
    variable text char(12);
    retval long;
    // In this function only the local variable "text" is accessible. The global variable "text" is "invisible".
    ...
}
```

3.5.4 Function parameters

You declare function parameters with a syntax that is similar to the one of global variables. But you use commas instead of semi-colons to separate the declarations. Only variables of type *variable* and *export* are permitted.

Variable type *variable*: The parameter is passed to the function and is then available as a local variable. Changes to the local variable do not affect the part of the program that calls the function ("call by value").

Variable type *export*: The parameter is passed to the function and is available as a local variable. Changes to the local variable affect the value in the part of the program that calls the function ("call by reference").

The naming conventions are the same as for local variables. Please also see the naming conventions in section 1.2.

Example

```
long calculate_value( par1 char(20), export par2 long )
{
    dprint( " Parameter: " || par1 || ", " || par2 );

    par1 = "SomeText";
    par2 = par2 * 2 + 17;

    return 1;
}

long main()
{
    value long;
    text char(20);
    retval long;

    value = 2;
    text = "Hello";
    retval = calculate_value( text, value );

    return value;
}
```

In the example, the statement "*par2* = *par2* * 2 + 17;" in the "*calculate_value*" function, also affects the contents of the variable "*value*" in the "*main*" function. The statement "*par1* = *SomeText*"; in the "*calculate_value*" function has no effect on the contents of the variable "*text*" in the "*main*" function.

3.5.5 Constants

Syntax

```
const <name> = <value>;
```

Meaning

<name>

The name of the constant. The name must begin with a letter, and is permitted to contain the letters from a – z, the digits from 0 – 9 and the “_” underscore symbol. Other special characters or umlauts are not allowed. Please also see the naming conventions in section 1.2.

<value>

Specifies the value of the constant. The constant can be of type *long*, *long64*, *double* or *char(n)*. An integer is interpreted as data type *long* if the value range is not exceeded. If the value range of data type *long* is exceeded, a constant of type *long64* is created.

Example

```
const    buffer_size = 1000;
variable buffer      char(buffer_size);
...
const    factor      = 2.5;
const    designation = "MyText";
...
for( i = 1 to buffer_size )
{
    ...
}
```

3.5.6 Implicit type conversions

HYDRA script performs implicit type conversions. Character strings can be converted into numbers or date values, if the character strings are correctly formatted. All of the other data types can also be converted into character strings. You can convert numerical values into date values and vice versa. If you convert floating point numbers into integers, the decimal places are cut off.

Interpretation of character strings

If HYDRA script must interpret character strings as numeric values, date or time stamp, the format of the character string is checked. If the string matches the format of the *datetime*, *date*, *double*, *long* or *long64* value, the character string is converted into the relevant data type. The following formats of character strings are supported. The formats are listed in order of priority. This means: If a character string matches the format of the data type *datetime*, it is interpreted as *datetime* and the system does not try to interpret the string as *double*, for example.

datetime

The string needs to have the format "mm/dd/yyyy hh:mm:ss.cc and may include trailing blanks.

date

The string needs to have the format "mm/dd/yyyy and may include trailing blanks.

double

The string must include the following components:

- Optional, leading blanks.
- Optional algebraic sign
- One or several numbers (pre-decimal places)
- Decimal separator dot or comma
- Optional numbers (decimal places)
- Optional trailing blanks

long / long64

The string must include the following components:

- Optional, leading blanks.
- Optional algebraic sign
- One or several numbers
- Optional trailing blanks

If the value range of data type *long* is respected, the string is interpreted as data type *long*. Otherwise, it is interpreted as *long64*.

If the implicit conversion of a character string into the required target data type is not possible, the HYDRA script is interrupted with a run time error. If incorrect character strings are expected, use the explicit conversion functions *char2<type>*, which are not canceled in case of an error. These function return the NULL value.

Special features with the *datetime* data type:

Special rules apply for the implicit conversion of *datetime* values into other data types:

datetime* -> *char(n)

The target variable includes the *datetime* value in the format "mm/dd/yyyy hh:mm:ss.ccc".

datetime* → *double

The target variable includes the *datetime* value as floating point number. In front of the decimal separator is the number of days since 01-JAN-1900, after the decimal point the time in days (0,5 → 12:00, 0,75 → 18:00 hours).

datetime* → *long* or *long64

The target variable includes the time in seconds from the *datetime* value.

datetime → date

The target variable includes the date from the *datetime* value.

char(n) → datetime ->

The format of the *char* variable is analyzed. If the string corresponds exactly to the format of a *datetime*, *date*, *double* or *long* value, the conversion rules are applied for the respective data type.

The valid formats are described with the arithmetic operators.

double → datetime

The floating point number is interpreted as time stamp. The *datetime* value is populated with the date and time.

long or long64 → datetime

The integer variable is interpreted as time in seconds. Only the time value is used to populate the *datetime* value. The date is set to 31-DEC-1899.

date → datetime

The date is set in the *datetime* value. The time is set to 00:00.

3.6 Exchanging data with the MPDV software that calls the script

Global import and export variables are used to exchange data between the HYDRA script and the MPDV software that calls the script. The declaration of global variables is described above.

You can also use callback functions. They are described below.

3.6.1 Import variables

Import variables are used to pass values from the calling MPDV software to the script.

For each user exit, separately defined import variables are available. The calling MPDV software tries to populate all defined import variables in the script. All import variables that you want to use in the script must be declared in the script. Only then, the calling MPDV software can successfully predefine the variables.

3.6.2 Export variables

Export variables are used both for passing data from the calling MPDV software to the script and for passing data from the script to the MPDV software that calls the script.

For the predefinition of values, the same rules apply as for import variables.

The content of export variables can be queried by the calling MPDV software at the end of the script or when a callback function is called.

For each user exit, separately defined export variables are available. The calling MPDV software tries to query the contents of all defined export variables in the script. All export variables that you want to use in the script must be declared in the script. Only then can they be successfully assigned in the script and queried by the calling MPDV software.

3.7 Definition of functions

Syntax

```
<data type of result> <function name> ( <function parameters> )  
{  
  <local variables>  
  ...  
  <instructions>  
  return <expression>;  
}
```

Example

```
long calculate_value( par1 char(20), export par2 long )  
{  
  dprint( " Parameter: " || par1 || ", " || par2 );  
  
  par1 = "SomeText";  
  par2 = par2 * 2 + 17;  
  
  return par2 - 10;  
}
```

Meaning

<data type of result>

The data type of the function result. The supported data types are listed above.

<function name>

The name of the function. The same rules are used for function names and for variable names.

See the naming conventions in section Naming conventions.



Functions may not have the same name as global variables or constants.

Local variables may not have the same name as previously defined functions.

<function parameters>

You can optionally declare several function parameters here. The declaration is described above.

<local variables>

You can optionally declare local variables here. The declaration is described above.



Local variables may not have the same name as previously defined functions.

<instructions>

The statements and instructions of the function are found here.

return <expression>;

Defines the return value of the function. The *return* must be at the end of the function. <expression> is a valid expression, e.g. a variable, a constant value or a calculation.

IMPORTANT (mainly in case of MPDV customization):

When using multi scripts, the following is required: If the calling program processes a return value of a function, the program must also consider the return value of the previous multi script function. Use the import variable **ERRORCODE** to do so. This mainly affects user exits executed before and after validation checks of dialogs. For more information and examples, refer to the technical documentation of the HYDRA script and the documentations of the relevant user exits.

3.7.1 The function "main"

The "main" function must be declared in each script. It is the first function to be called when the script is executed. The main function controls the program flow of the entire script. It is currently not possible to pass any parameters to the "main" function. Also, the return value of the script has no significance. The declaration of the "main" function is therefore often similar to the following example:

```
long main()
{
    ...
    <Instructions>
    ...
    return 0;
}
```

It is intended that in future versions of the HYDRA script, the calling MPDV software will be able to pass parameters to the "main" function and evaluate its return value.

3.8 Statements

The statements specify *which* action HYDSCR performs.

Statements are composed of keywords and expressions, as is explained below in detail for each of the available statements. *Expressions* are described in more detail in another section.

Statements are usually completed with a semi-colon.

Each statement can be either a simple or a composite statement. Composite statements are groups of statements and can include other composite statements, which are enclosed in curly brackets.

3.8.1 if / else (control structure)

This statement defines a conditional branch. It calculates an expression and then executes specific statements depending on the result.

Syntax

```
if (expression) instruction1  
[else instruction2]
```

Meaning

if

is a required keyword.

expression

is a required expression, which specifies which statement is executed by IF.

instruction1

is a required simple or composite statement, which executes IF *expression* is true (i.e. unequal null).

else

is an optional keyword.

instruction2

is an optional simple or composite statement, which executes IF *expression* is false (i.e. equal to null).

Examples

```
if(value < 0 )  
{  
    sign      = "-";  
    absolute = -value;  
}  
else  
{  
    sign      = "+";  
    absolute = value;  
}  
  
if(value < 0 )  
    note = "negative";  
else  
    note = "positive";  
  
if(value < 0 )  
    deficit = 1;  
  
...
```

Notes

1.If you want to use composite statements, you must enclose the statements in curly brackets.

3.8.2 for (control structure)

Overview

The FOR statement defines a loop. It executes a simple or composite statement again and again. The `loop_index` is incremented each time the loop is executed. If the stop condition is reached, the loop is exited and the statement following the loop is executed.

Syntax

```
for( loop_index = expression1 to expression2 [step const_value1] )  
instruction
```

Meaning

for

is a required keyword.

loop_index

is the name of a variable. The FOR statement uses this variable as `loop_index`, i.e. it increments the value of the variable each time the loop is executed. After each execution of the `loop_index`, the index is incremented by the value specified via the loop interval *const_value1*. If the loop interval is not specified, the `loop_index` is incremented by the value 1.

expression1

is a required expression, which specifies the start value of the index.

expression2

is a required expression, which specifies the end value of the index.

step

is an optional keyword.

const_value1

follows the optional keyword **step** and specifies the loop interval. The loop interval must be a positive numeric constant.

instruction

is a simple or composite statement.

3.8.3 while (control structure)

Overview

The WHILE statement defines a loop. It repeatedly executes a simple or composite statement, as long as an expression is true. If the expression is false, the loop is exited and the statement following the loop is executed.

Syntax

```
while (expression)
instruction
```

Meaning

while

is a required keyword.

expression

is a required keyword. As long as this expression is true, *while* executes the loop. If the expression becomes false, the first statement following the loop is executed.

instruction

is a simple or composite statement.

3.8.4 = (assignment)

Assigns a value to a variable.

Syntax

```
Variable [num_expression,num_expression] = expression;
```

Meaning

Variable

is a required variable name that has been declared beforehand.

num-expression

is an optional numerical expression or a list of one or two numerical expressions. The expressions define a substring of the CHAR variable, to which LET should assign a value. Substring operations can only be used together with a CHAR variable. They must be placed in square brackets.

=

is a required keyword.

expression

is an expression with a result that is assigned to the variable.

Notes

1. If a value with decimal places is assigned to a long or long64 variable, the decimal places are cut off without rounding.
2. For further information on conversions of data types, refer to section "3.5.6 Implicit type conversions".

Example

```
start = ( ( i - 1 ) * width ) + 1;
end = start + width - spaces - 1;
line1[start,end] = v_sometext;
i = 1;
```

3.8.5 pprint (output to log file for test purposes)

For test purposes, pprint outputs an expression to a log file. This function is designed mainly for internal use during the development and test phase of a script.

The name of the log file can be passed to the function. If the name of the log file remains empty, then the script name is used that is defined by the MPDV software. If the name of the log file is invalid, the name "hydscr.prt" is used. The log file is stored in the "err" directory of the system directory on the server (e.g. d:\mip1\1\err). The file size is monitored. If the defined maximum volume (status 16-OCT-2017: 1 MB) has been reached, the log file is renamed into "*.bak".

If the file name includes empty spaces, these are automatically replaced by underscores.

Syntax

```
pprint( file_name, expression );
```

Meaning

<i>pprint</i>	is a required key word.
<i>file_name</i>	is the name of the log file. See the rules listed above.
<i>expression</i>	is a required expression that is output on the screen.

Example

```
pprint( "../err/hydscr_X.prt",
        "Logging to file ../err/hydscr_X.prt." );
pprint( "%&$$.prt",
        "Logging to this file not possible due to invalid file name. Log to file ../err/hydscr.prt." );
pprint( "", "Log file name not specified. Use script name as file name." );
```

Results e.g. in log file:

```
22.02.05 11:16 PID 1888 "<script_name>": Logging to file ../err/hydscr_X.prt.
```

Date, time, process ID and script name are automatically added to the log text.

3.8.6 print(screen output)

Outputs an expression to the screen. This function is useful for scripts that are run by the interpreter `hydsr.exe/out`. This function does not output linefeeds; these must be output explicitly using the “\n” special character. This function always returns 0.

Syntax

```
long print( expression );
```

Meaning

<i>print</i>	is a required keyword.
<i>expression</i>	is a required expression that is output on the screen.

Example

```
ret = print( "Evaluation date " || eval_date || "\n" );
```

3.8.7 dprint (screen output for test purposes)

Outputs an expression to the screen for test purposes. This function is mainly designed for internal use by developers.

Syntax

```
dprint(expression );
```

Meaning

<i>dprint</i>	is a required key word.
<i>expression</i>	is a required expression that is output on the screen.

Example

```
dprint( " Parameter: " || par1 || ", " || par2 );
```

3.8.8 eprint (output in error log)

The *eprint* statement results in an output in the error log file of the calling MPDV software. A careful use is recommended. Only use it in a real error case because such error log files are sometimes checked by MPDV employees. If error log files are found, they usually refer to serious system errors.

The *pprint()* statement is available for application specific logging via scripts.

The error log file is stored in the "err" directory of the system directory (e.g. d:\mip1\1\err) on the server. It has the name of the calling MPDV software and the extension ".err".

Syntax

```
eprint( expression );
```

Meaning

<i>eprint</i>	is a required keyword.
<i>expression</i>	is a required expression, which is written to the error log file of the MPDV software.

Example

```
eprint( "Test of error logging " );
```

Results in an entry in the error log file:

```
03.02.2005 09:54:48.021      0 Script "<script_name>": Test of error logging
```

Date, time, SQL code and script name are automatically added in front of the error text.

3.8.9 system (system calls)

You can use this statement to call any other program.

Syntax

```
system( expression );
```

Meaning

<i>system</i>	is a required key word.
<i>expression</i>	is a required expression, which is passed to the system command as a string.

Notes

The **system** statement is only released for specific user exits. If you use the system statement in user exits where the statement is not released, the user exit is canceled with an error message.

MPDV cannot assume any responsibility if this command calls programs that can cause data loss and system inconsistencies! We recommend to use this function only to call well-known reports and lists and not to call programs that can actively change data.

The expression given after the "system" keyword is formatted as a command and executed on the operating system. This way, it is possible to execute any program on the server. You can use the keyword "sysresult" to access the return value of the operating system for the program called. This keyword is described in a separate section.

Under Windows as server platform, shell scripts are executed via a shell emulation (sh.exe). Calls of Linux programs with the ".out" file extension are converted into calls to Windows programs with the ".exe" extension.

You can start programs that are run in the background if you add a "&" character to the command (Linux convention).

Example

```
system( "hyt_ztnw.out 0 99999999 % % % " ||  
        " 01_2017 1 999 " );  
  
dprint( "Rasult: " || sysresult() );
```

3.8.10 sleep (waiting, execution pause)

Using this statement, the execution of the script pauses for a specified time. The system waits until the specified time has passed.

Syntax

```
sleep( milliseconds );
```

Meaning

<i>sleep</i>	is a required keyword.
<i>milliseconds</i>	time in milliseconds, which should be waited. See notes.

Notes

Carefully use the **sleep** statement. **sleep** is only useful in scripts which are executed in the script shell (test program). If you use sleep in frequently called user exits, the time to execute the program increases considerably.

Systems under Linux can currently only wait for complete seconds. The number of milliseconds, in this case, is rounded to the nearest complete second. Under Windows it is possible to wait for the exact number of milliseconds.

Example

```
...  
dprint( "Now it is ..." );  
sleep( 2000 );  
dprint( "... two seconds later." );  
...
```

3.8.11 sqlexec (executing SQL command)

This command executes an SQL command.

Syntax

```
sqlexec( expression );
```


SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

The **into** statement only changes the variables listed in the *variable list* if the previous SQL statement was executed without error (sqlcode() = 0).

With the select commands, the number of variables listed in the into statement must exactly match the number of selected columns.

During copying, the implicit type conversions, which are typical for the HYDRA script, are performed.

Example

See **sqlexec** statement.

3.9 Built-in functions

3.9.1 today (current date)

```
date today()
```

Meaning

Returns a date value with the current date.

3.9.2 now (current time)

```
long now()
```

Meaning

Returns the current time in seconds since 00:00 hours as an integer.

3.9.3 month (month from a date)

```
long month( date_expression )
```

Meaning

Returns the month of the date passed to the function.

3.9.4 day (day of a date)

```
long day( date_expression )
```

Meaning

Returns the day of the date passed.

3.9.5 year (year of a date)

Meaning

Returns the year of the date passed to the function.

3.9.6 weekday (weekday of a date)

```
long weekday( date_expression )
```

Meaning

Returns the weekday of the date passed. 0 = Sunday, 1 = Monday, ..., 6 = Saturday.

3.9.7 yearweek (calendar week of a date)

```
long yearweek( date_expression )
```

Meaning

Returns the calendar week of the transferred date according to ISO 8601 with a deviation from the standard for the first days of a year.



Deviation from ISO 8601

If the first part of the week in January of the year still belongs to the last calendar week of the previous year, the function does not return the value 52 or 53, but 0 (number zero). This allows sorting by year and calendar week and ensures that the combination of year and week is unique.

If your application requires the value 52 or 53 instead of 0, you can call the function again with result 0 and the last day of the previous year as parameter.

3.9.8 mdy (date from values for month, day and year)

```
date mdy ( month, day, year )
```

Meaning

Returns a date value, which is calculated from the month, day and year parameters.

3.9.9 add_bapi_val (adding ID with value to BAPI string)

```
char(n) add_bapi_val( bapi_string, id, value )
```

Meaning

This function attaches further IDs to a Bapi string with the transferred value. It returns the extended BAPI string as result.

Spaces at the beginning and end of the value are removed.

This function can also be used to compile lists (with headers):

Example 1:

```
//-----
// Build bapi string
bapi_str = "";
bapi_str = add_bapi_val( bapi_str, "DLG", "PNR.INSERT" );
bapi_str = add_bapi_val( bapi_str, "ZEIT", some_variable_or_expression );
date = "03/23/2004";
bapi_str = add_bapi_val( bapi_str, "DATE", date_var+1 );
dprint( "bapi_str: \"\" || bapi_str clipped || \"\" );
```

Example 2:

```
// -----
// build list
bapi_str = "";
bapi_str = add_bapi_val( bapi_str, "", "PNR" );
bapi_str = add_bapi_val( bapi_str, "", "NAME" );
bapi_str = add_bapi_val( bapi_str, "", "DATBIRTH" );
dprint( "LIST-Header: \"\" || bapi_str clipped || \"\" );
bapi_str = "";
bapi_str = add_bapi_val( bapi_str, "", "906000" );
bapi_str = add_bapi_val( bapi_str, "", "Doe, John" );
bapi_str = add_bapi_val( bapi_str, "", "08/10/1989" );
dprint( "LIST-Data : \"\" + bapi_str clipped || \"\" );
```

3.9.10 set_bapi_val (replacing/adding ID with value in BAPI string)

```
char(n) set_bapi_val( bapi_string, id, value )
```

Meaning

This function replaces the required entry in a Bapi string to the transferred value. The function returns the changed BAPI string as result. If the required entry is not available in the BAPI string, it is added.

Spaces at the beginning and end of the value are removed.

Example:

```
if( get_bapi_val( bapi_str, "ANR.ATK" ) = "076-374-A" )
{
    // If bapi_str does not contain id ANR.LART the entry will be appended
    bapi_str = set_bapi_val( bapi_str, "ANR.LART", "0263" );
}
```

3.9.11 get_bapi_val (identifying value of BAPI string via ID or position)

```
char(n) get_bapi_val( bapi_string, id )
```

Meaning

This function identifies the value from a BAPI string, which is specified via the ID passed. This value is returned as function result.

Notes

You can pass a **column number** in format "##123" as parameter *id* to extract specific columns of lists. The prefixed characters **##** specify that the access is made via column number. The column numbers start with 1. You can use additional modifiers to specify if only parts of the column are identified with dialog strings.

"## 12"	: Returns the complete content of the 12th column
"## 12 ai"	: Returns the acronym with the index of the 12th column
"## 12 a"	: Returns the acronym (without index) of the 12th column
"## 12 i"	: Returns the index of the 12th column
"## 12 v"	: Returns the value of the 12th column.

Example:

```
bapi_str = "DLG=TGERG.MODIFY|TGERG.PNR=906000|TGERG.BMK:1=3600|EINTRITT=01/15/1996";
dprint( "bapi_str: \"\" || bapi_str clipped || \"\" );

dlg = get_bapi_val( bapi_str, "DLG" );
dprint( "dlg \"\" || dlg || \"\" );

pnr_a = get_bapi_val( bapi_str, "TGERG.PNR" );
dprint( "pnr \"\" || pnr || \"\" );

bmk_01 = get_bapi_val( bapi_str, "TGERG.BMK:1" );
dprint( " bmk_01 \"\" || bmk_01|| \"\" );

date = get_bapi_val( bapi_str, "EINTRITT" );
dprint( "Date EINTRITT=\"\" || datum || \"\" );
```


3.9.12 test_bapi_val (checking whether ID is available in BAPI string)

```
long test_bapi_val( bapi_string, id )
```

Meaning

This function identifies whether the ID passed is included in a BAPI string.

If the ID is available, the function returns 1 (TRUE). If the ID is not available, the function returns 0 (FALSE).

Notes

A column number cannot be transferred as *id* parameter.

3.9.13 del_bapi_val (removing ID with value from BAPI string)

```
char(n) del_bapi_val( bapi_string, id )
```

Meaning

This function removes the transferred ID including the value from the BAPI string and returns the new BAPI string.

Notes

A column number cannot be transferred as *id* parameter.

3.9.14 hy_change_sep (changing separators in strings)

```
char(n) hy_change_sep(string, separator_from, separator_to, mask_charakter )
```

Meaning

This function replaces a separator with another separator in the string transferred. The function has been designed e.g. to convert a line from a CSV file with the semicolon separator into a string separated by a pipe, which then can be processed by `get_bapi_val()` or `get_list_column()`.

You can mask separators using the masking character transferred. You can use a backslash as masking character, for example. An empty masking character means that separators cannot be masked.

The function returns the string with changed separator.

Notes

- none -

3.9.15 sysresult (return value of a called program)

```
long sysresult()
```

Meaning

This function returns the value returned by the last system call, which has been made using the “system” statement.

Example

```
system( "hyt_ztnw.out 0 99999999 % % % " ||
        " 01 2000 1 999 " );

dprint( " result: " || sysresult() );
```

3.9.16 bv (embedding bind variable in SQL command)

```
char(n) bv( Variable );
```

This statement is used to embed variables as bind variables in SQL statements. If you frequently use SQL statements, which only include a different personnel number for example, the processing of the SQL statements is much faster using this bind variables. If complex statements are frequently used, the Oracle database in particular runs significantly faster with bind variables.

Meaning

bv is a required keyword.

variable is a required variable of the script, which is embedded as a bind variable in the SQL statement.

Example

The SQL statement with bind variables:

```
sqlxec( "select resps_area from hrmasterdata " ||
        " where personnelnumber = " || bv( pnr ) || ";" );
```

The SQL statement without bind variables:

```
sqlxec( "select resps_area from hrmasterdata " ||
        " where personalnummer = " pnr || ";" );
```

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

3.9.17 bvmnr (embedding machine number as bind variable in SQL command)

```
char(n) bvmnr( CharVariable );
```

Meaning

The statement is used similarly to “bv”. This statements also supports the specific formats of the machine numbers, which are specified in the basic settings as “numeric” or “alphanumeric”.

Meaning

<i>bvmnr</i>	is a required keyword.
<i>charvariable</i>	is a required variable of the script, which is embedded as a bind variable in the SQL statement. It must be a variable of type “char” that usually has a length of 40 characters.

Example

```
mnr char(40);
...
mnr = "1023";
...
sqlexec( "select designation from machines " ||
        " where masch_nr = " || bvmnr( mnr ) || ";" );
```

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

3.9.18 sqlcode (SQL error code)

```
long sqlcode()
```

Meaning

This function returns the SQL error code of the last SQL command, which has been executed using the "sql" statement.

sqlcode is a required keyword.

Example

See **sqlexec** statement.

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

The SQL error codes are different with the different database systems used (e.g. Informix, SQL server or Oracle). Some important SQL codes are standardized in the system.

These are:

0	Successful execution without errors.
100	No data record found (e.g. with select command)
239	Identified as not unique when a new data record is inserted (with insert command).

For information on the meaning of other error codes, refer to the documentation of your database system.

3.9.19 sqlerrormessage (error text of the database)

```
char(n) sqlerrormessage()
```

Meaning

This function returns the error text of the database system. You can receive an error message in plain text after SQL errors that have been identified via *sqlcode()*.

Notes

Not all database versions provide this error text. An up-to-date database interface is also required. If the database error text cannot be identified, the function returns an empty string.

3.9.20 SqlGetColNbr (number of columns in SQL result)

```
long SqlGetColNbr()
```

Meaning

This function returns the number of columns in the SQL result. Result data is returned when the SQL commands *select* and *fetch* are executed.

Meaning

SqlGetColNbr is a required keyword.

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

3.9.21 SqlColumn (transferring data from SQL command to variables)

This statement transfers single columns of the result data of an SQL command into variables of the script. Result data is returned when the SQL commands *select* and *fetch* are executed.

Syntax

```
char(x) SqlColumn( columnnumber );
```

Meaning

SqlColumn is a required keyword.

columnnumber is the number of the column that is read. Counting starts with 1.

Return value

char(x) The function returns a string with the contents of the column. Trailing blanks are removed. The string is as long as required by the contents. The string is formatted with respect to the data type used in the database, so that it can implicitly be converted into the relevant numeric data type of the script interpreter.

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

If the previous SQL command has not been executed without any errors (sqlcode() != 0), the function returns NULL.

When copied into the target variable, the implicit HYDRA script type conversions are performed.

3.9.22 sqlnumrows (number of changed data records)

```
long sqlnumrows()
```

Meaning

This function returns the number of changed data records when the UPDATE or DELETE commands are run.

Meaning

`sqlnumrows` is a required keyword.

Example

See **sqlexec** statement.

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

3.9.23 sqlstatement (last SQL command)

<code>char(n) sqlstatement()</code>

Meaning

This function returns the last SQL command that was executed.

Meaning

`sqlstatement` is a required keyword.

Example

See **sqlexec** statement.

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

3.9.24 sqlserial (data record number)

<code>long long64 sqlserial()</code>

Meaning

This expression returns the serial value of a newly added data record, if the relevant table includes a column of data type "serial" or "bigserial". These are integer columns, which are automatically assigned a sequence number and therefore always contain a unique value.

If the return value respects the value range of data type *long*, a value of data type *long* is returned. Otherwise, a value of data type *long64* is returned.

Meaning

`sqlserial` is a required keyword.

Example

```
sql( "insert into some_table " ||
      " ( col_1, col_2, ) " ||
      " values ( 9999, \"Test data\" );" );
dprint( "Serial value of inserted row: " || sqlserial() );
```

3.9.25 sqleroffset (position of an SQL error)

```
long sqleroffset()
```

Meaning

This function returns the position of the character in the SQL statement where an error first occurred.

`sqleroffset` is a required keyword, which must be written in lower case letters.

Example

```
long show_error()
{
    dprint( "Error No. " || (sqlcode() using "<<<<") || " occurred." );
    dprint( "SQL-Statement: " || sqlstatement() );
    dprint( "Error on position " || sqleroffset() );

    errcount = err_count + 1;

    return 0;
}
```

Notes

SQL statements are only released in specific user exits. If you use the statement in user exits where the statement is not released, the user exit is canceled with an error message.

With some database systems (e.g. Oracle), the interface does not provide information on the position of the error. These systems always return 0.

3.9.26 posc (searching for substring in string, case sensitive)

```
long posc(substr, string);
```

Meaning

posc searches for the substring *substr* in the string *string*. The parameters *substr* and *string* are of type char.

Meaning

If the substring is found, *posc* returns the integer index of the first character of *substr* in *string*. The index starts with 1 for the first character in the string. **posc** differentiates between upper and lower case letters. If *substr* is not available, the number 0 is returned.

Example

```
variable string      char(1000);
variable dlg_acronym char(100)
...

string = "DLG=PNR.INSERT|PNR.PNR=999999|PNR.FIR=..."

...

// Is ist an INSERT-Dialog?
dlg_acronym = get_bapi_val( string, "DLG" );
if( posc( ".INSERT", dlg_id ) > 0 )
{
    // Do something on INSERT
}
```

Notes

posc identifies blanks at the end of the string. If this is not required, you can use the operator **clipped**.

3.9.27 pos (searching for substring in string, not case sensitive)

```
long pos(substr, string);
```

Meaning

The **pos** function is similar to the **posc** function. But this function does not identify upper and lower case letters.



The **pos** function ignores upper/lower case, but umlauts of the character sets commonly used in the system are not supported properly!

3.9.28 strlen (string length)

```
long strlen( expression );
```

Meaning

strlen calculates the string length.

Meaning

strlen returns the string length **without trailing blanks**.

Example

```
len    long;
...

// result len = 5
len = strlen( " hello  " );

// result len = 5, too
len = strlen( " hello  " );

// result len = 6
len = strlen( " hello  " );
```

3.9.29 strsize (identifying the size of a char variable)

```
long strsize( char_variable );
```

Meaning

strsize returns the size that is declared for a char variable. To do this, a char variable or a char constant must be passed to the function.

Example

```
variable string1 char(100);
variable string2 char(500);
variable size    long;
...

// results to size = 100
size = strsize( string1 );

// results to size = 500
size = strsize( string2 );

// results to size = 5
size = strsize( "hello" );
```

3.9.30 strlwr (string to lower case letters)

```
char(n) strlwr( expression );
```

Meaning

Converts an expression to lower case.

expression

If necessary the expression is first converted into a string. The expression is converted into lower case letters.

Example

```
string char(1000);
...
//returns "door handle"
string = strlwr( "DOOR HANDLE" );
// returns "      2"
string char(2);
```

Notes

The umlauts of the character sets commonly used in the system are supported.

3.9.31 strupr (string to upper case letters)

```
char(n) strupr( expression );
```

Converts an expression to lower case.

Converts an expression to upper case letters. Further information can be found in the section on the **strlwr** function.

3.9.32 pow (exponentiation)

```
double pow( x, y );
```

Meaning

Exponential function y to base x (x^{**y}). "pow" calculates "x to the power of y".

- If executed without error: pow and powl return the calculated value x^{**y} .
- If x and y are 0, 1 is returned
- If x is real and < 0 , and if y is not an integer, NULL is returned.
- If x or y are NULL, NULL is returned.

With results of extraordinary size, the system can return NULL.

Meaning

x, y

Function arguments, see description above. If required, the two parameters are implicitly converted into numerical values before the calculation.

Example

```
variable x double;
variable y double;
variable r double;

...

// 5 ^ 2 gives 25
x = 5;
y = 2;
r = pow( x, y );

// 25 ^ 0.5 = square root of 25 = 5
x = 55;
y = 0.5;
r = pow( x, y );
```

3.9.33 fopen (open file)

```
long fopen( file_name char(n), mode char(n));
```

Meaning

fopen opens a file on the server and returns a handle for access to the open file. A maximum of 10 files of a script can be opened at the same time. If you try to open more than 10 files at the same time, the script is closed with a run time error.

file_name

Name of the file. The usual specifications, with absolute or relative paths, are allowed. Note that the current directory is usually the installation directory of the system. To separate directories, you can always use the forward slash "/", also under the operating system Microsoft Windows.

mode

Defines the mode of the open file. The following modes are common:

- wt** Creating a text file for write operations. If a file of this name already exists, it is overwritten.
- at** Creating a text file for write operations, or if the file already exists, opening the file to continue writing at the end of the file
- rt** Opening an existing text file, exclusively for read operations.

Return value

The function returns a file handle in the value range from 0 to 9, which can be used to access the open file with all further file operations. In case of an error, the function returns NULL. If an error occurs, you can get further information on the error reason using the functions *fileresult()* and *errno()*.

Example

```

filehandle long;
...

filehandle = fopen( "test.txt", "rt" );
if( filehandle is not NULL )
{
    // process file content
    ...

    ret = fclose( filehandle );
    filehandle = ""; // set to NULL
}
else
{
    dprint( "Invalid file name." );
}

...

```

Notes

You can also use the *fileresult()* function to check whether the file could be successfully opened. See also the example for the *fgetline()* function.

Use the function *hyfilepath()* to access files in the subdirectories of the different systems in the installation directory. The function enters automatically a system number in the file path.

3.9.34 fileresult (file operations result code)

```
long fileresult();
```

Meaning

fileresult: Similar to *sqlresult()*, this function returns the error code of the last file operation. It is mainly used to read data rows from text files.

Return value

0: The last function was performed successfully.
 -1: the file end has not been reached (EOF = end of file)
 -2: the file is not opened correctly or cannot be found.
 For further information on the reason
 use the function *errno()*.

Example

See *fprint()* function.

3.9.35 fprint / fprint_no_lf (output of a line into a file)

```

long fprint( handle long, text char(n) );
long fprint_no_lf( handle long, text char(n) );

```

Meaning

fprint outputs a text line into a file. A line break is automatically added to the text.

fprint_no_if outputs a text into a file. No line break is added to the text.

handle	Handle of the file. The handle must be valid. It must have been identified before via the <i>fopen()</i> function.
text	Text that is output. A line feed is automatically added.
Return value	NULL=error, otherwise the number of characters that is output. If an error occurs, you can get further information on the error reason using the functions <i>fileresult()</i> and <i>errno()</i> .

Example

```
variable ret long;
variable fh long;
variable line char(200);

fh = fopen( "test.dat", "wt" );
if( fh is not null )
{
    ret = fprint( fh, "This is the first line" );
    ret = fprint( fh, "This is a second line" );
    ret = fprint( fh, "And this is a third line " );

    ret = fclose( fh );
}

fh = fopen( "test.dat", "rt" );
while( fileresult() = 0 )
{
    line = fgetline( fh );
    if( fileresult() = 0 )
    {
        // Process line
        // ...
        dprint( line clipped );
    }
    else
    {
        // do not process line. End of file or some error occurred.
    }
}
```

3.9.36 fgetline (read line of file)

```
char(n) fgetline( handle long );
```

Meaning

fgetline reads a text line of an opened file. The line break is automatically removed.

handle	Handle of the file. The handle must be valid. It must have been identified before via the <i>fopen()</i> function.
Return value	The line which was read. If the variable where the line is inserted is not large enough, the line is cut off.

Example

See *fprint()* function.

Notes

You can use the function *fileresult()* to see if the function could successfully read the line. The handling is similar to the one of the SQL functions

```
sqlexec( "fetch <cursorname>;" )
```

and

```
sqlcode();
```

See also the example of function *fprint()*.

3.9.37 fflush (empty file write buffer)

```
long fflush( handle long );
```

Meaning

The operating system first buffers the file outputs in the RAM so that it does not have to access the data medium directly with every small output. **fflush** forces the physical writing of this buffer to the data medium.

handle	Handle of the file. The handle must be valid. It must have been identified before via the <i>fopen()</i> function.
Return value	1=OK, 0=error. If an error occurs, you can get further information on the error reason using the function <i>errno()</i> .

3.9.38 fclose (closing file)

```
long fclose( handle long );
```

Meaning

fclose closes a file on the HYDRA server using the handle that has been transferred.

handle	Handle of the file. The handle must be valid. It must have been identified before via the <i>fopen()</i> function.
Return value	1=OK, 0=error. If an error occurs, you can get further information on the error reason using the functions <i>fileresult()</i> and <i>errno()</i> .

Example

See *fopen()* function .

3.9.39 hyfilepath (HYDRA path with multi-system installation)

```
char(n) hyfilepath( filename char(n) );
```

Meaning

hyfilepath: The function automatically enters the system number X of its own current system in a file path. The system number is automatically entered for the following paths:

```
./prot/, ./err/, ./spool/, ./grafik/, ./custom/, ./inf_int/, ./sap_data/
.\prot\, .\err\, .\spool\, .\grafik\, .\custom\.\inf_int\, .\sap_data\
```

to:

```
./X/prot/, ./X/err/, ./X/spool/, ./X/grafik/, ./X/custom/, ./X/inf_int/, ./X/sap_data/
.\X\prot\, .\X\err\, .\X\spool\, .\X\grafik\, .\X\custom\, .\X\inf_int\, .\X\sap_data\
```

The function returns the other paths unchanged.

Example

See *rename()* function.

3.9.40 fsize (identifying file size)

```
long fsize( filename char(n) );
```

Meaning

fsize identifies the size of the file passed.

filename	Name of the file. To separate directories, you can always use the forward slash "/", also under the operating system Microsoft Windows. Substitute characters are not supported.
Return value	Size of the file in bytes. If the file does not exist, 0 is returned.

Example

```
...
ret = fsize( hyfilepath( "spool/list.txt" ) );
if( ret <= 0 )
{
    dprint( "File does not exist or is empty." );
}
...
```

3.9.41 rename (renaming file)

```
long rename( old_name char(n), new_name char(n) );
```

Meaning

rename changes the name of the file from *old_name* to *new_name*. If *new_name* specifies the name of a drive, it must be the same as in *old_name*. The directories in *old_name* and *new_name* need not necessarily be the same, so that a file can be moved from one directory to another with *rename*. Placeholders are not supported.

<i>old_name, new_name</i>	Names of the files. To separate directories, you can always use the forward slash "/", also under the operating system Microsoft Windows.
Return value	1=OK, 0=error. If an error occurs, you can get further information on the error reason using the function <i>errno()</i> .

Example

```
...
ret = rename( hyfilepath( "spool/list.txt" ), hyfilepath( "spool/verarb.txt" ) );
if( ret )
{
    // The file is available and may now be processed
    ...
    ret = unlink( hyfilepath( "spool/verarb.txt" ) );
}
...
```

3.9.42 unlink (deleting file)

```
long unlink( filename char(n) );
```

Meaning

Unlink deletes the file specified via filename. You can specify drive, path and file name via filename. Wild characters are not supported. Read-only files can not be deleted using this function. For these files, the read-only attribute must first be removed. A file must be closed before it can be deleted.

filename	Name of the file. To separate directories, you can always use the forward slash "/", also under the operating system Microsoft Windows.
Return value	1=OK, 0=error. If an error occurs, you can get further information on the error reason using the function <i>errno()</i> .

Example

```
...
ret = rename( hyfilepath( "spool/list.txt" ), hyfilepath( "spool/tempfile.txt" ) );
if( ret )
{
    // process temp file
    ...
    ret = unlink( hyfilepath( "spool/tempfile.txt" ) );
}
...
```

3.9.43 errno (system error number)

```
long errno();
```

Meaning

Returns the last system error number. This is used to analyze the errors that occurred with file operations. Important: You must call this function directly after the action that you want to analyze because each further statement can change the contents of *errno()*!

The operating system specifies the respective meaning of the error number. As an example, the following table shows the error codes of Windows. For the error numbers up to 34, the meaning is identical with Windows and Linux.

Value	Acronyms	Explanation
0	EZERO	No error occurred
1	EPERM	Operation not permitted
2	ENOENT	File not found (No such directory entry)
3	ESRCH	No such process
4	EINTR	Interrupted system call
5	EIO	I/O error
6	ENXIO	No such device or address
7	E2BIG	Arg list too long
8	ENOEXEC	Exec format error
9	EBADF	Bad file descriptor
10	ECHILD	No child processes
11	EAGAIN	Resource temporarily unavailable
12	ENOMEM	Not enough space
13	EACCES	Permission denied
14	EFAULT	Bad address
15	ENOTBLK	Block device required
16	EBUSY	Resource busy
17	EEXIST	File exists
18	EXDEV	Improper link
19	ENODEV	No such device
20	ENOTDIR	Not a directory
21	EISDIR	Is a directory
22	EINVAL	Invalid argument
23	ENFILE	Too many open files in system
24	EMFILE	Too many open files
25	ENOTTY	Inappropriate I/O control operation
26	ETXTBSY	Text file busy
27	EFBIG	File too large
28	ENOSPC	No space left on device
29	ESPIPE	Invalid seek
30	EROFS	Read only file system
31	EMLINK	Too many links
32	EPIPE	Broken pipe

33	EDOM	Domain error within math function
34	ERANGE	Result too large

3.9.44 file_get_first(), file_get_next(), filerresult(), fileclose()

You can use a set of functions, file_get_first(), file_get_next(), filerresult() and fileclose(), to evaluate text files. Using this function set, only one file can be open at a time in a script. Using this function, it is easy to read a text file row by row.



The function does not interact with other file handling functions, e.g. fprintf() or fgetline().

file_get_first()

```
char(n) file_get_first( file_name );
```

If another file has been opened before, this file is first closed. This function opens the text file passed and returns the first data row. The file remains open. You can use the function **filerresult()** to identify if the action was successful.

file_get_next()

```
char(n) file_get_next();
```

This function returns the next data row of the file that has been opened using the function **file_get_first("name")**. The file remains open. You can use the function **filerresult()** to identify if the action was successful.

filerresult()

```
long filerresult();
```

This function returns an error code that is set by the functions **file_get_first("name")** and **file_get_next()**:

0	:	No error occurred.
-1	:	End of file reached (EOF = end of file)
-2	:	File could not be opened.

fileclose()

```
fileclose();
```

This statement closes the file that has been opened using the function **file_get_first("name")**. No value is returned.

Example

```
bapi_str = file_get_first( "list.txt" );
while( fileresult() != 0 )
{
    // process bapi_str here
    bapi_str = file_get_next();
}
fileclose();
```

3.9.45 get_list_column (identifying value of a column from the data row)

```
char(x) get_list_column(char(x) header, char(x) data, char(x) key);
```

Meaning

This function uses the header to identify the position of the ID passed (key). The function then identifies the value of the field using the data row. This value is returned as function result.

Example

```
header = "DLG|TGERG.PNR|TGERG.BMK:1|EINTRITT|";
data = "TGERG.MODIFY|906000|3600|01/15/1996|";

dprint("bapi_str: \"\" || header clipped || \"\"");
dprint("bapi_str: \"\" || data clipped || \"\"");

dlg = get_list_column(header, data, "DLG");
dprint( "dlg \"\" || dlg || \"\" );

pnr_a = get_list_column(header, data, "TGERG.PNR");
dprint( "pnr \"\" || pnr || \"\" );

bmk_01 = get_list_column(header, data, "TGERG.BMK:1");
dprint( " bmk_01 \"\" || bmk_01 || \"\" );

date = get_list_column(header, data, "EINTRITT");
dprint( "Date EINTRITT=\"\" || date || \"\" );
```

3.9.46 set_list_column (setting value in a column in the data row)

```
char(x) set_list_column(char(x) header, char(x) data, char(x) key, char(x) value);
```

Meaning

This function uses the header to identify the position of the ID passed (key). The function then changes/sets the value in the data row. The changed data row is returned as function result.

Example:

```
header = "DLG|TGERG.PNR|TGERG.BMK:1|EINTRITT|";
data = "TGERG.MODIFY|906000|3600|01/15/1996|";
```

```
dprint("bapi_str: \"\" || header clipped || \"\");
dprint("bapi_str: \"\" || data clipped || \"\");

data = set_list_column(header, data, "DLG", "TGERG.UPDATE");
dprint("data neu \"\" || data || \"\");
```

3.9.47 char2long (converting char(n) to long or long64)

```
long|long64 char2long( expression );
```

Meaning

The function interprets the expression transferred as string and converts it into a long or long64 value. It identifies (in the following order):

- leading blanks and/or tabulators [ws]
- an optional algebraic sign [sn]
- a digit sequence [ddd]

The string must have the following format:

[ws] [sn] [ddd]

The conversion of characters is canceled with the first character that cannot be interpreted.

Return value:

If executed without errors the function returns the converted value of the string entered.

- If the value range of data type *long* is respected, the function returns a value of type *long*.
- If the value range of the data type *long* is exceeded, the function automatically returns a value of type long64. An overflow of *long64* is not checked (the results are not defined in this case).

In contrast to an implicit type conversion, an error does not result in a run time error with termination. If the string entered cannot be converted, the number 0 is returned.



If the string contains the value -2147483648, the return value is of type long and is interpreted as zero.

```
if( char2long(-2147483648) is null ) // allways true
```

Use the char2long64 function if you want to convert a 64-bit value where -2147483648 is a normal valid value.

If the string contains the value -9223372036854775808, the return value is of type *long64* and is interpreted as zero.

3.9.48 char2long64 (conversion of char(n) to long64)

```
long64 char2long64( expression );
```

Meaning

The function interprets the expression transferred as string and converts it into a long64 value. It identifies (in the following order):

- leading blanks and/or tabulators [ws]
- an optional algebraic sign [sn]
- a digit sequence [ddd]

The string must have the following format:

[ws] [sn] [ddd]

The conversion of characters is canceled with the first character that cannot be interpreted.

Return value:

If executed without errors the function returns the converted value of the string entered. An overflow of *long64* is not checked (the results are not defined in this case).

In contrast to an implicit type conversion, an error does not result in a run time error with termination. If the string entered cannot be converted, the number 0 is returned.

Use the *char2long64* function only if the string represents a 64-bit value and the return value is assigned to a long64 variable.



- If you assign the return value to another data type, overflows of the value range can occur at runtime.
- The detection of zero values does not work correctly if the source data type and the target data type do not match. For example, the value -2147483648 in a long64 type variable is interpreted as a normal valid value, while the same value in a long type variable causes the variable to be recognized as "is zero".

3.9.49 char2double (converting char(n) to double)

```
double char2double( expression );
```

Meaning

The function interprets the expression transferred as string and converts it into a double value. It identifies a floating point number from the following characters:

- optional leading blanks and/or tabulators
- an optional algebraic sign
- a sequence of figures
- an optional dot as decimal separator
- an optional sequence of figures after the decimal separator
- an optional exponent (e or E) including a negative or positive sign and an integer

The characters must match this format:

[whitespace] [sign] [ddd] [.] [ddd] [e|E[sign]ddd]

The conversion of characters is canceled with the first character that cannot be interpreted.

Return value:

If executed without errors the function returns the converted value of the string entered. 0 is returned if the entered string cannot be converted. In contrast to an implicit type conversion, an error does not result in a run time error with termination.

3.9.50 char2date (converting char(n) to date)

```
date char2date( expression );
```

Meaning

The function interprets the expression transferred as string and converts it into a date value. It identifies a date in the following formats:

- MM/DD/YYYY: normal format of a date
- "nnnnnn[.nn]": A numeric value is interpreted as a Julian date in days since 01.01.1970. Decimal places are ignored.

Return value:

If executed without errors the function returns the converted value of the string entered. "NULL" is returned if the string entered cannot be converted. In contrast to an implicit type conversion, an error does not result in a run time error with termination.

3.9.51 char2datetime (converting char(n) to datetime)

```
char(n) char2datetime( expression );
```

Meaning

The function interprets the transferred expression as a character string. It identifies the "mm/dd/yyyy hh:mm:ss.ccc" format.

- mm/dd/yyyy : date
- hh : hours
- nm : minutes
- ss : seconds
- ccc : milliseconds

The function returns a string that can always be used for implicit type conversion.

Return value:

If the execution is error-free

the function returns the parameter "expression".

In the event of an error

If the string entered does not match the correct format for *datetime*, an empty string (NULL) is returned.
The function does not lead to a runtime termination.

3.9.52 get_date (date from datetime)

```
char(10) get_date( datetime_value );
```

Meaning

The function returns the part including the date from the *datetime* value transferred.

Return value:

If executed without errors, the function returns the part including the date from the *datetime* value. If the variable transferred is not of data type *datetime*, the function returns NULL.

3.9.53 get_time (time from datetime)

```
double get_time( datetime_value );
```

Meaning

The function returns the part including the time from the *datetime* value transferred. Milliseconds are included as decimal places in the return value.

Return value:

If executed without errors, the function returns the part including the time from the *datetime* value. If the variable transferred is not of data type *datetime*, the function returns NULL.

3.9.54 date_time (datetime from date and time)

```
char(23) date_time( datum date|char(n)|long|double, zeit double );
```

Meaning

The function returns a *datetime* value using the date and time transferred. The date can be specified as value of the type *date*, *char(n)* or numerically as Julian date. Time can be transferred as floating point number including milliseconds.

If the date "**NOW**" is passed, the function returns the current time. If you use the Windows operating system, this time also includes milliseconds, provided that the script is not used in the environment of a deviating time zone.

```
my_now = date_time( "NOW", 0 );
```

Return value:

If executed without errors, the function returns a string in the format of a *datetime* value. The function returns NULL, if one of the parameters transferred is NULL or invalid.

3.9.55 hygetenv (access to environment variables and registry)

```
char(n) hygetenv( variable char(n), default_value char(n) );
```

Meaning

The function reads the environment variable using the registry and a default value.

Return value:

Value of the environment variable.

3.9.56 hysysinfo (system information on the server, database and software)

```
char(n) hysysinfo();
```

Meaning

Reading of system information. The system information is then available as BAPI string.

Examples:

Return with Linux:

```
HYDSCR.VER=84166|HYDRA.VER=8.10|HYDRA.PRJ=MPDV|HYDRA.SYSNR=4|DB.NAME=ORACLE|DB.VER=11.0|DB.ITF64BIT=0|HYTIMEZONE=|HYTIMESRVDIFF=0
|OS.SYSNAME=Linux|OS.NODENAME=linux11|OS.RELEASE=2.6.32.12-0.7-default|OS.VERSION=#1 SMP 2010-05-20 11:14:20
+0200|OS.MACHINE=x86_64|HYUNICODE=1|
```

Return with Windows:

```
HYDSCR.VER=84166|HYDRA.VER=8.30|HYDRA.PRJ=MPDV|HYDRA.SYSNR=7|DB.NAME=SQLSERVER|DB.VER=0.0|DB.ITF64BIT=1|HYTIMEZONE=|HYTIMESRVDIFF
=0|OS.SYSNAME=Windows_NT|OS.NODENAME=WIN2008-7|OS.RELEASE=6.1|OS.VERSION=7601 Service Pack 1|OS.MACHINE=x86|HYUNICODE=1|
```

Return value:

The system information is returned as BAPI string and include the following IDs:

Identification	Content
HYDSCR.VER	Version of the HYDRA script interpreter. This is a continuously growing, non-consecutive number. If the ID is not included, the version is 84166 or lower.
HYDRA.VER	Basic version of the system
HYDRA.PRJ	Project name included in the HYDRA basic settings.
HYDRA.SYSNR	System number
DB.NAME	Name of the database system (ORACLE / SQLSERVER)
DB.VER	Version number of the database (not with all DB systems)
DB.VER	Version number of the database (not with all DB systems)
DB.ITF64BIT	1: Interface to the database for 64-bit integer is available 0 or ID not available: 64-bit integer in the database interface is not available
HYTIMEZONE, HYTIMESRVDIFF	Information on the system's time zone.
HYUNICODE	1: Script runs in Unicode environment (the script file itself is UTF-8). 0 : Script runs in 8-bit character environment (ANSI).
OS.NODENAME	Network name of computer where the script is executed.
OS.MACHINE	Processor of computer where the script is executed.
OS.SYSNAME	Name of operating system: - "Windows NT" (also with recent Windows versions) - "Linux"
OS.RELEASE,	Version information on the operating system.

OS.VERSION

3.9.57 hy_read_ini_data (reading INI configuration)

```
char(n) hy_read_ini_data( ini_name char(n), ini_section char(n), ini_ident char(n),  
ini_usernummer char(n), default_value char(n) );
```

Meaning

Reads an entry from the INI configuration.

Return value:

If the entry is found it is returned. If the value is not found the default value is returned.

3.9.58 push_env_sql_sys(), pop_env_sql_sys ()

```
long push_env_sql_sys();  
long pop_env_sql_sys();
```

push_env_sql_sys() backs up the environment for SQL statements and system calls and stores them in a stack to this end.

pop_env_sql_sys() restores the environment for SQL statements and system calls from the stack.

The environment for SQL statements and system calls includes the internal memories, which provide the result for the functions `sqlcode()`, `sqlstatement()`, `sqlnumrows()`, `sqlserial()`, `sqlerrormessage()`, `sqleroffset()`, `into()` and `sysresult()`.

In case of a high degree of modularization, this function requires additional functions and includes. Util functions can back up the environment via `pop_env_sql_sys()` and restore it via `push_env_sql_sys()`. The util functions therefore guarantee that the current SQL code of the calling functions is not changed.

Important:

`push_env_sql_sys()` and `pop_env_sql_sys()` belong together and must always be executed in a balanced manner! If there is no balance (too many calls of `pop_env_sql_sys()`), then the script is canceled with an error message.

The function can back up a maximum of 20 environments. If there are more, the script is canceled with an error message.

Return value

The functions return the number of environments that are backed up and are in the stack.

Example

```
...  
// save SqlCode(), ...  
rc=push_env_sql_sys();  
  
sqlexec( "select some intermediate SQL statement ... ;" );  
    if (sqlCode() = 0)  
    {  
        some action ...  
    }  
  
// restore SqlCode(), ...  
rc=pop_env_sql_sys();  
  
...
```

3.9.59 set_dec_sep (setting decimal separator for „using“)

```
char(1) set_dec_sep( decimal_separator char(1) );
```

Meaning

You can use this function to set the decimal separator for the formatting of decimal numbers with the operator "using". The format string of "using" always uses a dot as decimal separator. You can use the function set_dec_sep(). The system then outputs a comma instead of a dot as decimal separator in the format string.

Only dot and comma are valid decimal separators.

Return value

The function returns the previously active decimal separator.



If you change the decimal separator, this change is valid for the entire script. If you have changed the decimal separator, you must reset the previous decimal separator as soon as possible to avoid that other functions of the script are affected.

Example

```
hydra basic;

long main()
{
    variable dec_sep_save char(1);
    variable some_string char (100);
    variable some_double double;

    some_double = 123.456;

    // Format double with komma as decimal separator
    dec_sep_save = set_dec_sep( "," );
    some_string = some_double using "####&.&&";
    dec_sep_save = set_dec_sep( dec_sep_save );

    dprint( "some_string: " || some_string );

    return 0;
}
```

Screen output:

```
some_string: 123,46
```

3.10 Operators

3.10.1 ascii (output of any ASCII characters)

Converts a numerical value into an ASCII character.

See also *ordinal*, to identify the ASCII code of a character.

Syntax

```
ascii num_expression
```

Meaning

ascii	is a required keyword.
num_expression	is a numerical expression

Example

The following statement is used to assign a £ pound symbol (ASCII code 156) to the variable "currency".

```
Currency = ascii 156;
```

3.10.2 ordinal (identifying ordinal value)

Converts to a numerical value which represents the ordinal value. This depends on the data type:

Data type	ordinal result
char(x)	Ascii code of the first character of the string
date	Days since 31-DEC-1899.
long	The number itself
double	NULL (undefined)

Syntax

```
ordinal expression
```

Meaning

`ordinal` is a required keyword.
`expression` is a random expression.

Example

The following code returns the ASCII code of the character "A".

```
Code = ordinal "A";
```

3.10.3 clipped and stripped (suppressing blanks)

clipped removes trailing blanks from a string.

stripped removes leading and trailing blanks from a string.

Syntax

```
char_expression clipped  
char_expression stripped
```

Meaning

`char_expression` is a required character expression.
`clipped/stripped` is a required key word.

Example

```
v_ort = (ad_nation clipped) || '-' || (ad_plz using "####") || " " || ad_ort;
```

3.10.4 Substrings ([and])

You can subscribe CHARACTER variables, i.e. you can make substrings. This can be done on the left or right hand side of the "=" assignment operator.

Syntax

Syntax:

```
char_variable[num_expression [,num_expression]]
```

Meaning

<i>char_variable</i>	is a required variable name, which has been declared as type CHAR.
<i>num-expression</i>	is an optional numerical expression or a list of one or two numerical expressions. They identify a substring of the CHAR variable. Substring operations can only be used together with a CHAR variable. They must be placed in square brackets.

Example

```
line = "abcdef";
line[4,5] = "xy";
line[6,6] = "z";
print( line[2,3] );
```

The example returns the output "bc". The string line is then filled with "abcxyz".



On the left hand side of an assignment, only the form is permitted that defines start and end position(`line[1,1] = "a";`).

3.10.5 Arithmetic operators

3.10.5.1 Basics

Arithmetic operators can be applied to all data types. If applied to strings, an implicit type conversion is performed, as described in section "3.5.6 Implicit type conversions". If one of the values to be compared is "NULL", the calculated result will always be "NULL".

<code><op> + <op></code>	Addition. When both operands are strings, the two strings are joined.
<code><op> - <op></code>	Subtraction
<code><op> * <op></code>	Multiplication
<code><op> / <op></code>	Division
<code><op1> modulo <op2></code>	rest of op1 divided by op2. Before calculation, the operands are rounded to integers to zero. The result has the arithmetic sign of op1
<code><op> <op></code>	Joining two strings. First, the numerical data types are implicitly converted into strings.

The use of `/` or `modulo` with 0 as the second operand results in an error. It is not possible to divide by 0.

See also the `pow(w,y)` function for exponentiation calculations.

3.10.5.2 Special features with the *datetime* data type

Not all possible combinations of operators and data types are permitted with the *datetime* data type. If a value of the data type *datetime* is on the left or right hand side of a binary operator, the following rules apply:

datetime* +/- *long

datetime* +/- *long64

datetime* +/- *double

long* + *datetime

long64* + *datetime

double* + *datetime

The numerical value is interpreted as the number of seconds. It is used to calculate the time stamp.
The result is a *datetime* time stamp.

datetime* - *datetime

The difference between the two time stamps is returned as *double* value, exact to the millisecond.

datetime* - *date

date* - *datetime

The *date* value is converted into a time stamp. The time is set to 00:00. The difference of the two time stamps is returned in seconds as *double* value, exact to the millisecond.

If a string is used to calculate a *datetime* value, the rules apply that are described in section "3.5.6 Implicit type conversions" including the interpretation of strings as numerical values of different numerical data types. For the calculation, the rules are used, which are described above for the relevant data type.

All other combinations of arithmetic operators and data types of the *datetime* data type lead to run time errors when the script is executed. The same also applies to each multiplication and division.

3.10.6 Logical comparison operators

Logical operators can be used with all data types. If different data types are compared, an implicit type conversion is performed. If one of the operands to be compared is NULL, the calculated result of the comparison will always be interpreted as false.

<code><op> = <op></code>	True if both operands are equal or both are NULL
<code><op> != <op> [or <>]</code>	True if both operands are not equal
<code><op> > <op></code>	True if the left operand is greater than the right.
<code><op> < <op></code>	True if the left operand is less than the right.
<code><op> >= <op></code>	True if the left operand is greater than or equal to the right.
<code><op> <= <op></code>	True if the left operand is less than or equal to the right.
<code><op> is NULL</code>	True if the operand is NULL.
<code><op> is not NULL</code>	True if the operand is not NULL.

3.10.7 Logical operators

Logical operators can be used with all data types. With logical operators, both operands are first converted into integers. A single operand is *true*, if it is not equal to 0.

<code><op> and <op></code>	True if both operands are true
<code><op> or <op></code>	True if at least one operand is true
<code>not <op></code>	True if the operand is false

3.10.8 using (formatting of dates to strings)

In some cases, the data must be carefully converted into specific formats in strings.

You can use this expression to format a numerical expression, a date or a time. You can use USING with a numerical expression to add decimal dots, to left-align or right-align numbers, to write negative numbers in brackets and to perform other formatting functions. You can use USING to convert a date into many different formats.

Syntax

```
expression1 using expression2
```

Meaning

<i>expression1</i>	is a required expression that specifies what is formatted by <i>using</i> .
<i>using</i>	is a required keyword.
<i>expression2</i>	is the required format character string, which specifies how <i>using</i> formats <i>expression1</i> . (See the "Notes" in this section.)

Notes

1. The format character string must be enclosed in quotation marks or it must be a variable or a constant.
2. If you try to display a number that is too long for the number of digits reserved, asterisks are displayed instead of the value to indicate oversize. The reserved number of digits is not automatically increased.

3.10.8.1 Formatting of numerical expressions

The format character string contains combinations of the following characters: & # . - +. The characters - + will "float". If a character floats, the system shows the character as single character that occurs several times as leading character in a string. This single character is output as far to the right as possible without affecting the number displayed.

& If you use this character, all positions in the output field, which are normally empty, are filled with zeros.

If you use this character, the empty positions in the output field remain empty. It can be used to specify the maximum width of a field.

< If you use this character, all numbers in the output field are aligned to the left.

. The dot is used as decimal separator. In a format string for a numerical expression, you can only use a dot. By default, the dot is issued as decimal separator. You can use the function `set_dec_sep()`. The system then outputs another character, e.g. a comma, instead of a dot as decimal separator.

- This character is a literal; USING displays it as a minus sign, if *expression1* is less than zero. If you enter several minus signs in the format string, the minus sign "floats" as far to the right as possible, without affecting the output number.

+ This character is a literal; USING displays it as a plus sign, if *expression1* is greater than or equal to zero. If you enter several plus signs in the format string, a single + "floats" as far to the right as possible, without affecting the output number.

3.10.8.2 Formatting of date expressions

The format string contains combinations of the characters d, m, and y, as shown in the following figure.

- dd Day of the month in the form of a number with two places (01-31)
- ddd Day of the week in the form of an abbreviation with three letters (Sun - Sat)
- mm Month in the form of a number with two places (01-12)
- mmm Month in the form of an abbreviation with three letters (Jan - Dec)
- yy Year in the form of a number with two places in the 20th century (00-99)
- yyyy Year as a number with 4 places (0001-9999)

3.10.8.3 Formatting of times and durations

Special format strings are used to format numeric values in seconds as times or durations, as shown in the following table:

Format string	character	Meaning
[h]h		hours (one or several letters "h"). Leading zeros are displayed for hours including up to two places.
nm		normal minutes (divided by 60)
ss		seconds
c [c]		Decimal places of seconds (milliseconds). The number of the letters "c" specifies the number of decimal places.
im[m]		industrial minutes (divided by 100). The number of the letters "m" specifies the number of decimal places.

And a special format character string is also available:

```
text = prot_dur using "$TIME";
```

This statement returns an output of the time with six digits in the format "123:59". With negative times, one digit of the number of hours is used for the sign "-": "-12:59".

Extended format specifications are also possible:

Format character string	Meaning
using "\$TIME i"	Forces the output in industrial minutes.
using "\$TIME n"	Forces the output in normal minutes.
using "\$TIME g"	Creates an output of time with ten digits for a number of hours of up to six digits for very long times: "-123456:59". This specification can be combined with the further format specifications "i" and "n".

3.10.8.4 Examples of format strings

In the following table, "X" is used as the visible symbol for spaces.

Format string	Numerical value	Formatted result
---------------	-----------------	------------------

"#####"	0	XXXXXX
"&&&&&"	0	00000
"<<<<"	0	NULL string
"#####"	123	XX123
"&&&&&"	123	00123
"<<<<"	123	123
"#####.###"	123,450	XX123.450
"&&&&&. &&&"	123,450	00123.450
"----.<<<"	-123,450	-123,450
"-----.###"	-123,450	XXXX-123.450
"&&&&&. &&&"	-123,450	00123.450
"\$TIME"	7140	XX1:59
"\$TIME"	-7140	
"\$TIME i"	7140	XX1.98
"\$TIME n"	7140	XX1:59
"hh:mm:ss.ccc"	7147.123	01:59:07.123
"hhhhhh:mm:ss.ccc"	7147.123	XXXXXX1:59:07.123
"hh,immm"	7147.123	01.98531
"hhhh,immm"	7147.123	XXX1.98531
"hh:mm:ss.ccc"	-7147.123	-1:59:07,123

The following examples show conversions of the date 24 December 2018.

Format string	Formatted result
"ddmmyy"	241218
"mmddyy"	122418
"ymmdd"	181224
"dd.mm.yy"	18.12.24
"dd/mm/yy"	24/12/18

"dd mm yy"	24 12 18
"dd-mm-yy"	24-12-18
"dd. mmm. yyyy"	24. Dec. 2018
"ddd, dd. mmm. Yyyy"	Thu, 24. Dec. 2018

3.11 The Callback function

Specific user exits provide a callback function. This function can be called from the script, e.g. in order to enable further data exchange between the MPDV software and the script. The callback function can also be called to trigger actions in the MPDV software.

The functionality of the callback function is documented separately for each user exit.

Not all user exits have a callback function. If a script tries to call a callback function, which is not defined, the script is canceled with a run time error.

Syntax

```
callback( <Function>, <Parameter> );
```

The function returns a specifically defined value for each user exit.

Meaning

callback	required keyword
<Function>	Expression, which is implicitly converted into a string in the callback function. A callback function of a user exit usually provides a range of functionalities. You use the <function> parameter to select the functionality.
<parameter>	Expression, which is implicitly converted into a string in the callback function. The callback function can use this expression as a parameter for the selected functionality.

3.11.1 Built-in Callback functions

HYDRA Script provides predefined callback functions. These functions provide access to frequently required system data, such as the basic settings.

3.11.1.1 Basic settings

Refer to the documentation of the basic settings for details on the meaning of the individual fields.

Callback function **HYDRA.SELECT**:

Parameter	Return value
PRJ	Project name
VER	Version
LEN:AUNR	Length of order number
LEN:AGNR	Length of operation number (Since the complete order number from MW 2.0 on has a complex structure, this size is no longer supported from MW 2.0 on)
LEN:KNR	Length of badge number

3.11.1.2 LLE basic settings

For detailed information on the meaning of the different fields, refer to the documentation of the LLE basic settings.

Callback function **LLE.SELECT**:

Parameter	Return value
"Options" tab	
OPT:AUS	Processing of scrap (Y/N)
VGZ:TR	Processing of standard setup time $[(m \cdot t_e) + tr]$ (Y/N)
FAKTTE	Specify t_e per XXX pieces
OPT:AKKMNR	Check if machine is suitable for piecework (Y/N)
LART:AKK	Piecework wage type
LART:KAR	Waiting period wage type
LART:EINARB	Practice wage type
LART:PZE	PZE wage type
"RPA times" tab	
OPT:IZAUSBMK	Calculate actual time from RPAs (Y/N)
IZ:BMK1 to IZ:BMK12	Calculate actual time from RPAs, RPA1 to RPA 12 (Y/N)
OPT:BMKZULART	Allocate RPA to wage types (Y/N)

RM:BMK1 to RM:BMK12	Upload part quantities for RPA1 to RPA12 (Y/N)
LART:BMK1 to LART:BMK12	Wage type for RPA1 to RPA0 12

3.11.1.3 Identifying the data type of table columns

Callback function **HYDRA.GET_DATA_TYPE**:

This callback function is used to identify the data type of a column in a database table.

The parameter contains the table name and the column name in form of a dialog string.

The data type is returned as char() variable. Example "decimal(4,2)". If the table or the column does not exist, a NULL string is returned.

Example

```
data_type = CallBack( "HYDRA.GET_DATA_TYPE", "TABLE=setup|COLUMN=version" );
if( (data_type stripped) != "decimal(4,2)" )
{
    ...
}
```

Example

For the efficient use of the add_bapi_val() function:

```
data_type = CallBack( "HYDRA.GET_DATA_TYPE", add_bapi_val( add_bapi_val( "",
                                                                "TABLE", "setup" ),
                                                                "COLUMN", "version" ) );
if( (data_type stripped) != "decimal(4,2)" )
{
    ...
}
```

3.11.1.4 Rounding of times

The callback function **ROUNDSEC** is used to round times and durations.

Parameter

TIME

Time in **seconds**.

REFTIME

Reference time in **seconds**. This is, for example, the planned start or end time according to the shift model in PZE or simply the number 0.

INT

Rounding interval **in seconds**.

LIMIT

Rounding limit **in seconds** (limit value, on which rounding up takes place).

Return value**Rounded time in seconds**

Rounding of times and durations. All times are specified in seconds.

Examples:

Time	Ref. time	Interval	Limit	Result
12:00:29	00:00:00	00:01:00	00:00:30	12:00:00
12:00:30	00:00:00	00:01:00	00:00:30	12:01:00
-00:01:29	00:00:00	00:01:00	00:00:30	-00:01:00
-00:01:30	00:00:00	00:01:00	00:00:30	-00:02:00
17:04:00	16:15:00	1:00:00	00:50:00	16:15:00
17:05:00	16:15:00	1:00:00	00:50:00	17:15:00
14:26:00	16:15:00	1:00:00	00:50:00	15:15:00
14:25:00	16:15:00	1:00:00	00:50:00	14:15:00
17:04:00	-7:45:00	1:00:00	00:50:00	16:15:00
17:05:00	-7:45:00	1:00:00	00:50:00	17:15:00
15:04:00	-7:45:00	1:00:00	00:50:00	14:15:00
15:05:00	-7:45:00	1:00:00	00:50:00	15:15:00



If you want to perform rounding to the nearest integer in case of negative durations, the rounding threshold is reversed when the time to be rounded is less than the reference time. In the Personnel Time Management (PZE), you must therefore subtract 24 hours from the reference time of working time models (e.g. start of "normal time" = 8:00). The reference time can also be negative.

Example

```

Clocking_time = CallBack( "ROUNDSEC",
                          "TIME="||dlg_time||"|REFTIME=0|INT=900|LIMIT=450|" );
...

```

3.12.1.1 Convert order number from SAP

Callback function **sap_to_hydra_ANR**:

This callback function is used to determine the components of the order number from the SAP order fields.

The parameter contains a mode and the SAP order fields.

The mode (MOD=) defines the value that the function must return.
Valid values are:

- "ANR" return of the entire order number
- "AGNR" return of the operation number
- "AUNR" return of the order header number

The function uses the SAP order fields to identify the order number. These SAP fields must therefore be transferred:

- 1) SAPAUNR = SAP order number
- 2) SAPSPLNR = SAP split number
- 3) SAPAFOLG = SAP operation sequence
- 4) SAPVGNR = SAP operation number
- 5) SAPUVGNR = SAP sub-operation
- 6) SAPKAPART = SAP capacity category

The identified part of the order number is returned in form of a char() variable. If the order number could not be identified, an empty value "" is returned.

Example

```
Hydra_anr = CallBack( "sap_to_hydra_ANR", "MOD=ANR|SAPAUNR=0815|SAPVGNR=0010|..." );
...
```

3.12.2 Examples

3.12.2.1 Identifying additional values

For example: in one user exit, a callback function is defined, which returns additional information for the HR master data.

Call

```
Area = CallBack( "PNR.SELECT", "PNR.PNR=906000|FIELD=AREA" );
```

The callback function reads the information on the person with personnel number 906000 from the database and returns the contents of the AREA field.

3.12.2.2 Triggering actions

In the example below, a user exit provides a callback function, which writes a data record to an interface file. For this purpose, the data record is assembled in the *line* string and then passed to the callback function.

```
line[ 1, 6] = personnelnumber;
line[ 7,10] = wagetype;
line[11,15] = (hours * 100) using "&&&&";
line[16,20] = (performance_rate * 100) using "&&&&";

ok = CallBack( "LLERCK.ADD", line );

if( not ok )
{
    ...
}
```

3.13 Interpreter hydscr

The hydscr.exe/.out program is available for the use of MPDV employees or experienced customers. You use this program to test scripts. Using this program any scripts can be tested and executed on the server, which are available in file form.

When the program is started, the variables to be imported can be specified.

Example of the import/export variables of a script:

```
...
import  imp_a    double;
import  imp_b    char(20);
export  exp_a    double;
export  exp_b    char(20);
...
```

Execution version 1 (BAPI string):

```
hydscr "DATEI=hydscr.hsc|IMP_A=123.123|IMP_B=Initial value imp_b|EXP_A=-123.123|EXP_B=Initial value exp_b|"
```

Execution version 2 (command line parameter):

```
hydscr hydscr.hsc 123.123 "Initial value imp_b" "123.123" "Initial value exp_b"
```

The program output shows the imported variables, the result of script execution and the variables to be exported.

```
D:\mip1>hydscr hydscr_f.hsc 123.123 "Initial value imp_b" "123.123" "Initial value exp_b"
Importing value "123.123" to variable "imp_a": 0
Importing value "Initial value imp_b" to variable "imp_b": 0
Importing value "123.123" to variable "exp_a": 0
Importing value "Initial value exp_b" to variable "exp_b": 0

Execution successful.

Exporting value "3600.000000" from variable "exp_a": 0
Exporting value "3600" from variable "exp_b": 0

D:\mip1>
```

If an error occurs during the reading, preparation or execution of the script, then the operation is canceled with an error message, which includes the number of the line with the error:

```
D:\mip1>hydscr hydscr_f.hsc 123.123 "Vorbelegung imp_b" "123.123" "Vorbelegung exp_b"  
*** parse error at "p_print" Line 295
```

The outputs of the script statements *dprint*, *pprint*, *eprint* and *print* are always shown on the screen additionally, if the script *hydscr* is executed.

4 Server Scripting – Generic User Exits

4.1 Overview

It is possible to use the generic user exits for BAPIs to control the processing of standard BAPIs. For this purpose, functions can be defined in the user exit, which are then called by the standard system and offer options of intervention.

The following options to intervene are available:

- You can change the dialog string. That means, you can control customer-specific pre-assignments and dependencies between fields.
- You can perform extended validation checks using additional SQL commands.
- You can manage further data via additional SQL statements.
- You can start additional processes by calling server programs (system calls).

4.2 Return values of the multi script functions executed

If a function has a return value that is evaluated by the calling program, the function in a script must know the return value of a function with the same name in a different multi script file that might have been executed previously. Only then, the function can react to this return value or forward the value and does not overwrite this value.

This is especially important with user exits for plausibility checks of BAPIs and dialogs.

Example:

A customer-specific user exit `b_anr_kunde.hsc` sets a customer-specific plausibility error 424 "Invalid XXX" in the function `bapi_check_before()`. If a multi script user exit `b_anr#pdv72#.hsc` also includes the function `bapi_check_before()`, then the function is executed after that. If this function now simply sends back the return value 0 without respecting the return value of the previous function, then the plausibility error previously determined by the customer is overwritten!

In a script, you can access the return value of the previous function using the **import variable ERRORCODE**. If no other multi script function has been run before, the variable is initialized with the value 0.

```

...
import ERRORCODE          long;
...

long dlg_check_before()
{
    ret          long;

    ret = ERRORCODE;

    // If no error has been detected before, the following is checked here,
    // if XXX is ok
    if( ret = 0 )
    {
        if( XXX = YYY )
        {
            ret = 1023; // P_DARF_A_NICHT_UNTERBRECHEN
        }
    }

    ...

    return ret;
}

```

4.3 Generic user exit for editing functions(BAPI)

Generate a script to define a BAPI. Script name and BAPI name are identical:

Bapi PNR.**** → Script "b_pnr*.hsc"

See separate documentation for details on server scripting.

Note:

Script file names must be completely lowercase. User exits with capital letters in the file name are not loaded!

SQL commands and system calls are allowed in the script.

The script has the following export and import parameters:

Parameter	Type	Content
DLG_DATA	Max. C32000	These export variable contains a dialog string. Individual fields can be read from this dialog string using the function get_Bapi_Val(DLG_DATA, "<acronym>") . Fields can be changed with the function set_Bapi_Val(DLG_DATA, "<Acronym>", value) . This change affects the standard function if it is performed before the plausibility checks (bapi_check_before).

ERRORTXT	Max. C32000	You can assign a free error message text to this export variable. When the error text is assigned, the return value RET=424 is automatically set regardless of the return value of the script. When the script function is finished, the processing of the BAPI is stopped and the error text ERR.TXT is transferred to the client.
RET_DATA	Max. C32000	This export variable contains the return string which is set by the application (dd_set_info). Similar to DLG_DATA, individual fields can be read or changed. This change will have an effect on the standard function and is also returned to the called process (e.g. console or terminal).
ERRORCODE	LONG	The import variable includes the current error code of previous processing and is important for the function bapi_end.
LIST_DATA	Max. C32000	The import variable is used by the user exit <i>modify_list_file_line()</i> und <i>append_list_file()</i> . The current content of a row of FILE specified in the DLG string is transferred to the functions.
LIST_LINE_NR	LONG	This import variable specifies the row number where the information in LIST_DATA comes from.

There might be some functions defined in the script, which are called before the standard processing. These function The functions do not necessarily have to be available; standard processing is then simply carried out.

Sequence	Function	Is called
1	bapi_check_before()	Starts the function before the validation check of the BAPI is executed.
2	bapi_check_after()	Starts the function before the plausibility check of the BAPI is executed.
3	bapi_action_before()	Starts the function before the BAPI is executed.
4	bapi_action_after()	Starts the function after the BAPI is executed.
5	bapi_end()	Starts the function, once BAPI processing has been completed (check, action). Even if BAPI processing was interrupted due to an error.
6	modify_list_file_line()	Starts the function after successfully processing the BAPI, if the dialog string includes the acronym DATEI=... . This function is called once for each row of the specified file and

		can be manipulated in the UserExit.
7	append_list_file()	Starts the function once, if all data rows from the file are transferred to the function modify_list_file_line(). You can use this function to attach additional data rows to the end of the file DATEI.

In general, you should bear in mind that the functions can only access fields transferred in the dialog string. Missing fields must be selected from the existing data record using SQL statements with the aid of the key fields.

Global variables defined in the script are permanent across a BAPI call.

Example:

You can save the old field value in a global variable in the function `bapi_check_before()`. The value is then available in the function `bapi_action_after()` and you can compare the old value with the new one.

Available call back functions:

Callback	Call
DLGCALLEXECUTE	Call the kernel function "DlgCallExecute" to execute a dialog (this can be a BAPI, SCMD, LIST or a message). <code>dlgdata char(32000);</code> <code>retdata char(32000);</code> <code>retdata = callback("DLGCALLEXECUTE" clipped, dlgdata);</code>
BAPICALLEXECUTE	Call the function "BapiCallExecute" of the kernel to execute a BAPI. <code>dlgdata char(32000);</code> <code>retdata char(32000);</code> <code>retdata = callback("BAPICALLEXECUTE" clipped, dlgdata);</code>
WRITEBATCHCALL	Call the function "WrteBatchCall" of the kernel. Dialog data is executed after the initial dialog is executed and the result is returned by the interface. If the DLG has a dot and the call is started via a hymwb, if not it is started with hymw. USR should be specified in the dialog data, otherwise the global system user 9999 would be set. <code>dlgdata char(32000);</code> <code>retdata char(32000);</code> <code>retdata = callback("DLGCALLEXECUTE" clipped, dlgdata);</code>

LISTOUTPUT	The data row transferred to the CallBack function is written in the DATEI (file from the dialog string). Call per row. This CallBack function can only be called from the user exits modify_list_file_line() and append_list_file().
CHECKPERSON	Plausibility check of the person: - Person exists? - Person locked? - Person has not yet joined? - Has the person left the company? Input parameter: param char(32000); Entries: PNR=<personnel number> or KNR=<card number> Return: result char(32000); Call: result = CallBack("CHECKPERSON", param clipped); Example: <pre>variable param char(32000); variable result char(32000); param = set_bapi_val("PNR", "123456"); // or // param = set_bapi_val("KNR", "4711"); result = CallBack("CHECKPERSON", param clipped); ret = get_bapi_val(result, "RET");</pre> The following information is included in a return string: a) error: - RET=<error number> - KT=<Error description short> - LT=<Error description long> b) If plausibility check is ok: - RET=0 - ASTUFE=<Info from HR master data: BDE authorizations> - MSTUFE=<Info from HR master data: MDE authorizations> - SSTUFE=<Info from HR master data : PZE authorizations>

The return string for a callback call "ret_val = callback("DLGEXEC", dlldata);" is extended by the server to include the missing entries RET, KT and LT.

4.3.1 Function "long main()"

Parameter

None.

Functions

This function must be available in the script, but has no required function. It can be used during the creation of the script to call the other functions in a test mode.

Return value

The function must return any value, but has no other purpose.

4.3.2 Function "long bapi_check_before()"

Parameter

None.

Functions

You can change the dialog data of the BAPI with this function.

Further plausibility checks can also be carried out before standard processing. These are usually checks that can be performed without database access and result from the dependence of the parameters transferred in the dialog string.

Return value

0 : all OK.

444 : All OK, the following standard plausibility checks are to be skipped. In this case the script function must perform all plausibility checks!

otherwise: error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.3.3 Function "long bapi_check_after()"

Parameter

None.

Functions

In this function further plausibility checks can take place, which are executed after standard processing.

Changes to the dialog data no longer affect the BAPI, since this data has already been transferred to internal variables.

Return value

0 : all OK.

otherwise: error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.3.4 Function "long bapi_action_before()"

Parameter

None.

Functions

Further SQL statements or system calls can be added to this function, which are carried out before the actual execution.

Changes to the dialog data no longer affect the BAPI, since this data has already been transferred to internal variables.

Return value

0 : all OK.

444 : All OK, the following standard processing should be skipped. In this case the script function must perform all processing steps!

otherwise: error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.3.5 Function "long bapi_end()"

Parameter

None.

Functions

After the BAPI has finished, regardless of whether it was successful (RET=0, BAPI was executed) or not (RET≠0, BAPI terminated with a plausibility error), the user exit bapi_end is called in the script b_<dialog>.hsc (hymw), where customer-specific extensions can be executed due to the error code.

Note: If transaction control is active, the database transaction of the BAPI is not yet completed at this point. Therefore the data changed in Bapi is not yet "visible" by external programs (system calls).

The error code (import variable) should be sent back as return code of the function.

```
long bapi_end()
{
    ret      long;
    ret = ERRORCODE;

    ...

    return ret;
}
```

Warning:

If the function bapi_end generally returns the return code "0", errors are not returned to the client or the calling BAPI!

Return value

0 : all OK.

otherwise: error code, the BAPI is exited. Changes that have already been made by standard processing are undone

(rollback of the transaction when transaction control is active)

NOTE:

The return value of any previously executed multiscript function must be included or transferred, see "1.2" on page 2 .

4.3.6 Function "long bapi_action_after()"

Parameter

None.

Functions

Further SQL statements or system calls can be added to this function, which are carried out after the actual execution. In contrast to function *bapi_end()*, the *bapi_action_after()* function is only called if no error has previously occurred in the BAPI processing.

Note: BAPIS lead to data changes in database transactions. The database transaction of the standard processing must be completed before this function is called to enable system calls to be executed in the function *bapi_action_after()*. This function can access the data just changed by the BAPI,

Return value

0 : all OK.

otherwise: error code, the BAPI is exited. Changes already made by standard processing are not reversed because the database transaction has already been completed.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.3.7 Function "modify_list_file_line()" und "append_list_file()"

Parameter

None.

Functions

After the BAPI is successfully completed, the functions check if the dialog string of the acronym DATEI=| (file) is available. If so, the user exit modify_list_file_line in the script b_<bapi>.hsc (hymwb) is called once for each row from the FILE. A data row can be changed or added in this user exit. The adoption of the changes is transmitted to the server via the Callback function LISTOUTPUT. If a data row is eliminated, the Callback function should not be called.

If all lines were transferred to the user exit modify_list_file_line, the user exit append_list_file is called once to attach additional data rows to the FILE. The Callback function LISTOUTPUT must be called for each additional row.

Important:

If only the user exit *append_list_file()* is used, the user exit *modify_list_file_line()* must also be implemented in the script and output all data rows via LISTOUTPUT.

Return value

0 : all OK.

otherwise: Error code, the changed to the DATEI (file) has not been accepted.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

Note:

The HYDRA script functions get_list_column() and set_list_column() can be used to process the dialog rows. See documentation on HYDRA script.

4.3.8 Example

The following example shows an user exit for the HR master data BAPI.

Bapi: PNR.xxx

Script file: b_pnr.hsc

```
/*-----*/
hydra basic;

export DLG_DATA char(10000);
export ERRORTXT char(300);
export RET_DATA char(10000);
import ERRORCODE long;
import LIST_DATA char(30000);
import LIST_LINE_NR long;
```

```

variable header char(30000);

/*-----*/
long bapi_check_before()
{
    ret    long;

    ret = ERRORCODE;

    /*-----*/
    /* Dialog data of bapi can be modified here */
    /*-----*/

    // For customer Department ALWAYS set to 4711
    DLG_DATA = set_bapi_val( DLG_DATA, "PNR.ABT", "4711" );

    /*-----*/
    /* Additional plausibility checks before standard checks */
    /*-----*/

    // Bei Kunde Personalnummern unter 1000 nicht erlaubt
    if( ret = 0 ) // Only if no error has been reported before by another multiscript
        // was set
    {
        if( get_bapi_val( DLG_DATA, "PNR.PNR" ) < 1000 )
        {
            ret = 1704; // Personnel number not valid
        }
    }

    /*-----*/
    /* By ret = 444 standard processing can be skipped */
    /*-----*/

    // No further plausibility checks for this customer
    // ret = 444;

    return ret;
}

/*-----*/
long bapi_check_after()
{
    ret    long;

    ret = ERRORCODE;

    if( ret = 0 )
    {
        /*-----*/
        /* Additional plausibility checks after standard checks */
        /*-----*/
        // Allow only areas which are also defined as cost centers
        sqlexec( "select 1 from cost kostenstelle (cost center) " ||
            "where kostenstelle = " || BV(get_bapi_val( DLG_DATA, "PNR.BER" )) || ";" );
        if( sqlcode() != 0 )
        {
            ret = 1882; // Invalid area
        }
    }

    return ret;
}

/*-----*/
long bapi_action_before()
{
    ret    long;

```

```

        ret = ERRORCODE;

if( ret = 0 )
{
    /*-----*/
    /* SQL statements and/or system calls which are inserted here are          */
    /* processed before standard processing                                   */
    /*-----*/

    /*-----*/
    /* By ret = 444 standard processing can be skipped                        */
    /*-----*/

    // Do not make any further database accesses for the customer
    // ret = 444;
}

return ret;
}

/*-----*/
long bapi_action_after()
{
    ret    long;

    ret = ERRORCODE;

if( ret = 0 )
{
    /*-----*/
    /* SQL statements and/or system calls which are inserted here are          */
    /* processed after standard processing                                   */
    /*-----*/
}

return ret;
}

/*-----*/
long bapi_end()
{
    ret    long;
    ret = ERRORCODE;

    /*-----*/
    // Depending on the error code, customer-specific actions can be
    // performed here
    /*-----*/

    return ret;
}

/*-----*/
long main()
{
    ret    long;

    // This function is only called when testing

    return ret;
}

/*-----*/

long modify_list_file_line()
{
    ret    long;

```

```
dummy char(1000);
    ret = ERRORCODE;

if(LIST_LINE_NR = 1)
    header = LIST_DATA;

    dummy = callback("LISTOUTPUT", LIST_DATA);

    return ret;
}

/*-----*/

/*
long append_list_file()
{
    ret    long;
    dummy char(1000);
    ret = ERRORCODE;

//  dummy = callback("LISTOUTPUT", list_data);

    return ret;
}
*/
```


4.4 Generic user exit for collection dialogs (DDI)

A script is created to define user exits in hymw based on the dialog (not to be confused with the event!).

Script name and dialog name are identical (DLG=...):

Dialog M_MST → Script "d_m_mst*.hsc"

See separate documentation for details on server scripting.

SQL commands and system calls are allowed in the script.

The script has the following export and import parameters:

Parameter	Type	Content
DLG_DATA	Max. C32000	These export variable contains a dialog string. Individual fields can be read from this dialog string using the function get_Bapi_Val(DLG_DATA, "<acronym>") . Fields can be changed with the function set_Bapi_Val(DLG_DATA, "<Acronym>", value) . This change affects the standard function if it is exited before the plausibility checks (<i>dlg_init_before</i> , <i>dlg_check_before</i>).
RET_DATA	Max. C32000	The export variable contains the return string, which is set by the application . Similar to DLG_DATA, individual fields can be read or changed. This change will have an effect on the standard function and is also returned to the called process (e.g. console or terminal).
ERRORTXT	Max. C32000	You can assign a free error message text to this export variable. When the error text is assigned, the return value RET=424 is automatically set regardless of the return value of the script. When the script function is finished, the processing of the BAPI is stopped and the error text ERR.TXT is transferred to the client.
ERRORCODE	LONG	The import variable contains the current error code from previous processing. This value is overwritten on the server with the return value of the called function.
LIST_DATA	Max. C32000	The import variable is used by the user exit <i>modify_list_file_line()</i> und <i>append_list_file()</i> . The current content of a row of FILE specified in the DLG string is transferred to the functions.
LIST_LINE_NR	LONG	This import variable specifies the row number where the information in LIST_DATA comes from.

Global variables, which are defined in the script, are *permanent*. This means that after the plausibility checks an (old) value of the data set to be changed, can be specified for a variable in the function After the actual execution, the variable is still valid and the old value can be compared with the new one.

There might be some functions defined in the script, which are called before the standard processing. These functions do not necessarily have to be available; standard processing is then simply carried out.

Sequence	Function	Is called
1	dlg_init_before()	This function is called before the dialog is initialized.
2	dlg_init_after()	This function is called after the dialog is initialized.
3	dlg_check_before()	The function is called before the plausibility check is executed for the dialog.
4	dlg_check_after()	The function is called after the plausibility check is executed for the dialog.
5	dlg_action_before()	This function is called before the posting is initialized.
6	dlg_action_after()	This function is called after the posting is initialized.
7	dlg_end()	The function is called at the end of the completed dialog processing (init, check, action), even if the update was terminated due to an error.
8	modify_list_file_line()	The function is called after successful dialog processing if the acronym FILE=... is present in the dialog string. This function is called once for each row of the specified file and can be manipulated in the UserExit.
9	append_list_file()	Starts the function once, if all data rows from the file are transferred to the function modify_list_file_line(). You can use this function to attach additional data rows to the end of the file DATEI.

In general, you should bear in mind that the functions can only access fields transferred in the dialog string. Missing fields must be selected from the existing data record using SQL statements with the aid of the key fields.

Available Callback functions

Callback	Call
DLGCALLEXECUTE	Call the kernel function "DlgCallExecute" to execute a dialog (this can be a BAPI, SCMD, LIST or a message). <pre>dlgdata char(32000); retdata char(32000); retdata = callback("DLGCALLEXECUTE" clipped, dlgdata);</pre>
BAPICALLEXECUTE	Call the function "BapiCallExecute" of the kernel to execute a BAPI. <pre>dlgdata char(32000); retdata char(32000); retdata = callback("BAPICALLEXECUTE" clipped, dlgdata);</pre>
WRITEBATCHCALL	Call the function "WrteBatchCall" of the kernel. Dialog data is executed after the initial dialog is executed and the result is returned by the interface. If the DLG has a dot and the call is started via a hymwb, if not it is started with hymw. USR should be specified in the dialog data, otherwise the global system user 9999 would be set. <pre>dlgdata char(32000); retdata char(32000); retdata = callback("DLGCALLEXECUTE" clipped, dlgdata);</pre>
LISTOUTPUT	The data row transferred to the Callback function is written in the DATEI (file from the dialog string). Call per row. This Callback function can only be called from the user exits modify_list_file_line() and append_list_file().
GETSCHICHTZEIT	The shift time in seconds is calculated via the Callback function using the shift calendar and returned to the calling UserExit. → Only HYMW The parameter dialog string must have the following information: <pre>MNR ... Machine number DATB ...Date of the starting time (Format: MM/DD/YYYY) ZEIB ... Starting time (Sec. since midnight) DATE ...Date of the end time(Format: MM/DD/YYYY) ZEIE ... End time (Sec. since midnight)</pre> <pre>variable ret_callback long; varibale parameter char(100); parameter = "MNR=TESTMA DATB=07/09/2007 ZEIB=21600 DATE=07/09/2007 ZEIE=50400 "; ret_callback = Callback("GETSCHICHTZEIT", parameter);</pre>
SYSGETINFO SYSADDINFO SYSSETINFO → Only HYMW	These Callback functions can be used to signal the terminal in dlg_action_after or dlg_end to reload a list, for example. The functions return the current information. Note: The hymw sets the information "LOAD=xxxx" after the update so that changes in dlg_action_after or dlg_end are possible. SYSGETINFO: returns the current INFO, the transferred parameter is ignored.

SYSSETINFO:

sets the current INFO completely with the transferred dialog data string, existing values are overwritten.

SYSADDINFO:

attaches the transferred dialog data to the current INFO. Values must always be ended with the pipe symbol.

Examples - reading the current INFO:

```
variable info char(30000);
info = CallBack("SYSGETINFO", "");
```

Set the signaling for the order and machine list, the existing info must be transferred:

```
info = CallBack("SYSSETINFO", info ||
"LOAD=ANR,MNR|");
```

Add a value:

```
rc = CallBack("SYSADDINFO", "VAR=VAL|");
```

Util function:

A Util function "SetLoadList()" was implemented to standardize the setting of new LOAD statements.

Old way of processing like

```
RET_DATA = set_bapi_val(RET_DATA, "LOAD", "ANR");
```

are obsolete and should not be used anymore!

```
/*
 * Function:      SetLoadList(<NameDerListe>);
 * Example:      ret = SetLoadList("ANR");
 * Return value:  always 0
 * Purpose:      Extends load instruction (Which lists should be reloaded by terminal)
 * Autor:        mpdv/08.08.2011
 */
long SetLoadList(list_to_be_added char(10))
{
    variable info char(30000);
    variable load char(255);
    variable i long;
    variable found long;
    variable list_included char(10);

    info = CallBack("SYSGETINFO", "");
    load = get_bapi_val(info, "LOAD");

    found = 0;

    if (load <> "")
    {
        load = hy_change_sep(load, ",", "|", "");
        i = 1;

        list_included = get_bapi_val(load, ("## " || (i using "<<<&")));
        while ((list_included <> "") and (found <> 1))
        {
            if (list_included = list_to_be_added)
                found = 1;

            i = i + 1;
            list_included = get_bapi_val(load, ("## " || (i using "<<<&")));
        }
    }

    if (found = 0)
    {
        if (load <> "")
            if (load[strlen(load)] != "|")
                load = load clipped || "|";

        load = load clipped || list_to_be_added clipped;

        load = hy_change_sep(load, "|", ",", "");
        info = set_bapi_val(info, "LOAD", load);
        info = CallBack("SYSSETINFO", info);
    }

    return 0;
}
```

CHECKPERSON	Plausibility check of the person: - Person exists? - Person locked? - Person has not yet joined? - Has the person left the company? Input parameter: param char(32000); Entries: PNR=<personnel number> or KNR=<card number> Return: result char(32000); Call: result = CallBack("CHECKPERSON", param clipped); Example: <pre>variable param char(32000); variable result char(32000); param = set_bapi_val("PNR", "123456"); // or // param = set_bapi_val("KNR", "4711"); result = CallBack("CHECKPERSON", param clipped); ret = get_bapi_val(result, "RET");</pre> The following information is included in a return string: a) error: - RET=<error number> - KT=<Error description short> - LT=<Error description long> b) If plausibility check is ok: - RET=0 - ASTUFE=<Info from HR master data: BDE authorizations> - MSTUFE=<Info from HR master data: MDE authorizations> - SSTUFE=<Info from HR master data : PZE authorizations>

4.4.1 Function "long main()"

Parameter

None.

Functions

This function must be available in the script, but has no required function. The function can be used during script creation to call the other functions for testing purposes.

Return value

The function must return any value, but has no other purpose.

4.4.2 Function "long dlg_init_before()"

Parameter

None.

Functions

You can change dialog date in this function.

Further initializations can also be carried out before standard processing.

Return value

0 : all OK.

otherwise : error code, processing is terminated.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.4.3 Function "long dlg_init_after()"

Parameter

None.

Functions

This function can be used for further initializations and executed after standard processing.

Changes to the dialog data no longer affect the further processing, since this data has already been transferred to internal variables.

Return value

0 : all OK.

otherwise : error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.4.4 Function "long dlg_check_before()"

Parameter

None.

Functions

You can change dialog date in this function.

Further plausibility checks can also be carried out before standard processing. These are usually checks that can be performed without database access and result from the dependence of the parameters transferred in the dialog string.

Return value

0 : all OK.

444 : All OK, the following standard plausibility checks are to be skipped. In this case the script function must perform all plausibility checks!

otherwise: error code, processing is terminated.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.4.5 Function "long dlg_check_after()"

Parameter

None.

Functions

In this function further plausibility checks can take place, which are executed after standard processing.

Changes to the dialog data no longer affect the further processing, since this data has already been transferred to internal variables.

Return value

0 : all OK.

otherwise: error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.4.6 Function "long dlg_action_before()"

Parameter

None.

Functions

Further SQL statements or system calls can be added to this function, which are carried out before the actual posting.

Changes to the dialog data no longer affect the further processing, since this data has already been transferred to internal variables.

Return value

0 : all OK.

444 : All OK, the following standard plausibility checks are to be skipped. In this case the script function must perform all processing steps!

otherwise: error code, the BAPI is exited.

NOTE:

The return value of any previously executed multiscript function must be transferred, see "1.2" on page 2 .

4.4.7 Function "long dlg_action_after()"

Parameter

None.

Functions

Further SQL statements or system calls can be added to this function, which are carried out after the actual execution.

Return value

0 : all OK.

otherwise: error code, posting is terminated. Changes that have already been made by standard processing are undone

(rollback of the transaction when transaction control is active)

NOTE:

The return value of any previously executed multiscript function must be included or transferred, see "1.2" on page 2 .

4.4.8 Function "long dlg_end()"

Parameter

None.

Functions

After the dialog has finished, regardless of whether it was successful (RET=0, posting was executed) or not (RET≠0, posting terminated with a plausibility error), the user exit dlg_end is called in the script d_<dialog>.hsc, where customer-specific extensions can be executed due to the error code.

The error code (import variable) should be sent back as return code of the function.

```
long dlg_end()
{
    ret      long;
    ret = ERRORCODE;

    return ret;
}
```

Warning:

If the function dlg_end generally returns the return code "0", errors are not returned to the client or the calling BAPI!

Return value

0 : all OK.

otherwise: error code, posting is terminated. Changes that have already been made by standard processing are undone

(rollback of the transaction when transaction control is active)

NOTE:

The return value of any previously executed multiscript function must be included or transferred, see "1.2" on page 2 .

4.4.9 Function "modify_list_file_line()" und "append_list_file()"

See section 4.3.7 Function "modify_list_file_line()" und "append_list_file()".

4.4.10 Example

The following example shows an user exit for the dialog **M_MST**.

Dialog: M_MST

Script file: d_m_mst.hsc

```
hydra basic;

export DLG_DATA char(10000);
export ERRORTXT char(200);
export RET_DATA char(10000);
import ERRORCODE long;
import LIST_DATA char(30000);
import LIST_LINE_NR long;

variable header char(30000);

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_init_before()
{
variable ret long;

    ret = ERRORCODE;

    dprint( "SCRIPT dlg_init_before");

    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_init_after()
{
    ret long;

    ret = ERRORCODE;
    dprint( "SCRIPT dlg_init_after");
    if( ret = 0 )
    {
        // actions
    }
    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_check_before()
{
```

```

ret    long;

    ret = ERRORCODE;
    dprint( "SCRIPT dlg_check_before");
    if( ret = 0 )
    {
        // actions
    }
    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_check_after()
{
    ret    long;

    ret = ERRORCODE;
    dprint( "SCRIPT dlg_check_after");
    if( ret = 0 )
    {
        // actions
    }
    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_action_before()
{
    ret    long;

    ret = ERRORCODE;
    dprint( "SCRIPT dlg_action_before");
    if( ret = 0 )
    {
        // actions
    }
    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_action_after()
{
    ret    long;

    ret = ERRORCODE;
    dprint( "SCRIPT dlg_action_after");
    if( ret = 0 )
    {
        // actions
    }
    return ret;
}
*/

/*-----*/
/* As long as a function does not matter, it should be commented out for
   performance reasons
long dlg_end()
{

```

```

ret    long;
ret = ERRORCODE;
dprint( "SCRIPT dlg_end" );

//-----
// Depending on the error code, customer-specific actions can be
// performed here
//-----
if( ret = XYZ )
{
    // actions
}

return ret;
}
*/

/*-----*/
long main()
{
    ret    long;

    // This function is only called when testing
    return ret;
}

/*-----*/
/* As long as a function does not matter, it should be commented out for
performance reasons
//-----
long modify_list_file_line()
{
    ret    long;
    dummy char(1000);

    ret = ERRORCODE;

    if(LIST_LINE_NR = 1)
        header = list_data;

    dummy = callback("LISTOUTPUT", list_data);

    return ret;
}
*/

/*-----*/

/*
long append_list_file()
{
    ret    long;
    dummy char(1000);

    ret = ERRORCODE;

    // dummy = callback("LISTOUTPUT", list_data);

    return ret;
}
*/

```

5 User Exit Reference

5.1 Overview

This document describes server user exits from MW 3.0. The document also indicates the required software versions for the user exits. If no software version is indicated, the user exit is available as of MW 3.0.

User exits are classified by product groups.

5.2 Objectives and guidelines for the use of script files

- Script files ensure the rapid, easy and low-risk possibility of changing and customizing applications without any compilation.
- Non-developers are also able to generate scripts.

5.3 HYDRA script language

A separate document (MDS-Server_scripting.pdf) describes the HYDRA script language. Among other things, the document MDS-Server_scripting.pdf deals with the following:

- HYDRA Script language elements
- Filing of script files
- Naming conventions for labels and script files
- Programming aids

5.4 Server user exits: Kernel

5.4.1 Modify dialog data

Name of user exit

hy_modify_dlgdata.hsc

Keywords

Modify dialog data

Function

This user exit enables you to change dialog data:

- from the terminal
- from the office client
- from the MLE inbound transactions
- via service interface

before parsing or triggering in order to forward this data to another place or to attach specific parameters.

You can change the export parameter *DLG_DATA* in the function *before_execute*. This parameter contains the overall, received dialog string, e.g:

Before:

```
"DLG=RES_AN|RES=4711|ANR=12345678010|MNR=00000100|...|USR=2115|DAT=02/03/2006|ZEI=15661|"
```

After:

```
"DLG=CUST_RESAN|RES=4711|ANR=12345678010|MNR=00000100|...|USR=2115|DAT=02/03/2006|ZEI=15661|"
```

Note: This is a user exit that can change general processing for several modules and must be used carefully. *hy_modify_dlgdata.hsc* is called for every input dialog and every service that calls legacy processing as wrapper. Inappropriate implementation of the user exit might have adverse effects on all system dialogs! For example: Use the appropriate user exit if you only want to change dialog data for MLE processing. If possible, only use the generic user exits *d_*.hsc* for input dialogs (*A_AN*, *A_UN*, etc.).

Implementation guidelines:

- If possible, always use the user exit *d_*.hsc* instead of *hy_modify_dlgdata.hsc*. As the user exit *d_*.hsc* is only called for the corresponding dialog and does not affect other dialogs. You can also use the user exits *d_*.hsc* to change dialog data via the function *dlg_init_before()*.
- Query the dialog ID (*DLG=xxx*) for every action in *hy_modify_dlgdata.hsc*.
- No database access without querying the dialog ID.
- Database access only for required dialog IDs.
- String commands (*DLG=SCMD;12*) and lists (*DLG=LIST;12*) only allow dialog data changes in *hy_modify_dlgdata.hsc*. You can only use the user exits *d_list_*.hsc* to change lists subsequently using the functions "*modify_list_file_line()*" and "*append_list_file()*".

Note: A customer uses the user exit *hy_modify_dlgdata* to add the ID

LOCKLST:EREIG=A_AB,A_UN,P_AB to the shift change dialogs (*A_ASW/A_AUN*) and the PZE dialogs (*P_KOM/P_GEH/P_AST*).

This activates a lock process to avoid parallel processing of identical posting events: CLOCK OUT and shift change logs off person.

(Double posting events led to double ADE log records).

Enable this processing/ID only after consultation with MPDV.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out			
hymwb.exe/out			
mle72imp.exe/out			
hyadeabg.exe			

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data
ERRORTTEXT	C30000	- not in use yet -

5.4.2 Logging of dialog data

Name of user exit

hyd_logging_dlg_<DLG>.hsc

Substitute the placeholder <DLG> with the actual dialog.

Examples:

A_TR → hyd_logging_dlg_a_tr.hsc

MNR.UPDATE → hyd_logging_dlg_mnr_update.hsc

Keywords

Modify logging data

Function

You can log the dialog data string at the end of a dialog, depending on the HYD_LOGGING configuration. You can use the user exit to extend the dialog data string in the script before logging.

This user exit enables you to change dialog data before they are entered in HYD_LOGGING.

You can change the export parameter *DLG_DATA* in the function *hyd_logging_data*. This parameter contains the overall, received dialog string, e.g:

Before:

```
"DLG=RES_AN|RES=4711|ANR=12345678010|MNR=00000100|...|USR=2115|DAT=02/03/2006|ZEI=15661|"
```

After:

```
"DLG=CUST_RESAN|RES=4711|ANR=12345678010|MNR=00000100|...|USR=2115|DAT=02/03/2006|ZEI=15661|"
```

Note: Only the contents of the string to be logged will be changed. The original dialog data remains unchanged.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out			hyd_logging.c
hymwb.exe/out			-"-
hyadeabg.exe			-"-
hymwcaq72.dll			-"-
b_hydscr.dll			-"-
hymwhyd72.dll			-"-
libbapi.dll			-"-
hymwmp172.dll			-"-
mle72imp.exe			-"-
tages_aw.exe			-"-
res_transfer.exe			-"-

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

5.4.3 Modifying batch call data

Name of user exit

hy_modify_batchcall.hsc

Keywords

Change dialog data of the batch file before they are written in the file.

Function

You can use this user exit to change or skip data of a batch file before the batch file is generated. So you can write changed data into the batch file.

You can change the export parameter *DLG_OUT* in this user exit. This parameter contains the dialog string you want to enter in the batch file. The parameter *DLG_IN* includes the triggering dialog string as additional parameter for the user exit.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out			hy_sys.c
hyadeabg.exe			
hymwcaq*.dll			
b_hydscr.dll			
hymwhyd72.dll			
hymwb.exe			
deso_pers_rep.exe			
egocheck.exe			
jumbo_ss.exe			
saagcheck.exe			
saaglist.exe			
lvr2hyd.exe			

Program	Version	Date	File(s)
hyl_nabu.exe			
hymwmp1*.dll			
mle72imp.exe			
hy_ups.exe			
tages_aw.exe			
hymwrsm*.dll			
hyresctl.exe			
hywtkupd.exe			
res_transfer.exe			

Import parameter

Parameter	Type	Content
DD_IN	C30000	Triggering dialog data

Export parameter

Parameter	Type	Content
DD_OUT	C30000	Dialog data to be entered into the batch file.
DO_DLGDATA	Integer	TRUE/FALSE (1/0) : Option to control whether to enter dialog data.

5.4.4 Modify event data

Name of user exit

hy_event_data.hsc

Keywords

Modify event data before they are entered (INSERT).

Function

Use this user exit to change or skip event data before they are generated. So you can write changed data to the database.

You can change the export parameter *DLG_OUT* in this user exit. This parameter contains the dialog string you want to enter in the batch file. The parameter *DLG_IN* includes the triggering dialog string as additional parameter for the user exit.

Import parameter

Parameter	Type	Content
none		

Export parameter

Parameter	Type	Content
KLASSE	C30000	Dialog data to be entered into the batch file.
EVENT	C80	Event
DLG	C80	Dialog
DAT	integer	Current date
ZEI	integer	Current time
TNR	integer	Terminal
ANR	C40	Order number (ANR = AUNR+AGNR)
CNR	C20	Batch number
PNR	C10	Personnel number
MGRP	C10	Machine group
MNR	C20	Machine
BPOS	C10	Operator position/function

5.4.5 Reload manager: Reload plug-ins

Name of user exit

reload_plugin_<TABLE>.hsc

TABLE ... Name of reload tables without the prefix "R_"

Examples: Reload table R_ADE_PROTOKOLL

- a) reload_plugin_ade_protokoll_<CUSTOMER>.hsc
 - b) reload_plugin_ade_protokoll#1#_<customer>.hsc
- ⇒ Customer-specific user exit with sequence control using multi scripting

Keywords

Archiving, hymwarc, reload, reload manager

Function

Use this user exit to edit the data to be reloaded via the Reload Manager before the data is provided in the corresponding reload table. To do so, use available SQL tools.

Case 1: **No** reload plugin scripts available for the table.

- Data is reloaded (as already implemented) directly from the export into the relevant reload table.

Case 2: Reload plugin scripts available for the table.

- Create a temporary reload table (schema identical with "real" reload table/current schema).
- Load data into temporary reload tables.
- Call all reload plugins for the table:
 - o UserExit file: reload_plugin_<table name>#<numbering>#.hsc
 - o Function: long reload_plugin()
 - o The <numbering> specifies the sequence of the plug-ins (standard multi scripting).
 - o Standard plug-ins are numbered 10, 20, 30, etc. You can define customer-specific plug-ins between the numbers.
 - o Use the variable ERRORCODE to request the status of the previous script in the script.
 - o The import variable PATCH_INFO_GEN provides information on the DB patch status when preparing the export.
 - o The import variable PATCH_INFO_ACT (from table SOFTWARE_STATUS → TYP=DBPATCH) provides information on the current DB patch status.
 - o Syntax PATCH_INFO_ACT
 - <PATCH1>=<Version>|<PATCH2>=<Version>|...
 - General: <SOFTWARE_STATUS.NAME>=SOFTWARE_STATUS.PRG_VERSION|...
 - Example: MW20_DBPATCHES=7.2.1.6|CAQ71_ADE72_INTEGRATION=7.2.1.3|...
 - o The plug-in determines if it must be executed.
 - o All changes are executed in the temporary reload table using SQL.
- After all plug-in scripts have been executed, the contents of the temporary reload table are transferred to the real reload table via SQL.
In drastic cases, the plug-ins can prevent the transfer to the reload area. In this case, the temporary reload table (including its contents) is rejected.
- If RET=0 or ERRORCODE=0 is set, data is transferred.
If Ret !=0 is set, data is not transferred.
- Error handling:
 - o Error handling required if reload plug-ins change data in such a way that errors occur when data is transferred into the reload table (e.g. duplicate keys; changed data, changed schema of temporary reload table, or similar).
 - o Error handling required, if the temporary reload table is rejected (determined by plug-in).
 - o Error handling required, if an error occurs in the plug-in.
 - o Error handling required, if the plug-in is faulty (syntax error due to changed scripting engine).

Note: Management of reload data remains unchanged.

Program(s) and source code files

Program	Version	Date	File(s)
b_hymwarc.dll / .so	8.1.1.8	2013-01-25	\\swq\entw8\entw72\hyd\src\b_hymwarc.cpp

Import parameter

Parameter	Type	Content
PATCH_INFO_GEN	C32000	Information about DB patch versions that were active when archive unload files were generated. Data are provided in the string as key value pairs and separated by a pipe character (" "). You can access data using the function get_bapi_val(). Example: <pre>LLE72=7.2.1.5 MW20_PZE_ZTNW_LISTE=7.2.1.1 CAQ_ISO_3951 DATA_CO=7.2.1.1 </pre>
PATCH_INFO_ACT	C32000	Information about current DB patch versions. Data derives from the table SOFTWARE_STATUS and is output as key value pairs separated by pipe characters (" ").
TEMPTABLE	C300	Name of the temporary DB table. You can integrate this name in the corresponding SQL statements. Examples: <pre>sqlexec ("select count(*) from " TEMPTABLE clipped "; "); sqlexec("update " TEMPTABLE clipped " " ...);</pre>
ERRORCODE	LONG	Error code transferred to the function. This corresponds to the return value of the previous script call in multi scripting.

5.5 Server user exits: ADE

5.5.1 Modification of the order list

5.5.1.1 Defining additional columns

Name of user exit

l_a_anr_list1.hsc

Keywords

Order list, sequencing list, List;11

Function

This user exit adds additional columns to the order list.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The user exit transfers the copy mode list to the script via the import variable.
- ❖ The import variable keys (char(200)) defines the processing.
 - The header transfers the value "HEADER".
 - The key values are transferred as BAPI string in the data row. Following acronym are currently supported:
 - ANR = Order number
 - MNR = Machine number
- ❖ The import variable zeile (char(8192)) transfers the header row/data row to the script.
- ❖ After executing the script, the export variable list_append (char(1000)) includes the string to extend the relevant row in the list.

After generating the header row, the user exit is requested with keys = "HEADER" and extends the header row with the contents of the export variable list_append.

After generating the data row, the second request takes place with keys = "ANR=...|MNR=..." and extends the data row also with the contents of the export variable list_append.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			hyd_bdel.c

Import parameter

Parameter	Type	Content
MOD	C1	Order list mode
KEYS	C200	"HEADER" or ANR and MNR as Bapi string
ZEILE	C8192	Header/data row
DLG_DATA	C30000	Dialog data string

Export parameter

Parameter	Type	Content
LIST_APPEND	C1000	String extending the header or data row.

5.5.1.2 Sort order of the sequencing list

Name of user exit

l_a_anr_list2.hsc

Keywords

Order list, sequencing list, List;11

Function

User exit defines sorting of the order list.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The import variable mod (char(1)) transfers the list mode to the script (for future use).
- ❖ The import variable mnr (char(20)) transfers the machine of the list to the script (for future use).
- ❖ After executing the script, the export variable *orderby* (char(1024)) includes the string with the new sort order (order by clause).
- ❖ The user exit is called before the header row is generated. If dynamic fields are added to the sort order, then use the acronym (not the database column) for the "order-by-clause" in the user exit. You also have to enter the acronym in the CTWINLAY.INI. If it is a standard field, then use the table qualifier and column name (e.g ab.auftrag_nr). If you use an unknown acronym and/or do not enter the entry in the CTWIN.INI, then the acronym is ignored.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			hyd_bdel.c

Import parameter

Parameter	Type	Content
MOD	C1	Order list mode
MNR	C20	Machine of the order list

Export parameter

Parameter	Type	Content
ORDERBY	C1024	String containing the new sort sequence.

Notes

If you want to sort data by default columns, you have to enter the alias in this database field.

→ Example: `ab.auftrag_nr`

If you want to sort data by dynamic fields (with AKRO=xxx), you have to enter the relevant acronym.

→ Example: Field `ab.spaet_end_dat` → `ANR_DATSE`

Complete example:

```
long main()
{
    variable ret_val      long;

    ret_val = 0;

    // Sorting by columns
    // ab.spaet_end_dat, ab.spaet_end_zeit, ab.auftrag_nr ; ";
    orderby = " ANR_DATSE, ANR_ZEISE, ab.auftrag_nr ";

    return ret_val;
}
```

5.5.1.3 Where clause of the sequencing list

Name of user exit

`l_a_anr_list3.hsc`

Keywords

Order list, sequencing list, List;11

Function

The user exit adds a where clause including additional selection criteria to the order list. You can use the user exit to add to the standard where clause.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The user exit transfers the copy mode list to the script via the import variable.
- ❖ The import variable union_nr (long) transfers the number of the union for the list statement to the script (for future use). Counting starts at 1. If the statement does not have a union, 0 is transferred.
- ❖ The import variable DLG_DATA (char(30000)) transfers the complete dialog data string of the list to the script.
- ❖ After executing the script, the export variable where (char(2048)) includes the string to extend the where clause.
- ❖ The user exit is called for each union before generating the DECLARE CURSOR. In the script, you can use the callback function to identify the aliases for the tables in the statement. To do so, start the function GET_TABLE_ALIAS and use the table name as parameter. Table names are not case sensitive.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			hyd_bdel.c

Import parameter

Parameter	Type	Content
MOD	C1	Order list mode
UNION	INT	The number of the union in the statement is starting with 1 or 0 if the statement does not include a union.
DLG_DATA	C30000	Dialog data string

Export parameter

Parameter	Type	Content
WHERE	C2048	String containing the extension for the WHERE clause.

5.5.1.4 Adding QM due date information

Name of user exit

I_a_anr_qm_list.hsc

Keywords

Order list, CAQ, QM, due date, list;11

Function

This user exit offers the HYDRA script standard functions to extend or change a list.

By default, the script shows due dates of the order based on open inspection points of relating inspection steps, if the parameter `AKRO` contains at least one of the values `PRUEFSTAT:BEZK`, `PRUEFSTAT:FARBE` and/or `PRUEFSTAT:MINUTEN`.

In this case the list columns with the same names are filled with values. The list columns `PRUEFSTAT:BEZK` and `PRUEFSTAT:FARBE` are empty by default. The column `PRUEFSTAT:MINUTEN` is added.

You can overwrite the user exit, if required, to determine separate due dates and to forward these dates to the terminal.

Program(s) and source code files

Program	Version	Date	File(s)
hymw	8.1.1.578	2016-03-23	hyd_bdel.c
l_a_anr_qm_list.hsc	8.1.1.78223	2016-03-23	l_a_anr_qm_list.hsc

5.5.2 Extending the machine list

5.5.2.1 Adding QM due date information

Name of user exit

l_m_ml_qm_list.hsc

Keywords

Machine list, CAQ, QM, due dates, list;10

Function

This user exit offers the HYDRA script standard functions to extend or change a list.

By default, the script shows due dates of the machine based on open inspection points of relating inspection steps, if the parameter `AKRO` contains at least one of the values `PRUEFSTAT:BEZK`, `PRUEFSTAT:FARBE` and/or `PRUEFSTAT:MINUTEN`.

In this case the list columns with the same names are filled with values. The list columns `PRUEFSTAT:BEZK` and `PRUEFSTAT:FARBE` are empty by default. The column `PRUEFSTAT:MINUTEN` is added.

You can overwrite the user exit, if required, to determine separate due dates and to forward these dates to the terminal.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmde72	8.1.1.105	2016-03-23	l_mnr.c
l_m_ml_qm_list.hsc	8.1.1.78225	2016-03-23	l_m_ml_qm_list.hsc

5.5.3 Extending the ANR Bapi

5.5.3.1 Changing data in production method (production variant) identification

Name of user exit

b_anr_modify_fertvar.hsc

Keywords

Accept/transfer operation

Function

Adding the user exit "b_anr_modify_fertvar.hsc" to the ANR Bapi. This function allows you to edit/reject the data deriving from the production variant (production method).

The user exit is not permitted to execute SQL commands. But you can edit all transfer parameters as part of the user exit before they will be transferred to the OP. Use the specific transfer parameter "ACCEPT_VALS" if you want to prevent the data from being transferred to the OP!

Program(s) and source code files

Program	Version	Date	File(s)
hymw			b_anr.c
b_anr.dll			

Import parameter

Parameter	Type	Content
ANR	C40	Order number.
VOM_PPS	Long	Order comes from PPS (Yes = 1; No = 0)

Export parameter

Parameter	Type	Content
ATK	C40	Article number
MNR	C20	Machine number
MGRP	C20	Machine group
WNR	C40	Tool number
FERTVAR	Long	Internal ID of the production variant
ACCEPT_VALS	Long	Transfer data to OP (Yes=1, No=0)

5.5.4 Dialog processing HYMW

5.5.4.1 Calculation of the average actual cycle

Name of user exit

calc_istzyklus.hsc

Keywords

Calculation of the average actual cycle when orders are interrupted or logged off.

Function

Adding the user exit "calc_istzyklus.hsc" when orders are interrupted or logged off. Use this user exit to overwrite the default calculation of the average actual cycle. The default calculation formula is: $\text{istzyklus} = \text{bmk11} / \text{hub_gesamt} * 1000$.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The import variable *hub_gut* (double) includes the strokes/clocks recorded for the yield quantity.
- ❖ The import variable *hub_gesamt* (double) transfers all the recorded clocks/strokes.
- ❖ The import variables *bmk1* - *bmk12* (long) transfer all resource performance accounts.
- ❖ The export variable *ist_zyklus* (long) includes the average actual cycle calculated in the script. This average actual cycle is stored in the order status after executing the script.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			d_a_ab.c

Import parameter

Parameter	Type	Content
hub_gut	Double	The cycles/strokes/clocks so far recorded for the yield quantity.
hub_gesamt	double	So far collected cycles/strokes/clocks.
bmk1 - bmk12	Long	RPA01 to RPA12

Export parameter

Parameter	Type	Content
ist_zyklus	Long	Calculated actual cycle

5.5.4.2 Identification of LLE default values for ADE collection

Name of user exit

d_lle_daten.hsc

Keywords

Target te, te, wage type, premium group, lle_daten

Function

LLE data is written into the posting entries during ADE data collection. The ADE log records then include these LLE data.

Use this user exit to change the posting entries before writing the LLE data.

You can execute SQL commands and system calls in the user exit.

Warning:

You should avoid time consuming activities for this user exit as it is highly demanded during data collection. Improper implementation of this user exit might have adverse effects on the overall performance of data collection.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.out			hyd_utl.c

Import parameter

Parameter	Type	Content
DLG_DATA	char(30000)	Overall dialog data
DLG	char(79)	Dialog ID
EVENT	char(79)	Triggering event
MNR_MNR	char(20)	Machine/workplace
MNR_MGRP	char(20)	Machine/workplace: Group
MNR_ART	char(1)	Machine/workplace: Type group/single workplace
MNR_KST	char(10)	Machine/workplace: Cost center
MNR_BEZK	char(8)	Machine/Workplace: Name
MNR_BEZL	char(40)	Machine/workplace: Comment
MNR_PRKZ	char(1)	Machine/workplace: Incentive wages indicator
MNR_LEIGRAD	long	Machine/workplace: Performance level
ANR_ANR	char(40)	Operation
ANR_ATK	char(40)	Operation: Article
ANR_AUNR	char(40)	Operation: Order number (without OP)
ANR_AGNR	char(40)	Operation: Operation number (without order number)

ANR_RMNR	char(40)	Operation: Confirmation number
ANR_AUART	char(5)	Operation: Order type
ANR_AGBEZ	char(40)	Operation: Name
ANR_ATKBEZ	char(40)	Operation: Article name
ANR_LART	char(4)	Operation: Wage type
ANR_TE	double	Operation: Target t_e
ANR_TR	double	Operation: Target t_r
ANR_TEB	double	Operation: Target t_{eb}
ANR_TRB	double	Operation: Target t_{rb}
FU01	date	Operation: User field
...		
FU06	date	Operation: User field
FU07	long	Operation: User field
...		
FU22	long	Operation: User field
FU23	double	Operation: User field
...		
FU28	double	Operation: User field
FU29	char(1)	Operation: User field
...		
FU44	char(1)	Operation: User field
FU45	char(10)	Operation: User field
...		
FU50	char(10)	Operation: User field
FU51	char(20)	Operation: User field
...		
FU64	char(20)	Operation: User field
FU65	char(40)	Operation: User field
FU66	char(40)	Operation: User field
PNR_PNR	char(10)	Person: personnel number
PNR_KNR	char(10)	Person: Badge number
PNR_ANRGK	char(40)	Person: Waiting period OP
PNR_KST	char(10)	Person: Regular cost center
PNR_PGRP	char(20)	Person: Employee group
PNR_PRKZ	char(1)	Person: premium indicator
PNR_LEISTGRP	char(10)	Person: Regular premium group
PNR_BPOS	char(10)	Person: Regular operator function

PNR_LPKZ	char(10)	Person: Regular wage/premium indicator
PNR_LART	char(4)	Person: Regular wage type
PNR_LGRP	char(4)	Person: Regular wage group
PNR_MNR	char(20)	Person: Regular workplace

Export parameter

Parameter	Type	Content
LART	char(4)	Assigned wage type
TE	double	Assigned t_e
TR	double	Assigned t_r
TEB	double	Assigned t_{eb}
TRB	double	Assigned t_{rb}
LEISTGRP	char(10)	Assigned premium group By default, the premium group is only identified for machines with premium indicator G and for event A_AN from dialog data or from the premium group assignment. By default, the system also enters the premium group of the logged in order into the personnel postings. Consequently, identical premium groups are assigned for order postings and personnel postings.

Example


```

/*****
* d_lle_daten.hsc
* Purpose:      User exit to identify LLE specifications
*               to be entered in ADE log records
* Notes:
* Date:         $Date: 2010/05/25 00:00:00 $
* Revision:     $Revision: 1.0 $
*****/
* History
* $Log$
*
*****/

hydra basic;

import ERRORCODE          long;

// import DLG_DATA          char(30000); // Overall dialog data
import DLG                char(79) ; // dialog ID
import EVENT              char(79) ; // Resolved event
import MNR_MNR            char(20) ; // Machine/Workplace
import MNR_MGRP           char(20) ; // Machine/Workplace: Group
/*
import MNR_ART            char(1)  ; // Machine/Workplace: Type group/single workplace
import MNR_KST            char(10) ; // Machine/Workplace: Cost center
import MNR_BEZK           char(8)  ; // Machine/workplace: Name
import MNR_BEZL           char(40) ; // Machine/Workplace: Comment
import MNR_PRKZ           char(1)  ; // Machine/Workplace: Incentive wage indicator
import MNR_LEIGRAD        long     ; // Machine/Workplace: Performance level
import ANR_ANR            char(40) ; // Operation
*/
import ANR_ATK            char(40) ; // Operation: Article
/*
import ANR_AUNR           char(40) ; // Operation: Order number (without operation)
import ANR_AGNR           char(40) ; // Operation: Operation number (without order number)
import ANR_RMNR           char(40) ; // Operation: Confirmation number
import ANR_AUART          char(5)  ; // Operation: Order type
import ANR_AGBEZ          char(40) ; // Operation: Name
import ANR_ATKBEZ         char(40) ; // Operation: Article name
import ANR_LART           char(4)  ; // Operation: Wage type
import ANR_TE             double   ; // Operation: Target te
import ANR_TR             double   ; // Operation: Target tr
import ANR_TEB            double   ; // Operation: Target teb
import ANR_TRB            double   ; // Operation: Target trb
import FU01               date     ; // Operation: User field
import FU02               date     ; // Operation: User field
import FU03               date     ; // Operation: User field
import FU04               date     ; // Operation: User field
import FU05               date     ; // Operation: User field
import FU06               date     ; // Operation: User field
import FU07               long     ; // Operation: User field
import FU08               long     ; // Operation: User field
import FU09               long     ; // Operation: User field
import FU10               long     ; // Operation: User field
import FU11               long     ; // Operation: User field
import FU12               long     ; // Operation: User field
import FU13               long     ; // Operation: User field
import FU14               long     ; // Operation: User field
import FU15               long     ; // Operation: User field
import FU16               long     ; // Operation: User field
import FU17               long     ; // Operation: User field
import FU18               long     ; // Operation: User field
import FU19               long     ; // Operation: User field
import FU20               long     ; // Operation: User field
import FU21               long     ; // Operation: User field
import FU22               long     ; // Operation: User field
import FU23               double   ; // Operation: User field
import FU24               double   ; // Operation: User field
import FU25               double   ; // Operation: User field
import FU26               double   ; // Operation: User field
import FU27               double   ; // Operation: User field
import FU28               double   ; // Operation: User field
import FU29               char(1)  ; // Operation: User field
import FU30               char(1)  ; // Operation: User field
import FU31               char(1)  ; // Operation: User field
import FU32               char(1)  ; // Operation: User field
import FU33               char(1)  ; // Operation: User field
import FU34               char(1)  ; // Operation: User field
import FU35               char(1)  ; // Operation: User field
import FU36               char(1)  ; // Operation: User field
import FU37               char(1)  ; // Operation: User field
import FU38               char(1)  ; // Operation: User field
import FU39               char(1)  ; // Operation: User field
import FU40               char(1)  ; // Operation: User field
import FU41               char(1)  ; // Operation: User field
import FU42               char(1)  ; // Operation: User field
import FU43               char(1)  ; // Operation: User field
import FU44               char(1)  ; // Operation: User field
import FU45               char(10) ; // Operation: User field
import FU46               char(10) ; // Operation: User field
import FU47               char(10) ; // Operation: User field
import FU48               char(10) ; // Operation: User field
import FU49               char(10) ; // Operation: User field
import FU50               char(10) ; // Operation: User field
import FU51               char(20) ; // Operation: User field
import FU52               char(20) ; // Operation: User field
import FU53               char(20) ; // Operation: User field
import FU54               char(20) ; // Operation: User field

```

```

import FU55          char(20) ; // Operation: User field
import FU56          char(20) ; // Operation: User field
import FU57          char(20) ; // Operation: User field
import FU58          char(20) ; // Operation: User field
import FU59          char(20) ; // Operation: User field
import FU60          char(20) ; // Operation: User field
import FU61          char(20) ; // Operation: User field
import FU62          char(20) ; // Operation: User field
import FU63          char(20) ; // Operation: User field
import FU64          char(20) ; // Operation: User field
import FU65          char(40) ; // Operation: User field
import FU66          char(40) ; // Operation: User field
import PNR_PNR       char(10) ; // Person: Personnel number
import PNR_KNR       char(10) ; // Person: Badge number
import PNR_ANRGK     char(40) ; // Person: Waiting period OP
import PNR_KST       char(10) ; // Person: Regular cost center
import PNR_PGRP      char(20) ; // Person: Employee group
import PNR_PRKZ      char(1) ; // Person: Premium indicator
import PNR_LEISTGRP  char(10) ; // Person: Regular premium group
import PNR_BPOS      char(10) ; // Person: Regular operator function
import PNR_LPKZ      char(10) ; // Person: Regular wage/premium indicator
import PNR_LART      char(4) ; // Person: Regular wage type
import PNR_LGRP      char(4) ; // Person: Regular wage group
import PNR_MNR       char(20) ; // Person: Regular workplace
*/

export LART          char(4) ; // Assigned wage type
export TE            double ; // Assigned te
export TR            double ; // Assigned tr
export TEB           double ; // Assigned teb
export TRB           double ; // Assigned trb
// export LEISTGRP   char(10) ; // Assigned premium group

/*-----*/
long main()
{
variable ret long;
variable atk_te    double;
variable atk_tr    double;
variable atk_teb   double;
variable atk_trb   double;
variable atk_lart  char(4);
variable atk_masch_nr char(20);
variable atk_mgruppe char(20);

//-----
// Notes for premium group:
// By default the premium group can only be identified for machines with premium
// indicator G and for A-AN events from the dialog data or
// from the premium group assignment.
// By default, the premium group
// of the logged in order is also assigned to personnel postings. This ensures consistency of premium groups:
// between order postings and
// related personnel postings.
//
// if( (MNR_PRKZ = "G" ) and (EVENT="A-AN" ) )
// {
//   dprint( "identify premium group here in the script:" );
//   // ...
// }
// else
// {
//   dprint( "Leave premium group from logged in OP: " ||
//         (LEISTGRP clipped) || "." );
// }
//
//-----

ret = ERRORCODE;

dprint( "Search for article specifications\"" || (ANR_ATK clipped) || "\" at the machine\"" || (MNR_MNR clipped) ||
      "\"", Group "\" || (MNR_MGRP clipped) || "\" (" || (DLG clipped) || "/" || (EVENT clipped) || ")." );

sqlexec( "select nvl( soll_te, 0 ) + nvl( soll_te2, 0) soll_te, " ||
        " nvl( soll_tr, 0 ) soll_tr, " ||
        " nvl( soll_teb, 0 ) soll_teb, " ||
        " nvl( soll_trb, 0 ) soll_trb, " ||
        " lohnart, " ||
        " masch_nr, " ||
        " mgruppe " ||
        " from u_lle_atk_vorgaben " ||
        " where artikel = " || BV(ANR_ATK) ||
        " and (masch_nr = " || BV(MNR_MNR) || " or masch_nr is null ) " ||
        " and (mgruppe = " || BV(MNR_MGRP) || " or mgruppe is null ) " ||
        " order by nvl( masch_nr, \"\") desc;" ); // Sorting: Empty machine numbers at the back.
into( atk_te, atk_tr, atk_teb, atk_trb, atk_lart, atk_masch_nr, atk_mgruppe );

if( sqlcode() = 0 )
{
dprint( " Gefunden: te=" || atk_te || ", tr=" || atk_tr || ", teb=" || atk_teb || ", trb=" || atk_trb || ", " );
dprint( " Lohnart=\"" || atk_lart || "\", Maschine=\"" || (atk_masch_nr clipped) ||
      "\"", Maschinengruppe=\"" || (atk_mgruppe clipped) || "\"." );

TE = atk_te;
TR = atk_tr;
TEB = atk_teb;
TRB = atk_trb;
if( atk_lart is not null )
{
LART = atk_lart;
}
}

```

```

    }
}
else
{
    dprint( " No article specifications found." );
}

    return ret;
}

/*-----*/

```

5.5.4.3 Assignment of user fields in the ADE log and order backlog

Name of user exit

dd_usrfld_ab.hsc for order backlog fields

dd_usrfld_ap.hsc for ADE log fields

Keywords

User fields in the order backlog and ADE log, order-related postings.

Function

The relevant user exit (see above) is called and the data is stored in the database during the generation of an ADE log record or an order dialog function (A_TR, A_MR, A_BE, A_AB, A_UN, A_P_AN and A_AN).

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The import variable *dd* transfers dialog data.
- ❖ The export variables fu_d_01 to fu_d_06 transfer the date user fields.
- ❖ The export variables fu_n_07 to fu_n_22 transfer the integer user fields.
- ❖ The export variables fu_f_23 to fu_f_28 transfer the double user fields.
- ❖ The export variables fu_c_29 to fu_c_66 transfer the char user fields.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			d_a_an.h, d_ade.c, d_a_ab.c, d_a_an.c, d_a_tr.c, d_a_ab.c

Import parameter

Parameter	Type	Content
DD	C8000	Dialog data calling the dialog.
DIALOG	C80	Dialog that triggered the user exit.
EREIGNIS	C80	Event that triggered the user exit.
ANR	C40	OP from the event or ADE log.
MNR	C20	Machine from the event or ADE log.
PNR	C10	Person from the event or ADE log
CNR	C20	Batch from the event or ADE log

Export parameter

Parameter	Type	Content
FU_D_01 to FU_D_06	Date	Date user fields
FU_N_07 to FU_N_22	Long	Integer user fields
FU_F_23 to FU_F_28	Double	Double user fields
FU_C_29 to FU_C_44	C1	Char user fields
FU_C_45 to FU_C_50	C10	Char user fields
FU_C_51 to FU_C_64	C20	Char user fields
FU_C_65 to FU_C_66	C40	Char user fields

5.5.4.4 User exit after INSERT of ADE log record

Name of user exit

dd_ap_afterinsert.hsc

Keywords

User fields, ADE log record, order-related postings.

Function

User exit requested with the ADE log record reference after inserting the ADE log record.

- ❖ User exit requested with the ADE log record reference after inserting the ADE log record. The import variable DLG_DATA transfers the dialog data.
- ❖ The user exit is also called if data is recalculated via the event maintenance.
- ❖ If HYMW is active, the user exit is always reinitialized. This means that changes in the user exit take effect immediately without having to restart the system on the server.
- ❖ You can use SQL and Bapi calls in the user exit.
- ❖ Timeout is set to 30 seconds.
- ❖ Warning: This user exit might have adverse effects on the performance of data collection.
- ❖ WARNING: The user exit is not requested for automatically generated partial confirmations (e.g. because of automatically calculated quantities).
- ❖ Warning: you require at least the hymw version 7.2.1.456 and 8.1.1.451 in order to use this user exit also for B records from waiting period processing. In this case, the function main_karenz() is requested and not the function main(). You can request the function main() in main_karenz(). The reason for this specific function is ensuring backward compatibility if this user exit is now also activated for waiting period records.
- ❖ WARNING: The userexit can be used for automatically generated T-records for waiting periods (e.g. for calculated quantities) from hymw Version 7.2.1.457 and 8.1.1.460 onwards. In this case, the function main_auto_tr() is requested and not the function main(). You can request the function main() in main_auto_tr(). The reason for this specific function is ensuring backward compatibility if this user exit is now also activated for automatic T-records.
- ❖ WARNING: The user exit does not ensure that the data changed in the log record will be available when uploading data to the PPS.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			d_ade.c hyd_usrexit_usrfld.c hyd_karz.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data calling the dialog.
BUCH_MODE	INT	Posting mode (online/recalculation/...)
AP_VERWEIS	INT	Reference to the inserted log record
AP_SATZART	C1	Record type of the inserted log record (T/U/H/...)
DLG	C10	Dialog that triggered the user exit.
EVENT	C10	Event that triggered the user exit.
ANR	C40	OP from the event
MNR	C20	Machine from the event

Parameter	Type	Content
PNR	C10	Person from the event
CNR	C20	Batch from the event

Requested functions

Parameter	Content
long main()	Is requested for ADE log records triggered by input dialogs.
long main_karenz()	From hymw version 7.2.1.456 or 8.1.1.451 (08.07.11): This function is requested for ADE log records from personnel waiting period postings. Only the import parameters DLG_DATA, BUCH_MODE, AP_VERWEIS and AP_SATZART are completed with values. Other import parameters remain empty.
long main_auto_tr()	From hymw version 7.2.1.467 and 8.1.1.460 (16.09.11): Requested for automatically generated log records of type T. In this case, the triggering event completes the import parameters with values.
long check_recomputing()	RECALCULATION/CANCELLATION By default, the user exit is only executed with ONLINE data collection. From hymw version 7.2.1.458 and 8.1.1.452 onwards you can decide if you want to execute the user exit for recalculation/reversal posting. If the function check_recomputing() is available and the function returns RET=1, then the user exit is even executed for NON-ONLINE postings.

5.5.5 Extending the machine status list - Defining additional columns

Name of user exit

I_m_sl_list.hsc

Keywords

Machine status list, malfunction list, List;16

Function

Extending the machine status list with the user exit "I_m_sl_list.hsc". Use this user exit to add columns to the machine status list.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The user exit transfers the copy mode list to the script via the import variable.
- ❖ The import variable keys (char(200)) defines the processing.
 - The header transfers the value "HEADER".
 - The key values are transferred as BAPI string in the data row. Following acronym are currently supported:
 - MNR = Machine number
 - MST= Status
 - ZUNR = Assignment number
 - MSTTNR = Status number
 - PKENN = Control
- ❖ After executing the script, the export variable list_append (char(1000)) includes the string to extend the relevant row in the list.
- ❖ The import variable zeile (char(8192)) transfers the header row/data row to the script.

After generating the header row, the user exit is requested with keys = "HEADER" and extends the header row with the contents of the export variable list_append.

After generating the data row, the second request takes place with keys = "MNR=...|MST=...|ZUNR=...|MSTTNR=...|PKENN=..." and extends the data row also with the content of the export variable list_append.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			l_mnr.c

Import parameter

Parameter	Type	Content
MOD	C1	Mode of the machine status list
KEYS	C200	"HEADER" or ANR and MNR as Bapi string
ZEILE	C8192	Header/data row
DLG_DATA	C8192	Dialog data string

Export parameter

Parameter	Type	Content
LIST_APPEND	C1000	String extending the header or data row.

5.5.6 Extending the data cursor for HYASPROT

Name of user exit

hyasprot.hsc

Keywords

Hyasprot, order shift log

Function

Adding columns to the order shift log. Use the user exit to add the columns, tables and required joins to the standard data cursor.

Carry out the following steps in the user exit:

- Enter additional columns in the export parameter SPALTEN (columns).
- Enter the additional tables in the export parameter TABELLEN (tables) if the tables are not available in the standard data cursor.
- Enter the required relations between tables in the export parameter JOINS.
- Enter the data types of the additional columns in the export parameter DATENTYPEN (data types).

The user exit must fill the export parameter *header_data* with the names of the additional columns.

Note: Do not execute an SQL statement in the user exit.

Program(s) and source code files

Program	Version	Date	File(s)
hyasprot.out			

User exit: pre_work()

Export parameter

Parameter	Type	Content
select_list	C1000	Database column names with table alias (single columns separated with comma, no comma after the last column). e.g.: mk.user_c_29, mk.leistgrad
table_reference	C1000	Database tables with table alias (comma separated tables, no comma after the last table). e.g.: maschinen_status u_ms, auftrag_status u_ast

Parameter	Type	Content
		<u>Note:</u> - Variable <i>table_reference</i> can remain empty, if you want to add columns from tables that are included in the standard data cursor. - Set "u_" in front of the alias to avoid any overlapping with the program's namespace. <u>Available tables:</u> - ade_protokoll ap - auftrags_bestand ab (operation) - auftrags_bestand ak (order header) - maschinen mk - ade_auftragsarten aa
where_clause	C1000	Extension of "WHERE" clause e.g.: <code>mk.masch_nr = u_ms.masch_nr and ab.auftrag_nr = u_ast.auftrag_nr</code>
header_data	C32000	You can add further column headings for the header to this data string. e.g.: <code>ZusatzDaten1 ZusatzDaten2 ZusatzDaten3 </code> <u>Note:</u> You must finish additional column names with a pipe character ().
additional_data	C32000	This data string stores the user exit's additional data. The data has the following format: e.g.: <code>KEY1=DATA1 KEY2=DATA2 KEY3=DATA3 </code> The fields DATA1 ... DATA _n are filled by a fetch and are available in this variable.
prog_parameter	C32000	This data string includes the command line parameters.

User exit: fetch_data()

Import parameter

→ All variables from the standard fetch are provided to the user exit in read-only mode.

Parameter	Type	Content
FETCH_AB_ANR FETCH_AP_SART FETCH_AP_VERWEIS FETCH_AB_SGR_GUTB FETCH_AB_SGR_GUTP FETCH_AB_SGR_GUTS		

Parameter	Type	Content
FETCH_AB_SGR_GUTB		
FETCH_AB_ATK		
FETCH_AB_ATKBEZ		
FETCH_AP_EGR_GUTB		
FETCH_AP_EGR_GUTP		
FETCH_AP_EGR_GUTS		
FETCH_AP_EGR_GUTT		
FETCH_AP_EGR_AUSB		
FETCH_AP_EGR_AUSP		
FETCH_AP_EGR_AUSS		
FETCH_AP_EGR_AUST		
FETCH_AP_EGR_NCHB		
FETCH_AP_EGR_NCHP		
FETCH_AP_EGR_NCHS		
FETCH_AP_EGR_NCHT		
FETCH_AP_ABBR_GRD		
FETCH_AB_SZ		
FETCH_AB_SGE_B		
FETCH_AB_SGE_P		
FETCH_AB_SGE_S		
FETCH_AB_SGE_T		
FETCH_AB_TLG		
FETCH_AP_EGR_PRBB		
FETCH_AP_EGR_PRBP		
FETCH_AP_EGR_PRBS		
FETCH_AP_EGR_PRBT		
FETCH_AB_RUEZ		
FETCH_AP_BMK01		
FETCH_AP_BMK02		
FETCH_AP_BMK03		
FETCH_AP_BMK04		
FETCH_AP_BMK05		
FETCH_AP_BMK06		
FETCH_AP_BMK07		
FETCH_AP_BMK08		
FETCH_AP_BMK09		
FETCH_AP_BMK10		
FETCH_AP_BMK11		
FETCH_AP_BMK12		
FETCH_AP_EGR_DAUER		
FETCH_AP_DATB		
FETCH_AP_ZEIB		
FETCH_AP_DATE		
FETCH_AP_ZEIE		
FETCH_MK_MNR		
FETCH_MK_MGRP		
FETCH_MK_BEZK		
FETCH_MK_BEZL		
FETCH_MK_KST		
FETCH_AP_TNR		
FETCH_MK_TYP		
FETCH_MK_BDEJMOD		
FETCH_AB_AGBEZ		

Parameter	Type	Content
FETCH_AB_AGPOS FETCH_AB_OPT_CNR FETCH_AA_KAT FETCH_AB_AUART FETCH_AA_ICON FETCH_AB_SAMMEL FETCH_AB_SPLIT		

Note: Information about the origin of the fields:

- AA ... ADE_AUFTRAGSARTEN
- AB ... AUFTRAGS_BESTAND
- AP ... ADE_PROTOKOLL
- MK ... MASCHINEN
- MA ... intermediate table

Export parameter

→ All variables from the standard totals flag are provided to the user exit.

Parameter	Type	Content
OUTDATA_MNR, OUTDATA_MBEZK OUTDATA_MBEZL OUTDATA_MGRP OUTDATA_KST OUTDATA_ANR OUTDATA_ATK OUTDATA_ATKBEZ, OUTDATA_SGR_GUT_BAS OUTDATA_SGR_GUT_PRI OUTDATA_SGR_GUT_SEK OUTDATA_SGR_GUT_TER OUTDATA_AGR_GUT_BAS OUTDATA_AGR_GUT_PRI OUTDATA_AGR_GUT_SEK OUTDATA_AGR_GUT_TER OUTDATA_AGR_AUS_BAS OUTDATA_AGR_AUS_PRI OUTDATA_AGR_AUS_SEK OUTDATA_AGR_AUS_TER OUTDATA_AGR_LEN_BAS OUTDATA_AGR_LEN_PRI OUTDATA_AGR_LEN_SEK OUTDATA_AGR_LEN_TER OUTDATA_EGR_PRBB OUTDATA_EGR_PRBP OUTDATA_EGR_PRBS OUTDATA_EGR_PRBT OUTDATA_SGR_BMK11 OUTDATA_AGR_BMK01 OUTDATA_AGR_BMK02 OUTDATA_AGR_BMK03 OUTDATA_AGR_BMK04		

Parameter	Type	Content
OUTDATA_AGR_BMK05 OUTDATA_AGR_BMK06 OUTDATA_AGR_BMK07 OUTDATA_AGR_BMK08 OUTDATA_AGR_BMK09 OUTDATA_AGR_BMK10 OUTDATA_AGR_BMK11 OUTDATA_AGR_BMK12 OUTDATA_AGR_DAUER OUTDATA_DAT OUTDATA_SKNR OUTDATA_SOLLTAKTE OUTDATA_AG_BEZ OUTDATA_AUNR OUTDATA_AFOLG OUTDATA_AGNR OUTDATA_UAGNR OUTDATA_SPLNR OUTDATA_KATEGORIE OUTDATA_ME_BAS OUTDATA_ME_PRI OUTDATA_ME_SEK OUTDATA_ME_TER OUTDATA_SYMBOL OUTDATA_AUFTRAG_ART OUTDATA_DATB OUTDATA_ZEIB		

Parameter	Type	Content
additional_data	C32000	The values from the additionally requested columns in <code>AP pre_work()</code> (=> depending on <code>select_list</code>, <code>table_reference</code> and <code>where_clause</code>) are assigned to the variables in the <code>additional_data</code> string. e.g.: <pre>select_list = mk.user_c_29, mk.leistgrad additional_data = maschine_info1= maschine_info2= </pre> <u>Result:</u> → additional_data = maschine_info1={value from mk.user_c_29} maschine_info2={value from mk.leistgrad} <u>Note:</u> The table fields are always transferred as string values.
output_data	C32000	Provide the additional columns to be output in this data string. The sequence of data must correspond to that of the columns added to the variable <code>header_data</code>. You must finish the values with a pipe character (). e.g.: 20 Muster Maschine 16.5
write_data	C1	The user exit can use this flag to define whether a calculated

Parameter	Type	Content
		data record is written to the output file. Possible values: Y.... The full data record (standard fields and fields specific to the user exit) is written to the output file (by default). N... No data is written in the output file.

User exit: `calculated_sum ()`

This user exit allows you to change the data after having calculated the data. All fetch and output variables are available.

User exit: `last_chance ()`

This (last) user exit can write additional rows into the output file.

- The header line is transferred in read-only mode.
- Data string with values: The user exit must complete the standard fields and the fields specific to the user exit. Separate the individual lines by "\n".

5.5.7 Extending the data cursor for HYPSPROT

Name of user exit

hypsprot.hsc

Keywords

Hypsprot, personnel shift log, staff shift log

Function

Adding columns to the personnel shift log. Use the user exit to add the columns, tables and required joins to the standard data cursor.

Carry out the following steps in the user exit:

- Enter additional columns in the export parameter SPALTEN (columns).

- Enter the additional tables in the export parameter TABELLEN (tables) if the tables are not available in the standard data cursor.
- Enter the required relations between tables in the export parameter JOINS.
- Enter the data types of the additional columns in the export parameter DATENTYPEN (data types).

The user exit must fill the export parameter *header_data* with the names of the additional columns.

Note: Do not execute an SQL statement in the user exit.

Program(s) and source code files

Program	Version	Date	File(s)
hypsprot.out			

User exit: pre_work()

Export parameter

Parameter	Type	Content
select_list	C1000	Database column names with table alias (single columns separated with comma, no comma after the last column). e.g.: <code>mk.user_c_29, mk.leistgrad</code>
table_reference	C1000	Database tables with table alias (comma separated tables, no comma after the last table). e.g.: <code>maschinen_status u_ms, auftrag_status u_ast</code> <u>Note:</u> - Variable <i>table_reference</i> can remain empty, if you want to add columns from tables that are included in the standard data cursor. - Set "u_" in front of the alias to avoid any overlapping with the program's namespace. <u>Available tables:</u> - ade_protokoll ap - auftrags_bestand ab (operation) - auftrags_bestand ak (order header) - maschinen mk - ade_auftragsarten aa
where_clause	C1000	Extension of "WHERE" clause e.g.: <code>mk.masch_nr = u_ms.masch_nr and ab.auftrag_nr = u_ast.auftrag_nr</code>
header_data	C32000	You can add further column headings for the header row to this data string. e.g.: <code>ZusatzDaten1 ZusatzDaten2 ZusatzDaten3 </code>

Parameter	Type	Content
		Note: You must finish additional column names with a pipe character ().
additional_data	C32000	This data string stores the user exit's additional data. The data has the following format: e.g.: KEY1=DATA1 KEY2=DATA2 KEY3=DATA3 The fields DATA1 ... DATAn are filled by a fetch and are available in this variable.
prog_parameter	C32000	This data string transfers command line parameters (one-to-one) to the user exit.

User exit: fetch_data()

Import parameter

→ All variables from the standard fetch are provided to the user exit in read-only mode.

Parameter	Type	Content
FETCH_AB_ANR FETCH_AP_SART FETCH_AP_VERWEIS FETCH_AB_SGR_GUTB FETCH_AB_SGR_GUTP FETCH_AB_SGR_GUTS FETCH_AB_SGR_GUTB FETCH_AB_ATK FETCH_AB_ATKBEZ FETCH_AP_EGR_GUTB FETCH_AP_EGR_GUTP FETCH_AP_EGR_GUTS FETCH_AP_EGR_GUTT FETCH_AP_EGR_AUSB FETCH_AP_EGR_AUSP FETCH_AP_EGR_AUSS FETCH_AP_EGR_AUST FETCH_AP_EGR_NCHB FETCH_AP_EGR_NCHP FETCH_AP_EGR_NCHS FETCH_AP_EGR_NCHT FETCH_AP_ABBR_GRD FETCH_AB_SZ FETCH_AB_SGE_B FETCH_AB_SGE_P FETCH_AB_SGE_S FETCH_AB_SGE_T		

Parameter	Type	Content
FETCH_AB_TLG FETCH_AP_EGR_PRBB FETCH_AP_EGR_PRBP FETCH_AP_EGR_PRBS FETCH_AP_EGR_PRBT FETCH_AB_RUEZ FETCH_AP_BMK01 FETCH_AP_BMK02 FETCH_AP_BMK03 FETCH_AP_BMK04 FETCH_AP_BMK05 FETCH_AP_BMK06 FETCH_AP_BMK07 FETCH_AP_BMK08 FETCH_AP_BMK09 FETCH_AP_BMK10 FETCH_AP_BMK11 FETCH_AP_BMK12 FETCH_AP_EGR_DAUER FETCH_AP_DATB FETCH_AP_ZEIB FETCH_AP_DATE FETCH_AP_ZEIE FETCH_MK_MNR FETCH_MK_MGRP FETCH_MK_BEZK FETCH_MK_BEZL FETCH_MK_KST FETCH_AP_TNR FETCH_MK_BDEJMOD FETCH_AB_AGBEZ FETCH_AB_AGPOS FETCH_AB_OPT_CNR FETCH_AA_KAT FETCH_AB_AUART FETCH_AA_ICON FETCH_AB_SAMMEL FETCH_AB_SPLIT		

Note: Information about the origin of the fields:

- AA ... ADE_AUFTRAGSARTEN
- AB ... AUFTRAGS_BESTAND
- AP ... ADE_PROTOKOLL
- MK ... MASCHINEN

Export parameter

→ All variables from the standard totals flag are provided to the user exit.

Parameter	Type	Content
OUTDATA_MNR OUTDATA_MBEZK OUTDATA_MBEZL		

Parameter	Type	Content
OUTDATA_MGRP		
OUTDATA_KST		
OUTDATA_ANR		
OUTDATA_ATK		
OUTDATA_ATKBEZ		
OUTDATA_PNR		
OUTDATA_PNAME		
OUTDATA_PVORNAME		
OUTDATA_PGRP		
OUTDATA_SGR_GUT_BAS		
OUTDATA_SGR_GUT_PRI		
OUTDATA_SGR_GUT_SEK		
OUTDATA_SGR_GUT_TER		
OUTDATA_AGR_GUT_BAS		
OUTDATA_AGR_GUT_PRI		
OUTDATA_AGR_GUT_SEK		
OUTDATA_AGR_GUT_TER		
OUTDATA_AGR_AUS_BAS		
OUTDATA_AGR_AUS_PRI		
OUTDATA_AGR_AUS_SEK		
OUTDATA_AGR_AUS_TER		
OUTDATA_AGR_LEN_BAS		
OUTDATA_AGR_LEN_PRI		
OUTDATA_AGR_LEN_SEK		
OUTDATA_AGR_LEN_TER		
OUTDATA_EGR_PRBB		
OUTDATA_EGR_PRBP		
OUTDATA_EGR_PRBS		
OUTDATA_EGR_PRBT		
OUTDATA_SGR_BMK11		
OUTDATA_AGR_BMK01		
OUTDATA_AGR_BMK02		
OUTDATA_AGR_BMK03		
OUTDATA_AGR_BMK04		
OUTDATA_AGR_BMK05		
OUTDATA_AGR_BMK06		
OUTDATA_AGR_BMK07		
OUTDATA_AGR_BMK08		
OUTDATA_AGR_BMK09		
OUTDATA_AGR_BMK10		
OUTDATA_AGR_BMK11		
OUTDATA_AGR_BMK12		
OUTDATA_AGR_DAUER		
OUTDATA_BPOS		
OUTDATA_BPOS_BEZK		
OUTDATA_BPOS_BEZL		
OUTDATA_DAT		
OUTDATA_SKNR		
OUTDATA_LPKZ		
OUTDATA_AUNR		
OUTDATA_AFOLG		
OUTDATA_AGNR		
OUTDATA_UAGNR		

Parameter	Type	Content
OUTDATA_SPLNR OUTDATA_KATEGORIE OUTDATA_ME_BAS OUTDATA_ME_PRI OUTDATA_ME_SEK OUTDATA_ME_TER OUTDATA_SYMBOL OUTDATA_AUFTRAG_ART		

Parameter	Type	Content
additional_data	C32000	The values from the additionally requested columns in AP <i>pre_work()</i> (=> depending on select_list, table_reference and where_clause) are assigned to the variables in the <i>additional_data</i> string. e.g.: <pre>select_list = mk.user_c_29, mk.leistgrad additional_data = maschine_info1= maschine_info2= </pre> Result: <pre>→ additional_data = maschine_info1={value from mk.user_c_29} maschine_info2={value from mk.leistgrad} </pre> <u>Note:</u> The table fields are always transferred as string values.
output_data	C32000	Provide the additional columns to be output in this data string. The sequence of data must correspond to that of the columns added to the variable <i>header_data</i>. You must finish the values with a pipe character (). e.g.: 20 Muster Maschine 16.5
write_data	C1	The user exit can use this flag to define whether a calculated data record is written to the output file. Possible values: Y.... The full data record (standard fields and fields specific to the user exit) is written to the output file (by default). N... No data is written in the output file.

User exit: **calculated_sum ()**

This user exit allows you to change the data after having calculated the data. All fetch and output variables are available.

User exit: **last_chance ()**

This (last) user exit can write additional rows into the output file.

- The header line is transferred in read-only mode.
- Data string with values: The user exit must complete the standard fields and the fields specific to the user exit. Separate the individual lines by "\n".

5.5.8 Overriding the HYDRA basic settings with machine configuration

Name of user exit

hyd_mnr_getconfig.hsc

5.5.8.1 Processing merged operations at the terminal

Keywords

Merged operation(s)

Function

In the basic settings of the system, the processing of merged operations is specified system-wide.

- J: Creation using office client, homogeneous distribution
- N: Creation using office client, inhomogeneous distribution
- A: Creation using the terminal (distribution by single OPs)
- V: Creation using the terminal (distribution in proportion to the standard times of the single OPs)
- M: Creation using the terminal (distribution in proportion to the target quantities of the single OPs)

In the function `hyd_mnr_getconfig_optsag()` the setting can be changed including machines at runtime, provided that an option for the formation of merged operations at the terminal has been selected in the basic settings. The user exit is not executed if merged operations are built with the office client.

Example:

- The actual times recorded for the merged operation are to be divided up at a workplace and configured as an "individual workplace" in proportion to the standard times of the individual operations.
- The actual times recorded for the merged operation are to be divided up at a workplace configured as a "group workplace" according to the number of single operations.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe / .out			

Import parameter

Parameter	Type	Content
MNR	C20	Machine number
MGRP	C1	Machine group number
TNR	N	Terminal number
PNR	C10	Person: personnel number

Export parameter

Parameter	Type	Content
RET_DATA	C1000	BDE: Processing of merged operations (HYDRA.OPT:SAG) --> Recorded times are split A: ... according to the number of single OPs V: ... in relation to the standard times of single OPs M: ... in relation to the default quantities of single OPs

5.5.9 Extension of the BDE archiver

Name of user exit

hybdearc_modifyarcdata.hsc

Keywords

hybdearc

Function

Use the user exit "hybdearc_modifyarcdata.hsc" to change/complement/correct objects subsequently, once the archiver has archived these objects.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ The import variable ONLINE_TABLE (char(100)) transfers the currently processed live table to the script.
- ❖ The import variable ARCHIVE_TABLE (char(100)) transfers the currently processed archive table to the script. The name and path of the unload file are transferred here in case of archiving action 'X'unload!
- ❖ The import variable TMP_TABLE (char(100)) transfers the name of the temporary table to the script. The column "schluessel" (key) of this temporary table includes all references of the currently processed data of the online and archive table.
- ❖ The import variable KEY_COLUMN (char(100)) transfers the reference field of the online and archive table to the script. Use this reference field to join to the "schluessel" field of the temporary table containing the current references.
- ❖ The import variable MODE (char(1)) transfers the current archiving action ('M'ove, 'D'elete, 'X'unload) to the script.
- ❖ The export variable TA_STATUS (long) transfers the status of the current transaction to the script (0 = Commit; != 0 = Rollback). You can overwrite this status in the script and trigger a rollback of the current transaction!

The user exit is called at the end of each transaction for every object to be archived. At this point in time, the actual archiving action has been executed and the data is located in the archive tables, Xunload files or has already been deleted!

Program(s) and source code files

Program	Version	Date	File(s)
hybdearc			hybdearc.c

Import parameter

Parameter	Type	Content
ONLINE_TABLE	C100	Processed live table
ARCHIVE_TABLE	C100	Populated archive table or path and name of the unload file
TMP_TABLE	C100	Temp. table containing the processed references (refer to description).
KEY_COLUMN	C100	Reference field of the archive and online table (see description)
KEY_D_COLUMN	C100	Reference field of the dependent table
MODE	C1	Current archiving mode ('M'ove, 'D'elete, 'X'unload)

Export parameter

Parameter	Type	Content
-----------	------	---------

Parameter	Type	Content
TA_STATUS	Long	Current TA status: 0 : Commit unequal 0 : Rollback.

5.5.10 3.9 Extension of the function ade_auto_verarb_insert

Name of user exit

ade_auto_verarb.hsc

Keywords

ade_auto_verarb

Function

Use this user exit to prevent data records from being inserted in the ade_auto_verarb table. If, for example, you want to schedule an order only once, you can use this user exit to check whether the table entry is allowed.

Program(s) and source code files

Program	Version	Date	File(s)
b_anr.dll			b_anr.c
b_afolg.dll			b_afolg.c
b_hls.dll			b_hls.c

Import parameter

Parameter	Type	Content
KEY1	C10	This field specifies whether the order header or the complete order number is stored in KEY2. "AUNR" (only for activity = "T")

Parameter	Type	Content
		"ANR" (only for activity = "R")
KEY2	C40	The content of KEY2 depends on KEY1 <AUNR> for KEY1="AUNR" <ANR> for KEY1="ANR"
AKTION	C1	T: Scheduling R: Check resources B: Batch job "update order data" b: Batch job "update order data" in process
DIALOG	C20	Where does the table data derive from (name of the BAPI/dialog)

Export parameter

Parameter	Type	Content
DO_INSERT	N	1 → Enter data in the table ade_auto_verarb {set by default} 0 → Reject entry

5.5.11 Extension of the data cursor for the maintenance of postings (DQADEPRO)

Name of user exit

dqadepro.hsc

Keywords

dqadepro

Function

- Extension of the standard "Where clause" for data selection to narrow down data.

Carry out the following steps in the user exit:

- Modify the "Where clause" to further restrict selection options in the pre_work() function
- Add further columns to the "select" of the maintenance of postings. Use the user exit to add the columns, tables and required joins to the standard data cursor.

Carry out the following steps in the user exit:

- Enter the data to be selected additionally in the export parameter SELECT_LIST.
- Enter the additional tables in the export parameter TABLE_REFERENCE if the tables are not available in the standard data cursor.
- Enter the required relations between tables in the export parameter WHERE_CLAUSE.
- Enter the columns to be displayed in the export parameter OUTPUT_DATA.
The parameter ADDITIONAL_DATA transfers the data from the "Select" to the user exit.
- Define the columns to be output in the export parameter HEADER_DATA.

Program(s) and source code files

Program	Version	Date	File(s)
dqadepro.out			

User exit: pre_work()

Import parameter

Parameter	Type	Content
DLG_DATA	C1000	This parameter calls the maintenance of postings

-

Export parameter

Parameter	Type	Content
select_list	C1000	Database column names with table alias (single columns separated with comma, no comma after the last column).
table_reference	C1000	Database tables with table alias (comma separated tables, no comma after the last table). <u>Note:</u> Variable <i>table_reference</i> can remain empty, if you want to add columns from tables that are included in the standard data cursor. <u>Existing tables with Alias:</u> - ade_protokoll ap - auftrags_bestand ab - auftrag_status ast - maschinen m - personalstamm pst - auftrags_zusatz az - auftrags_bestand ab2 (order header) - ade_auftragsarten auart
where_clause	C1000	Extension of "WHERE" clause e.g.: and ap.kostenstelle in (select kostenstelle from

Parameter	Type	Content
		kstst_tab where usr= " BV(bearb stripped) ") "
header_data	C32000	You can add further column headings for the header row to this data string. e.g.: ZusatzDaten1 ZusatzDaten2 ZusatzDaten3 <u>Note:</u> You must finish additional column names with a pipe character ().
additional_data	C32000	This data string stores the user exit's additional data. The data has the following format: e.g.: KEY1=DATA1 KEY2=DATA2 KEY3=DATA3 The fields DATA1 ... DATAn are filled by the Fetch and are available in this variable.

User exit: fetch_data()

Import parameter

Parameter	Type	Content
DLG_DATA	C1000	This parameter calls the maintenance of postings
VERWEIS_DATA	INT	

-

Export parameter

Parameter	Type	Content
additional_data	C32000	The values from the additionally requested columns in AP <i>pre_work()</i> (=> depending on select_list, table_reference and where_clause) are assigned to the variables in the <i>additional_data</i> string. e.g.: select_list = mk.user_c_29, mk.leistgrad additional_data = maschine_info1= maschine_info2= <u>Result:</u> ➔ additional_data = maschine_info1={value from mk.user_c_29} maschine_info2={value from mk.leistgrad} <u>Note:</u> The table fields are always transferred as string values.
output_data	C32000	Provide the additional columns to be output in this data string. The sequence of data must correspond to that of the columns added to the variable <i>header_data</i>.

Parameter	Type	Content
		You must finish the values with a pipe character (). e.g.: 20 Muster Maschine 16.5

5.5.12 Modification of event maintenance data (HYEEDIT)

Name of user exit

hyeedit.hsc

Keywords

Hyeedit, event maintenance

Function

- Adjust dialog data with Insert / Update before processing
-> hyeedit_modify_dlgdata()
- Adjust table data before starting the recalculation
-> hyeedit_before_calc()
- Carry out activities after recalculation
(e.g. update a customer-specific table after successful recalculation process)
-> hyeedit_after_calc()

Program(s) and source code files

Program	Version	Date	File(s)
hyeedit.out			

Import parameter

Call parameters of the event maintenance:

Parameter	Type	Content
HYEED_FKT	C100	Function you used to call the event maintenance -INSERT -UPDATE -DELETE
HYEED_TABLE	C100	Name of the temporary table of the event maintenance

Parameter	Type	Content
HYEED_USR	INT	HYDRA user
HYEED_DATB	INT	Parameter used when calling the event maintenance: From date
HYEED_DATE	INT	Parameter used when calling the event maintenance: To date
HYEED_ZEIB	INT	Parameter used when calling the event maintenance: From time
HYEED_ZEIE	INT	Parameter used when calling the event maintenance: To time
HYEED_MNR	C40	Parameter used when calling the event maintenance: Machine number
HYEED_ANR	C40	Parameter used when calling the event maintenance: Order number
HYEED_PNR	C40	Parameter used when calling the event maintenance: Personnel number
HYEED_JOBID	INT	Parameter used when calling the event maintenance: Job number (specific processing for calling the batch mode)
HYEED_NOCALCSK	INT	Parameter when calling the event maintenance (internal parameter): Do not recalculate shift
HYEED_NOCALCPERS	INT	Parameter used when calling the event maintenance (internal parameter): Do not recalculate personnel postings
HYEED_TABDAT	C100	Parameter used when calling the event maintenance (internal parameter): - Specifies which tables are to be taken into account Default = JJJJ: Digit 1=ADE, Digit 2=ADEP, Digit 3=MDE, Digit 4=LOS (batch)
HYEED_PARAPERS	C40	Parameter used when calling the event maintenance: "J" - include parallel personnel postings

User exit: hyeedit_modify_dlgdata ()

Import parameter

Parameter	Type	Content
All call parameters of the event maintenance		See above

Export parameter

Parameter	Type	Content
HYEED_DLGDATA	C1000	Dialog data of the event

User exit: hyeedit_before_calc ()

Import parameter

Parameter	Type	Content
All call parameters of the event maintenance		See above

Import parameter

User exit: hyeedit_after_calc ()

Import parameter

Parameter	Type	Content
All call parameters of the event maintenance		See above
HYEED_HYDDI_RET	INT	Recalculation return value by HYMW

5.5.13 Setup change

Name of user exit

ruestw.hsc

Keywords

Setup change

Function

Use this user exit to change the output file of setup changes. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
ruestw.out			ruestw.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.5.14 PZE (IN/OUT) controls BDE / waiting period processing

Name of user exit

pze_controls_ade.hsc

Keywords

PZE controls BDE, waiting period processing, staff is not logged off in BDE during break.

Function

This user exit specifies:

- Function: `is_active_for_dlg()`:
If the person is logged off from and on to workstations due to PZE clockings (example: do not log off staff from workstations if breaks are clocked in PZE)
- Function: `is_no_op_interruption_karenz_active()`
If the operation is interrupted when all people are logged off from the workstation due to PZE clockings (example: unmanned production)

Note:

When using batch data processing, a minimum version of `hymwmp1725.dll/so 8.1.1.228` must be active in order for the processing/control of the user exit `pze_controls_ade.hsc` to also apply to the output batches communicated at the operation.

.Program(s) and source code files

Program	Version	Date	File(s)
<code>hymw.out/.exe</code>			SP11
<code>hymwmp1.so/.exe</code>	8.1.1. 228	2020-01-20	

Import parameter

Parameter	Type	Contents:																											
DLG_DATA	char(10000)	Parameter string in dialog data format.																											
SUBDLG	char(80)	Clocked event (e.g. P_AST, P_PAU, P_FGR)																											
DLG	char(80)	Executed event leading to the person's new clocking status. The system determines this event from the clocked event and, if necessary, from the current status of the person. Examples: <table> <tr> <th>Clocked=SUBDLG</th><th>Status</th><th>DLG</th></tr> <tr> <td>P_AST</td><td>Present</td><td>P_GEH</td></tr> <tr> <td>P_AST</td><td>Absent</td><td>P_KOM</td></tr> <tr> <td>P_PAU</td><td>Present</td><td>P_GEH</td></tr> <tr> <td>P_PAU</td><td>Absent</td><td>P_KOM</td></tr> <tr> <td>P_FGR</td><td>Present</td><td>P_GEH</td></tr> <tr> <td>P_FGR</td><td>Absent</td><td>P_KOM</td></tr> <tr> <td>P_KOM</td><td>-</td><td>P_KOM</td></tr> <tr> <td>P_GEH</td><td>-</td><td>P_GEH</td></tr> </table>	Clocked=SUBDLG	Status	DLG	P_AST	Present	P_GEH	P_AST	Absent	P_KOM	P_PAU	Present	P_GEH	P_PAU	Absent	P_KOM	P_FGR	Present	P_GEH	P_FGR	Absent	P_KOM	P_KOM	-	P_KOM	P_GEH	-	P_GEH
Clocked=SUBDLG	Status	DLG																											
P_AST	Present	P_GEH																											
P_AST	Absent	P_KOM																											
P_PAU	Present	P_GEH																											
P_PAU	Absent	P_KOM																											
P_FGR	Present	P_GEH																											
P_FGR	Absent	P_KOM																											
P_KOM	-	P_KOM																											
P_GEH	-	P_GEH																											

KNR	char(10)	Badge number of the person carrying out the clocking.
-----	----------	---

Export parameter

There are no export parameters. The return value of the called function specifies system behavior:

Function is_active_for_dlg()

Return value 0:

The person is not logged off from the workstations if due to the PZE clocking the person is set to the status "absent" or "break".

Return value 1 (default):

Activates the standard processing according to the basic settings.

Function is_no_op_interruption_karenz_active()

Return value 0 (set by default):

Activates standard processing. The OP is interrupted if the PZE event logs off the last person from the OP.

Return value 1

The OP is not interrupted but continues (unmanned production) even if the PZE event logs off the last person from the OP.

Example

```
//-----
// Script: pze_controls_ade.hsc
// Descr.:
//-----
// $Revision: 1.00000 $
// $Date: 2016/09/01 12:00:00 $
//-----
hydra basic;

//-----
// constants
//-----

//-----
// IMPORT/EXPORT variables
//-----
import SUBDLG char(80); // Original clocking P KOM, P GEH, P AST, P PAU, P FGR
import DLG char(80); // Event that is finally executed (P_KOM, P_GEH)
import KNR char(10); // Badge id of person
import DLG_DATA char(32000); // Complete dialog data string

//-----
// global variables
//-----

//-----
// function get_pnr_from_dlg_data
//-----
char(10) get_pnr_from_dlg_data(dlgdata char(30000), export pnr_infotext_3 char(40))
{
    variable pnr char(10);

    pnr = "";
    pnr_infotext_3 = "N";
}
```

```

// Get HR master data by badge id
if ((pnr clipped) is null)
{
    sqlexec
    (
        "select " ||
        " person_nr char " ||
        ",infotext_3 " ||
        " from " ||
        " personalstamm " ||
        " where " ||
        " karten_nummer = " || BV(KNR clipped) || " " ||
        ";"
    );
}

if (sqlCode() = 0)
{
    into(pnr, pnr_infotext_3);
    dprint("Person id by HR master data: [" || (pnr clipped) ||
        "]" to badge=[" || (KNR clipped) || "] INFO3[" || (pnr_infotext_3 clipped) || "]);";
}
else
{
    // Process replacement badges
    sqlexec
    (
        "select" ||
        " personalstamm.person_nr_char " ||
        ",personalstamm.infotext_3 " ||
        " from " ||
        " zks_ausweis " ||
        ",personalstamm " ||
        " where " ||
        " zks_ausweis.ausweis_nr = " || BV(KNR clipped) || " " ||
        " and zks_ausweis.art = 'E' " ||
        " and zks_ausweis.aktiv = 'J' " ||
        " and (zks_ausweis.gueltig_von <= today or zks_ausweis.gueltig_von is null) " ||
        " and (zks_ausweis.gueltig_bis >= today or zks_ausweis.gueltig_bis is null) " ||
        " and zks_ausweis.person_nr = personalstamm.personalnummer " ||
        ";"
    );

    if (sqlCode() = 0)
    {
        into(pnr, pnr_infotext_3);
        dprint("Person id by replacement badge: [" || (pnr clipped) ||
            "]" of badge=[" || (KNR clipped) || "] INFO3[" || (pnr_infotext_3 clipped) || "]);";
    }
}
}

return pnr;
}

//-----
// function is_active_for_dlg
//-----
long is_active_for_dlg()
{
    variable ret long;
    variable pnr char(10);
    variable pnr_infotext_3 char(40);

    ret = 1; // default: TRUE

    dprint("[is_active_for_dlg] { DLG[" || DLG clipped || "] SUBDLG[" || SUBDLG clipped ||
        "]" DLG_DATA[" || DLG_DATA clipped || "]);";

    // Get INFOTEXT 3 from hr master data
    pnr = get_pnr_from_dlg_data(DLG_DATA, pnr_infotext_3);

    // Log off staff when clocking breaks and log on staff automatically at end of the break?
    // Y ... Yes / N ... No
    if ((pnr_infotext_3 clipped) = "N")
    {
        if ((SUBDLG clipped = "P_PAU") and
            ((DLG clipped = "P_KOM") or (DLG clipped = "P_GEH")))
        {
            dprint("[is_active_for_dlg] PZE cotrolls ADE is turned of in case of start/end of break.");
            ret = 0; // FALSE
        }
    }

    dprint("[is_active_for_dlg] } RET[" || ret using "<<<<<<<<<" || "]);";
    return ret;
}

//-----
// function is_no_op_interruption_karenz_active
//-----
// long is_no_op_interruption_karenz_active()
// {
//     variable ret long;
//     ret = 0; // default: FALSE
//     dprint("[is_no_op_interruption_karenz_active] { DLG[" || DLG clipped ||
//         "]" SUBDLG[" || SUBDLG clipped || "] DLG_DATA[" || DLG_DATA clipped || "]);";
// }

```



```
//
//  dprint("[is_no_op_interruption_karenz_active] } RET[" || ret using "<<<<<<<&" || "]");
//  return ret;
// }

//-----
// main function
//-----
long main()
{
    // only for testing
    return 0;
}
```

5.6 Server user exits - LLE - incentive wages

The user exits of the standard product group LLE are marketed as LLE-FBL products and documented in the corresponding manual. The following description only refers to general internal technical conditions.

5.6.1 Identifying the wage type of a time ticket

Name of user exit

Isl00000.hsc

Keywords

Wage type, LLE, piecework, time wage, time ticket

Function

LLC individual allocation: Identifying the wage type as part of time ticket calculation

In standard processing, the wage type of a time ticket is transferred from the original BDE personnel posting (B record).

The wage type of the BDE personnel posting (B record) is identified from the wage type defined in the operation. Use the following user exit to change the wage type that is identified via standard processing.

Note that the identified wage type specifies if piecework or time wage applies. Also refer to the LLE basic settings and explanations in the document LLE-BPL and the sections following.

Program(s) and source code files

Program	Version	Date	File(s)

Import parameter

The standard document LLE-FPL deals with the import and export parameters.

5.6.2 Identifying the time type of a time ticket

Name of user exit

Isz00000.hsc

Keywords

Wage type, LLE, piecework, time wage, time ticket

Function

LLE individual allocation: Identifying the time type (piecework, time wage, overhead costs, group premium ...) as part of time ticket calculation.

Program(s) and source code files

Program	Version	Date	File(s)

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

Parameter	Type	Content
n/a		

5.6.3 Recalculating time tickets

Name of user exit

Isv00000.hsc

Keywords

LLE, piecework, time wage, time ticket

Function

LLE individual allocation: you can use the user exit for a time ticket that is pre-calculated by default.

Program(s) and source code files

Program	Version	Date	File(s)

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

Import parameter

Parameter	Type	Content
n/a		

Export parameter:

Parameter	Type	Content
n/a		

5.6.4 Group allocation Step 1: Distribution of data in premium accounts

Name of user exit

lvp00000.hsc

Keywords

Group premium, group piecework, LLE time ticket, group result, premium account, premium accounts

Function

LLE group allocation: Allocation of entered data to premium accounts

Program(s) and source code files

Program	Version	Date	File(s)
---------	---------	------	---------

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

Import parameter

Parameter	Type	Content
n/a		

5.6.5 Group allocation Step 2: Calculation of group results

Name of user exit

lpb00000.hsc

Keywords

Group premium, group piecework, LLE time ticket, group result, premium account, premium accounts

Function

LLE group allocation: calculation of daily and monthly group results from premium accounts.

Program(s) and source code files

Program	Version	Date	File(s)
hyl7komp.out	6.5.1.1	2003-02-12	hyl7komp.c hyl7grpu.c hyl_sc_util.c
hyl7gret.out	6.5.1.1	02/2003	hyl7gret.c hyl7grpu.c hyl_sc_util.c

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

Import parameter

Parameter	Type	Content
LEISTGRP_LART_*	*	Master data of wage type, refer to section further ahead. (Available from November/2005) (Available as of November/2005)

5.6.6 Group allocation: Assigning group results to individual time tickets

Name of user exit

hyl_leistgrp2tls.hsc

Keywords

Group premium, group piecework, LLE time ticket, group result, premium account, premium accounts

Function

This user exit has so far only been used for customer-specific purposes. Use this user exit to assign group results to individual time tickets. It takes up a lot of performance. You should use the user exit only if it is unavoidable. Normally, you should assign group results to the individual person based on the person's amount of time devoted to this group result (personal group participation).

Program(s) and source code files

Program	Version	Date	File(s)

Import and export parameter

Import parameters:

Parameter	Type	Content
n/a		

Export parameter:

Parameter	Type	Content
n/a		

5.6.7 Time period results for persons and premium groups

Name of user exit

hyl_pnrperiod_calc.hsc Function final_calc()
 hyl_prgrperiod_calc.hsc Function final_calc()

Keywords

LLE time period and monthly results

Function

Calculation of time period and monthly results for persons and premium groups.

Program(s) and source code files

Program	Version	Date	File(s)
hyl_compute72.exe	8.1.1.75	SP6 11/2014	

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

-

5.6.8 LLE info function on PZE terminal

Name of user exit

hyl_info.hsc

Keywords

LLE info; information function on the terminal

Function

Use this user exit to add further rows with incentive wage data to the PZE information function of the PZE terminal.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.out			hyd_scmd.c hyl_sc_util.c

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

-

5.6.94.7 LLE interface – data collection

Name of user exit

lrck1000.hsc

Keywords

PZW 8.1 - Interface to payroll

Function

Data collection for LLE payroll system interface.

Program(s) and source code files

Program	Version	Date	File(s)
hyl7rck.out			hyl7rck.c

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

-

5.6.10 4.7 LLE interface – data output

Name of user exit

Lrck2000.hsc

Keywords

PZW 8.1 - Interface to payroll

Function

Output of the LLE payroll interface data collected with the previously mentioned user exit.

Program(s) and source code files

Program	Version	Date	File(s)
hyl7rck.out			hyl7rck.c

Import and export parameter

The standard document LLE-FPL deals with the import and export parameters. Only the import and export parameters that were not available from the beginning are listed here to document the corresponding software versions.

5.6.11 Active PZE/ADE comparison - RPA distribution and changing data

Name of user exit

hyadeabg.hsc

Keywords

The data of ADEPRO.UPDATE and ADEPRO.INSERT can be changed in the user exit. You can change here the RPA distribution.

Function before_adepero_update()

Use the user exit *before_adepero_update()* to change RPA distribution before saving the synchronized ADE times.

In case a log record must be changed due to a deviation between ADE-PZE, the ADE record is changed by default that all RPAs and personal RPAs are reset to zero and RPA 11 includes the compared duration (DAUER).

The value for PBMKA11 (RPA) is also set to the changed duration. If the option "Proportionate RPA posting in personnel postings" is set in the basic settings, PBMK11 is set to the proportional value. The value for PDAUER (duration) is always set to a proportional value.

Note on RPA/PBMK - distribution

The comparison can increase or decrease the values for DAUER/PDAUER (duration). Please bear this in mind when distributing on the RPA/PBMK. It is also possible to edit the values inconsistently in the correction mask for "Order-related postings". The value for DAUER/PDAUER can deviate from the totals of the single RPAs.

If the function `before_adepto_update()` is available in the user exit `hyadeabg.hsc`, this function is called. The original ADEPRO record (`ADEPRO_SELECT`) and the changes (`ADEPRO_UPDATE`) are transferred to the function. You can change the new ADEPRO record in the user exit. You can also set the flag `UPDATE_REQUIRED` to specify whether an update is made in any case (1) or not (0). If this flag is not set (-1), the program control determines whether or not an update is required. You can use the `SIGN_REQUIRED` flag in the user exit to control whether the ADEPRO.SIGN BAPI should be called:

`SIGN_REQUIRED=1` => Always call Sign
`SIGN_REQUIRED=0` => Do not call Sign

If `SIGN_REQUIRED` is not set (-1), the program control also calculates whether a Sign is necessary.

Function `before_adepto_insert()`

The comparison inserts completely new B records in the table `ADE_PROTOKOLL` if necessary, if gap filling is configured. These B records are created via `ADEPRO.INSERT`.

The user exit `before_adepto_insert()` makes it possible to modify the data before creating the B records.

Depending on the gap-filling configuration, either overhead cost operations or predecessor operations are used to fill gaps.

If the function `before_adepto_insert ()` is available in the user exit `hyadeabg.hsc`, this function is called. The new ADEPRO record to be created (`ADEPRO_UPDATE`) is transferred to the user exit. In this use case, the `ADEPRO_SELECT` variable is an identical copy of the `ADEPRO_UPDATE` variable.

You can change the new ADEPRO record to be created in the user exit.

The flag UPDATE_REQUIRED can also be set in the user exit. If the variable UPDATE_REQUIRED is set to the value 0, the BAPI ADEPRO.INSERT is not executed. If the variable is not set specifically, the program runs ADEPRO.INSERT.

Callback functions (hyadeabg.hsc):

You can use the built-in cache functions for better performance in the user exit. The cache is managed in the program hyadeabg in a map. The map is restarted after a person changes.

- **SetCacheValue**: set the value in the cache
- **GetCacheValue**: Get a value from the cache
- **DelCacheValue**: delete the value in the cache
- **ClearCache**: delete all values in the cache

SetCacheValue:

The callback function expects the parameters KEY and VALUE as dialog data. The value for KEY and VALUE have a maximum length of 50 characters.

```
retVal = CallBack( "SetCacheValue", "KEY=CACHE_FILLED|VALUE=1");  
retVal = CallBack( "SetCacheValue", "KEY=ANR|VALUE=" || anr clipped );
```

The callback returns the long value of 0, if the KEY and CALUE was correctly parsed in the dialog data. If one of the identifications is not existent, then 1661 is returned.

Note:

The cache should manage whether the cache is already initialized for this person or whether the necessary values are set for this person. This example uses the marker CACHED_FILLED.

GetCacheValue:

The callback function expect the parameter KEY.

```
cacheFilled = CallBack( "GetCacheValue", "KEY=CACHE_FILLED" );  
if (cacheFilled is null)  
{  
    //fill cache ...  
}  
  
mnr = CallBack( "GetCacheValue", "KEY=MNR" );  
if (mnr is not null)  
    ADEPRO_UPDATE = set_Bapi_Val(ADEPRO_UPDATE, "MNR", mnr);
```

If the KEY is not found in the cache, the callback call returns the value null.

DelCacheValue:

The callback function expect the parameter KEY.

```
retVal = CallBack( "DelCacheValue", "KEY=test2" );
```

The callback returns the long value of 0, if the KEY was correctly parsed in the dialog data. If this acronym is not available, the 1661 is returned.

ClearCache:

callback function does not expect a parameter.

```
retVal = CallBack( "ClearCache", "" );
```

The callback always returns the long value 0.

Program(s) and source code files

Program	Version	Date	File(s)
hyadeabg.exe/out	8.1.1.31	2019-05-17	hyabgbapi.cpp

Import parameter

Parameter	Type	Content

Export parameter

Parameter	Type	Content
ADEPRO_SELECT	char(32000)	Data of the original ADEPRO record as dialog string with acronyms of the BAPI ADEPRO.
ADEPRO_UPDATE	char(32000)	Data to be changed for ADEPRO.UPDATE/.INSERT (preset with the standard changes, see above)
UPDATE_REQUIRED	long	Controls whether an ADEPRO.UPDATE/.INSERT should take place -1: Check by standard logic* 1 : Execute ADEPRO.UPDATE/.INSERT 0 : Do not execute ADEPRO.UPDATE/.INSERT
SIGN_REQUIRED	long	Controls whether ADEPRO.SIGN is executed -1: Check by standard logic 1 : Execute ADEPRO.SIGN Do not execute 0 : ADEPRO.SIGN

* The standard logic checks if there are differences to the original record. The following values are checked: EGR:BMK*, EGR:PBMK*, EGR:DAUER and EGR:PDAUER. If changed are detected, an update is forced. There is no original record for *Insert*. This means that *Insert* is always forced.

5.6.12 Active ADE/PZE comparison - after daily personal results

Name of user exit

hyadeabgpnrdat.hsc

Keywords

User exit after completion of daily personal results.

Function after_comparision()

Use the function `after_comparision()` to carry out further activities after daily personal results have been completed.

Program(s) and source code files

Program	Version	Date	File(s)
hyadeabg.exe/out	8.1.1.15	2012-03-21	

Import parameter

Parameter	Type	Content
PNR	long	
DAT	date	
ADEPROCHANGED	char(1)	J/N

Export parameter

Parameter	Type	Content

Callback function:

BAPICALLEEXECUTE

Enables to execute BAPI dialogs.

5.6.13 Active ADE/PZE comparison – Where clause

Name of user exit

hyadeabgsql.hsc

Keywords

User exit to change the Where clause when importing ADE logs.

Function add_where_clause_ade_bookings()

Use the function add_where_clause_ade_bookings() to change the Where condition selecting the affected BDE personnel postings (B records). You can filter out the B records that:

- you do not want to compare
- should not have an effect on the comparison of other B records.

The table Clause includes the following tables:

ade_protokoll	ap
auftrags_bestand	ab

Program(s) and source code files

Program	Version	Date	File(s)
hyadeabg.exe/out	8.1.1.26	2014-12-01	

Import parameter

Parameter	Type	Content
MOD	long	1: Looks for the latest B record outside the evaluation period in the past. 2: Selects all B records in the evaluation period (standard mode) 3: Selects all U/E records in the evaluation period This mode is only executed if the special option "ADE/PZE comparison: comparison of U/E records with labor time" (ANP-000354) is enabled. 4: Specifies additional B records of other personnel who need to be integrated due to the selection of U/E records (mode 3). This mode is only executed if the special option "ADE/PZE comparison: comparison of U/E records with labor time" (ANP-000354) is enabled.
PNR	char(10)	Evaluated person
MNR	char(20)	Machine number: Parameter is only filled for MOD=3 4
ANR	char(40)	Order number: Parameter is only filled for MOD=3 4
PDATB	long	Start date: Evaluation period
PZEIB	long	Start time: Evaluation period
PDATE	long	End date: Evaluation period
PZEIE	long	End time: Evaluation period

Export parameter

Parameter	Type	Content
UX_WHERE	char(10000)	Where - extension

5.6.14 Labor time comparison (list)

Name of user exit

hyl_comp_hr_mf.hsc

Keywords

Labor time comparison, hy_pzs

Function

Use this user exit to change the output file of the labor time comparison. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hy_pzs.exe out	8.1.1.25	07.07.2016 → SP 12	

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.7 Server user exits - MLE

5.7.15.1 Extension of data transfer (MLE72IMP)

5.7.1.1 Modifying the data string (SDATA)

Name of user exit

mle_modifysapdata_in(_MESTYP).hsc

An attempt is always made initially to link the user exit specific to mestyp. If this one is not available, the global user exit will be linked.

Keywords

Changing the SDATA string before it is processed through MLE72IMP.

Function

- ❖ This user exit edits the data from the *hysap_inbound_data.sdata* field, before this data is actually converted in the context of MLE templates. Consequently, you can change the original data string for processing.
- ❖ The user exit has the following callback functions:
=> DO_BAPI : Allows an internal Bapi call. Use the parameter MOD=PROT to document the Bapi call and return code of the Bapi in the MLE72IMP log.

Program(s) and source code files

Program	Version	Date	File(s)
mle72imp			mle72imp.c

Import parameter

Parameter	Type	Content
-----------	------	---------

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS)
MESFCT	C3	Message functions (e.g. APP)
VARIANTE	C30	Variant (e.g. U:HY72PPS_001)
BAPI	C40	Bapi dialog (e.g. ANR.INSERT)
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01)
SAP_SDATA	C1000	Data string (SDATA)

Export parameter

Parameter	Type	Content
SAP_SDATA	C1000	Changed data string (SDATA)
VERARB	LONG	Processing code: Possible values 0 = carry on with default/standard processing (by default) 1000 = Exit hierarchy 1001 = Exit segment 1002 = Exit minor (sub segments) 1004 = Exit Bapi

5.7.1.2 Processing the data string (SDATA)

Name of user exit

mle_convsapdata_in(_MESTYP).hsc

Keywords

Converts the SAP data field SAPDATA in a dialog string

An attempt is always made initially to link the user exit specific to mestyp. If this one is not available, the global user exit will be linked.

Function

- ❖ This user exit processes the data from the hysap_inbound_data.sdata field instead of the MLE configuration. It creates a dialog string directly from the data string of a segment, which is executed by the mle72imp program.
- ❖ Unlike the "mle_verarbseg_in.hsc" user exit, this user exit is triggered AFTER calling the segment configuration. Thus, the MLE template requires a valid segment configuration!

- ❖ The user exit has the following callback functions:
=> DO_BAPI : Allows an internal Bapi call. Use the parameter MOD=PROT to document the Bapi call and return code of the Bapi in the MLE72IMP log.



Programs and source code files

Program	Version	Date	File(s)
mle72imp			mle72imp.c

Import parameter

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS)
MESFCT	C3	Message functions (e.g. APP)
VARIANTE	C30	Variant (e.g. U:HY72PPS_001)
BAPI	C40	Bapi dialog (e.g. ANR.INSERT)
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01)
SAP_SDATA	C1000	Data string (SDATA)

Export parameter

Parameter	Type	Content
DLG	C8000	Dialog data string Minimum versions: Refer to the table "Program(s) and source code files"
SAP_SDATA	C1000	Data string (SDATA) Minimum versions: Refer to the table "Program(s) and source code files"

5.7.1.3 Changing the dialog string

Name of user exit

mle_modifydlgstr_in(_MESTYP).hsc

An attempt is always made initially to link the user exit specific to mestyp. If this one is not available, the global user exit will be linked.

Keywords

Change the generated dialog string

Function

- ❖ This user exit edits the dialog string created by the MLE processing. It is therefore possible to subsequently change the created dialog string.
- ❖ The user exit has the following callback functions:
=> DO_BAPI : Allows an internal Bapi call. Use the parameter MOD=PROT to document the Bapi call and return code of the Bapi in the MLE72IMP log.
- ❖

Programs and source code files

Program	Version	Date	File(s)
mle72imp			mle72imp.c

Import parameter

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS) Minimum versions: Refer to the table "Program(s) and source code files"
MESFCT	C3	Message functions (e.g. APP) Minimum versions: Refer to the table "Program(s) and source code files"
VARIANTE	C30	Variant (e.g. U:HY72PPS_001) Minimum versions: Refer to the table "Program(s) and source code files"
BAPI	C40	Bapi dialog (e.g. ANR.INSERT) Minimum versions: Refer to the table "Program(s) and source code files"
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01) Minimum versions: Refer to the table "Program(s) and source code files"

Parameter	Type	Content
DLG	C8000	Generated dialog string Minimum versions: Refer to the table "Program(s) and source code files"
TID	C30	Transaction number Minimum versions:

Export parameter

Parameter	Type	Content
DLG	C8000	Revised dialog string Minimum versions: Refer to the table "Program(s) and source code files"
VERARB	LONG	Processing code: Possible values 0 = carry on with default/standard processing (by default) 1000 = Exit hierarchy 1001 = Exit segment 1002 = Exit minor (sub segments) 1004 = Exit Bapi

5.7.1.4 Converting a field

Name of user exit

mle_convfield_in(_MESTYP).hsc

An attempt is always made initially to link the user exit specific to mestyp. If this one is not available, the global user exit will be linked.

Keywords

Converts a specific field of the data string

Function

- ❖ The data string position of the field to be converted is transferred to this user exit. The user exit uses this information to identify the field value and/or to populate/calculate the field. The full value is expected as return, as to how it can be integrated in the dialog string (e.g. ANR.BEZ=XXX).

- ❖ The user exit has the following callback functions:
=> DO_BAPI : Allows an internal Bapi call. Use the parameter MOD=PROT to document the Bapi call and return code of the Bapi in the MLE72IMP log.



Programs and source code files

Program	Version	Date	File(s)
mle72imp	MW 3.0		mle72imp.c

Import parameter

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS) Minimum versions: Refer to the table "Program(s) and source code files"
MESFCT	C3	Message functions (e.g. APP) Minimum versions: Refer to the table "Program(s) and source code files"
VARIANTE	C30	Variant (e.g. U:HY72PPS_001) Minimum versions: Refer to the table "Program(s) and source code files"
BAPI	C40	Bapi dialog (e.g. ANR.INSERT) Minimum versions: Refer to the table "Program(s) and source code files"
FELD	C40	Field acronym (e.g. ANR.BEZ) Minimum versions: Refer to the table "Program(s) and source code files"
DLG_DATA	C8000	Dialog string generated up to now Minimum versions: Refer to the table "Program(s) and source code files"
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01) Minimum versions: Refer to the table "Program(s) and source code files"
SAP_SDATA	C1000	Data string (SDATA) Minimum versions:

Parameter	Type	Content
		Refer to the table "Program(s) and source code files"
POS_VON	N	Position From Minimum versions: Refer to the table "Program(s) and source code files"
POS_BIS	N	Position To Minimum versions: Refer to the table "Program(s) and source code files"

Export parameter

Parameter	Type	Content
ZIEL	C1000	The value to be included in the dialog string Minimum versions: Refer to the table "Program(s) and source code files"
VERARB	LONG	Processing code: Possible values 0 = carry on with default/standard processing (by default) 1000 = Exit hierarchy 1001 = Exit segment 1002 = Exit minor (sub segments) 1003 = Exit Acronym 1004 = Exit Bapi

5.7.1.5 Processing a segment

Name of user exit

mle_verarbseg_in_(MESTYP).hsc

An attempt is always made initially to link the user exit specific to mestyp. If this one is not available, the global user exit will be linked.

Keywords

The user exit creates up to three executable DLG strings from the data string (SDATA) of a segment.

Function

- ❖ This user exit processes the data from the `hysap_inbound_data.sdata` field instead of the MLE configuration. The user exit directly creates executable dialog strings from the data string. The program `mle72imp` gets and executes these dialog strings.
- ❖ Unlike the “`mle_convsapdat_in.hsc`” user exit, this user exit is triggered BEFORE calling the segment configuration. Therefore, you can also process MESTYPES that have not stored any valid segment configuration in the MLE template.
- ❖ The user exit has the following callback functions:
=> `DO_BAPI` : Allows an internal Bapi call. Use the parameter `MOD=PROT` to document the Bapi call and return code of the Bapi in the MLE72IMP log.
- ❖

Program(s) and source code files

Program	Version	Date	File(s)
mle72imp			mle72imp.c

Import parameter

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS)
MESFCT	C3	Message functions (e.g. APP)
VARIANTE	C30	Variant (e.g. U:HY72PPS_001)
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01)
SAP_SDATA	C1000	Data string (SDATA)

Export parameter

Parameter	Type	Content
DLG1	C8000	Dialog data string 1
DLG2	C8000	Dialog data string 2
DLG3	C8000	Dialog data string

5.7.2 Extension of the upload/confirmation (MYERPRCK/MPLRFRCK)

5.7.2.1 Modifying the upload/confirmation (MESTYP)

Name of user exit

mle_modifyrckdata-<MESTYP>_out.hsc

Keywords

Execute special upload actions for the transferred message type (Mestyp). For example:

mle_modifyrckdata_E2BPTIMETICKET_out.hsc

Function

- ❖ This user exit is linked dynamically according to the transferred message type (MESTYP) and therefore the user exit executes only actions for this message type. Example:
mle_modifyrckdata_E2BPTIMETICKET_out.hsc
- ❖ The user exit can execute SQL operations and has the following callback functions:
 - => DO_RCK : Permits the autonomous generation of any number of upload records.
 - => CreateMasterData : Create master segment (refer to C function having the same name)
 - => CreateOutboundDataEx : Create child segment (refer to C function having the same name)
 - => CommitMasterData : Authorize master segment (refer to C function having the same name)
 - => double_to_sapexp : Convert double values into SAP exponential format
 - => time_to_hhmmss : Convert seconds into a string of the format "hhmmss"
 - => ShowMessage : Generate log entry (refer to C function having the same name)
- ❖ Use the return parameter DS_MELDEN (TRUE(1), FALSE(0) or SPAETER (later) (2)) to specify if the data string should be confirmed/uploaded via MYERPRCK!
- ❖ Use the transfer parameter PROG_PARAMS to transfer optional program parameters in the DD format to the user exit. These parameters are forwarded 1:1 from the "/UE_PARAMS=" program parameter of the myerprck.exe/out program.
For example:
→ myerprck.scr -z /MESTYP=TEST_MESTYP /UE_PARAMS="MODE=ALL|TEST=TRUE"
- ❖ Additional user exit mle_myerprck_action_after: The user exit is recalled after closing the ADE log cursor and an attempt is made to go to the mle_myerprck_action_after function. You can perform concluding actions in this function, e.g., execute a CommitMasterData.
- ❖ Additional user exit mle_myerprck_action_before: This user exit is called before opening the ADE log cursor and an attempt is made to go to the mle_myerprck_action_before function. You can take preparatory measures in this function.
It goes without saying that context-related information is not available here (SAP_SDATA, PARAMS).

Program(s) and source code files

Program	Version	Date	File(s)
myerprck			myerprck.c
mplrfrck			mplrfrck.c

Import parameter

Parameter	Type	Content
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01)
SAP_SDATA	C1000	Data string (SDATA)
VERWEIS	N	Reference of the triggering log record
PARAMS	C8000	Transfer parameters in the DD format (from the internal data structure)
PROG_PARAMS	C100	Program parameters from "/UE_PARAMS="

Export parameter

Parameter	Type	Content
SAP_SDATA	C1000	Changed data string (SDATA)
SAP_SDATA_LEN	N	Length of the data string (for file port length restriction)
DS_MELDEN	N	1 (TRUE) = The main program uploads the data record. 0 (FALSE) = The main program does NOT upload the data record. 2 (SPAETER)/later = The current interface run does not upload the data record. But the data record is processed once more in a subsequent interface run.
MERK_VARIABLE	C1000	Memory variable for backing up additional information between the interface runs.

5.7.2.2 Modifying the data string (SDATA)

Name of user exit

mle_modifysapdata_out.hsc

Keywords

Changing the SDATA string after it has been processed by MYERPRCK.

Function

- ❖ This user exit revises the data string (SDATA) created by MYERPRCK.
- ❖ The user exit can execute SQL operations and has the following callback functions:
 - => DO_RCK : Permits the autonomous generation of any number of upload records.
 - => CreateMasterData : Create master segment (refer to C function having the same name)
 - => CreateOutboundDataEx : Create child segment (refer to C function having the same name)
 - => CommitMasterData : Authorize master segment (refer to C function having the same name)
 - => double_to_sapexp : Convert double values into SAP exponential format
 - => time_to_hhmmss : Convert seconds
 - => ShowMessage : Generate log entry (refer to C function having the same name)
- ❖ Use the return parameter DS_MELDEN (TRUE(1), FALSE(0) or SPAETER (later) (2)) to specify if the data string should be confirmed/uploaded via MYERPRCK!
- ❖ Additional user exit mle_myerpck_action_after: The user exit is recalled after closing the ADE log cursor and an attempt is made to go to the mle_myerpck_action_after function. You can perform concluding actions in this function, e.g., execute a CommitMasterData.
- ❖ Additional user exit mle_myerpck_action_before: This user exit is called before opening the ADE log cursor and an attempt is made to go to the mle_myerpck_action_before function. You can take preparatory measures in this function.
It goes without saying that context-related information is not available here (SAP_SDATA, PARAMS).
- ❖

Program(s) and source code files

Program	Version	Date	File(s)
myerpck			myerpck.c
mplrfrck			mplrfrck.c

Import parameter

Parameter	Type	Content
SEGNAM	C30	Segment name (e.g. HY72_AU_HD_01)
VERWEIS	N	Reference of the triggering log record
PARAMS	C8000	• Upload via ADE_PROTOKOLL ◦ Key fields for the log record in the dialog data format • Upload with HYD_LOGGING (program parameter /LOGGING)

Parameter	Type	Content
		○ Not in use • Upload of changed orders ○ Key fields of the changed record
PROG_PARAMS	C100	Program parameters from "/UE_PARAMS=" Use the transfer parameter PROG_PARAMS to transfer optional program parameters in the DD format to the user exit. These parameters are forwarded 1:1 from the "/UE_PARAMS=" program parameter of the myerprck.exe/out program. For example myerprck.scr -z /MESTYP=TEST_MESTYP /UE_PARAMS="MODE=ALL TEST=TRUE"

Export parameter

Parameter	Type	Content
SAP_SDATA	C1000	Changed data string (SDATA)
SAP_SDATA_LEN	N	Length of the data string (for file port length restriction)
DS_MELDEN	N	1 (TRUE) = The main program uploads the data record. 0 (FALSE) = The main program does NOT upload the data record. 2 (SPAETER)/later = The current interface run does not upload the data record. But the data record is processed once more in a subsequent interface run.

5.7.2.3 Executing the confirmation/upload (MESTYP)

Name of user exit

mle_rckmestyp-<MESTYP>_out.hsc

Keywords

Execute upload actions for the transferred message type (Mestyp). For example:

mle_rckmestyp_TEST_MESTYP_out.hsc

Function

- ❖ This user exit is linked dynamically according to the transferred message type (MESTYP) and therefore the user exit executes only actions for this message type. Example :
mle_rckmestyp_TEST_MESTYP_out.hsc. In contrast to other upload user exits, this user exit carries out the entire upload process (that means the data selection, conversion and upload). The actual upload program myerprck.exe is only used as framework for the upload !
- ❖ The user exit can execute SQL operations and system commands. The user exit has the following callback functions:
--> DO_RCK : Permits the autonomous creation of any number of upload records.=>
CreateMasterData : Create master segment (refer to C function having the same name)
=> CreateOutboundDataEx : Create child segment (refer to C function having the same name)
=> CommitMasterData : Authorize master segment (refer to C function having the same name)
=> double_to_sapexp : Convert double values into SAP exponential format
=> time_to_hhmmss : Convert seconds
=> ShowMessage : Generate log entry (refer to C function having the same name)
- ❖ The timeout for the user exit has been deactivated. So the user exit has an arbitrary runtime.
- ❖ Use the transfer parameter PROG_PARAMS to transfer optional program parameters in the DD format to the user exit. These parameters are forwarded 1:1 from the "/UE_PARAMS=" program parameter of the myerprck.exe/out program.
For example:
→ myerprck.scr -z /MESTYP=TEST_MESTYP /UE_PARAMS="MODE=ALL|TEST=TRUE"
- ❖ Use the transfer parameters ANZAHL_DS, ANZAHL_ERROR, ANZAHL_UNKNOWN to provide the calling program myerprck with information that is directly integrated in the generated log record!

Please note:

As the confirmation/upload can access any data basis, you should use the newly created transfer parameter /NOLOCK in the corresponding confirmation/upload script (myerprck.scr). Usually, the confirmation/upload program checks whether it should be started or whether an ADE lock entry exists in hyd_lock. This makes sense only if the confirmation/upload also accesses ADE data (ade_protokoll), otherwise there is no need to check/set this lock!

Program(s) and source code files

Program	Version	Date	File(s)
myerprck			myerprck.c

Import parameter

Parameter	Type	Content
PROG_PARAMS	C100	Program parameters from "/UE_PARAMS="

Export parameter

Parameter	Type	Content
ANZAHL_DS	N	Number of processed data records
ANZAHL_ERROR	N	Number of errors
ANZAHL_UNKNOWN	N	Number of warnings

5.7.2.4 Sort sequence and SELECT statement of the upload program myerprck

Name of user exit

mle_myerprck_modify_verwtable.hsc

Keywords

Select statement and/or sort sequence of the uploaded data records.

Function

- ❖ Use this user exit to override the default sorting of the upload program.
- ❖ Likewise, you can add further columns, which can also be used in sorting, to the SELECT statement.
- ❖ The timeout for the user exit has been deactivated. So the user exit has an arbitrary runtime.

Program(s) and source code files

Program	Version	Date	File(s)
myerprck			myerprck.c erprueck.c

Import parameter

Parameter	Type	Content
tmp_table	C1000	Name of the temporary table temp_erp_adeprot
mestyp	C30	Message type of the call

Export parameter

Parameter	Type	Content
order_by	C255	order by
sql_statement	C1000	SQL command that is executed once per program run
add_select_val	C1000	You can define additional columns for the table ade_protokoll. You can use these columns in the order_by.

5.7.3 Extension of CAQ confirmation/upload (CAQRCK)

5.7.3.1 Modifying the upload/confirmation (MESTYP)

Name of user exit

mle_modify_caqrck_data_<MESTYP>_out.hsc

Keywords

Execute special upload actions for the transferred message type (MESTYP) e.g.:

mle_modify_caqrck_data_ZQMPRU_out.hsc

Function

- ❖ This user exit is linked dynamically according to the transferred message type (MESTYP) and therefore the user exit executes only actions for this message type. Example :
mle_modify_caqrck_data_ZQMPRU_out.hsc
- ❖ The user exit can execute SQL operations and has a callback function which permits the user exit to create any number of upload records independently.
- ❖ Use the return parameter DS_MELDEN (TRUE/FALSE) to specify if the data string should be confirmed/uploaded via CAQRCK!

Program(s) and source code files

Program	Version	Date	File(s)
caqrck			caqrck.c

Import parameter

Parameter	Type	Content
SEGNAM	C30	Segment name
SAP_SDATA	C1000	Data string (SDATA)
VERWEIS	N	Reference of the triggering log record
PARAMS	C8000	Transfer parameters in the DD format (from the internal data structure)

Export parameter

Parameter	Type	Content
SAP_SDATA	C1000	Changed data string (SDATA)
DS_MELDEN	N	TRUE / FALSE

5.7.3.2 Modifying the data string (SDATA)

Name of user exit

mle_modify_caqrck_data.hsc

Keywords

This user exit changes the data string (SDATA) after it has been processed by CAQRCK.

Function

- ❖ Use this user exit to change the data string (SDATA) created by CAQRCK before it is written in the interface.
- ❖ The user exit can execute SQL operations and has a callback function which permits the user exit to create any number of upload records independently.
- ❖ Use the return parameter DS_MELDEN (TRUE/FALSE) to specify if the data string should be confirmed/uploaded via CAQRCK.

Program(s) and source code files

Program	Version	Date	File(s)
caqrck			caqrck.c

Import parameter

Parameter	Type	Content
SEGNAM	C30	Segment name

Parameter	Type	Content
SAP_SDAT	C1000	Data string (SDAT)
VERWEIS	N	Reference of the triggering log record
PARAMS	C8000	Transfer parameters in the DD format (from the internal data structure)

Export parameter

Parameter	Type	Content
SAP_SDAT	C1000	Changed data string (SDAT)
DS_MELDEN	N	TRUE / FALSE

5.7.4 Extension of the file port (hyalesrv)

5.7.4.1 Modifying data from input file

Name of user exit

mle_modifyfiledata_in.hsc

Keywords

Revises the input data of an interface file (inbound) before inserting this data in inbound tables.

Function

- ❖ The user exit can execute SQL operations.
- ❖ Use the return parameter DS_MELDEN (TRUE(1) or FALSE(0) to specify if the current line of the input file is written to the inbound tables!
- ❖ Use the transfer parameter EDIDD40 to revise the complete structure of the input file. This structure includes controlling information and user data (payload) (SDAT).

Program(s) and source code files

Program	Version	Date	File(s)
hyalesrv	6.5.1.27	2004-11-22	hyalesrv.c

Import parameter

Parameter	Type	Content
MESTYP	C30	Message type (e.g. HY72PPS). This corresponds to the file name without extension.

Export parameter

Parameter	Type	Content
EDIDD40	C1064	The modified line from the input file.
DS_MELDEN	N	1 (TRUE) = Current line is incorporated 0 (FALSE) = Current line is NOT incorporated

5.7.4.2 Modifying data of the output file

Name of user exit

mle_modifyfiledata_out.hsc

Function

- ❖ The user exit can execute SQL operations.

Program(s) and source code files

Program	Version	Date	File(s)
hysapupl			

Import parameter

Parameter	Type	Content

Export parameter

Parameter	Type	Content
-----------	------	---------

Parameter	Type	Content
DD40_DATA	C1064	Modified line
SIZE	N	Buffer length

Use this user exit to manipulate the current data row which is to be written to the output file.

5.7.4.3 Modifying the output file

Name of user exit

mle_modifyfiledata_out_complete.hsc

Function

- ❖ The user exit can execute SQL operations.
- ❖ The user exit is permitted to carry out system calls.

Program(s) and source code files

Program	Version	Date	File(s)
hysapupl	8.1.1.99	2016-03-23	

Import parameter

Parameter	Type	Content
PID	C50	Process ID and program start time
TID	C30	Transaction ID of the data record
IDX	Long	Database reference of the data record in the outbound table.
SEGNAM	C30	Segment name specified as the parameter in hysapupl /SEGNAM=<SEGNAM>
FILENAME	C300	The complete interface path of the work file (including file name)

Export parameter

Parameter	Type	Content

When you execute the user exit, all interface operations have been completed and the file is about to be moved from the working directory into the interface directory. Use the import parameter FILENAME to change the file before it is copied.

5.7.4.4 Modifying the name of the output file

Name of user exit

mle_modifyfilename_out.hsc

Function

- ❖ The user exit can execute SQL operations.
- ❖ The user exit is permitted to carry out system calls.

Program(s) and source code files

Program	Version	Date	File(s)
hysapupl	8.1.1.99	2016-03-23	

Import parameter

Parameter	Type	Content
PID	C50	Process ID and program start time
SEGNAM	C30	Segment name specified as the parameter in hysapupl /SEGNAM=<SEGNAM>
FILEPATH	C300	Directory path of the interface directory
FILEPATH_WORK	C300	Directory path of the working interface directory
TID	C30	Transaction ID of the data record

Export parameter

Parameter	Type	Content
FILENAME	C300	File name for the interface directory (without path)

Use the user exit to specify the file name in the interface directory. This overwrites all parameter effects and functions like /DATE_FILE (integration of date and time in the file name). The user exit's value applies.

5.7.5 Extension of the upload client (hysapupl.exe/out)

5.7.5.1 Modifying data transferring alerts to SAP (internal use only)

Name of user exit

hysap_modify_alert_call.hsc

Keywords

Use this user exit to change the data of alerts sent to SAP and to change the function module name dynamically (by default = SALERT_CREATE).

Consequently, the user exit is called for the header record of an escalation (escalation ID) as well as for every single data record of the escalation (escalation data). The parameter RECORD_TYPE determines whether it is a header record or a sub-record.

Function

- ❖ Use the transfer parameter RFC_CALL to transfer the name of the function module.
- ❖ The parameters IP_CAT, IP_ALIAS and IP_APPLICATION_GUID directly change the value of the import parameters having the same names pertaining to the function module "SALERT_CREATE". You can only change these parameters if it is a header record.
- ❖ The parameters ELEMENT, VALUE, TYPE and ELEMLength directly change the values of the fields having the same names in the SAP table of the function module "SALERT_CREATE". These values only have an effect if it is a sub-record.

Program(s) and source code files

Program	Version	Date	File(s)
hysapupl			hysapupl.c

Import parameter

Parameter	Type	Content
VERWEIS	N	Reference of the appropriate data record in hysap_out_data
RECORD_TYPE	C1	Record type : H=header record / C=sub-record

Export parameter

Parameter	Type	Content
RFC_CALL	C31	Name of the function module to be called (by default = SALERT_CREATE) Note : You can only change this value if it is a header record (RECORD_TYPE = H).
IP_CAT	C30	Import parameter 1 of the function module "SALERT_CREATE". By default, this parameter includes the escalation ID (e.g. MST-MST_MALFUNCTION_CONTINUE). Note : You can only change this value if it is a header record (RECORD_TYPE = H).
IP_ALIAS	C80	Import parameter 2 of the function module "SALERT_CREATE". Note : You can only change this value if it is a header record (RECORD_TYPE = H).
IP_APPLICATION_GUID	C32	Import parameter 3 of the function module "SALERT_CREATE". Note : You can only change this value if it is a header record (RECORD_TYPE = H).
ELEMENT	C32	Name of the element. Corresponds to the acronym (e.g. MST_MNR). Note: This field corresponds to the field of the same name in the SAP table pertaining to the function module. This field is only populated with values and processed if it is a sub-record (RECORD_TYPE = C).
VALUE	C255	Element value Note: This field corresponds to the field of the same name in the SAP table pertaining to the function module. This field is only populated with values and processed if it is a sub-record (RECORD_TYPE = C).

Parameter	Type	Content
TYPE	C1	Data type of the element. Note: This field corresponds to the field of the same name in the SAP table pertaining to the function module. This field is only populated with values and processed if it is a sub-record (RECORD_TYPE = C).
ELEMLENGTH	C3	Length of the element. Note: This field corresponds to the field of the same name in the SAP table pertaining to the function module. This field is only populated with values and processed if it is a sub-record (RECORD_TYPE = C).

5.7.6 Extension of the QM upload client (hysapqmc.exe/out)

Name of user exit

hysap_qmidi_qirf_send_requirements_get_dat2.hsc

Keywords

Use this user exit to add customer-specific data to the structure I_QAILS that is used to request QM-IDI data. Use this structure to transfer selection options to SAP. SAP uses this data to check whether QM-IDI data must be transferred or not.

The main purpose of this user exit is to support integration of QM-IDI in other interfaces (e.g. HY72PPS or PP-PDC). Up to now, the program has firmly specified how to evaluate the data provided by these interfaces and how to transfer this data into the structure I_QAILS (example: If the source message type is "PPCC2RECORDER" when calling hysapqmc.exe/out, the order number derives from the segment E2BP_PP_PDC_OPERA2000). This user exit, however, provides more flexibility. You can now use any source message type with any segment structures.

For this purpose, the user exit is provided with the SDATA string. Then you can copy any positions in the user exit and transfer these to the structure I_QAILS.

The user exit offers greater flexibility for QM-IDI when using source message types. But you can still also implement the user exit if you only use the CAQ product group (stand-alone) and the QM-IDI interface.

Function

- ❖ The user exit can execute SQL operations.

- ❖ Use the return parameter DS_MELDEN (TRUE(1) or FALSE(0) to specify if the current line of the source IDoc included in the table hysap_inbound_data should be used to call the function module.
If you do not set DS_MELDEN = TRUE for a data record of the source IDoc:
 - no function module will be called
 - no new transaction (no new IDoc) will be generated.

❖

Program(s) and source code files

Program	Version	Date	File(s)
hysapqmc.exe out			

Import parameter

Parameter	Type	Content
MESTYP_IN	CHAR30	Source message type (value of program parameter /MESTYP) for which the program was called e.g. PPCC2RECORDER or ZPPORDER. The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.
MESTYP_OUT	CHAR30	Target message type (value of program parameter /MESTYP_OUT) you used to call the program.
TID_IN	CHAR30	Transaction number (value of the program parameter /TID). You called the program for processing this transaction number. This field always remains empty if you use the stand-alone CAQ option. The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.
MESFCT_IN	CHAR3	Message function (value of the program parameter /MESFCT) you used to call the program. The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.
VERWEIS	N	Reference of the appropriate data record in hysap_inbound_data. The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.
VARIANTE	CHAR30	Variant (value of the program parameter /VARIANTE) you used to call the program.
SEGNAM	CHAR30	Segment name that is currently being processed. The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.

Parameter	Type	Content
SDATA	CHAR2000	Data of the field sap_sdata from hysap_inbound_data The parameter only includes a value if a source IDoc exists, i.e. if you do NOT use the CAQ stand-alone option.

Export parameter

Parameter	Type	Content
SATZART	CHAR3	Value of the structure I_QAILS
LOSNR_VON	NUMC12	Value of the structure I_QAILS
LOSNR_BIS	NUMC12	Value of the structure I_QAILS
PLNFL	CHAR6	Value of the structure I_QAILS
VORNR_VON	CHAR4	Value of the structure I_QAILS
VORNR_BIS	CHAR4	Value of the structure I_QAILS
VORGWERK	CHAR4	Value of the structure I_QAILS
SUBSYS	CHAR6	Value of the structure I_QAILS
PRPLATZ	CHAR8	Value of the structure I_QAILS
PRPLATZWRK	CHAR4	Value of the structure I_QAILS
MATNR	CHAR18	Value of the structure I_QAILS
DATUM_VON	DATS (YYYYM MDD)	Value of the structure I_QAILS
DATUM_BIS	DATS (YYYYM MDD)	Value of the structure I_QAILS
PRUEFSTAT	CHAR1	Value of the structure I_QAILS
ART	CHAR8	Value of the structure I_QAILS
HERKUNFT	CHAR2	Value of the structure I_QAILS
CHARG	CHAR10	Value of the structure I_QAILS
AUFNR_VON	CHAR12	Value of the structure I_QAILS
AUFNR_BIS	CHAR12	Value of the structure I_QAILS
LIFNR	CHAR10	Value of the structure I_QAILS
KUNNR	CHAR10	Value of the structure I_QAILS
MBLNR	CHAR10	Value of the structure I_QAILS
MAXLOSANZ	NUMC4	Value of the structure I_QAILS

5.7.7 HR-PDC uploads and downloads

5.7.7.1 Downloading mini HR master DNPERSO (sap45ein)

Name of user exit

hr_pdc_dnperso.hsc

Keywords

HR-PDC download, HR master data, DNPERSO

Function

Use this user exit to modify data when downloading the HR master from SAP (DNPERSO). Use this user exit, for example, to

- prepopulate fields with values when HR masters are created for the first time
- assign customer field values to nearly any HR master fields.

The function main() of the user exit is not used. Downloading the mini HR master from SAP affects several data objects in the system (HR master, ZKS badges, PZE access authorizations, ZKS profile assignments). Therefore, a separate function is used for every data object. These functions are described in the sections below.

The user exit supports SQL statements and system calls.

Program(s) and source code files

Please refer to the sections that follow, as the individual functions are not implemented together!

Import parameter

Parameter	Type	Content
SDATA	C1000	Data content of the SAP interface with the record structure DNPERSO.
CUST1	C20	Customer-specific field 1 (also included in SDATA)
CUST2	C40	Customer-specific field 2 (also included in SDATA)
MODUS	C10	INSERT or UPDATE: Use the mode INSERT if a person is created for the first time. Use the mode UPDATE if existing HR master versions are updated or additional versions are inserted

Parameter	Type	Content
		on the basis of existing HR master versions.
DLG_DATA	C30000	Dialog data of the dialog *.SAPDATA of the corresponding object (including additional options of the interface from ALE customizing)
DATA_OLD	C4000	Dialog string with acronyms and data already available in the database.

Export parameter

Parameter	Type	Content
DATA_NEW	C4000	Dialog string with acronyms and new data specified by SAP interface data.

5.7.7.2 Modifying HR master data

Program(s) and source code files

Program	Version	Date	File(s)
sap45ein.out			sap45ein.c
lib\b_pnr.dll			b_pnr.c

Function

The function *modify_data_pnr()* is called in the user exit.

The HR master data is transferred to the user exit as dialog data for the *PNR* object. Use the user exit to change the following data fields. The IDs are identical to those in the default HR master interface and, as a result, they are described in detail in the EIS-LUG document.

PNR.PLAUS:	PNR.PARAM:1H	PNR.STRASSE
SMENGE	PNR.PARAM:1I	PNR.PSTDSATZ
PNR.PLAUS:PNRANAG	PNR.PARAM:1J	PNR.SSTUFE:n (1 to 19)
PNR.ABT	PNR.RSTUFE	PNR.TAETIGKEIT
PNR.AUSTRITT	PNR.DLSTUFE	PNR.TZGRAD
PNR.ASTUFE:n (n=1 to 19)	PNR.ULSTUFE	PNR.TEL:FIR
PNR.SPERR:BDE	PNR.FGSTUFE	PNR.TEL:PRIVAT
PNR.FIR	PNR.RESEINAUS	PNR.DATB:TMP
PNR.BER	PNR.WARTSTUFE	PNR.DATE:TMP
PNR.DGBERECHT	PNR.MASSSTUFE	PNR.TITEL
PNR.EINTRITT	PNR.ANRGK	PNR.OPG
PNR.ETGAWDAT	PNR.OPT:PABSKE	PNR.UPG

PNR.FAMSTAND	PNR.LPKZ	PNR.URLANSR:NORM
PNR.GEBDAT	PNR.BPOS	PNR.URLANSR:DAT
PNR.GESCHLECHT	PNR.LART	PNR.ENTLTMOD:MEHRARB
PNR.BESCHVERH	PNR.LGRP	PNR.ORT:WOHN
PNR.GLZJMOD	PNR.OPT:PZEADEABGL	PNR.URLANSR:ZUSATZ
PNR.INFOTXT:n (1 to 15)	PNR.ANTFAKTLBON	PNR.ANMELDMAX
PNR.INFOTXT:n (17 to 20)	PNR.PNAME	PNR.EMAIL:FIR
PNR.INFOWERT:n (1 to 5)	PNR.PVORNAME	PNR.EMAIL:PRIVAT
PNR.INFODAT:n (1 to 5)	PNR.PGRP	PNR.MOBILTEL:FIR
PNR.KST	PNR.PGRPNEU	PNR.MOBILTEL:PRIVAT
PNR.VAB	PNR.PIN	PNR.PAGER:FIR
PNR.OPT:AVGAZVERB	PNR.PKREIS	PNR.KONS
PNR.AVGAZ	PNR.PLZ	PNR.PNR:VGS
PNR.ENTLJMOD	PNR.PRGRP	PNR.OPT:NSTMP
PNR.MEHRMNR	PNR.PRKZ	PNR.KST:TMP
PNR.MSTUFE:n (1 to 19)	PNR.SCHZARTJMOD	PNR.FIR:TMP
PNR.NATION	PNR.URLANSR:SONDER	PNR.DATB:TMP
PNR.OPT:AGWAUTO	PNR.SPERR:PZE	PNR.DATE:TMP
PNR.PARAM:1E	PNR.BDEJMOD	
PNR.PARAM:1F	PNR.SMNR	
PNR.PARAM:1G		

The other fields described for the HR master interface cannot and must not be changed. See below:

Field	reason, why not changeable
PNR.PNR	Key Personnel Number
PNR.KNR	Key Badge Number
PNR.DATB	Key Start of Validity
PNR.DATE	Automatic End of Validity
PNR.ASTUFE	Available as PNR.ASTUFE:n
PNR.MSTUFE	Available as PNR.MSTUFE:n
PNR.SSTUFE	Available as PNR.SSTUFE:n
PNR.INFOTXT:16	SAP Source System: assigns the person to the SAP system
PNR.PARAM:2	Not available
PNR.OPT:SAPVERARB	Internal SAP processing
PNR.OPT:SAPTEVTGR	Internal SAP processing
PNR.BEARB	Automatic
PNR.BEARBDAT	Automatic
PNR.BEARBZEI	Automatic
...	

5.7.7.3 Uploading time events UPTEVEN (sap45rck)

Name of user exit

hyt_hr_pdc_conf11.hsc

Keywords

HR-PDC upload, time events, clockings, UPTEVEN, CONF11

Function

The function `modify_out_data()` is called for each data segment. The function `main()` of the user exit is not used.

Use this user exit to modify data when uploading time events (UPTEVEN). You can change and add field contents.

You may also change the segment name and the entire structure of the data record. So you can use this interface to upload time events also to other time management systems apart from SAP-HR or SAP-HCM.

The user exit supports SQL statements and system calls.

Refer to the HR-PDC documents and the below example for further information on the import/export parameters and interface contents.

Program(s) and source code files

Program(s) and source code files

Program	Version	Date	File(s)
sap45rck.exe out	8.1.1.33	June 2017, Service Pack 12	hyt_sc_util.c, sap45rck.c

Import parameter

Parameter	Type	Content
SYS_PARAMS	C500	Command line parameters of the interface program. Use these parameters to transfer additional information to the user exit, e.g. for Scheduler calls.
PNR_PNR	N	Personnel number
PNR_KNR	C10	Badge number
PNR_FIR	C4	Company from the HR master data
PNR_KST	C10	Cost center from the HR master.
STMP_EVENT	C3	Type of time event.
STMP_TS	datetime	Time stamp of time event.
STMP_TNR	N	Number of the terminal where the time event was recorded.
STMP_FGR	C4	Absence reason
STMP_KST	C10	Cost center entered in the clocking record.

Parameter	Type	Content
STMP_BEM	C40	Clocking comment.
STMP_TYPE	C1	Internal type of clocking pair (K/k/g/D/F)
STMP_INTERNAL_ID	N	Internal data record number

Export parameter

Parameter	Type	Content
SOURCE_SYS	C10	SAP source system that is supposed to receive the time event.
SAP_SEGNAM	C30	Segment
SDATA	C1000	Data record
SDATA_LEN	N	Enter the length of the output string here, if you want to output data in a file port. As otherwise, the information about the number of trailing blanks gets lost during MLE transmission.
OUTPUT_DS	N	1: By default, the data record is written into open outbound transactions. This is the default. 0: The data record is not written into open outbound transactions, but identified as being uploaded/confirmed.

Example

```

hydra basic;
/* -----
Script : hyt_hr_pdc_conf11.hsc
Descr. : Modifying CONF11 Interface HR PDC

$Revision: 1.10000 $
$Date: 2017/05/11 00:00:00 $

$Log$

----- */

//-----
import SYS_PARAMS      char(500); // Command line options of program
import PNR_FIR         char(4);   // HR master data: Company
import PNR_KST         char(10);  // HR master data: Cost center
import PNR_PNR         long;      // HR master data: Personnel number
import PNR_KNR         char(10);  // HR master data: Badge ID
import STMP_EVENT      char(3);   // Clocking: Time event type (see comments below)
import STMP_TS         datetime;  // Clocking: Time stamp
import STMP_TNR        long;      // Clocking: ID of the shop floor terminal
import STMP_FGR        char(4);   // Clocking: Absence reason
import STMP_KST        char(10);  // Clocking: Cost center of clocking
import STMP_BEM        char(40);  // Clocking: Commentary
import STMP_TYPE       char(1);   // Clocking: Type of clocking
// (K: Attendance (Clock In + Clock Out),
// k: Only Clock-in,
// g: Only Clock-out,
// F: Absence,
// D: Business trip,
// d: Start of Business trip)
import STMP_INTERNAL_ID long;     // Clocking: Internal id (stempelsaetze.verweis)
export SOURCE_SYS      char(10); // Output source system
export SAP_SEGNAM      char(30);  // Output segment name
export SDATA           char(1000); // Output data string
export SDATA_LEN       long;      // Length of output data string (hydra
// script does not preserve trailing blanks)
export OUTPUT_DS       long;      // Output data to interface:
// 1=Yes (default), 0:No

//-----

```

```

long modify_out_data() //
{
    new_SDATA      char(1000);
    posting_sign    char(2);
    type           char(3);

    // Time Event Types
    //-----
    // P01 Automatic status (clock-in or clock-out) (Please note: automatic
    // status clocking records are transferred as record type P10 clock-in
    // or P20 clock-out if customized accordingly)
    // P02 Break of automatic status (start or end of break)
    // P03 Business trip auto status (start or end offsite work)
    // P04 Work at home auto status (start or end offsite work at home)
    // P05 Access log (interim entry)
    // P10 Clock-in (Please note: automatic status clocking records are
    // transferred as record type P01 if customized accordingly)
    // P11 Change of payment or cost center information
    // P15 Start of break
    // P20 Clock-out (Please note: automatic status clocking records are
    // transferred as record type P01 if customized accordingly)
    // P25 End of break
    // P30 Start of business trip
    // P35 Start offsite work at home
    // P40 End of business trip
    // P45 End offsite work at home

    // Structure of SDATA for time event types P01 to P45
    //-----
    // Field name      [from,to] Meaning
    //-----
    // SOURCE_SYS      [ 1, 10] Logical system
    // TEVENTTYPE      [ 11, 13] Time event type
    // TERMINALID      [ 14, 17] ID of the shop floor terminal if posted on
    // terminals, else the first four letters of the
    // HYDRA user name are transferred for events
    // manually recorded at office client.
    // LOGDATE         [ 18, 25] Clocking date
    // LOGTIME         [ 26, 31] Clocking time
    // PHYSDATE        [ 32, 39] Date on which the event was written to the interface.
    // PHYSTIME        [ 40, 45] Time at which the event was written to the interface.
    // TIMEID_NO       [ 46, 53] Badge number, constant 0. The personnel number is
    // used instead.
    // PERNO           [ 54, 61] Personnel number
    // ATT_ABS_REASON  [ 62, 65] Absence reason. In order for absence reasons to be
    // transferred, PZE has to be entered as HR subsystem
    // as of version 4.5.
    // OBJECT_TYPE     [ 66, 67] SAP object type
    // OBJECT_ID       [ 68, 75] SAP object ID
    // COMP_CODE       [ 76, 79] Company key
    // COSTCENTER      [ 80, 89] Cost center of clocking
    // ORDER           [ 90,101] SAP internal order (no HYDRA operation!)
    // WBS_ELEMENT     [102,125] SAP project (work breakdown structure)
    // CUSTOMER_FIELD_1 [126,145] Customer specific field 1. Free for customer
    // specific processing.
    // CUSTOMER_FIELD_2 [146,185] Customer specific field 2. Free for customer
    // specific processing.

    // Implementation for Connection to LOGA
    // -----

    // check time event type
    type = SDATA[11,13];
    if ( (type = "P10") or
        (type = "P20") or
        (type = "P30") or
        (type = "P40") )
    {
        // change segnam
        SAP_SEGNAM = "U_CUST_CLOCKING";

        // build structure U_CUST_CLOCKING
        new_SDATA = "NT1300110000000000";
        new_SDATA[18, 22] = char2long(PNR_KNR) using "&&&&&"; // badge number
        new_SDATA[23, 28] = SDATA[20,25]; // LOGDATE YYMMDD
        new_SDATA[29, 34] = SDATA[26,31]; // LOGTIME
        if (type = "P10")
        {
            posting_sign = "01";
        }
        else if (type = "P20")
        {
            posting_sign = "00";
        }
        else if (type = "P30")
        {
            posting_sign = "20";
        }
        else if (type = "P40")
        {
            posting_sign = "21";
        }
        new_SDATA[35, 36] = posting_sign; // posting sign

        // set new sap_sdata
        SDATA = new_SDATA;
        // set new length of segment
        SDATA_LEN = 36;
        dprint("SDATA = [" || SDATA clipped || "]");
    }
}

```

```

    dprint("SDATA_LEN = [" || SDATA_LEN using "<<<&" || "]" );
    OUTPUT_DS = 1;
}
else
{
    // do not send this clocking type to interface
    OUTPUT_DS = 0;
}
return 0;
}

//-----
long main() // dummy
{
    return 0;
}
//-----

```

5.7.8 Extension of the program hysap_dp (SAP Dispatcher)

5.7.9 Sort sequence of the data cursor

Name of user exit

mle_modify_dp_orderby_in.hsc

Keywords

Use this user exit to modify the sort sequence of the data cursor.

Function

You can use this user exit to change the standard sorting of the data cursor to transfer data from MLE interfaces.

The user exit may execute SQL commands and has the following interface to the system:

- ❖ After executing the script, the export variable *orderby* (char(1024)) includes the string with the new sort order (order by clause).
- ❖ This user exit mle_modify_dp_orderby_in.hsc only becomes active after a restart of the service "MIP1 ECS Inbound Dispatcher" or "HYDRA MLE Inbound Dispatcher".

Program(s) and source code files

Program	Version	Date	File(s)
hysap_dp			hysap_dp.c

Import parameter

none

Export parameter

Parameter	Type	Content
orderby	C1024	String containing the new sort sequence.

Default sorting of the cursor: *order by 11 desc, 14, 15, 16*

You can use all columns of the tables hysap_inbound_ctrl and hysap_dist_mod. But you cannot use the column name. Column assignment and numbering look as follows:

Sort number	Column name
1	hic.TA_ID
2	hic.TA_TYPE
3	hic.SAP_MESTYP
4	hic.SAP_MESCOD
5	hic.SAP_MESFCT
6	hic.SAP_IDOCTYP
7	hic.TA_Lines
8	hic.TA_LDone
9	hic.TALError
10	hic.TA_LUnknown
11	hdm.DM_PRIO
12	hdm.DM_CMD
13	hdm.DM_CMDPARAM
14	hic.TA_Status
15	hic.TA_WorkDate
16	hic.TA_WorkTime
17	hic.SAP_CRETIM
18	hic.VERWEIS
19	hic.SAP_DOCNUM
20	hic.ta_logsys
21	hic.ta_savdate
22	hic.ta_savtime
23	hic.sap_tabnam
24	hic.sap_mandt
25	hic.sap_docrel
26	hic.sap_status
27	hic.sap_direct
28	hic.sap_outmod
29	hic.sap_exprss
30	hic.sap_test
31	hic.sap_cimtyp
32	hic.sap_std
33	hic.sap_stdvrs
34	hic.sap_stdmes
35	hic.sap_sndpor
36	hic.sap_sndprt
37	hic.sap_sndpfc
38	hic.sap_sndprn
39	hic.sap_sndsad
40	hic.sap_sndlad
41	hic.sap_rcvpor
42	hic.sap_rcvppt
43	hic.sap_rcvpfc
44	hic.sap_rcvprn
45	hic.sap_rcvsad
46	hic.sap_rcvlad
47	hic.sap_credat
48	hic.sap_refint
49	hic.sap_refgrp
50	hic.sap_refmes

51	hic.sap_arckey
52	hic.sap_serial
53	hic.param1
54	hic.param2
55	hic.bearb
56	hic.bearb_date
57	hic.bearb_time
58	hdm.dm_direct
59	hdm.dm_desc
60	hdm.dm_sap_mestyp
61	hdm.dm_sap_idotyp
62	hdm.dm_sap_cimtyp
63	hdm.dm_sap_mescod
64	hdm.dm_sap_mesfct
65	hdm.dm_sourcesys
66	hdm.dm_sap_segnam01
67	hdm.dm_sap_segnam02
68	hdm.dm_sap_segnam03
69	hdm.dm_sap_segnam04
70	hdm.dm_sap_segnam05
71	hdm.dm_sap_segnam06
72	hdm.dm_sap_segnam07
73	hdm.dm_sap_segnam08
74	hdm.dm_sap_segnam09
75	hdm.dm_sap_segnam10
76	hdm.dm_sap_test
77	hdm.dm_dest
78	hdm.dm_keepdays
79	hdm.dm_param1
80	hdm.dm_param2
81	hdm.bearb
82	hdm.bearb_date
83	hdm.bearb_time

5.8 Server user exits: MPL

5.8.1 Setting batch status (STKOMBI)

Name of user exit

mpl_c_sta_init.hsc

Keywords

Batch status, batch class, quality status, material status

Function

- ❖ While reporting/posting batches in MPL, this user exit creates all the relevant statuses of the batch from a combined status. The user exit is called during batch change (CA_AB), batch generation (C_GEN) and while setting batch status (C_STA).
- ❖ The user exit is active only if the STKOMBI=<value> ID is transferred through the DD command (e.g. DLG=C_GEN|STKOMBI=A|...).

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			d_mplutil.c
hymw.exe			

Import parameter

Parameter	Type	Content
STKOMBI	C1	Combined status from DD command
DLG	C30	Dialog
ANR	C40	Operation
MNR	C10	Machine
CNR	C20	Batch number
STA	C1	Current batch status
MATST	C1	Current material status
QSTMANU	C1	Current, manual quality status
CKL	C1	Current batch class

Export parameter

Parameter	Type	Content
Status	C1	Batch status
mat_status	C1	Material status
q_status_manu	C1	Manual Quality status
klasse	C1	Batch class

5.8.2 Extension of ZWAU/ZWEI goods movement confirmation/upload (MPLRFRCK)

See MLE for further information.

5.8.3 Material movement (C_MBEW)

Name of user exit

hyo_c_mbew_init.hsc

Keywords

Batch number, C_MBEW

Function

- ❖ This user exit can override the calculation of the lot number for the command C_MBEW Customer-specific.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			d_c_mbew.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30	Dialog

Export parameter

Parameter	Type	Content
CNR	C20	Batch number

Name of user exit

hyo_c_mbew_exec1.hsc
hyo_c_mbew_exec2.hsc

Keywords

Saving the values of the batch with the C_MBEW dialog.

Function

- ❖ These user exits can store the values on the batch with the C_MBEW command. The hyo_c_mbew_exec1.hsc user exit is called before standard processing. Standard processing is not executed if the user exit provides a return code of 0.
- ❖ The hyo_c_mbew_exec2.hsc user exit is called after standard processing.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			d_c_mbew.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30	Dialog
CNR	C20	Batch number

Export parameter

Parameter	Type	Content
-		

5.8.4 Setting the retrograde consumption type (backflush)

Name of user exit

mpl_get_consumption_type.hsc

mpl_get_consumption_type_ce_ab.hsc

Keywords

This user exit sets the processing type for retrograde consumption calculation (backflushing).

Function

- ❖ The control indicators of the material type (stock indicator) and components (consumption type) are overridden if the user exit is active.
- ❖ The user exit is active only if the VERBRAUCH_ART field of the material component has the value 'U'.
- ❖ The mpl_get_consumption_type.hsc user exit controls only the retrograde posting of input materials (backflushing).
- ❖ The mpl_get_consumption_type_ce_ab.hsc user exit controls only the posting of input materials when batches are logged off (backflushing).

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			d_verb.c d_ce_ab.c

Import parameter

Parameter	Type	Content
ANR	C40	Operation
ATK	C40	Article
CNR	C20	Batch number
SLP	C10	BOM item
MATTYP	C10	Material type (batch)

Export parameter

Parameter	Type	Content
VERBR_ART	C1	Type of consumption calculation Valid values: 'R' – Retrograde consumption recording (backflushing) 'W' – Retrograde / goods movements only (backflushing)

5.8.5 Processing surplus consumption quantities when logging off batches

Name of user exit

mpl_get_ce_ab_consumption_surplus.hsc

Keywords

The user exit processes surplus consumption quantities.

Function

This user exit is called before setting the negative remaining quantity of the batch to 0.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp1*			d_c_stat.c

Import parameter

Parameter	Type	Content
ANR	C40	Operation
MNR	C20	Machine
ATK	C40	Article
ARTIKEL_BEZ	C80	Article name
SLP	C10	BOM item
MATTYP	C10	Material type (batch)
CNR	C20	Batch number
VERB_MENGE	DEC	Total consumption quantity to be posted
M_VERB_MENGE	DEC	Surplus consumption quantity
EINH	C3	Unit

5.8.6 Changing data while generating goods movements

Name of user exit

mpl_cmm_e_before_insert.hsc → goods-in

mpl_cmm_a_before_insert.hsc → goods-out

Keywords

Use the user exit to change the data of goods movements (event_mlb table) before this data is inserted in the database.

Function

- ❖ If the user exit is active, you can change all data of goods movements before this data is inserted in the database.

Callback functions (mpl_cmm_a_before_insert.hsc):

- BAPICALLEEXECUTE: any BAPI calls (from version 7.2.1.72/8.1.1.77)

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			d_c_mm.cpp

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data string

Import/export parameter

Parameter	Type	Content
EVENT	C10	Event e.g. CA_AB
EV	C10	Goods out/goods in CMM_A/CMM_E or return transfer/posting CMM_R
DLG	C10	Dialog e.g. CA_WL
DAT	INT	Date
ZEI	INT	Time
TNR	INT	Terminal user
ANR	C40	OP
MNR	C10	Machine
CNR	C20	Batch number

Parameter	Type	Content
DLL	C20	Throughput batch
PNR	C10	Person
ATK	C40	Article
ATKBEZ	C40	Article designation
SLP	C10	BOM item
MATTYP	C10	Material type
CST	C1	Batch status
CKL	C1	Batch class
MATST	C1	Material status
QST	C1	Quality status
ZLO	C10	Storage location
FIR	C10	Company
LAGORT	C10	PPS storage location
LAGPZ	C10	PPS storage bin
SAPCNR	C10	SAP batch
SAPANR	C10	SAP order
BWART	C3	Transaction type (e.g. 101)
OPTEAUS	C1	Option "Finally issued"
OPTENDLIEF	C1	Option "Delivery completed"
OPTRCK	C1	Confirm/upload Y/N
EGR1-10	DEC	Quantity fields
TYP1-10	C3	Type of quantity (e.g. VGR)
GR1-10	C4	Reason
EINH1-10	C3	Unit
LAGORT_Q	C10	Issuing PPS storage location (only for user exit mpl_cmm_e_before_insert; with dialog CMM_A the issuing and receiving storage location are always identical) Minimum versions: MW30 : hymwmp1725 → 8.1.1.124
ZLO_Q	C10	Issuing storage location (only for user exit mpl_cmm_e_before_insert; with dialog CMM_A the issuing and receiving storage location are always identical) Minimum versions: MW30 : hymwmp1725 → 8.1.1.124
ATTR1	C20	Attribute field in the table EVENT_MLB. Note: By default, ATTR6 is used for the material type.
ATTR2	C20	
ATTR3	C30	
ATTR4	C20	

Parameter	Type	Content
ATTR5 ATTR6 ATTR7 ATTR8 ATTR9	C20 C10 C10 C20 C30	- ATTR1 is set with upload attribute PALNR for CMM_E Minimum versions: - MW30 : hymwmp1725 → 8.1.1.156
FU_D_01 to FU_D_06	Date	Date user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_N_07 to FU_N_22	Long	Integer user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_F_23 to FU_F_28	Double	Double user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_C_29 to FU_C_44	C1	Char user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_C_45 to FU_C_50	C10	Char user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_C_51 to FU_C_64	C20	Char user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc
FU_C_65 to FU_C_66	C40	Char user fields Minimum versions: MW30 : hymwmp1725 → 8.1.1.175 Only applies to mpl_cmm_a_before_insert.hsc

5.8.7 Itemizing the generated batch number

Name of user exit

hylosnrgen.hsc

Keywords

Itemizing the batch number when generating batch numbers (e.g. output batch)

Function

- ❖ Use this user exit to change batch number generation in the server.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172			mpl_util.c

Import parameter

Parameter	Type	Content
losnr_len	NUM	Length of the batch number
losnr_prefix	C2	Prefix of batch number for production batches from setup
Terminal number	NUM	Terminal user

Export parameter

Parameter	Type	Content
CNR	C20	Batch number

5.8.8 Processing consumption quantities of input batches

Name of user exit

mpl_publish_batch_consumption.hsc

Keywords

Processing consumption quantities of input batches.

Function

- ❖ This user exit provides information about the consumption quantity shortly before the quantity is deducted from the input batch.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmp172	8.1.1.163		

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Original dialog data
DLG	C80	Dialog
ANR	C40	Order number
MNR	C40	Machine number
ATK	C40	Article
CNR	C20	Batch number
SLP	C10	BOM item
VERB_MENGE	DEC	Consumption quantity

Export parameter

5.9 Server user exits – PZW Personnel TimeManagement

5.9.1 General import/export parameters

5.9.1.1 HR master data for evaluations

HR master data is required frequently. Hence, this data is available in a uniform structure in several scripts. The prefix may differ according to the script.

Parameter	Type	Contents:
<Prefix>FIR	char(4)	Company
<Prefix>PNR	long	Personnel number
<Prefix>KNR	char(10)	Badge number
<Prefix>BER	char(8)	Area
<Prefix>KST	char(10)	Cost center
<Prefix>PNAME	char(40)	Last name
<Prefix>PVORNAME	char(20)	First name
<Prefix>EINTRITT	date	Date of joining
<Prefix>AUSTRITT	date	Date of leaving
<Prefix>AVGAZ	long	Average working time in seconds
<Prefix>OPT_AVGAZVERB	char(1)	Option "Post average working time"
<Prefix>URLANSPR_NORM	long	Annual leave entitlement in days times 10 (e.g. 300 for 30 days leave entitlement)
<Prefix>URLANSPR_SONDER	long	Annual special leave entitlement in days times 10 (e.g. 300 for 30 days leave entitlement)
<Prefix>URLANSPR_ZUSATZ	long	Annual additional leave entitlement in days times 10 (e.g. 300 for 30 days leave entitlement)
<Prefix>URLANSPR_DAT	date	Effective date of the annual leave
<Prefix>PKREIS	char(8)	Employee subgroup
<Prefix>NATION	char(3)	Nationality
<Prefix>SPERR_PZE	char(1)	Blocking ID PZE (S/"
<Prefix>BESCHVERH	char(1)	Salaried/Non-salaried employee ID (A/G)
<Prefix>GEBDAT	date	Date of birth
<Prefix>KST_TMP	char(10)	Temporary cost center
<Prefix>FIR_TMP	char(4)	Company of the temporary cost center
<Prefix>DATB_TMP	date	Beginning of validity for the temporary cost center
<Prefix>DATE_TMP	date	End of validity for the temporary cost center
<Prefix>ETGAWDAT	date	Date of first allocation/evaluation
<Prefix>ABREDAT	date	Date of latest evaluation
<Prefix>TZGRAD	long	Part-time rate in percentages times 1000 (50000 for 50%)
<Prefix>TAETIGKEIT	char(20)	Activity
<Prefix>TEL_FIR	char(20)	Telephone / Company
<Prfix>INFO01	char(40)	Configurable information field
<Prfix>INFO02	char(40)	Configurable information field
<Prfix>INFO03	char(40)	Configurable information field
<Prfix>INFO04	char(40)	Configurable information field
<Prfix>INFO05	char(40)	Configurable information field

<Prfix>INFO06	char(40)	Configurable information field
<Prfix>INFO07	char(40)	Configurable information field
<Prfix>INFO08	char(40)	Configurable information field
<Prfix>INFO09	char(40)	Configurable information field
<Prfix>INFO10	char(40)	Configurable information field
<Prfix>INFO11	char(20)	Configurable information field
<Prfix>INFO12	char(20)	Configurable information field
<Prfix>INFO13	char(20)	Configurable information field
<Prfix>INFO14	char(20)	Configurable information field
<Prfix>INFO15	char(20)	Configurable information field
<Prfix>INFO16	char(40)	Configurable information field
<Prfix>INFO17	char(40)	Configurable information field
<Prfix>INFO18	char(40)	Configurable information field
<Prfix>INFO19	char(40)	Configurable information field
<Prfix>INFO20	char(40)	Configurable information field
<Prfix>INFO21	long	Configurable information field
<Prfix>INFO22	long	Configurable information field
<Prfix>INFO23	long	Configurable information field
<Prfix>INFO24	long	Configurable information field
<Prfix>INFO25	long	Configurable information field
<Prfix>INFO26	date	Configurable information field
<Prfix>INFO27	date	Configurable information field
<Prfix>INFO28	date	Configurable information field
<Prfix>INFO29	date	Configurable information field
<Prfix>INFO30	date	Configurable information field
<Prefix>DATB	date	Validity start of the HR master data
<Prefix>DATE	date	Validity end of the HR master data
<Prefix>VERWEIS	long	Reference of the data record in the database (from MW 2.0.2)

5.9.2 Data output: interface to payroll accounting

5.9.2.1 User exit hylobuprint

Name of user exit

hylobuprint.hsc

Keywords

HYD-LUG, hylobu.out, hylobu.exe, payroll interface

Function

The user exit allows you to edit the text line to be output in the interface transferring the PZE monthly wage types to payroll accounting. The function "main" is called.

Program(s) and source code files

Program	Version	Date	File(s)
hylobu.out	8.1.1.227	2014-12-17	hylobu.c hyt_sc_util.c

As of MW 3.0 the user exit is executed before data is output in outbound transaction to SAP-HR for the formats HYSAP_SEGNAM_HR_UPEXTWT and HYSAP_SEGNAM_HR_IT2010 (available with hylobu.exe|out as of version 8.1.1.227 / 17 December 2014).

Available with hylobu.exe|out as of version 8.1.1.236 / 01 December 2015:

If the export parameter LOBU_DATA is empty in this user exit, the data row is neither displayed in the file nor in SAP outbound transactions.

Import parameter

The following import variables of the script transfer HR master data fields with the prefix "PNR_":

Parameter	Type	Contents:
PNR_*	misc.	HR master data for evaluations (see separate chapter)

The ID of the interface format is also transferred as import variable:

Parameter	Type	Contents:
FORMAT	char(max.100)	Interface format, e.g., "HYDRA" or "C-LOHN" for C-Lohn, XL/XXL.

Export parameter

Parameter	Type	Contents:
LOBU_DATA	char(max.10000)	Output string of a data row according to the interface documentation

LOBU_CLIP_DATA	char(1)	The spaces at the end of the output string are removed if this export variable is set to "Y". This is required for interfaces having separators between the columns and variable record lengths.
----------------	---------	--

5.9.2.2 User exit hyt_lobu_lart

Name of user exit

hyt_lobu_lart.hsc

Keywords

HYD-LUG, hylobu.out, hylobu.exe, payroll interface

Function

In the interface transferring the PZE monthly wage types to payroll, this user exit allows you to edit the single lines of the interface file.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Unlike the previous user exit *hylobuprint*, the global variables of this user exit are persistent. This means that the content of global variables remains while processing the single rows.

Program(s) and source code files

Program	Version	Date	File(s)
hylobu.out			hylobu.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.2.3 User exit hyt_lobu_fz

Name of user exit

hyt_lobu_fz.hsc

Keywords

HYD-LUG, hylobu.out, hylobu.exe, absence interface to payroll accounting

Function

In the interface transferring absences to payroll accounting, this user exit allows you to edit the single rows of the interface file. Only use this user exit if absences are output in a separate file. Use the previously described user exit if absences are stored in the same file as the monthly wage types.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Unlike the userexit hylobuprint, the global variables of this user exit are persistent. This means that the content of global variables remains while processing the single rows.

Program(s) and source code files

Program	Version	Date	File(s)
hylobu.out			hylobu.c

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.3 Work day evaluation, pre-calculation and post-calculation

Name of user exit

hyt_tagesaw.hsc

Keywords

Work day evaluation, tages_aw, daily closing

Function

Use this user exit to change the results for a person and day before and after the work day evaluation. The database is normally accessed in this context. Before the work day evaluation, you can block persons or return an error. After the work day evaluation, you can edit the work day result and the clocking records. From MW 2.0 onwards, you can also set an error after the workday evaluation. This error leads to the current transaction being reset and thus the modifications and results of the work day evaluation are rejected.

The functions "before_calc()" and "after_calc()" are provided. The two functions are called only if a person is evaluated (requirements: the person is subject to evaluation, not locked and there are no errors). The after_calc() function is called even if an error occurs during the work day evaluation.

The global variables of this user exit are persistent. This means, that global variables can be set in the function "before_calc()" and their content can be processed in the function "after_calc()". Please keep in mind to initialize the global variables in the function "before_calc()", as otherwise, the values of the previously evaluated person are still set.

Program(s) and source code files

Program	Version	Date	File(s)
tages_aw.out			tages_aw.c

Program	Version	Date	File(s)
			hyt_sc_util.c

Import parameter

The following import variables of the script transfer HR master data fields with the prefix "PNR_":

Parameter	Type	Contents:
PNR_*	misc.	HR master data for evaluations (see specific chapter)

The following variable is also transferred in order to specify the evaluation date:

Parameter	Type	Contents:
ABREDAT	date	Evaluation date

Export parameter

Parameter	Type	Contents:
ERG	long	Result: 0=OK, 1=error, 2=blocked, 3=not required This parameter is always assigned to 0 before the work day evaluation. Change this variable to prevent the evaluation. After the work day evaluation, this variable indicates whether the evaluation was successful or an error has occurred. From MW 2.0 on, the work day evaluation will run in one transaction. Thus, you can set an error after a successful evaluation and reject the modifications and results.
MELDNR	long	If an error is set in the user exit, use this parameter to set the message No. for the error message in the listing of messages. The program writes the message for available error messages (messages with *). If MELDNR remains 0, no error message is generated and the user exit must do it on its own.

No callback functions

5.9.4 Wage type posting, pre-calculation and post-calculation

Name of user exit

hyt_lartverb.hsc

Keywords

Wage type posting, work day evaluation, weekly evaluation, PZE posting, wage types, wage type postings, editing of work day results

Function

Use this user exit to edit the results for a person and evaluation period before and after the work day evaluation. The database is normally accessed in this context. The results of the work day evaluation and wage type posting (weekly evaluation) are available for the user exit in the database. You can evaluate and possibly modify the results subsequently.

The functions "before_calc()" and "after_calc()" are provided. The two functions are called only if a person is evaluated (requirements: the person is subject to evaluation, not locked and there are no errors).

The global variables of this user exit are persistent. This means, that global variables can be set in the function "before_calc()" and their content can be processed in the function "after_calc()". Please keep in mind to initialize the global variables in the function "before_calc()", as otherwise, the values of the previously evaluated person are still set.

Program(s) and source code files

Program	Version	Date	File(s)
woche_aw.out			woche_aw.c hyt_sc_util.c

Import parameter

The following import variables of the script transfer HR master data fields with the prefix "PNR_":

Parameter	Type	Contents:
PNR_*	misc.	HR master data for evaluations (see specific chapter)

The following variables are transferred to determine the evaluation period:

Parameter	Type	Contents:
AW_DATB	date	Start of the evaluated period. This is the start of the overtime period or sub-period at the end of the month.
AW_DATE	date	End of the evaluated period. This is the least/earliest value from:

		1. Date when the overtime period ends 2. Date when the sub-period ends at the end of the month 3. The day evaluated by the work day evaluation
MEHRARBPER_DATB	date	Start of the full overtime period.
MEHRARBPER_DATE	date	End of the full overtime period.

Export parameter

Parameter	Type	Contents:
-	-	-

Callback function char(n) PZE_LOCK()

The callback function PZE_LOCK is used to make an entry in the lock table or to check whether the object is already locked. The parameters of the function are the 1 to 5 lock keys as dialog acronyms KEY:1 to KEY:5. If no keys are provided, the first key is set to "-".

Return value:

"" = null Lock could be set, the object had not been locked by someone else.

Otherwise: The object had already been locked by someone else.

 The return string indicates who locked the object.

Callback function PZE_UNLOCK

Use this callback function to undo your lock. The parameters correspond to the PZE_LOCK callback function. The return value of the function is undefined.

Example of callback functions PZE_LOCK and PZE_UNLOCK

```
variable lockresult char(200);
...
lockresult = CallBack( "PZE_LOCK", "KEY:1="||hyfilepath(METZ_FILENAME)||"|" );

if( lock result is null )
{
    // Locking successful: Action ...
    ...
    lockresult = CallBack( "PZE_UNLOCK", "KEY:1="||hyfilepath(METZ_FILENAME)||"|" );
}
else
{
    dprint( "Error: Object locked by "||lockresult clipped || "." );
}
```

5.9.55.1 Month evaluation, pre- and post-allocation

Name of user exit

hyt_monataw.hsc

Keywords

Monthly evaluation, month_aw, monthly wage types, monatlohnarten_aw, accounting

Function

Use this user exit to edit the results for a person and monthly period before and after the monthly evaluation. The database is normally accessed in this context. Before the monthly evaluation, you can create daily wage types in the *pzebuchung* table (PZE posting). These wage types are integrated in the monthly result during the monthly evaluation. After the monthly evaluation, you can edit the monthly result and the monthly wage types.

The functions "before_calc()" and "after_calc()" are provided.

The global variables of this user exit are persistent. This means, that global variables can be set in the function "before_calc()" and their content can be processed in the function "after_calc()". Please keep in mind to initialize the global variables in the function "before_calc()", as otherwise, the values of the previously evaluated person are still set.

Program(s) and source code files

Program	Version	Date	File(s)
---------	---------	------	---------

Program	Version	Date	File(s)
hymonaw.out			hymonaw.c hyt_sc_util.c

Import parameter

The following import variables of the script transfer HR master data fields with the prefix "PNR_":

Parameter	Type	Contents:
PNR_*	misc.	HR master data for evaluations (see specific chapter)

The following variables are transferred to determine the evaluation period:

Parameter	Type	Contents:
MONATPER_DATB	date	Start of the monthly period.
MONATPER_DATE	date	End of the monthly period.

Export parameter

Parameter	Type	Contents:
-	-	-

No callback functions

5.9.6 Monthly evaluation, processing of account limits

Name of user exit

hyt_kontogrenze.hsc

Keywords

Restrict accounts, monthly evaluation, monat_aw, ktogren, account limit, settlement

Function

Use this user exit to process PZE account limits. The user exit is executed directly after every single account limit has been processed, even before the specified wage type is saved in the database and allocated to a target account that might be configured.

The user exit is also called for account limits, which are configured, but that do not enforce a limit (e.g. flexible hours amount to 10.00 hours. There is a limit which cuts off hours if 30 hours are exceeded. The limit does not involve changes to the account. But the user exit is started). In this case BUCH_LART is empty and BUCH_WERT is 0.

In the user exit you can change the limiting wage type BUCH_LART and the value BUCH_WERT posted onto the wage type.

The function "after_account_limit ()" is provided.

The global variables of this user exit are persistent. Please keep in mind to initialize the global variables, if required, as otherwise the values of the previously evaluated person are still set.

Program(s) and source code files

Program	Version	Date	File(s)
hymonaw.out			ktogren4.c hyt_sc_util.c

Import parameter

The following import variables of the script transfer HR master data fields with the prefix "PNR_":

Parameter	Type	Contents:
PNR_*	misc	HR master data for evaluations (see specific chapter)
MONATPER_DATB	date	Start of the monthly period.
MONATPER_DATE	date	End of the monthly period.
PZEKTOG_KTO	N	Number of the limited account
PZEKTOG_KTOG_MIN	N	Lower account limit (in seconds or evaluated with account factor).
PZEKTOG_KTOG_MAX	N	Upper account limit (in seconds or evaluated with account factor)
PZEKTOG_LART_MIN	C4	Wage type for lower account limit (empty: limit is inactive. "0": limit active, do not post on wage type) "0": limit is active; do not post on wage type).
PZEKTOG_LART_MAX	C4	Wage type for upper account limit (empty: limit is inactive. "0": limit active, do not post on wage type) "0": limit is active; do not post on wage type).
PZEKTOG_KTOG_UMRFAKT	N	With day accounts: conversion factor: hours per day (in seconds)
PZEKTOG_VERB	C1	Processing indicator: B=limit/K=constant value

Export parameter

Parameter	Type	Contents:
BUCH_LART	C4	Wage type that has been limited.
BUCH_WERT	N	Value posted onto the wage type in seconds (negative value if the minimum limit is applied!)

No callback functions

Example:

```

hydra basic;

/* -----
Script : hyt_kontogrenze.hsc
Description: User exit processing PZE account limits.

$Revision: 1.0 $
$Date: 2010/08/04 00:00:00 $

$Log: hyt_kontogrenze.hsc $

----- */

/*- Variable declaration -----*/
import ERRORCODE          long;      // Return code of previous multi-script function calls

// Import data: person
//import PNR_PNR          long;      // Personnel number
//...

// Import data: evaluation period
//import MONATPER_DTB      date;      // Start of accounting period
//import MONATPER_DATE     date;      // End of accounting period

import PZEKTOG_KTO         long;      // Number of the limited account
import PZEKTOG_KTOG_MIN    long;      // lower account limit (in seconds or evaluated with account factor.)
import PZEKTOG_KTOG_MAX    long;      // Upper account limit (in seconds or evaluated with account factor)
import PZEKTOG_LART_MIN    char(4);   // wage type for lower account limit (empty: Limit not active.) "0": Limit active,
do not post to wage type)
import PZEKTOG_LART_MAX    char(4);   // wage type for upper account limit (empty: Limit not active.) "0": Limit active,
do not post to wage type)
import PZEKTOG_KTOG_UMRFAKT long;      // For day accounts: conversion factor: hours per day (in seconds)
import PZEKTOG_VERB        char(1);   // Processing ID: B=limit/K=constant value

export BUCH_LART           char(4);   // Limited wage type
export BUCH_WERT long;      // Wage type value in seconds (negative, if the minimum limit was applied)

```

```
//-----  
// Function is called after processing each account limit  
//-----  
long after_account_limit()  
{  
variable ret long;  
variable lart_auswahl_kz char(1);  
  
ret = ERRORCODE;  
  
dprint( "PZEKTOG_KTO : " || PZEKTOG_KTO using "+<<<<<<<" );  
dprint( "PZEKTOG_KTOG_MIN : " || PZEKTOG_KTOG_MIN using "+<<<<<<<||" (" || (PZEKTOG_KTOG_MIN using "$TIME") || )" );  
dprint( "PZEKTOG_KTOG_MAX : " || PZEKTOG_KTOG_MAX using "+<<<<<<<||" (" || (PZEKTOG_KTOG_MAX using "$TIME") || )" );  
dprint( "PZEKTOG_LART_MIN : " || PZEKTOG_LART_MIN );  
dprint( "PZEKTOG_LART_MAX : " || PZEKTOG_LART_MAX );  
dprint( "PZEKTOG_KTOG_UMRFAKT : " || PZEKTOG_KTOG_UMRFAKT using "+<<<<<||" (" || (PZEKTOG_KTOG_UMRFAKT using "$TIME") || )" );  
dprint( "PZEKTOG_VERB : " || PZEKTOG_VERB );  
dprint( "BUCH_LART : " || BUCH_LART );  
dprint( "BUCH_WERT : " || (BUCH_WERT using "+<<<<<<<||" (" || (BUCH_WERT using "$TIME") || )" );  
  
// Special processing of flexible working hours:  
// 1) Monthly account 4,2 h expires : --> normal account limit  
// 2) Monthly account 2,5 between 4,2 h : Total including 2,5 h which are credited on the flexible working hours account  
// --> Special processing  
// 1) Monthly account 0 - 2,5 h expire : --> normal account limit  
// 4) Monthly account less than 0 will be offset with flexible hours: --> normal account limit  
  
// Identifies, if the wage type is configured for special processing.  
lart_auswahl_kz = "";  
if( ( BUCH_LART is not null ) and  
( BUCH_LART != "0" ) and  
( PZEKTOG_VERB = "B" ) and  
( BUCH_WERT > 0 ) )  
{  
sqlexec( "select auswahl_kz from lohnarten where lohnart = " || BV(BUCH_LART) || ";" );  
into( lart_auswahl_kz );  
}  
  
if( lart_auswahl_kz = "G" )  
{  
// It is the limited described above 2). You have to add the account limit  
// in order to have the total calculated including the  
  
2,5h  
dprint( "Post the total account balance to the wage type." );  
BUCH_WERT = BUCH_WERT + PZEKTOG_KTOG_MAX;  
  
dprint( "BUCH_LART : " || BUCH_LART );  
dprint( "BUCH_WERT : " || (BUCH_WERT using "+<<<<<<<||" (" || (BUCH_WERT using "$TIME") || )" );  
}  
else  
{  
dprint( "user exit does not intervene." );  
}  
  
return ret;  
}  
  
/*-----*/  
/* Main function (only for test purposes) */  
/*-----*/  
long main()  
{  
return 0;  
}  
  
/*-----*/
```

5.9.7 Information display at the terminal

Name of user exit

hyt_pzeinfo.hsc

Keywords

PZE information display, PZE terminal, terminal information, account balances, time balance

Function

Use this user exit to add additional lines to the PZE information display (at the beginning or end). The info lines are formatted in the user exit independently of the terminal and then formatted by the software according to terminal type.

Use the function "append_info()" to prefix lines. Use the function "append_info()" to append lines.

Use the function "modify_info()" to change existing lines, e.g. to change account balances.

Please

note:

The list of authorized badges (LIST;27) indicates the account name for Benzing terminals and the CTP-340 terminal. For these terminals the name must be unique for all employees and, as a result, it cannot be set subject to the respective employee. This applies even if the info is displayed online at the CTP-340 terminal.

Program(s) and source code files

Function	Program	Version	Date	File(s)
append_info ()	hymw.out			hyd_scmd.c hyt_sc_util.c
prepend_info()	hymw.out			hyd_scmd.c hyd_pzel.c
modify_info	hymw.out			hyd_scmd.c hyt_sc_util.c

Import parameter

Parameter	Type	Contents:
PNR	long	Personnel number
LEN_DIS_ADD_ROWS	long	Max. length of the export variable DIS_ADD_ROWS
DLG_DATA	char(30000)	Dialog data you used to request the information display.

Export parameter

Parameter	Type	Contents:
DIS_ADD_ROWS	char(3000)	Dialog string used to add additional lines to the PZE info display or to change the lines. The content depends on the called script function. Please see the below section.

5.9.7.1 Functions prepend_info() and append_info()

The user exit completes the export parameter DIS_ADD_ROWS as dialog string to add additional rows to the PZE info display.

Format: "BEZ:1=x|WERT:1=y|BEZ:2=a|WERT:2=b|..."

To add 1, 2 or more lines: The number has always to start at 1, irrespective of whether the lines are to be prepended or appended.

5.9.7.2 Function modify_info()

The export parameter is prepopulated with the name and value to be displayed. In addition, the export parameter also includes further information on the account to be displayed:

Format: "BEZ=x|WERT=y|..."

The line is not displayed if BEZ (name) and WERT (value) are empty.

DIS_ADD_ROWS also provides the following information as Bapi values. Further processing is not affected if you change these values.

Identifier	Type	Description
PZEKTO.KTO	N	Number of the PZE account to be displayed.
PZEKTO.BEZL	C40	Name of the account
PZEKTO.BEZK	C6	Short name of the account
PZEKTO.ART	C1	T= day account Z= time account
PZEKTO.FAKT	N	Factor for day accounts
LINENR	N	Number of the output line in the terminal info

		dialog
--	--	--------

5.9.8 Display online balances during clocking

Name of user exit

hyt_pze_online_balance.hsc

Keywords

Online display of balances, display of balances during clocking, PZE information display, PZE terminal, terminal information, time balances, balance status.

Function format_balance()

The user exit is requested with the string command 22 "sc_pers_status". The user exit is only requested if the display of balances is activated at the terminal and the terminal acronym "OPT.BALANCE=1|" is transferred to the string command.

[!]: The online balance display is only available in the Windows terminal CTAIP.

Use an authorization key (at terminal = license) TNR-OSA to activate this option for the terminal.

You can format a string in the user exit. This string shows the balance at the terminal during clocking.

The function format_balance() is called in the user exit.

The user exit must assign the export parameter BALANCE_TEXT with the balance text to be displayed. Eight accounts are transferred. You can add up the accounts or show any of these 8 accounts separately. The display must be tested on the AIP.

You can also perform an online daily evaluation that calculates the current balance up to the current point in time.

Program(s) and source code files

Function	Program	Version	Date	File(s)
sc_pers_status()	hymw.out			hyd_scmd.c hyt_sc_util.c

Import parameter

Parameter	Type	Contents:
PNR	long	Personnel number
DLG_DATA	char(n)	Other dialog data the terminal has added to the fixed parameters of the string command.
LATEST_EVALUATION	date	Date of latest evaluation. Date of the last, successful labor time calculation.
ACCOUNT01 to ACCOUNT08	long	Balances of the person's eight accounts. The unit is "seconds" for time accounts. The unit is "days" for daily accounts. Depending on the number of configured decimal places, the value will be multiplied by the factors 10, 100 or 1000.
BALANCE_TEXT_LEN	char(n)	Max. length of text displaying the balance

Export parameter

Parameter	Type	Contents:
BALANCE_TEXT	char(n)	Text that is showed for the balance

5.9.9 Planning data source

Name of user exit

hyt_pzeplanung.hsc

Keywords

PZE planning data source, planning data source, hy_plan

Function

The user exit enables you to make changes to the output file of the PZE planning data source. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hy_plan.out			hy_plan.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.10 Data source of account planning

Name of user exit

hyt_kontoplanung.hsc

Keywords

Account planning data source, data source for account planning, hytkplan

Function

The user exit enables you to make changes to the output file of the account planning data source. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
---------	---------	------	---------

Program	Version	Date	File(s)
hytkplan.out			hytkplan.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.11 Attendance/absence overview

Name of user exit

hyt_anablist.hsc

Keywords

PZE attendance and absence overview, attendance overview, hyt_anab

Function

The user exit allows you to change the output file of the attendance and absence overview. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hyt_anab.out			hyt_anab.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.12 Labor time statistics

Name of user exit

hyt_tagesstatistik.hsc

Keywords

PZE labor time statistic, day statistic, hyt_stat

Function

Use this user exit to change the output file for labor time statistics. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hyt_stat.out			hyt_stat.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.9.13 Time sheet

Name of user exit

hyt_zeitnachweis.hsc

Keywords

Time sheet, time sheet archive, time statement, maintenance of workday results, hyt_znw7

Function

Use this user exit to change the output files of the time sheet. The user exit is also used for the generation of time sheet archives and for the generation of the data source for the clocking list in the "maintenance work day results"/editing of labor times function. User exits are not processed when time sheets are read out from time sheet archives.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

This is the standard user exit to change list files.

This user exit is special as not only one single, external data source is processed by it but three data sources:

- Headers
 - Headers start with "KENN_K|"
 - Data rows start with "K|"
- Clockings
 - Headers start with "KENN_S|"
 - Data rows start with "S|"
- Monthly wage types
 - Headers start with "KENN_M|"
 - Data rows start with "M|"

The processing order varies subject to the purpose of the time sheet that has been started:

- "normal" time sheet:

There are three, separate files for headers, clockings and monthly wage types that the user exit processes one after the other and independently of each other. This leads to the following order:

- Header row "header"
- Data row "header" of all people, then "append_list_file"
- Header row "clockings"
- Data row "clockings" of all people, then "append_list_file"
- Header row "monthly wage types"
- Data row "monthly wage types" of all people, then "append_list_file"

Do not use direct relations between headers, clockings and monthly wage types as the persistent variables of a user exit do not apply to several files.

- Archiving of the time sheet

When time sheets are archived, one single file is generated for each person and month. This file includes the headers and data rows for the header data, clockings and monthly wage types.

Consequently, this results in the following processing sequence in the user exit:

- Header line "Header "
- Header line "Clockings "
- Header line "Monthly wage types "
- Data rows "Header" for one person
- Data rows "Clockings" for one person
- Data rows "Monthly wage types" for one person, then "append_list_file"

- Generation of the clocking list for "maintenance of work day results/editing of labor times".

In this case, the system only generates the file that includes a person's clockings within the requested period. This results in the following processing order in the user exit:

- Header line "Clockings"
- Data rows "Clockings" for one person, then "append_list_file"

This special time sheet can be identified by the time sheet number "ZNWL=10000|" that is displayed in DLG_DATA.

A user exit that should behave in the same manner for all three cases must comply with these conditions. See example below.

Program(s) and source code files

Program	Version	Date	File(s)
hyt_znw7.out			hyt_znw7.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

Example

```

hydra basic;

// -----
//
// Userexit Time Sheet
//
// -----

import DLG_DATA      char(10000);
import LIST_LINE_NR long;
import LIST_DATA      char(10000);

variable header_kopf char(2000);
variable header_stmp char(2000);
variable header_mola char(2000);

//-----
long modify_list_file_line()
{
    dummy char(10000);
    ret long;

    ret = 0;

    //-- Header data -----
    if( LIST_DATA[1,7] = "KENN_K|" )
    {
        dprint( "--> Header data" );
        header_kopf = LIST_DATA;
    }
    else if( LIST_DATA[1,2] = "K|" )
    {
        dprint( "--> Header data" );
        // Action ...
        // x = get_list_column( header_kopf, LIST_DATA, "XXX" );
        // LIST_DATA = set_list_column( header_kopf, LIST_DATA, "XXX",  x + 2 );
    }
    //-- Clockings -----
    else if( LIST_DATA[1,7] = "KENN_S|" )
    {
        dprint( "--> Header of clockings" );
        header_stmp = LIST_DATA;
    }
    else if( LIST_DATA[1,2] = "S|" )
    {
        dprint( "--> Clocking data" );
        // Action ...
        // x = get_list_column( header_stmp, LIST_DATA, "XXX" );
        // LIST_DATA = set_list_column( header_stmp, LIST_DATA, "XXX",  x + 2 );
    }
    //-- monthly wage types -----
    else if( LIST_DATA[1,7] = "KENN_M|" )
    {
        dprint( "--> Header of the monthly wage types" );
        header_mola = LIST_DATA;
    }
}

```

```

    }
    else if( LIST_DATA[1,2] = "S|" )
    {
        dprint( "--> Monthly wage types" );
        // Action ...
        // x = get_list_column( header_mola, LIST_DATA, "XXX" );
        // LIST_DATA = set_list_column( header_mola, LIST_DATA, "XXX", x + 2 );
    }
    //-----

    dummy = callback( "LISTOUTPUT", LIST_DATA clipped );

    return ret;
}

//-----
//long append_list_file()
//{
//    return 0;
//}

/*-----*/
/* Main function (only for test purposes) */
/*-----*/
long main()
{
    return modify_list_file_line();
}

/*-----*/

```

5.9.14 HR master data download from SAP

The section dealing with the MLE product group describes the user exit for downloading HR master data from SAP via HR-PDC.

5.9.15 Uploading time events to SAP

The section dealing with the MLE product group describes the user exit for uploading time events (clocking records) to SAP via HR-PDC.

5.10 Server user exits: CAQ

5.10.1 User exits in the context of operation and order events

5.10.1.1 Searching for a QM operation (after logging on an operation)

Name of user exit

ade_caq_a_an_search_op.hsc

Keywords

After logging on an operation, you want to identify if a QM operation is already available.

Function

- ❖ Call this user exit if a new QM operation is to be generated after logging on an operation.
- ❖ Requirement: enter the column "PAN_AU/A_AN" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that QM operations are created when generating inspection requirements.
- ❖ Searching for the QM operation is skipped if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA. In this case, no QM operation will be generated either.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
-----------	------	---------

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating the QM operation

5.10.1.2 Generating a QM operation (after logging on an operation)

Name of user exit

ade_caq_a_an_create_op.hsc

Keywords

A QM operation (not yet existing) is to be generated, once an operation is logged on.

Function

- ❖ Call this user exit if you want to generate a new QM operation and searching for a QM operation failed (see previous section).
- ❖ Requirement: enter the column "PAN_AU/A_AN" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that QM operations are created when generating inspection requirements.
- ❖ The QM operation is not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating the QM operation

5.10.1.3 Searching for a QM operation (by changing the order status)

Name of user exit

ade_caq_ast_change_search_op.hsc

Keywords

After changing the order status, you want to identify if a QM operation is already available.

Function

- ❖ Call this user exit if a new QM operation is to be generated after changing the order status.
- ❖ Requirement: enter the column "PAN_AU/A_ST" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that QM operations are created when generating inspection requirements.
- ❖ Searching for the QM operation is skipped if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA. In this case, no QM operation will be generated either.
- ❖ Please note:
License "ADE-CAQ2" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the changed order status)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating the QM operation

5.10.1.4 Generating a QM operation (by changing the order status)

Name of user exit

ade_caq_ast_change_create_op.hsc

Keywords

A QM operation (not yet existing) is to be generated, once the order status has been changed.

Function

- ❖ Call this user exit if you want to generate a new QM operation and searching for a QM operation failed (see previous section).
- ❖ Requirement: enter the column "PAN_AU/A_ST" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that QM operations are created when generating inspection requirements.
- ❖ The QM operation is not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the changed order status)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating the QM operation

5.10.1.5 Generating inspection requirements (triggered by operation logon)

Name of user exit

ade_caq_a_an_create_pan.hsc

Keywords

An inspection requirement is to be generated, once an operation has been logged on.

Function

- ❖ Call this user exit, if you want to generate an inspection requirement, once an operation has been logged on.
- ❖ The inspection requirement is not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection requirements

5.10.1.6 Inspection requirements and inspection steps generated (triggered by operation logon)

Name of user exit

ade_caq_a_an_create_pan_and_pau_complete.hsc

Keywords

An inspection requirement has been generated after the logon of an operation. Then (if configured):

- all QM operations are created
- all associated inspection steps are created.

Function

- ❖ This user exit is called, if you generate an inspection requirement including all associated inspection steps after logging on an operation.
- ❖ This user exit is not called, if the inspection plan setting "generate inspection step + characteristic" is set to "operation logon". The parameter "generate inspection step and characteristic" must be set to "when generating the inspection requirement".
- ❖ The user exit is executed for every inspection requirement that is created (except for the above-mentioned restrictions).
This user exit is not executed if you do not generate the inspection requirement (e.g. using SKIP=1).
- ❖ Please note:
License "ADE-CAQ2" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72	8.1.1.186		hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection requirements
DLG_RET	C32000	Return data of generating inspection requirements
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

5.10.1.7 Generating inspection steps for an operation (when generating inspection requirements, triggered by operation logon)

Name of user exit

ade_caq_a_an_create_pau_pan_create.hsc

Keywords

You want to generate one or several inspection steps, once an operation has been logged on.

This user exit only applies to inspection plans if

- these plans are configured to generate inspection steps upon the generation of inspection requirements.

Function

- ❖ Call this user exit, if you want to generate one or several inspection steps, once an operation has been logged on.
- ❖ Requirement: enter the columns "PAN_AG/A_AN" or "PAN_AU/A_AN" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that characteristics are created when generating inspection requirements.
- ❖ Inspection steps are not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection steps.

5.10.1.8 Generating inspection steps for an operation (when logging on the OP, triggered by operation logon)

Name of user exit

ade_caq_a_an_create_pau_a_an.hsc

Keywords

You want to generate one or several inspection steps, once an operation has been logged on.

This user exit only applies to inspection plans if:

- these plans are configured to generate inspection steps when the operation is logged on.

Function

- ❖ Call this user exit, if you want to generate one or several inspection steps, once an operation has been logged on.
- ❖ Requirement: enter the column "PAN_AG/A_AN" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that characteristics are created when the operation is logged on.
- ❖ Inspection steps are not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "ADE-CAQ2" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection steps.

5.10.1.9 Generating inspection requirements (triggered by changing the order status)

Name of user exit

ade_caq_ast_change_create_pan.hsc

Keywords

An inspection requirement is to be generated, once the order status has been changed.

Function

- ❖ Call this user exit if an inspection requirement is to be generated after changing the order status.
- ❖ The inspection requirement is not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "ADE-CAQ2" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the changed order status)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection requirements

5.10.1.10 Generating inspection steps for an operation (when generating inspection requirements, triggered by order status change)

Name of user exit

ade_caq_ast_change_create_pau.hsc

Keywords

You want to generate one or several inspection steps, once the order status has been changed.

This user exit only applies to inspection plans if

- these plans are configured to generate inspection steps upon the generation of inspection requirements.

Function

- ❖ Call this user exit, if you want to generate one or several inspection steps, once the order status has been changed.
- ❖ Requirement: enter the column "PAN_AU/A_ST" in the "area/order type configuration".
- ❖ In addition, you have to configure the corresponding inspection plan in such a way that characteristics are created when generating inspection requirements.
- ❖ Inspection steps are not generated if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.
- ❖ Please note:
License "**ADE-CAQ2**" is required.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the changed order status)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating inspection steps.

5.10.1.11 Logging on an inspection step to a workplace (triggered by operation logon)

Name of user exit

ade_caq_a_an_dev_logon.hsc

Keywords

After logging on an operation, you want to log on an inspection step to a workplace.

Function

- ❖ Call this user exit, if you want to log on an inspection step to a workplace, once an operation has been logged on.
- ❖ The inspection step is not logged on to the workplace if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logon)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog logging on the inspection step to the workplace

5.10.1.12 Logging off an inspection step from a workplace (triggered by operation logoff)

Name of user exit

ade_caq_a_ab_dev_logoff.hsc

Keywords

You want to log off an inspection step from a workplace after logging off an operation.

Function

- ❖ Call this user exit, if you want to log off an inspection step from a workstation, once an operation has been logged off.

- ❖ The inspection step is not logged off from the workplace if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the operation logoff)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog logging off the inspection step from the workplace

5.10.1.13 Interrupting an inspection step at a workplace (triggered by an interrupted operation)

Name of user exit

ade_caq_a_ab_dev_interrupt.hsc

Keywords

You want to interrupt an inspection step at a workplace after interrupting an operation.

Function

- ❖ Call this user exit, if you want to interrupt an inspection step at a workstation, once an operation has been interrupted.
- ❖ The inspection step is not interrupted at the workplace if the user exit sets the dialog parameter SKIP=1 in the parameter DLG_DATA.

Program(s) and source code files

Program	Version	Date	File(s)
---------	---------	------	---------

Program	Version	Date	File(s)
hymwcaq72			hymwcaq72.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the interrupted operation)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog interrupting the inspection step at the workplace

5.10.2 User exit for inspection requirements and inspection steps

5.10.2.1 Filtering of characteristics when generating a new inspection requirement

Name of user exit

caq_after_pan_insert.hsc

Keywords

This user exit is immediately available, once a new inspection requirement has been generated.

Function

- ❖ The user exit is called, when the inspection requirement has just been created. But the corresponding detail data are not yet available.
- ❖ Assign the dialog parameter CPAN.MOD:AFOLIST to the parameter DLG_DATA to specify the characteristics (of the previously identified inspection plan) you want to integrate to generate the substructures. Use commas to separate the OP sequences of characteristics.
- ❖ This user exit is only reasonable if
 - you directly generate the inspection requirement and all substructures using the dialog CPAN.INSERT.
 You should not use this user exit if inspection requirements are generated by the ADE/CAQ integration (e.g. by logging on the operation or by changing the order status).

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_cpan.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Reference to the currently generated inspection requirement. The following parameters are included: - CPAN.RECTYP - CPAN.BER - CPAN.PANNR

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data parameters optionally restricting the search for characteristics. Only the parameter CPAN.MOD:AFOLIST is supported here.

5.10.2.2 Generating a QM operation by creating an inspection step

Name of user exit

ade_caq_cpau_ag_insert.hsc

Keywords

After generating an inspection step, you want to create a corresponding QM operation.

Function

- ❖ If you change the parameter DLG_DATA, you can affect the generation of the QM operation.
- ❖ The escalation ANR.INSERT_CAQ_AG will not be generated if
 - the parameter DLG_DATA includes the dialog parameter CPAU.MOD:IGNOREADEERR with value 1 and
 - errors occur while the QM operation is generated.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_cpau.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the generation of the QM operation)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog completing the inspection step

5.10.2.3 Completing inspection steps by evaluating the inspection requirement

Name of user exit

caq_cpan_beur_call_cpau_abs.hsc

Keywords

You want to complete an inspection step, once an inspection requirement has been evaluated.

Function

- ❖ An inspection requirement is evaluated, for example, as part of completing the inspection requirement.
- ❖ If you change the parameter DLG_DATA, you can affect completion of the inspection step.
- ❖ You cannot prevent the inspection step from being completed.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_cpan.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog leading to the evaluation of the inspection requirement)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog completing the inspection step

5.10.2.4 Completing an inspection step by logging it off from a workstation

Name of user exit

caq_cpau_devab_call_cpau_abs.hsc

Keywords

You want to complete the inspection step, once this inspection step has been logged off from a workstation.

Function

- ❖ If you change the parameter DLG_DATA, you can affect completion of the inspection step.
- ❖ You cannot prevent the inspection step from being completed.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_cpau.c

Import parameter

Parameter	Type	Content
DLG_DATA_SRC	C32000	Triggering dialog (dialog logging off the inspection step from the workstation)

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog completing the inspection step

5.10.3 User exit for entries in the CAQ number pool

5.10.3.1 Generating escalations upon completion

Name of user exit

caq_cpanump_esk_completed.hsc

Keywords

You want to generate an escalation, once a number pool entry has been completed (e.g. an inspection point).

Function

- ❖ This user exit is called, before the escalation CPANUMP.COMPLETED is triggered due to the completion of a number pool entry.
- ❖ Change the parameter DLG_DATA, for example, in order to add new dialog parameters to the escalation.
- ❖ You cannot prevent the escalation from being generated.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_cpanump.c

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog generating escalations

5.10.3.2 Generating entries

5.10.3.3 Editing entries

5.10.4 User exits calculating measured values for characteristics

5.10.4.1 Identify variable values

Name of user exit

caq_mm_ber_var_ersetzen.hsc

Keywords

You want to identify a measured value for a calculated characteristic. In this context, you want to determine a value for an unknown variable.

Function

- ❖ Call this user exit if you want to identify the value for a variable called VAR.
- ❖ Use the following syntax to enter the variables in the formula: VAR:<Reference>:<Value>
- ❖ The export variable RET_DOUBLE returns the calculation result.
- ❖ If the parameter ERR_TEXT includes an error message, the system returns details about errors that occurred while identifying the value.

Program(s) and source code files

Program	Version	Date	File(s)
hymwcaq72			b_caqbase.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Details about the variable are indicated here: • VAR:NAME → variable name (fixed: VAR) • VAR:IDTYP → reference of the variable • VAR:VALUE → reference value

Export parameter

Parameter	Type	Content
ERR_TEXT	C250	Error message
RET_DOUBLE	double	Identified variable value

5.10.5 Server user exits for MDI measurement recording

5.10.5.1 Characteristic found

Name of user exit

hy_cmdilrv_characteristic_found.hsc

Keywords

A characteristic relevant to MDI has been found.

Function

- ❖ This user exit is called if a relevant characteristic was found during the server-supported collection of MDI measured values.
- ❖ If you change the parameter MDICFG, you can adjust the associated MDI configuration.
- ❖ If a blank string is assigned to the parameter MDICFG, the detected characteristic will be skipped when collecting MDI measured values.

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.
RECTYP	C50	Data type of the detected characteristic.
BER	C10	Area of the detected characteristic.
PANNR	Long	Inspection requirement number of the detected characteristic.
PAUNR	Long	Inspection step number of the detected characteristic.
OP sequence	Long	OP sequence of the detected characteristic.

Parameter	Type	Content
PMID	C50	Number of the test equipment assigned to the detected characteristic.
RESFAM	C20	Resource family assigned to the detected characteristic.

Export parameter

Parameter	Type	Content
MDICFG	C50	Use this MDI configuration to query measured values.

5.10.5.2 Before saving a measured value

Name of user exit

hy_cmdilrv_before_capture.hsc

Keywords

A dialog has been set up for a measured value to be saved. The measured value is about to be stored.

Function

- ❖ This user exit is called before a measured value is stored in the server-supported collection of MDI measured values.
- ❖ If you change the parameter DLG, you can adjust the measured value to be saved.
- ❖ The measured value will not be saved if you assign a blank string to the parameter DLG. In this case, the measured value will not be deleted from the MDI server buffer.

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.
MDIDATA	C32000	Data of the measured value to be saved. The associated MDI server provides this data.

Export parameter

Parameter	Type	Content
DLG	C32000	Dialog saving the measured value.

5.10.5.3 After saving a measured value

Name of user exit

hy_cmdilrv_after_capture.hsc

Keywords

An attempt was made to save a measured value.

Function

- ❖ This user exit is called after a (possibly failed) attempt to save a measured value as part of the server-supported collection of MDI measured values
- ❖ You can assign the parameter SKIP_ERRLOG to a value greater than 0 if you do not want to record a failed attempt in the error log.

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.
MDIDATA	C32000	Data of the measured value to be saved. The associated MDI server provides this data.
DLG	C32000	Dialog used for saving the measured value.
RETDLG	C32000	Return string indicating that the measured value has been saved.

Export parameter

Parameter	Type	Content
SKIP_ERRLOG	long	Flag suppressing output in the error log.

5.10.5.4 Before generating an inspection point

Name of user exit

hy_cmdilrv_before_ip_creation.hsc

Keywords

A dialog has been set up for a new inspection point. The inspection point is about to be saved.

Function

- ❖ This user exit is called before a new inspection point is created for the server-supported collection of MDI measured values.
- ❖ If you change the parameter DLG, you can adjust the inspection point to be generated.
- ❖ The inspection point will not be generated if you assign a blank string to the parameter DLG.

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.

Export parameter

Parameter	Type	Content
DLG	C32000	Dialog generating the inspection point.

5.10.5.5 After generating an inspection point

Name of user exit

hy_cmdilrv_after_ip_creation.hsc

Keywords

An attempt was made to generate an inspection point.

Function

- ❖ This user exit is called after a (possibly unsuccessful) attempt to create a new inspection point for the server-supported collection of MDI measured values.
- ❖ You can assign the parameter SKIP_ERRLOG to a value greater than 0 if you do not want to record a failed attempt in the error log.

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.
DLG	C32000	Dialog used for the generation of the inspection point.
RETDLG	C32000	Return string indicating that the inspection point has been generated.

Export parameter

Parameter	Type	Content
SKIP_ERRLOG	Integer	Flag suppressing output in the error log.

5.10.5.6 After completing the process for an inspection step

Name of user exit

hy_cmdilrv_after_inspectionstep_loop.hsc

Keywords

You have completed all activities to collect MDI measured values for an inspection step relevant to MDI.

Function

- ❖ This user exit is called up after all actions to save the measured values of an inspection step have been executed for the server-supported collection of MDI measured values.

- ❖ This user exit allows you to use different editing functions (e.g. complete inspection points).

Program(s) and source code files

Program	Version	Date	File(s)
hy_cmdilrv			hy_cmdilrv.c

Import parameter

Parameter	Type	Content
CMDPARAMS	C32000	Command line parameter contents of the program call This parameter is available to hy_cmdilrv only from version 8.1.1.17.
RECTYP	C50	Data type of the inspection step
BER	C10	Area of the inspection step
PANNR	Long	Inspection requirement number of the inspection step
PAUNR	Long	Inspection step number
AFO_LIST_MDI	C32000	Comma-separated list of OP sequences of inspection step characteristics relevant to MDI.

5.10.6 CAQ list extensions

5.10.6.1 control chart

Name of user exit

I_caq_controlchart_list_sql.hsc

Keywords

Requesting sample data to display control charts.

Function

This user exit is called when requesting sample data for displaying control charts.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out	8.1.1.608	2017-02-17	hyd_caql.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data used to request control chart data.

Export parameter

Parameter	Type	Content
SQL_REQUEST	C32000	SQL statement used to request sample data for displaying control charts. The request will be cancelled with RET=101 (no data available) if you clear this parameter.

5.10.6.2 Histogram

When requesting statistical data, you can choose to automatically identify how many classes the histogram should include.

For each class of the histogram, the system separately determines how many single values are included per class.

Separate SQL statements are executed for each of these requests. You can use one of the following two user exits to change these SQL statements.

User exits cannot affect additional SQL queries that are executed if you use option 1196.

Name of user exit

I_caq_histogram_get_classes_sql.hsc

Keywords

Identifying the number of classes for the histogram.

Function

This user exit is only executed if the number of classes displaying the histogram is not set by an optional parameter.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out	8.1.1.608	2017-02-17	hyd_caql.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data used to request histogram data.

Export parameter

Parameter	Type	Content
SQL_REQUEST	C32000	SQL statement used to identify the number of classes. The request will be cancelled with RET=101 (no data available) if you clear this parameter.

Name of user exit

I_caq_histogram_list_sql.hsc

Keywords

Identifying the number of single values included in a histogram class.

Function

This user exit is called when identifying data for each class included in the histogram.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out	8.1.1.608	2017-02-17	hyd_caql.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data used to request histogram data.

Export parameter

Parameter	Type	Content
SQL_REQUEST	C32000	SQL statement that is used to identify data for the current histogram class. The request will be cancelled with RET=101 (no data available) if you clear this parameter.

5.10.6.3 Statistics

Name of user exit

l_caq_statistics_list_sql.hsc

Keywords

Requesting data for online statistics calculation.

Function

This user exit is called when requesting data for online statistics calculation.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe/out	8.1.1.608	2017-02-17	hyd_caqi.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data used to request control chart data.

Export parameter

Parameter	Type	Content
SQL_REQUEST	C32000	SQL statement used to request data for online statistics calculation. The request will be cancelled with RET=101 (no data available) if you clear this parameter.

5.11 Server user exits: ESK

Note:

The documentation dealing with user exits for the escalation management describes further specific user exits for the escalation management.

5.11.1 Overriding KEY fields in the escalation configuration

Name of user exit

esk_handle_<DLG>.hsc

Replace the placeholder <ESKMSG.ESKID> with the escalation ID.

Example:

ANR.REGISTER_REMARK → esk_handle_anr_register_remark.hsc

Keywords

Controlling escalations.

Function: esk_identify_key_values

Escalation management allows you to define special key fields that prevent the escalation from being triggered again. If an escalation is encountered for the first time, no more escalations will be triggered until the escalation status changes. The file db_sql/hyeskregvar.lod loads these key fields into the HYDRA database.

Use this user exit to override the key fields already configured in the database. All KEY fields are cleared in the esk_handle_anr_register_remark.hsc example. Consequently, the ANR.REGISTER_REMARK escalation is triggered every time it occurs.

Program(s) and source code files

Program	Version	Date	File(s)
hyeskmgr.exe/out			hyeskmgr.c

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	ESKMSG.KEY:1 to ESKMSG.KEY:5
MSGTEXT	C1024	Message string of the escalation

5.11.2 Extension by cyclic requests (escalations)

Name of user exit

esk_cyc_event_<Event>.hsc

Replace the placeholder <EVENT> with the actual event.

Example:

ESK event = MST.STATUS_SUMMARY → <event> = MST_STATUS_SUMMARY (the dot in the ESK event is replaced with an underscore)

→ esk_cyc_event_MST_STATUS_SUMMARY.hsc

Key

words

New cyclic escalations

Function: process_event()

Subject to configuration, this function is called at regular intervals. In this function use the callback function PROCESS_ESCALATION to trigger the escalation.

Note:

The user exit function has to return 0, as otherwise the user exit is called every 10 seconds. If successful (RET=0), the user exit is only called every X minutes (subject to the configuration of the escalation).

Program(s) and source code files

Program	Version	Date	File(s)
hyeskcyc.exe			

Import parameter

Parameter	Type	Content
EVENT_ID	C40	Reference to a registered event
EVENT_NAME	C100	Description of the event
CYCL_DAYS	LONG	Number of days between 2 cycles
CYCL_SECS	LONG	Number of seconds between 2 cycles
CYCL_INTERVALL	LONG	Interval
LAST_CHECK_DATE	C10	Date of the last check
LAST_CHECK_TIME	LONG	Time of the last check
REFERENCE	LONG	Reference to the escalation event

Export parameter

Parameter	Type	Content

Note:

The user exit is only called unless the EVENT has been processed by default.

Example:

```

hydra basic;

/*****
 * Script : esk_cyc_event_u_anr_target_quantity_reached.hsc
 * Deser. : Userexit cycle escalations
 *      Check for operations where target quantity is reached.
 *
 * $Revision: 1.1 $
 * $Date: 2017/01/01 01:01:01 $
 *****/
* History
* $Log: esk_cyc_event_anr_u_target_quantity_reached.hsc $
* Revision 1.1 2017/01/01 01:01:01 nn
* Initial revision
*
*****/

import EVENT_ID      char(40);
import EVENT_NAME    char(100);
import CYCL_DAYS     long;
import CYCL_SECS     long;
import CYCL_INTERVALL long;
import LAST_CHECK_DATE char(10);
import LAST_CHECK_TIME long;
import REFERENCE     long;

variable MESSAGE     char(410);

//-----
long callback_process_escalation()
{
    variable ret      long;
    variable DLG_DATA char(511);
    ret = 0;

    DLG_DATA = "DLG=ESKMSG.INSERT|ESKMSG.ESKID=" ||

```

```

        (EVENT_ID stripped) || "|" ||
        (MESSAGE stripped) || "|" ||
        "ESKMSG.VERWEIS=" || (REFERENCE using "<<<<<&" || "|");

    ret = CallBack("PROCESS_ESCALATION", (DLG_DATA stripped));
    return ret;
}

//-----
long process_event()
{
    ret          long;
    machine_id   char(20);
    start_date   date;
    start_time   long;
    prot_dat     date;
    prot_zeit    long;
    bmk_07       long;
    bmk_08       long;
    yield_quan   long;
    target_quan  long;
    res_tool     char(40);
    article      char(40);
    operation_id char(40);

    ret = 0;

    dprint(">>>>>>> esk_cyc_event.hsc - process_event() - Event[" || (EVENT_ID stripped) || "]);

    MESSAGE = set_bapi_val(MESSAGE, "ANR.MNR", ""); // Machine no.
    MESSAGE = set_bapi_val(MESSAGE, "ANR.ANR", ""); // Order/Operation
    MESSAGE = set_bapi_val(MESSAGE, "ANR.WNR", ""); // Tool no (ID of main ressource with ressource type WNR)
    MESSAGE = set_bapi_val(MESSAGE, "ANR.ATK", ""); // Article no (of operation)
    MESSAGE = set_bapi_val(MESSAGE, "ANR.SGR:GUT", ""); // Target quantity of operation
    MESSAGE = set_bapi_val(MESSAGE, "ANR.EGR:GUT", ""); // Current yield quantity of operation at the time of escalation
    MESSAGE = set_bapi_val(MESSAGE, "ANR.DATB", ""); // (latest) Time of log on of operation (date and time)
    MESSAGE = set_bapi_val(MESSAGE, "ANR.ZEIB", "");

    // Check on all running operations
    sqlexec
    (
        " declare esk_yield_curs cursor for " ||
        " select hb.prot_dat, " ||
        "       hb.prot_zeit, " ||
        "       hb.start_date, " ||
        "       hb.start_time, " ||
        "       hb.subkey1, " ||
        "       ast.gut_pri, " ||
        "       ab.operation_id, " ||
        "       ab.article, " ||
        "       ab.soll_menge_pri, " ||
        "       ab.res_tool " ||
        " from hybuch hb, " ||
        "       auftrag_status ast, " ||
        "       auftrags_bestand ab " ||
        " where hb.key_type = 'A' " ||
        "       and hb.subkey2 = ast.operation_id " ||
        "       and ab.operation_id = ast.operation_id " ||
        " ;" );
    ret = sqlcode();

    if (ret = 0)
    {
        sqlexec("open esk_yield_curs;");
        ret = sqlcode();

        if (ret != 0)
        {
            dprint("ERROR: open esk_yield_curs - [" || (ret using "<<<<<&" || "]);
        }

        while( ret = 0 )
        {
            sqlexec("fetch esk_yield_curs;");
            ret = sqlcode();

            if ((ret != 0) and (ret != 100))
            {
                dprint("ERROR: fetch esk_yield_curs - [" || (ret using "<<<<<&" || "]);
            }

            if (ret = 0)
            {
                into( prot_dat,
                    prot_zeit,
                    start_date,
                    start_time,
                    machine_id,
                    yield_quan,
                    operation_id,
                    article,
                    target_quan,
                    res_tool );

                // Check on overproduction
                dprint( "AG [" || operation_id clipped || "] Current yield: " || yield_quan using "<<<<<&" ||
                    " ; Target quantity : " || target_quan using
                    "<<<<<&");
            }
        }
    }
}

```

```

        if(yield_quan > target_quan)
        {
            dprint("Target quantity was reached !!");
            MESSAGE = set_bapi_val(MESSAGE, "ANR.MNR", machine_id clipped); // Machine no.
            MESSAGE = set_bapi_val(MESSAGE, "ANR.ANR", operation_id clipped); // Order/Operation
            MESSAGE = set_bapi_val(MESSAGE, "ANR.WNR", res_tool clipped); // Tool no (ID of main ressource with ressource type
WNR)
            MESSAGE = set_bapi_val(MESSAGE, "ANR.ATK", article clipped); // Article no (of ooperation)
            MESSAGE = set_bapi_val(MESSAGE, "ANR.SGR:GUT", target_quan); // Target quantity of operation
            MESSAGE = set_bapi_val(MESSAGE, "ANR.EGR:GUT", yield_quan); // Current yield quantity of operation at hte time of
escalation
            MESSAGE = set_bapi_val(MESSAGE, "ANR.DATB", start_date); // (latest) Time of log on of operation (date and
time)
            MESSAGE = set_bapi_val(MESSAGE, "ANR.ZEIB", start_time);

            ret = callback_process_escalation();

            if (ret = 100)
                ret = 0;
        }
    }
}

sqlxexec("close esk_yield_curs;");
}
else
{
    dprint("ERROR: declare esk_yield_curs cursor - [" || (ret using "<<<<<&") || "]);
}

dprint("<<<<<<<< esk_cyc_event.hsc - process_event() - Event[" || (EVENT_ID stripped) || "]);

return 0; // Function always has to return 0. Otherwise execution time of escalation is not updated-
}

//-----
long main() // Only for test
{
    variable ret long;
    ret = 0;
    return ret;
}

//-----

```

5.11.3 Modify escalation data before processing

Name of user exit

esk_handle_< ESKMSG.ESKID >.hsc

Replace the placeholder <ESKMSG.ESKID> with the actual escalation ID.

Keywords

Modifying escalation data.

Function: esk_modify_message

Use this user exit to easily customize standard escalations. For this purpose, you can modify escalation data before it is actually evaluated and processed.

This user exit takes effect prior to the evaluation of the escalation, i.e. before different conditions are checked. Consequently, you can add specific information to the message string to make available customized selection criteria in the escalation configuration.

Note:

Currently, this user exit can be accessed from two different positions:

- 1) use the function ESKSendValidMessage() of the cyclic escalation agent
- 2) use the function ESKProcessMessageInsert() of the escalation manager.

In case of a cyclic escalation, this user exit might be executed twice (at first by the escalation agent and, if the condition was true, by the escalation manager as well). A corresponding mode is transferred in the parameter DLG_DATA (see below) to be able to recognize in the user exit from which function it was accessed.

Program(s) and source code files

Program	Version	Date	File(s)
hyeskmgr.exe/out			hyeskmgr.c
hyeskcyc.exe/out			hyesklib.c

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	The acronym MOD indicates the function from which the user exit was called. This is only for information purposes. MOD: Possible values: - SEND_VALID_MESSAGE (from hyeskcyc) - SEND_MESSAGE (from hyeskmgr)
MSGTEXT	C1024	Message string of escalation that can be modified.

5.11.4 Changing e-mail address(es)

Name of user exit

esk_handle_main.hsc

Keywords

Modifying e-mail addresses

Function: esk_modify_mail_address

Use this user exit to customize the data required for sending e-mails.



At the moment this user exit is only used for sending an e-mail to an SMS gateway (ESK-SMSGW). This SMS gateway then sends the contents of the e-mail as SMS (text message) to the mobile number (mostly indicated in the subject line of the e-mail).

You can add further types of dispatch to this user exit in future. But you have to take note of the parameter NOTIFICATION.TYPE before you change data.

```
...
long esk_modify_mail_address()
{
    if( get_bapi_val( DLG_DATA, "NOTIFICATION.TYPE" ) = "SMS" )
    {
        // Change content of MAIL.* parameters for sending SMS via e-mail gateway
        DLG_DATA = set_bapi_val( DLG_DATA, "MAIL.FROMNAME", "CompanyNotifier" );
        // ...
    }

    return 0;
}
...
```

Program(s) and source code files

Program	Version	Date	File(s)
hyeskmgr.exe/out			hyeskmgr.c

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	You can change the following parameters in the dialog string (the default values for ESK-SMSGW are indicated in parentheses): MAIL.FROM=e-mail address of the escalation manager from the basic settings of escalation

Parameter	Type	Content
		<i>management (esk_setup.smtp_sender)</i> - MAIL.FROMNAME= <i>Name of the escalation manager (esk_setup.smtp_sendername or program description of hyeskmgr.exe/out)</i> - MAIL.TO=<i>e-mail address of the SMS gateway from path configuration (hy_path.p_url_path)</i> - MAIL.TONAME=<i>the recipient's name from HR master data</i> - MAIL.SUBJECT=<i>Mobile number (company) from HR master data</i> - MAIL.HOSTNAME=<i>SMTP server from path configuration (hy_path.p_host)</i> - MAIL.PORT=<i>SMTP port from path configuration (hy_path.p_port)</i> DLG_DATA also includes the following values (that cannot be changed): - NOTIFICATION.TYPE= Possible values: "SMS": Indicates that an SMS is sent by sending an e-mail to the SMS gateway. - PNR=<i>The recipient's personnel number (personalstamm. personalnummer)</i>
MSGTEXT	C1024	The message string of the escalation that cannot be modified

5.11.5 Save escalation message

Name of user exit

esk_handle_< ESKMSG.ESKID >.hsc

Replace the placeholder <ESKMSG.ESKID> with the escalation ID. Replace dots in the escalation ID with underscores in the user exit name.

Example:

ANR.REGISTER_REMARK → esk_handle_anr_register_remark.hsc

Example script:

```

long esk_save_message_ext ()
{
    DLG_DATA = set_bapi_val(DLG_DATA, "SAVE.MSG:EXT", "J");

    return 0;
}

```

Keywords

Controlling escalations.

Function: esk_save_message_ext

Use this user exit to specify if the complete escalation message is saved in the table [esk_event_msgext](#). By default, the escalation message is not saved.

You can save up to a maximum of 1000 characters.

Note: Once the escalation message has been archived, the archiving process deletes the data records pertaining to this escalation from the table [esk_event_msgext](#).

Program(s) and source code files

Program	Version	Date	File(s)
hyeskmgr.exe/out	8.1.1.62	2016-09-15	hyeskmgr.c

Import parameter

Parameter	Type	Content
-	-	-

Export parameter

Parameter	Type	Content
DLG_DATA	C32000	If SAVE.MSG:EXT=J is returned, the escalation string is saved.
MSGTEXT	C1024	Not used

5.12 Server user exits: HYD-SIG

5.12.1 Additional information for signature check/collection

Name of user exit

hyd_sig_getdata.hsc

Keywords

Select variables for conditions of the signature check

Function: getdata

As part of the signature check (HYD-SIG) you can specify conditions defining if the action (login, data maintenance) requires certain signatures. The system identifies the values for the placeholders in the conditions from the dialog data. The *getdata()* function of this user exit is called if the dialog data do not provide information on all placeholders. Use the *getdata()* function to determine the value for such placeholders. For example, the function selects the value from the database.

For further information about the signature check/collection, refer to the document dealing with "signature collection".

Note: By default, HYDRA provides a sample implementation of the user exit:

<Instance>\1\custom\userexit\hyd_sig_getdata_0.hsc

You can activate this user exit by renaming: <Instance>\1\custom\userexit\hyd_sig_getdata.hsc.

Program(s) and source code files

Program	Version	Date	File(s)
hymw.exe			hyd_sig.c
hyadeabg.exe			hyd_sig.c
b_work.dll			hyd_sig.c
mle72imp.exe			hyd_sig.c
tages_aw.exe			hyd_sig.c
res_transfer.exe			hyd_sig.c

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Sent dialog data
VARNAME	C255	Variable name

Export parameter

Parameter	Type	Content
VARDATA	C40	Value for the variable name (return value)

5.13 Server user exits: PDV

5.13.1 Specification list search

The parameters for searching for the specification list are transferred to the user exit

`hp_cfgm_mod_spec_search.hsc` at first. Consequently, you can change the single parameters or even disable searching for the specification list.

Name of user exit

`hp_cfgm_mod_spec_search.hsc`

Keywords

User exit, which is called by the configuration monitor PDV 7.2, to modify the parameters of the specification list search or to deactivate the search.

Function

- ❖ This user exit is called, before searching for the specifications of a process parameter from the specification list.
- ❖ The user exit supports system calls and SQL queries.
- ❖ The user exit provides the following options:
 - => Change the parameters that are provided in the dialog format in the variable `CURRENT_LINE`
 - => Use the export variable `SKIP_SEARCH` to determine whether or not the configuration monitor is supposed to search for a specification list entry.

Program(s) and source code files

Program	Version	Date	File(s)
hp_cfgm			hp_cfgm.cpp / h, PPHandler.cpp / h, Specification.cpp / h

Export parameter

Parameter	Type	Content
CURRENT_LINE	C8000	Current parameters in the dialog format that are used for the specification list search.
SKIP_SEARCH	long	Flag specifying whether or not the configuration monitor should search for a specification list entry. 1 = search other = do not search

5.13.2 Writing interface file for data collection

The database objects call the user exit: `hp_cfgm_mod_list.hsc` before returning the string.
Description:

Name of user exit

`hp_cfgm_mod_list.hsc`

Keywords

User exit, which is called by the configuration monitor PDV 7.2, to modify the interface file for PDV 7.2 data collection (modify, extend, reduce).

Function

- ❖ This user exit is called before writing the interface file for PDV 7.2 data collection.
- ❖ The user exit supports system calls and SQL queries.
- ❖ The user exit provides the following options:
 - => Change a line of the file. => The data row to be changed is included in the export variable CURRENT_LINE.
 - => The current context of the row can be extracted from the CONTEXT import variable, which is available in the dialog data format.
 - > Use the IS_HEADER import variable to check whether CURRENT_LINE is a header or data row.
 - > The export variable WRITE_LINE specifies whether or not the configuration monitor is supposed to write the current row.

Program(s) and source code files

Program	Version	Date	File(s)
---------	---------	------	---------

Program	Version	Date	File(s)
hp_cfgm			hp_cfgm.cpp / h, PPHandler.cpp, Event.cpp / h, LogicChannel.cpp / h, ProcessParameter.cpp / h, Setup.cpp / h , GlobalInformation.cpp / h

Import parameter

Parameter	Type	Content
CONTEXT	C8000	Dialog with current data context
IS_HEADER	long	Defines whether CURRENT_LINE is a header (IS_HEADER = 1) or if it is a data row

Export parameter

Parameter	Type	Content
WRITE_LINE	long	Flag specifying whether or not the line is to be written in the interface file; yes or no: (1= yes, other = no)
CURRENT_LINE	C8000	Returned row that the configuration monitor writes 1:1 to the interface file.

5.14 Server user exits: WRM

5.14.1 Control mounting of resources (RES_EIN)

Name of user exit

wrm_res_ein_wnr.hsc

Keywords

Check mounting of resources for validity and errors and define subsequent status.

Function

Use this user exit to intervene in the default processing of the RES_EIN dialog and to:

- reject the mounting/installation and
- define a subsequent status for the sub-resource

You can change the export parameter *RETCODE* in the function *action_allowed()*. This parameter directly affects validation of the dialog. If *RETCODE* != 0 is set, processing of RES_EIN is interrupted with this error code!

You can change the export parameters RESSTA_T und RESSTA_WL in the function *define_new_status()*. The export parameters RESSTA_T and RESSTA_WL can be changed by the function *define_new_status()*.

Note:

If the initiated status change fails (e.g. as an invalid status is set), processing will not be interrupted and no error message will be displayed. Consequently, the user exit is responsible for setting valid statuses!

Program(s) and source code files

Program	Version	Date	File(s)
hymwwrm72.dll/so			D_res_ein.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data
RESSTA_M	long	Current resource status of the parent resource

Export parameter

Parameter	Type	Content
RESSTA_T	long	Resource status of the sub-resource. Is assigned to the current status when called. The resource status of the sub-resource is only set to RESSTA_T if the parameter RESSTA_WL=1 is set.
RESSTA_WL (is only evaluated in the function <i>define_new_status()</i>)	long	Controls whether the status of the daughter resource is to be changed or not :1 : 1 : Change status.

Parameter	Type	Content
		: Do not change status
RETCODE (is only evaluated in the function <i>action_allowed()</i>)	long	Defect code If you set a value !=0, processing of the RES_EIN dialog is cancelled with this error code!

5.14.2 Control demounting of resources (RES_AUS)

Name of user exit

wrm_res_aus_wnr.hsc

Keywords

Check demounting of resources for validity and define subsequent status.

Function

Use this user exit to intervene in the default processing of the RES_AUS dialog and to:

- reject the demounting/removal and
- define a subsequent status for the sub-resource

You can change the export parameter *RETCODE* in the function *action_allowed()*. This parameter directly affects validation of the dialog. If *RETCODE* != 0 is set, processing of RES_AUS is interrupted with this error code!

You can change the export parameters *RESSTA_T* and *RESSTA_WL* in the function *define_new_status()*. If the parameter *RESSTA_WL*=1 is set:

- the resource is demounted/removed and
- the status of the sub-resource is set to the value defined in the export parameter *RESSTA_T*!

Note:

If the initiated status change fails (e.g. as an invalid status is set), processing will not be interrupted and no error message will be displayed. Consequently, the user exit is responsible for setting valid statuses!

Program(s) and source code files

Program	Version	Date	File(s)
hymwwrm72.dll/so			D_res_aus.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data
RESSTA_M	long	Current resource status of the parent resource

Export parameter

Parameter	Type	Content
RESSTA_T	long	Resource status of the sub-resource. Is assigned to the current status when called. The resource status of the sub-resource is only set to RESSTA_T if the parameter RESSTA_WL=1 is set.
RESSTA_WL (is only evaluated in the function <i>define_new_status()</i>)	long	Controls whether the status of the daughter resource is to be changed or not :1 : 1 : Change status. : Do not change status
RETCODE (is only evaluated in the function <i>action_allowed()</i>)	long	Defect code If you set a value !=0, processing of the RES_AUS dialog is cancelled with this error code!

5.15 Server user exits: MES-Cockpit

5.15.1 Exporter – extension of existing objects

Name of user exit

hybkfexp_<object name>.hsc

Replace <object name> with one of the three default objects “workplace”, “order” or “pparam”.

Keywords

Adding individual KPIs to the MESC exporter objects.

Function

Use this user exit to add your own basic KPIs to the XML files that MESC exporter creates for its objects. This is supported for the export of LOG and CUR data. Distinguish data provision in the user exit subject to the REF_TYPE.

The used function is *main()*.

Every block requires header data. These headers specify to which object the KPIs belong.

The HEADER callback function writes the header data. They always consist of a KEYNAME and a VALUE.

It is mandatory to write the following values in the header of the corresponding objects:

Workplace

KEY NAME	Type	Description
workplace_id	String	Workplace number
Costcenter	String	Cost center of the workplace
workplace_group	String	Machine group
ref_date	Date	Reference date (shift date) for basic KPIs.
shift_no	Short	Shift number

Order

KEY NAME	Type	Description
order_id	String	Order number
Costcenter	String	Cost center of order
ref_date	Date	Reference date (shift date) for basic KPIs.
shift_no	Short	Shift number

Pparam

KEY NAME	Type	Description
workplace_id	String	Workplace number
pparam_id	String	Process characteristic
ref_date	Date	Reference date (shift date) for basic KPIs.
shift_no	Short	Shift number

Example: Writing header data of a block for order 000047110010 with cost center BDE 100, shift 1 and reference date 08/20/2009

```

ret long;
data char(2000);

data="";
data = add_bapi_val( data, "KEYNAME", "order_id" );
data = add_bapi_val( data, "VALUE", "000047110010" );
ret = CallBack( "HEADER", data clipped );

data="";
data = add_bapi_val( data, "KEYNAME", "costcenter" );
data = add_bapi_val( data, "VALUE", "BDE100" );
ret = CallBack( "HEADER", data clipped );

data="";
data = add_bapi_val( data, "KEYNAME", "ref_date" );
data = add_bapi_val( data, "VALUE", "08/20/2009" );
ret = CallBack( "HEADER", data clipped );

data="";
data = add_bapi_val( data, "KEYNAME", "shift_no" );
data = add_bapi_val( data, "VALUE", 1 );
ret = CallBack( "HEADER", data clipped );

```

Use the callback function DATA to write the actual basic KPIs of a block.

There are two types of basic KPIs.

- KPIs without index (e.g. count_setup for setup processes)
- KPIs with index (e.g. count_dist_class with index 1, 2, ... for disturbance classes 1, 2, ...)

You need the two parameters KEYNAME and VALUE to write KPIs without index. The parameter INDEX is additionally required for KPIs with index.

Example: Writing a KPI *count_setup* without index and the value 5

```
data="";
data = add_bapi_val( data, "KEYNAME", "count_setup" );
data = add_bapi_val( data, "VALUE", 5 );
ret = CallBack( "DATA", data clipped );
```

Example: Writing a KPI *count_dist_class* with index 1 and value 3

```
data="";
data = add_bapi_val( data, "KEYNAME", "count_setup" );
data = add_bapi_val( data, "INDEX", 1 );
data = add_bapi_val( data, "VALUE", 3 );
ret = CallBack( "DATA", data clipped );
```

Use the callback function NEXTBLOCK to write the current block to the file and to continue with the next block.

Example:

```
ret = CallBack( "NEXTBLOCK", "" );
```

Program(s) and source code files

Program	Version	Date	File(s)
hybkfexp.exe/out			Hyd\src\hybkfx*, hyd\src\BKF*

Import parameter

Parameter	Type	Content
REF_TYPE	C10	Includes C for cur, L for log, A for acc and M for mdata
START_DATE	date	Includes the start date for the period of required data selection (only with log)
END_DATE	date	Includes the end date for the period of required data selection

Export parameter

none

5.15.2 Exporter: export separate objects

If you call the Exporter for an object that does not belong to the three default objects “workplace”, “order” and “pparam”, processing will be performed via a user exit.

In the user exit for separate objects you also have to differentiate by the imported reference type. But all four types are possible (L = log, C = cur, A = acc, M = mdata).

The process for the reference types L and C is identical to that of enhancing existing objects. Only header data is processed for the reference types A and M. Therefore, the process for enhancements can be applied. However, key figures do not have to be written. If so, they are ignored.

5.16 Server user exits: MDE

5.16.1 Machine status depends on parallel status

Name of user exit

The name of the user exit is defined in the configuration of the status type for the parallel status (table: RES_STATUS_TYPE).

Keywords

Identifying and, if necessary, setting the machine status subject to the active, parallel status.

Function

Use the dialogs RES_STB and RES_STE to set and/or finish the parallel statuses. Use the user exit described in this section to make assignments to the machine status subject to the available parallel status.

HYMW searches for the function *mst_determination()* in the configured user exit.

If the function returns a value greater than 0 via the variable *MST* and the variable *ERRORCODE* does not document an error (equals 0), a batch call sets the machine status using the dialog *M_MST*. In case an error occurs while processing the batch call, this error will be documented in the dialog error log and in the system log.

Program(s) and source code files

Program	Version	Date	File(s)
hymwmde72.dll / .so			d_res_stb_ste.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

Export parameter

Parameter	Type	Content
MST	long	Identified machine status The dialog M_MST sets the identified machine status if: - MST > 0 - ERRORCODE equals 0 - MNR is specified in the dialog data.
ERRORCODE	long	Defect code See MST

Function requested in the user exit

```
long mst_determination()
```

5.16.2 User exit after INSERT of MDE log record

Name of user exit

```
dd_mdep_afterinsert.hsc
```

Keywords

User exit that is requested with the MDE log record reference after having inserted the MDE log record.

Function

- ❖ The import variable DLG_DATA transfers dialog data.
- ❖ The user exit is also active if data is recalculated via the event maintenance.
- ❖ The user exit is always reinitialized. This means that changes in the user exit take effect immediately without having to restart the system on the server.
- ❖ You can use SQL and Bapi calls in the user exit.
- ❖ Timeout is set to 30 seconds.
- ❖ Warning: This user exit might have adverse effects on the performance of data collection.
- ❖ WARNING: The user exit does not ensure that the data changed in the log record will be available when uploading data to the PPS.

Program(s) and source code files

Program	Version	Date	File(s)
hymw			hyd_usrexit_usrfld.c

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data calling the dialog.
BUCH_MODE	INT	Posting mode (online/recalculation/...)
MDEP_VERWEIS	INT	Reference to the inserted log record
DLG	C10	Dialog that triggered the user exit.
EVENT	C10	Event that triggered the user exit.
MNR	C20	Machine from the event
PNR	C10	Person from the event
M_STATUS	INT	Machine status
DATUM ("date")	INT	Date
ZEI	INT	Time

Requested functions

Parameter	Content
long main()	

Parameter	Content
long status_korrektur()	

5.16.3 Extended status evaluation (hym_stat72)

Name of user exit

hym_stat72.hsc

Keywords

Adding further columns to the machine status evaluation.

Function: modify_list_file_line

You can add further fields to the generated list.

Program(s) and source code files

Program	Version	Date	File(s)
hym_stat72.exe/out	8.1.1.15	2013-11-08	hym_stat72.cpp

5.17 Server user exits: HLS

5.17.1 Configuration of planning component

Name of user exit

hls_planungskomp_setup.hsc

Keywords

Configuration of the planning component before scheduling/automatic planning.

Function

The planning component requires defined identifiers for specific algorithms or configuration settings. The planning component must be provided with these identifiers during initialization. Use the function `pk_set_konfig()` to transfer additional configurations to the planning component.

Example 1: Configure the scheduling of planned OPs like unplanned OPs.

```
DLG_DATA = add_bapi_val(DLG_DATA, "TERMINIERUNG_EINGEPLANTE_AG", "1");
```

Example 2: Algorithm A for scheduling

```
DLG_DATA = add_bapi_val(DLG_DATA, "SCHEDULEALGORITHM", "A");
```

Program(s) and source code files

Program	Version	Date	File(s)
b_hls.dll/so			b_hls.cpp

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

Export parameter

Parameter	Type	Content
-----------	------	---------

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

5.17.2 Saving planned data

Name of user exit

modify_hls_script.hsc

Keywords

Saving planned data of the client (MOC, console). This user exit is not used for planning/scheduling on the server.

Function

The system sends the BAPI HLS.SAVE when saving changes to planning. Use this user exit to change every saved operation.

Callback functions:

- BAPICALLEEXECUTE: any BAPI calls (from version 7.2.0.170/8.1.0.171)

Program(s) and source code files

Program	Version	Date	File(s)
b_hls.dll/so			b_hls.cpp

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data
PLANMODUS	C2	HLS.EINGEPLANT/HLS.UMGEPLANT/HLS.AUSGEPLANT
ANR	C40	Operation
MNR	C40	Workplace
MGRP	C40	Group

Parameter	Type	Content
WNR	C40	Tool
FERTVAR	LONG	Internal ID of the production variant

Export parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

Note: This BAPI can save a large amount of operations in one go. Therefore, it is essential for the performance to avoid time-consuming functions (SQL, BAPIs) and to look for other options.

5.17.3 Saving changed planned data after planning/scheduling on the server

Name of user exit

modify_hls_schedule.hsc

Keywords

Used for automatic planning/scheduling

Function

After scheduling or automatic planning on the server, all orders / operations changed by the planning are saved. This user exit is called for each order/operation after the data was saved.

Callback functions:

- BAPICALLEXECUTE: Any BAPI calls

Program(s) and source code files

Program	Version	Date	File(s)
b_hls.dll/so	8.1.0.171	2012-08-17	b_hls.cpp

Import parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

Export parameter

Parameter	Type	Content
DLG_DATA	C30000	Dialog data

5.18 Server user exits: ZKS - Access Control System

5.18.1 8 List of access authorizations

Name of user exit

hyz_zut.hsc

Keywords

ZKS, list of access authorizations, hyz_zut

Function

Use this user exit to change the output file of the list of access authorizations. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hyz_zut.out			hyz_zut.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number
LIST_DATA	char(10000)	Current line of the file

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.18.2 Access log

Name of user exit

hyz_zpr.hsc

Keywords

ZKS, access log, hyz_zpr

Function

Use this user exit to change the output file of the access log. This is the standard user exit to change list files.

The function "modify_list_file_line()" is called for each line in the file. The function "append_list_file()" is called after all lines have been processed. You can add a total line, for example.

Program(s) and source code files

Program	Version	Date	File(s)
hyz_zpr.out			hyz_zpr.c

Import parameter

Parameter	Type	Contents:
DLG_DATA	char(10000)	Parameter string in dialog data format.
LIST_LINE_NR	long	Current line number

LIST_DATA	char(10000)	Current line of the file
-----------	-------------	--------------------------

Export parameter

Parameter	Type	Contents:

There are no export parameters. Use the callback function "LISTOUTPUT" to write back a modified line. This callback function also allows you to insert additional lines. If you do not call the callback function for a specific line, this line will be deleted from the output file.

5.19 Server user exits: ETD

5.19.1 Log additional information for reprinting

Name of user exit

hyapptnr.hsc

Keywords

Label printing: Log additional values for reprinting

Function: after_log_insert

If the option "Log" is set in the label assignment, then the master data is handed over from the terminal to the server and logged by the program hyapptnr in the tables [hyd_prn_log/](#) [hyd_prn_logdet](#). This data is required for reprinting labels. The user exit **hyapptnr.after_log_insert** allows you to perform further activities after both tables had been filled.

Program(s) and source code files

Program	Version	Date	File(s)
hyapptnr.exe/out	8.1.1.41	2013-06-26	hyapptnr.cpp/ hyappux.cpp

Import parameter

Parameter	Type	Content
DLG_DATA	C32000	Dialog data sent by the terminal
VERWEIS	LONG	Reference to the prepared label reprint log record hyd_prn_log.verweis

5.19.2 Additional activities after creating the reprint file

Name of user exit

hyapptnr.hsc

Keywords

Create label reprint file.

Function: after_reprint

You can carry out further activities after creating the label reprint file.

Program(s) and source code files

Program	Version	Date	File(s)
hyapptnr.exe/out	8.1.1.39	2013-05-27	hyapptnr.cpp/ hyappux.cpp

Import parameter

Parameter	Type	Content
MASTER_DATA	C32000	Master data read from the database.
DLG_DATA	C32000	Command line
VERWEIS	LONG	Reference to the label reprint log record hyd_prn_log.verweis

5.19.3 Extended reprint list file (terminal)

Name of user exit

hyettnls.hsc

Keywords

Modify the label printing list for the terminal.

Function: modify_list_file_line

You can add additional fields for the terminal to the list showing the logged label print jobs.

Program(s) and source code files

Program	Version	Date	File(s)
hyapptnr.exe/out	8.1.1.39	2013-05-27	hyapptnr.cpp/ hyappux.cpp

6 Creating PDM-BAPIs using HYDRA Script

6.1 Overview

You can use the MES Development Suite to define BAPIs via HYDRA script. The complete definition is then made in the HYDRA script. There are two implementation stages with the definition:

1. Creating a basic structure

This basic structure provides the methods INSERT, UPDATE, DELETE, COPY, LIST, SELECT, NEW, LOCK and UNLOCK and is sufficient for most applications.

2. Extended options

You can overwrite and add to standard methods. You can also define additional methods.

In the sections that follow, the possible options are described using an example. In the example, the printing of labels is configured. For different labels, you must therefore define the size, the number of printed labels and the day from which the configuration applies.



Note: The definitions for namespaces, scopes and other requirements, which are described in the general documentation of the server scripting and the database, must be respected!



PDM dialogs are a proven technology. But you should only use PDM dialogs for clients that cannot call services of the WSP (Web Service Provider), e.g. the client AIP. If possible, you should create services because this technology is more forward-thinking.



The results of LIST bapis are written in files on the server. The client passes the file name with a relative path to the server. The server creates the file. The client then loads the file and can process it.

The file should be created in the spool directory.

Note: The file name must be unique per client. Only then, the server will not overwrite files of another request. If unique file names are not guaranteed, processes can be blocked on the server because these processes must access the same file.

You can use the following methods to assign unique file names:

- Integrate a unique number per client in the file name (e.g. with AIP use the user number = terminal number + 2000)
- Integrate the current time stamp in the file name.

Examples:

- With user number: FILE=../spool/myfile**2043**.dat|

- With time stamp (format: MonDDhmmssMMM with milliseconds):
FILE=.:spool/myfileDec31235959999.dat|

6.2 Using the Server Scripting

You use the Server Scripting to define a BAPI. The script file is named after the BAPI.

- BAPI: U_LABEL.*
- Script file: u_label_<customer id>@local.hsc

Script file names are usually in lower case letters. The BAPI itself is written in upper case letters when it is called.

A separate documentation describes the general server scripting details.

6.3 Basic structure

You use a simple basic structure to configure a BAPI. The definition via basic structure provides the dialogs SELECT, NEW, INSERT, UPDATE, MODIFY, DELETE, LIST, COPY, LOCK and UNLOCK that the BAPI can use.

The table name and the BAPI name is specified. You also define the database fields. An acronym links these fields to the input fields on the client. You specify the datatype for internal buffers and you use so-called constraints to specify specific ways of processing and plausibility checks.

```
hydra basic;

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = Callback( "SetTabName" , "u_label" );

    // -----
    // Define fields of table and BAPI
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    // -----
    // Table column | Acronym | Index | Data type | Constraint "
    // -----
    ret = Callback( "AddElement", "name" | NAME | | char(40) | KEY NOTNULL | " );
    ret = Callback( "AddElement", "name" | NAME | Z | char(40) | KEY NOTNULL ZIEL | " );
    ret = Callback( "AddElement", "groesse" | GROESSE | | double | NOTNULL | " );
    ret = Callback( "AddElement", "anzahl" | ANZ | | long | >0 | " );
    ret = Callback( "AddElement", "gueltig_ab" | DATV | | date | NOTNULL | " );
    ret = Callback( "AddElement", "verweis" | VERWEIS | | long | SERIAL | " );
    ret = Callback( "AddElement", "bearb_date" | | | date | DAT | " );
    ret = Callback( "AddElement", "bearb_time" | | | long | ZEI | " );
    ret = Callback( "AddElement", "bearb" | | | char(10) | BEARB | " );

    return 0;
}
```

You require the variable **ret** to receive the return value of the callback functions, but this variable need not be further processed in the script.

The function **main()** initializes the BAPI.

The callback function **CallBack("SetTabName"** specifies the name of the database table to be edited. The function includes only one single parameter, the table name. You can also leave out this call. In this case, the table name is generated using the BAPI name.

The callback function **CallBack("AddElement"** defines the BAPI fields. The function includes several parameters that are separated by the pipe character "|":

6.3.1 Parameters of the callback function AddElement

1: Table column

The first parameter includes the column of the database table.

2: Acronym

The second parameter includes the acronym (=field ID). This acronym is used for the fields in the applications on the client. Some acronyms have indexes that must be specified in the parameter that follows. You must use upper case letters for the acronyms.

3: Index

The third parameter includes the acronym index if it is used. Otherwise the parameter is empty. Also use upper case letters for the index.

4: Data type

The fourth parameter includes the data type for the internal storage of dialog data. The data types of the HYDRA script are available: **long**, **double**, **char(n)**, **date** and **datetime**. For details on these data types, refer to the HYDRA script documentation. You can use these data types to easily integrate the database data types (e.g. smallint via long or decimal(3.1) via double).

5: Constraints

The fifth parameter includes optional constraints. You can list several constraints separated by spaces. The following constraints are available:

Constraints

Constraint	Description
KEY	Identifies a key field. Key fields are used to derive the where clause. Key fields are also returned after INSERT and UPDATE bapis in the return string. The clients can then use the returned keys to identify the new data record also if the keys were assigned automatically (e.g. fields with constraint SERIAL).
SERIAL	Identifies the field as serial field. When a data record is created, its value is

	automatically assigned as a sequence number. If this field is used as key, you must also specify the "KEY" constraint ("KEY SERIAL"). In this case, the other fields must not have the KEY constraint. If they had a KEY constraint, the UPDATE BAPI would ignore them. Fields with constraint SERIAL are returned after INSERT and UPDATE bapis in the return string. The clients can then use the returned keys to identify the new data record also with the automatically assigned serial value.
NOTNULL	The field may not be NULL.
>0	Field must be >0.
ZIEL	Identifies a field as target field for the copy BAPI
MOD=E	Is always used in combination with the constraints "ZIEL KEY". It specifies that the respective field is only used as key field with copy mode 'Single'. For the other modes ('F', 'G'), this constraint is not included in the where clause and therefore there can be multiple results. (e.g. „ZIEL KEY MOD=E“)
BEARB	Specifies the field as "Modified by" field (special processing logic in the program). This field does not include an acronym. In the dialog string, the acronym "BEARB" is then specified without BAPI prefix.
ZEI	Specifies the field as "Processing time" field (special processing logic in the program). This field does not include an acronym. In the dialog string, the acronym "ZEI" is then specified without BAPI prefix.
DAT	Specifies the field as "Modified on" field (special processing logic in the program). This field does not include an acronym. In the dialog string, the acronym "DAT" is then specified without BAPI prefix.
NOVERARB	Specifies that the field is not processed (e.g. for customizations)
TIME	This constraint should be used for columns that include times or durations. If TIME is specified, the dialog string can use seconds, normal time or industrial minutes for the times/durations (12:34:56 or 12,5822 or 45296).
MNR	Identifies the field as machine number. Depending on the basic settings, it is regarded as an alphanumeric field or interpreted numerically and then filled with leading zeros when parsing from the dialog string.

	If a field has the acronym MNR and is of data type char(20), then it is automatically treated as if the constraint MNR were specified.
MGRP	Identifies the field as machine group. Depending on the basic settings, it is regarded as an alphanumeric field or interpreted numerically and then filled with leading zeros when parsing from the dialog string. If a field has the acronym MGRP and is of data type char(20), then it is automatically treated as if the constraint MGRP were specified.
PNR	Identifies the field as personnel number. Depending on the basic settings, it is filled with leading zeros when parsing from the dialog string. If a field has the acronym PNR and is of data type char() with a length of 10 characters or more, then it is automatically treated as if the constraint PNR were specified.

6.4 Extended options

6.4.1 Definition of a dialog list in the basic structure

You can define a list of dialogs in the basic structure using the callback function **AddDlg**. This way, the system can decide very early which BAPIs are defined. You can also specify additional processing options.

If the dialog list is not defined, all dialogs available in the basic structure are enabled in the standard processing.


```

hydra basic;

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName" , "u_label" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Special?|"
    // -----
    ret = CallBack( "AddDlg", "LOCK      |      |" );
    ret = CallBack( "AddDlg", "UNLOCK   |      |" );
    ret = CallBack( "AddDlg", "INSERT   |      |" );
    ret = CallBack( "AddDlg", "UPDATE   |      |" );
    ret = CallBack( "AddDlg", "DELETE   |SPECIAL CHECKLOCK|" );
    ret = CallBack( "AddDlg", "MYFKT    |      |" ); // Special Dlg

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    // -----
    // "Table column |Acronym |Index |Data type|Constraint "
    // -----
    ret = CallBack( "AddElement", "name      |NAME      |      |char(40)|KEY NOTNULL|" );
    ret = CallBack( "AddElement", "name      |NAME      |Z      |char(40)|KEY NOTNULL ZIEL|" );
    ret = CallBack( "AddElement", "groesse   |GROESSE   |      |double |NOTNULL|" );
    ret = CallBack( "AddElement", "anzahl    |ANZ       |      |long    |>0|" );
    ret = CallBack( "AddElement", "gueltig_ab|DATV      |      |date    |NOTNULL|" );
    ret = CallBack( "AddElement", "verweis   |VERWEIS   |      |long    |SERIAL|" );
    ret = CallBack( "AddElement", "bearb_date|          |      |date    |DAT|" );
    ret = CallBack( "AddElement", "bearb_time|          |      |long    |ZEI|" );
    ret = CallBack( "AddElement", "bearb     |          |      |char(10)|BEARB|" );

    return 0;
}

```

In the example above, the dialogs LOCK, UNLOCK, INSERT, UPDATE, DELETE and MYFKT are defined for the BAPI U_LABEL. For the dialogs that are available via the basic structure (see section basic structure), the dialog definition activates the standard processing if the option SPECIAL is not specified.

If the option SPECIAL is specified, then the dialog is not executed using the standard processing, but the script function **performSpecDlg()** is called that is then responsible for the complete processing.

If the option **CHECKLOCK** is specified, the system checks if the data record is locked before calling the script function **performSpecDlg()**.

6.4.2 Import and export variables

Export variables

Parameters	Type	Contents
DLG_DATA	C30000 (max.)	This variable contains the dialog string. Individual fields can be read from this dialog string using the function get_Bapi_Val(DLG_DATA, "<acronym>") .
ERRORTTEXT	C200	You can assign a free error message text to this export variable. When the script function is finished, the processing of the BAPI is stopped and the error text is passed to the client. If the standard processing has already changed data, the transaction is undone.

LOCK_KEY_1 to LOCK_KEY_5	C40	LOCK keys for the function setLockKeys that can be overwritten. See description.
-----------------------------	-----	---

6.4.3 Error handling

6.4.3.1 Creating error messages

To create error messages in a customer BAPI, you assign a free error text to the export variable **ERRORTXT**.

In some functions that can be overwritten (e.g. **CheckKeys**), the return value of the function has a meaning that is explained in the respective function description.

The error text is issued in the error file mentioned below. The name of the error file is returned in the return string:

```
RET=424|KT=Fehler im Bapi|LT=Fehler im User-Bapi (siehe Protokoll)|ERR.TXT=Invalid person id|ERR.DATEI=./spool/userbapro.1109|
```

6.4.3.2 Output in log file

Outputs with the command *dprint* are automatically written in a log file. Immediately after executing the dialog, the log file is available on the server in the spool directory with the name "userbapro.<UserNo>".

Note: In case of a multi-system installation, the spool directory is in the sub-directory with the system number (example: d:\mip\1\spool\userbapro.109). This log file is overwritten with each BAPI of the user.

Example

```
// -----
long CheckKeys()
{
    ret          long;
    dialog       char(20);
    bearb        char(10);
    verantw_bereich char(15);

    ret = CallBack( "Inherited", "" );

    dialog = get_Bapi_Val( DLG_DATA, "DLG" );

    sqlexec( "select verantw_bereich from personalstamm " ||
            " where personalnummer = " || BV(CallBack( "GetField", "personalnummer" )) ||
            " and firmen_nummer = " || BV(CallBack( "GetField", "firmen_nummer" )) || ";" );
    into( verantw_bereich );

    bearb = get_Bapi_Val( DLG_DATA, "BEARB" );

    ret = CallBack( "CheckAuthVAB", bearb || "|" || verantw_bereich || "|" || dialog );

    if (ret != 0)
    {
        dprint( "No Authorisation for VAB " || verantw_bereich || ", Dialog " || dialog );
    }
    return ret;
}
// -----
```

Output in log file

```
16.09.04 11:23 Start BAPI "PNRZVG2.UPDATE"
No Authorisation for VAB CompanyAdmin , Dialog PNRZVG2.UPDATE
```

16.09.04 11:23 End BAPI "PNRZVG2.UPDATE"

In addition to the outputs using *dprint*, you can also create outputs in any log files using the function *pprint*. See also the description of the HYDRA script language.

6.4.3.3 Logging of SQL or system errors

SQL and system errors are automatically logged in log files in the "err" directory on the server. The log file name is "hymw.<UserNr>.err" or "hymwb.<UserNr>.err" according to the program called (example: d:\mip1\1\err\hymw.1109.err). Note: In case of a multi-system installation, the err directory is in the sub-directory with the system number (example: d:\mip2\2\err\hymw.1109.err).

6.4.4 Additional joins with SELECT and LIST

As an example, we want to output the name with SELECT and LIST in addition to data that includes the personnel number. This requires several steps:

Defining the join tables

Using the instruction **CallBack("SetJoinTables", "<Join Tables>");** you define the tables that are joint to the main table. Assign an alias to each table. Note: the main table must automatically be assigned the alias *bt1*. Example:

```
ret = CallBack( "SetJoinTables",
               " outer gltzeitjhresmodell am, " ||
               " outer personalstamm p "      );
```

Defining the join clause

The join clause specifies the relation between the main table and the join tables. The join conditions must provide unambiguous joins, i.e. for each data record of the main table, not more than one data record of the join table is found. The join clause is defined using the instruction **JoinClause = new HyString("<Join Clause>");** With SELECT and LIST, the join clause is added to the where condition. The clause must therefore start with the respective Boolean operators. The column names must be unambiguous. Therefore you must assign the alias of the tables to the columns. Again, the main table must automatically be assigned the alias *bt1*.

Example:

```
ret = CallBack( "SetJoinClause",
               " and bt1.personalnummer = p.personalnummer (+) " ||
               " and bt1.firmen_nummer = p.firmen_nummer (+) " ||
               " and bt1.az_modell = am.modellnummer (+) " ||
               " and am.jahr (+) = year(today) " );
```

Defining the join fields

The function **CallBack("AddElement"** defines not only the normal fields, but also the join fields of the BAPI. The join fields are identified via the constraint *"JOINED"*. The column names must be unambiguous. Therefore the alias of the join table must be specified for the column name. Again, the main table must automatically be assigned the alias *bt1*. Example:

```
ret = CallBack( "AddElement", "am.bez_ausf      |BEZ      |AZJMOD|char(20) |JOINED      ||" );
ret = CallBack( "AddElement", "p.person_name  |NAME      |PNR      |char(40) |JOINED      ||" );
ret = CallBack( "AddElement", "p.person_vorname|VORNAME|PNR      |char(40) |JOINED      ||" );
```

6.4.5 Sorting with LIST

With LIST-BAPI, it is often useful to sort data before output. To this end, you can specify the order-by-clause as SQL fragment.

The instruction **ret = CallBack("SetOrderByClause", "<order by clause>");** defines the order-by-clause that is added to the SQL statement of the LIST-BAPI. Assign an alias to each table. Note: the main table must automatically be assigned the alias *bt1*. Example:

```
ret = CallBack( "SetOrderByClause",
               " order by bt1.firmen_nummer, bt1.personalnummer");
```

6.4.6 Additional where clauses

The instruction **ret = CallBack("SetAdditionalWhereClause", "<where clause>");** can define additional conditions for the where clause of the SQL statement. These additional conditions are added to the where clause with each SQL statement. For example, it is possible that data records are not actually deleted, but only marked as deleted. The data records marked as deleted may not be used by any statement. (Note: To proceed as described you must not only extend the where clause. You must also overwrite the SQL statement to delete a data record via update of the "deleted" identifier.)

Assign an alias to each table. Note: the main table must automatically be assigned the alias *bt1*. Example:

```
ret = CallBack( "SetAdditionalWhereClause",
               " and (bt1.geloescht <> \"J\" or bt1.geloescht is null )");
```

6.4.7 Available BAPI standard functions in the script

You can call specific functions of the standard BAPIs using **CallBack("FctName", "");** .

GetField

Parameters

Table field, see *AddElement* in the section "Basic structure".

Functions

The function identifies the internal value of the field specified by the name of the table field.

Return value

Field value, null if not available.

Example

```
anzahl = CallBack( "GetField", "anzahl" );
```

6.4.7.1 GetAkronym

Parameters

Acronym and index, see AddElement in the section "Basic structure". Acronym and index are separated by the pipe character.

Functions

The function identifies the internal value of the field specified by the acronym and the index.

Return value

Field value, null if not available.

Example

```
ziel_anzahl = CallBack( "GetAkronym", "ANZ|Z" );
```

6.4.7.2 GetSerial

Parameters

None.

Functions

The function identifies the internal value of a serial value that might exist. In the basic structure, this value is specified using the constraint SERIAL.

Return value

Value of serial field, null if not available.

6.4.7.3 SetField

Parameters

Table field, see AddElement in the section "Basic structure". New field value. Table field and new value are separated by the pipe character.

Functions

The function sets the internal value of the field specified by the name of the table field.

Return value

New field value, null if not available.

Example

```
neue_anzahl = CallBack( "SetField" ,"anzahl|27" );
```

6.4.7.4 SetAkronym

Parameters

Acronym and index, see AddElement in the section "Basic structure". New field value. Acronym, index and new value are separated by the pipe character.

Functions

The function sets the internal value of the field specified by the acronym and the index.

Return value

New field value, null if not available.

Example

```
neue_anzahl = CallBack( "SetAkronym", "ANZ|Z|27" );
```

6.4.7.5 ParseKeys

Parameters

None.

Functions

The function parses the key fields from the dialog string to the internal variables. Key fields are specified using the constraint "KEY".

Return value

Always 0.

6.4.7.6 ParseData

Parameters

None.

Functions

The function parses the data fields from the dialog string to the internal variables. Data fields are **not** specified using the constraint "KEY".

Return value

Always 0.

6.4.7.7 ParseAll

Functions

The function parses all fields from the dialog string to the internal variables.

Return value

Always 0.

6.4.7.8 ParseSet

Functions

You use this function to write all values of the internal buffers into the return string of the BAPI. The client can then use these values to fill the fields of the editing dialog. You mainly use this function for the dialogs NEW, SELECT and LOCK.

Return value

Always 0.

6.4.7.9 ParseSetKeys

Functions

You use this function to write the values of the key fields of the internal buffers into the return string of the BAPI. The client can then use these values. You mainly use this function for the dialogs INSERT, UPDATE and COPY.

Return value

Always 0.

6.4.7.10 SetValidFromToIns

Parameters

None.

Functions

You use this function with versioned master data (see constraints VDATB and VDATE). You must call this function to overwrite the function *InsertDB*.

You use this function to automatically change the validity period of the changed and of the previous and subsequent data records.

Return value

SQL code.

6.4.7.11 SetValidFromToDel

Parameters

None.

Functions

You use this function with versioned master data (see constraints VDATB and VDATE). You must call this function to overwrite the function *DeleteDB*.

You use this function to automatically change the validity period of the changed and of the previous and subsequent data records.

Return value

SQL code.

6.4.7.12 CheckAuthVAB

Parameters

- 1) User
- 2) Responsibility area
- 3) Mode

Functions

The function checks if the user specified is authorized for the responsibility area. If a mode is specified, the function also checks if the user is authorized to perform this operation (display, use, insert, modify, delete). As a mode, the dialog can be passed.

Return value

0 = OK, the user is authorized for the responsibility area and in
the mode specified
otherwise = error code, the user is not authorized.

Example

```
// -----
long CheckKeys()
{
    ret          long;
    dialog       char(20);
    bearb        char(10);
    verantw_bereich char(15);

    ret = CallBack( "Inherited", "" );

    dialog = get_Bapi_Val( DLG_DATA, "DLG" );

    sqlexec( "select verantw_bereich from personalstamm " ||
            " where personalnummer = " || BV(CallBack( "GetField", "personalnummer" )) ||
            " and firmen_nummer = " || BV(CallBack( "GetField", "firmen_nummer" )) || ";" );
    into( verantw_bereich );

    bearb = get_Bapi_Val( DLG_DATA, "BEARB" );

    ret = CallBack( "CheckAuthVAB", bearb || "|" || verantw_bereich || "|" || dialog );

    if (ret != 0)
    {
        dprint( "No Authorisation for VAB " || verantw_bereich || ", Dialog " || dialog );
    }
    return ret;
}

// -----
```

6.4.7.13 CheckAuthVAB_PNR

Parameters

- 1) User
- 2) Personnel number

Functions

The function checks if a person exists with the personnel number specified and if the user passed is authorized for the responsibility area of this person.

Return value

0 = OK, the person exists and the user is authorized for the person's responsibility area
otherwise = error code, the person does not exist or the user is not authorized for the person's responsibility area.

Availability

This BAPI standard function is available as of April 2006.

Example

```
// -----
long CheckKeys()
{
    ret          long;
    bearb        char(10);
    pnr          char(10);

    ret = CallBack( "Inherited", "" );

    bearb = get_Bapi_Val( DLG_DATA, "BEARB" );
    pnr   = get_Bapi_Val( DLG_DATA, "PNR" );

    ret = CallBack( "CheckAuthVAB_PNR", bearb || "|" || pnr );

    if (ret != 0)
    {
        dprint( "Person " || pnr || " not available or no authorisation for VAB" );
    }
    return ret;
}

// -----
```

6.4.8 Functions that can be overwritten

You can define the functions listed in the following in the script. These functions overwrite the standard function of the HYDRA BAPIs. In each function, the standard function can be called using the callback function **CallBack("Inherited" , "");** . The following options are therefore possible:

- You can perform an action before executing the standard function.
- You can perform an action after executing the standard function.
- You can completely replace the standard function.

If you completely replace the standard function (no "Inherited" is called), it is important to cover the complete functionality in the script because otherwise the function of the whole BAPI cannot be guaranteed.

The return value described in the following is evaluated by the BAPI. This description also applies for the return value of the standard function (with **CallBack("Inherited" , "");**)

6.4.8.1 Call sequences

INSERT

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
ParseAll	Check		
CheckData	Check	X	X
InsertDB	Action	X	X
In InsertDB: SetValidFromToIns	Action		
ParseSetKeys	Action		

UPDATE

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
SetLockKeys	Check	X	
CheckLock	Check		
ParseAll	Check		
CheckData	Check	X	X
UpdateDB	Action	X	X
ParseSetKeys	Action		

NEW

None. Using the function "main()", the columns can be preassigned. See separate section.

SELECT

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
ParseSet	Action		

MODIFY

The case "UPDATE":

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
SetLockKeys	Check	X	
CheckLock	Check		
ParseAll	Check		
CheckData	Check	X	X
UpdateDB	Action	X	X
ParseSetKeys	Action		

The case "INSERT":

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
ParseAll	Check		
CheckData	Check	X	X
InsertDB	Action	X	X
SetValidFromToIns	Action		
ParseSetKeys	Action		

DELETE

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
SetLockKeys	Check	X	
CheckLock	Check		
DeleteDB	Action	X	X
In DeleteDB: SetValidFromToDel	Action		

LIST

With LIST, the functions cannot be overwritten. But it is possible to process additional selection conditions in the main function.

```
// -----
long main()
{
    datum date;
    where char(1000);
    dummy char(1);
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum5" );

    ...

    ret = CallBack( "AddElement", "p.person_vorname|PVORNAME|      |char(40) |JOINED      |First name of person" );

    if( get_bapi_val( DLG_DATA, "DLG" ) = "U_PNRRAUM5.LIST" )
    {
        where = "";

        datum = get_bapi_val( DLG_DATA, "U_PNRRAUM5.DAT:BIS" );

        if( datum != "" )
            where = where clipped || " and gueltig_von <= " || BV( datum );

        datum = get_bapi_val( DLG_DATA, "U_PNRRAUM5.DAT:VON" );

        if( datum != "" )
            where = where clipped || " and gueltig_bis >= " || BV( datum );

        if( where != "" )
            ret = CallBack( "SetAdditionalWhereClause", where clipped );
    }
    else
    {
        // Set field "geloescht" (deleted) to value "N"
        dummy = CallBack( "SetField", "geloescht|N" );
    }

    return 0;
}
```

Using the dialog parameter MAXLISTROWS, you can additionally limit the number of rows with LIST!

COPY

With copy, the options are limited:

Function	Phase	Can be overwritten	Inherited possible
ParseAll	Check		
CheckKeysforCopy	Check	X	X
SetValidFromToIns	Action		

LOCK

Function	Phase	Can be overwritten	Inherited possible
ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
SetLockKeys	Check	X	

UNLOCK

Function	Phase	Can be overwritten	Inherited possible
----------	-------	--------------------	--------------------

ParseKeys	Check		
CheckKeys	Check	X	X
SelectDB	Check	X	X
SetLockKeys	Check	X	

Special

Function	Phase	Can be overwritten	Inherited possible
ParseAll	Check		
SetLockKeys	Check	X	
CheckLock	Check		
ParseAll	Action		
performSpecDlgCheck	Action	X	
performSpecDlgAction	Action	X	

6.4.8.2 CheckKeys

Parameters

None.

Functions

The function must check if all key fields are specified. The dialog has already taken over these key fields into the internal variables.

In the standard, the function is called with the BAPIs LOCK, UNLOCK, SELECT, INSERT, UPDATE and DELETE.

Return value

0 : The key fields are OK.

otherwise: Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.3 CheckKeysforCopy

Parameters

None.

Functions

This very powerful function performs all plausibility checks for the COPY-BAPI. The function checks if the source data is available and if the target data does not yet exist. The data itself is not checked because the data to be copied has already been checked. New data from the dialog string is not processed with copy.

In the standard, the function is called with the BAPI COPY.

Return value

0 : Plausibility is OK.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.4 CheckData

Parameters

None.

Functions

The function must check if all data fields are okay. The standard function only checks the constraints. If further checks must be performed, e.g. check of references in other tables or authorizations for responsibility areas, these checks must be performed here in the script in this function after calling the Inherited function.

In the standard, the function is called with the BAPIs INSERT and UPDATE. The internal field variables then already contain the values that must be inserted into the database.

Return value

0 : The key fields are OK.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.5 SetLockKeys

Parameters

None.

Functions

You use this function for the specific assignment of the lock keys. The lock keys are used to lock data records and to check for locked data records.

The function supports 5 lock keys. The lock keys are passed to the script in the export variables LOCK_KEY_1 to LOCK_KEY_5. The standard function preassigns these keys that can then be overwritten in the script.

The standard function assigns the keys as follows:

- The BAPI name is assigned to the first lock key.
- The content of the SERIAL field is assigned to the second lock key if a SERIAL is specified in the constraints. Otherwise the content of the first KEY field is assigned.
- The contents of the KEY fields 2 to 4 are assigned to the lock keys 3 to 5.

Overwriting the lock keys can be useful if all or several data records must be locked to edit one single data record in order to guarantee data consistency.

Important: This function does not support calling *Inherited* because it is not useful to have no lock at all. The function is therefore only called after the standard processing and can change the preassigned lock keys subsequently.

Return value

Any value. Is not further processed.

6.4.8.6 SelectDB

Parameters

None.

Functions

This function executes the data base access **select**. If you call the callback **CallBack("Inherited" , "");** the standard access is performed. You can use own statements to replace it. Before and after, you can also perform further database actions and changes of the internal buffers.

Return value

0 : OK, data record found.

100: : Data record not found

otherwise: Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.7 InsertDB

Parameters

None.

Functions

This function executes the data base access **insert**. If you call the callback **CallBack("Inherited" , "")**; the standard access is performed. You can use own statements to replace it. Before and after, you can also perform further database actions and changes of the internal buffers.

Important: If the standard function is replaced and if you work with versioned master data (constraints VTATB and VDATE), then the callback function **CallBack("SetValidFromToIns", "")** must be called in the function **InsertDB** .

Return value

0 : OK, data record found.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.8 UpdateDB

Parameters

None.

Functions

This function executes the data base access **update**. If you call the callback **CallBack("Inherited" , "")**; the standard access is performed. You can use own statements to replace it. Before and after, you can also perform further database actions and changes of the internal buffers.

Return value

0 : OK, data record found.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.9 DeleteDB

Parameters

None.

Functions

This function executes the data base access **delete**. If you call the callback **CallBack("Inherited" , "");** the standard access is performed. You can use own statements to replace it. Before and after, you can also perform further database actions and changes of the internal buffers.

Important: If the standard function is replaced and if you work with versioned master data (constraints VTATB and VDATE), then the callback function **CallBack("SetValidFromToDel", "")** must be called in the function **DeleteDB** .

Return value

0 : OK, data record found.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.4.8.10 PerformSpecDlgCheck and PerformSpecDlgAction

Parameters

None.

Functions

You use these functions

- to completely overwrite one of the standard dialogs defined via the basic structure. You use the definition of the dialog list in the basic structure to implement this.
- to define own dialogs that are not available as standard dialog in the basic structure. You use the definition of the dialog list in the basic structure to implement this.

To overwrite standard dialogs, specify the option **SPECIAL** for the standard dialog in the dialog list. Dialogs that are not available in the basic structure are automatically processed using the script functions **PerformSpecDlgCheck** and **PerformSpecDlgAction**. If the system must check before start of processing if the data record to be processed is locked, then the option **CHECKLOCK** must be specified in the definition of the dialog list.

PerformSpecDlgCheck: includes the plausibility checks

PerformSpecDlgAction: includes the actual action

If you use the script function **PerformSpecDlgAction**, note the following: This function must "do everything itself", including the database transaction processing and the parsing of return values via the callbacks ParseKeys and ParseSet.

Return value

0 : OK, data record found.

otherwise : Standard error number (from Inherited)

The function can always return the value of the Inherited function or 0. You must write the error messages specific to the BAPI in plain text in the ERRORTXT variable. This way, the messages can be displayed in plain text on the client.

6.5 Tips and tricks

6.5.1 Set default values of fields

After the definition of the fields in the basic structure, you can set default values of fields using the callback function **SetField** or **SetAkronym** in the script function **main()**.

```
neue_anzahl = CallBack( "SetField" ,"anzahl|1" );
```

6.5.2 Versioned master data

In the system, some master data have a "Valid from" date. Different versions of this data exist. For example, the HR master data or the working time day types of the time and attendance module. Note the following for this type of master data:

In addition to the required key columns, the column "Valid from" is added. This column usually has the name "gueltig_von" in the database and the data type "date". You must specify this column using the constraints VDATB and KEY.

Another column "gueltig_bis" of type "date" is added to the table. This column is specified using the constraint VDATE. The column is automatically edited by the BAPI and is used to easily join the keys and a date value in the table.

If you want to use the COPY dialog, you must additionally define the "Valid from" column as ZIEL column (target).

Example**Table structure**

```
create table pnr_zielvorgabe2
(
  personalnummer integer,
  firmen_nummer char(4),
  gueltig_von date,
  gueltig_bis date,
  bemerkung char(40),
  ziel_vorgabe decimal(18,6),
  az_modell integer,
  geloescht char(1),
  bearb_date date,
  bearb_time integer,
  bearb char(10)
);

create index pnr_zvg21 on pnr_zielvorgabe2 (nicht unique wg. geloescht )
(personalnummer, firmen_nummer, gueltig_von );
```

BAPI definition

```
// ----- "Tabellen-Feld" |Akronym|Index |Datentyp |Constraint |Kommentar" -----
// -----
ret = CallBack( "AddElement", "personalnummer |PNR | |long |KEY NOTNULL |Person|" );
ret = CallBack( "AddElement", "personalnummer |PNR |Z |long |KEY NOTNULL ZIEL|Person|" );
ret = CallBack( "AddElement", "firmen_nummer |FIR | |char(4) |KEY NOTNULL |Company Code|" );
ret = CallBack( "AddElement", "firmen_nummer |FIR |Z |char(4) |KEY NOTNULL ZIEL|Company Code|" );
ret = CallBack( "AddElement", "gueltig_von |DATB | |date |KEY VDATB |Valid from|" );
ret = CallBack( "AddElement", "gueltig_von |DATB |Z |date |KEY VDATB ZIEL|Valid from|" );
ret = CallBack( "AddElement", "gueltig_bis |DATE | |date |VDATE |Valid to|" );
ret = CallBack( "AddElement", "bemerkung |BEM | |char(40) |NOTNULL |Comment|" );
ret = CallBack( "AddElement", "ziel_vorgabe |ZVG | |double |>0 |Personal objectives|" );
ret = CallBack( "AddElement", "az_modell |AZJMOD | |long |>0 |Working Time Model|" );
ret = CallBack( "AddElement", "bearb_date | | |date |DAT |" );
ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |" );
ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |" );
ret = CallBack( "AddElement", "am.bez_ausf |BEZ |AZJMOD|char(20) |JOINED |" );
ret = CallBack( "AddElement", "p.person_name |NAME |PNR |char(40) |JOINED |" );
ret = CallBack( "AddElement", "p.person_vorname|VORNAME|PNR |char(40) |JOINED |" );
```

6.6 Tutorial

6.6.1 Task

Using the Development Suite, you must create a table that shows a person and the room where the person is working. And the personnel number must be assigned to a room number.

6.6.2 Simple version

In the simple version, only the customer-specific table must be maintained.

Database table

The database table is created in the tablespace U_USERCFG. An SQL file "u_pnrraum1.sql" is created that is executed using the HYDRA SQL interpreter "hysql".

SQL file (Oracle syntax)

```
create table u_pnr_raum
(
  firma char(4),
  person integer,
  raumnummer decimal(5,2),
  bemerkung char(100),
  bearb_date date,
  bearb_time integer,
  bearb char(10)
) TABLESPACE u_usercfg;
```

SQL file (MSSQL syntax)

```
create table "hydadm".u_pnr_raum
(
    firma            char(4),
    person           integer,
    raumnummer       decimal(5,2),
    bemerkung        char(100),
    bearb_date       date,
    bearb_time       integer,
    bearb            char(10)
)
on u_usercfg;
```

Execution of the SQL file

```
/usr/hydra72>hysql.out u_pnrraum.sql
```

BAPI definition

The basic structure described above is sufficient for the definition of the BAPI:

```
hydra basic;

// -----
//
// Tutorial
// Assignment room -- person
//
// version 1: Simple
//
// -----

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Options |Comment"
    // -----
    ret = CallBack( "AddDlg", "SELECT | | |" );
    ret = CallBack( "AddDlg", "NEW | | |" );
    ret = CallBack( "AddDlg", "INSERT | | |" );
    ret = CallBack( "AddDlg", "UPDATE | | |" );
    ret = CallBack( "AddDlg", "MODIFY | | |" );
    ret = CallBack( "AddDlg", "DELETE | | |" );
    ret = CallBack( "AddDlg", "LIST | | |" );
    ret = CallBack( "AddDlg", "COPY | | |" );
    ret = CallBack( "AddDlg", "LOCK | | |" );
    ret = CallBack( "AddDlg", "UNLOCK | | |" );

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    //
    // "Table column |Acronym |Index |Data type|Constraint |Comment"
    // -----
    ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
    ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
    ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
    ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
    ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
    ret = CallBack( "AddElement", "bemerkung |BEM | |char(100) | |Comment|" );
    ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
    ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
    ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );

    return 0;
}
```

6.6.3 Version including display of personnel data

In this version, the person's name is displayed with SELECT-BAPI and LIST-BAPI. And with LIST-BAPI, a sorting by the room number is performed in addition.



See the note on unique file names in section "Overview".

Database table

The database table is identical to the one of the simple version. It is not changed.

BAPI definition

With the BAPI definition, the basic structure is extended as described in section "Additional joins with SELECT and LIST". See the rows marked in gray.

```

hydra basic;

// -----
//
// Tutorial
// Assignement room -- person
//
// Version 2: With additional joins
//
// -----

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum" );

    ret = CallBack( "SetJoinTables", "outer personalstamm p" );
    ret = CallBack( "SetJoinClause", " and bt1.person = p.personalnummer (+) " ||
                                     " and bt1.firma = p.firmen_nummer (+) " );

    ret = CallBack( "SetOrderByClause", "order by bt1.raumnummer" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Options |Comment"
    // -----
    ret = CallBack( "AddDlg", "SELECT | | |" );
    ret = CallBack( "AddDlg", "NEW | | |" );
    ret = CallBack( "AddDlg", "INSERT | | |" );
    ret = CallBack( "AddDlg", "UPDATE | | |" );
    ret = CallBack( "AddDlg", "MODIFY | | |" );
    ret = CallBack( "AddDlg", "DELETE | | |" );
    ret = CallBack( "AddDlg", "LIST | | |" );
    ret = CallBack( "AddDlg", "COPY | | |" );
    ret = CallBack( "AddDlg", "LOCK | | |" );
    ret = CallBack( "AddDlg", "UNLOCK | | |" );

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    //
    // "Table column |Acronym |Index |Data type|Constraint |Comment"
    // -----
    ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
    ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
    ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
    ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
    ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
    ret = CallBack( "AddElement", "bemerkung |BEM | |char(100) | |Comment|" );
    ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
    ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
    ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );
    ret = CallBack( "AddElement", "p.person_name |PNAME | |char(40) |JOINED |First name of person|" );
    ret = CallBack( "AddElement", "p.person_vorname |PVORNAME | |char(40) |JOINED |Last name of person|" );

    return 0;
}

```

6.6.4 Version including date-dependent assignment

In the next version, the person is assigned to the room and the date is additionally specified. To this end, a "Valid from" date is specified on the client. As a result, you obtain a chronological order. And you can also plan the room occupancy in advance.

Database table

For this version, you must create a new table with the required data fields. The changes to the previous version are marked in gray.

SQL file (Oracle syntax)

```
create table u_pnr_raum3
(
  firma          char(4),
  person         integer,
  gueltig_von    date,
  gueltig_bis    date,
  raumnummer     decimal(5,2),
  bemerkung      char(100),
  bearb_date     date,
  bearb_time     integer,
  bearb          char(10)
) TABLESPACE u_usercfg;
```

Execution of the SQL file

```
/usr/mipl>mysql.out u_pnrraum.sql
```


BAPI definition

The BAPI of the previous section is extended. The method described in section "Versioned master data" is used. See the rows marked in gray.

```

hydra basic;

// -----
//
// Tutorial
// Assignment room -- person
//
// Version 3: Limited validity
//
// -----

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum3" );

    ret = CallBack( "SetJoinTables", "outer personalstamm p" );
    ret = CallBack( "SetJoinClause", " and btl.person = p.personalnummer (+) " ||
                                     " and btl.firma = p.firmen_nummer (+) " );

    ret = CallBack( "SetOrderByClause", "order by btl.raumnummer" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Options |Comment"
    // -----
    ret = CallBack( "AddDlg", "SELECT | | |" );
    ret = CallBack( "AddDlg", "NEW | | |" );
    ret = CallBack( "AddDlg", "INSERT | | |" );
    ret = CallBack( "AddDlg", "UPDATE | | |" );
    ret = CallBack( "AddDlg", "MODIFY | | |" );
    ret = CallBack( "AddDlg", "DELETE | | |" );
    ret = CallBack( "AddDlg", "LIST | | |" );
    ret = CallBack( "AddDlg", "COPY | | |" );
    ret = CallBack( "AddDlg", "LOCK | | |" );
    ret = CallBack( "AddDlg", "UNLOCK | | |" );

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    // -----
    // "Table column |Acronym |Index |Data type|Constraint |Comment"
    // -----
    ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
    ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB | |date |KEY NOTNULL VDATB|valid from|" );
    ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
    ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB |Z |date |KEY NOTNULL VDATB ZIEL|Target valid from|" );
    ret = CallBack( "AddElement", "gueltig_bis |DATE | |date |VDATE |valid to|" );
    ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
    ret = CallBack( "AddElement", "bemerkung |BEM | |char(100) | |Comment|" );
    ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
    ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
    ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );
    ret = CallBack( "AddElement", "p.person_name |PNAME | |char(40) |JOINED |First name of person|" );
    ret = CallBack( "AddElement", "p.person_vorname |PVORNAME | |char(40) |JOINED |Last name of person|" );

    return 0;
}

```

6.6.5 Extended check "Person available"

This version additionally checks if the person assigned does exist. To this end, a standard function is overwritten in the BAPI.

Database table

The database table does not change. It is identical to the previous version.

BAPI definition

The BAPI of the previous section is extended. The function `CheckKeys()` is overwritten. See the rows marked in gray.

```

hydra basic;

// -----
//
// Tutorial
// Assignment room -- person
//
// Version 4: Custom plausibility check
//
// -----

export ERRORTTEXT char(200);
export DLG_DATA char(30000);

// -----
long CheckKeys()
{
    ret          long;
    dialog        char(80);

    ret = CallBack( "Inherited", "" );

    if( ret = 0 )
    {
        dialog = get_Bapi_Val( DLG_DATA, "DLG" );

        if( pos( ".INSERT", dialog ) != 0 )
        {
            sqlexec( "select person name from personalstamm " ||
                    " where personalnummer = " || BV(CallBack( "GetField", "person" )) ||
                    " and firmen_nummer = " || BV(CallBack( "GetField", "firma" )) || ";" );

            if( sqlcode() != 0 )
            {
                ERRORTTEXT = "Person does not exist";
            }
        }
    }

    return ret;
}

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum" );

    ret = CallBack( "SetJoinTables", "outer personalstamm p" );
    ret = CallBack( "SetJoinClause", " and btl.person = p.personalnummer (+) " ||
        " and btl.firma = p.firmen_nummer (+) " );

    ret = CallBack( "SetOrderByClause", "order by btl.raumnummer" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Options |Comment"
    // -----
    ret = CallBack( "AddDlg", "SELECT | | |" );
    ret = CallBack( "AddDlg", "NEW | | |" );
    ret = CallBack( "AddDlg", "INSERT | | |" );
    ret = CallBack( "AddDlg", "UPDATE | | |" );
    ret = CallBack( "AddDlg", "MODIFY | | |" );
    ret = CallBack( "AddDlg", "DELETE | | |" );
    ret = CallBack( "AddDlg", "LIST | | |" );
    ret = CallBack( "AddDlg", "COPY | | |" );
    ret = CallBack( "AddDlg", "LOCK | | |" );
    ret = CallBack( "AddDlg", "UNLOCK | | |" );

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    // -----
    // "Table column |Acronym |Index |Data type|Constraint |Comment"
    // -----
    ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
    ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB | |date |KEY NOTNULL VDATB|valid from|" );
    ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
    ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB |Z |date |KEY NOTNULL VDATB ZIEL|Target valid from|" );
    ret = CallBack( "AddElement", "gueltig_bis |DATE | |date |VDATE |valid to|" );
    ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
    ret = CallBack( "AddElement", "bemerkung |BEM | |char(100) | |Comment|" );
    ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
    ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
    ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );
    ret = CallBack( "AddElement", "p.person_name |PNAME | |char(40) |JOINED |First name of person|" );
    ret = CallBack( "AddElement", "p.person_vorname |PVORNAME | |char(40) |JOINED |Last name of person|" );

    return 0;
}

```

```
}
```

6.6.6 When delete, only mark as deleted

This version does not really delete the data records, but the data records are only marked as deleted. The delete actions that have been performed can be retraced if necessary. To this end, a standard function is overwritten in the BAPI.

Database table

For this version, you must create a new table with the required data fields. The changes to the previous version are marked in gray.

SQL file (Oracle syntax)

```
create table u_pnr_raum5
(
  firma          char(4),
  person         integer,
  gueltig_von    date,
  gueltig_bis    date,
  geloescht      char(1),
  raumnummer     decimal(5,2),
  bemerkung      char(100),
  bearb_date     date,
  bearb_time     integer,
  bearb          char(10)
) TABLESPACE u_usercfg;
```

Execution of the SQL file

```
/usr/mipl>hysql.out u_pnrraum.sql
```

BAPI definition

The BAPI of the previous section is extended. The function DeleteDB is overwritten and an additional where clause is specified. See the rows marked in gray.

```
hydra basic;

// -----
//
// Tutorial
// Assignment room -- person
//
// Version 5: Keep deleted data sets as hidden
//
// -----

export ERRORTTEXT char(200);
export DLG_DATA char(30000);

// -----
long DeleteDB()
{
    ret long;

    // Do not really delete data set, mark it only as deleted
    sqlexec( "update u_pnr_raum5 set " ||
        " geloescht = \"J\" " || ", " ||
        " bearb_date = " || bv( today() ) || ", " ||
        " bearb_time = " || bv( now() ) || ", " ||
        " bearb = " || BV( get_bapi_val( DLG_DATA, "BEARB" ) ) ||
        " where ( geloescht <> \"J\" or geloescht is null ) " ||
        " and person = " || BV( CallBack( "GetField", "person" ) ) ||
        " and firma = " || BV( CallBack( "GetField", "firma" ) ) ||
        " and gueltig_von = " || BV( CallBack( "GetField", "gueltig_von" ) ) ||
        "; " );

    ret = sqlcode();

    dprint( sqlnumrows() using "<<<<&" || " dat set(s) marke das deleted. Code " || ret using "<<<<<<&" );

    if( sqlnumrows() != 1 )
    {
        ERRORTTEXT = sqlnumrows() using "<<<<&" || " dat set(s) marke das deleted. Code " || ret using "<<<<<<&";
        ret = 100;
    }

    // Update validity of onther data sets of same key
    if( ret = 0 )
    {
        ret = CallBack( "SetValidFromToDel", "" );
    }

    return ret;
}

// -----
long CheckKeys()
{
    ret long;
    dialog char(80);

    ret = CallBack( "Inherited", "" );

    if( ret = 0 )
    {
        dialog = get_Bapi_Val( DLG_DATA, "DLG" );

        if( pos( ".INSERT", dialog ) != 0 )
        {
            sqlexec( "select person_name from personalstamm " ||
                " where personalnummer = " || BV( CallBack( "GetField", "person" ) ) ||
                " and firmen_nummer = " || BV( CallBack( "GetField", "firma" ) ) || "; " );

            if( sqlcode() != 0 )
            {
                ERRORTTEXT = "Person does not exist";
            }
        }
    }

    return ret;
}

long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum5" );

    ret = CallBack( "SetJoinTables", "outer personalstamm p" );
    ret = CallBack( "SetJoinClause", " and btl.person = p.personalnummer (+) " ||
        " and btl.firma = p.firmen_nummer (+) " );
}
```

```

ret = CallBack( "SetOrderByClause", "order by bt1.raumnummer" );

ret = CallBack( "SetAdditionalWhereClause",
    " and (bt1.geloescht <> \"J\" or bt1.geloescht is null )");

// -----
// Define dialogs
// -----
// "Dialog |Options |Comment"
// -----
ret = CallBack( "AddDlg", "SELECT | | |" );
ret = CallBack( "AddDlg", "NEW | | |" );
ret = CallBack( "AddDlg", "INSERT | | |" );
ret = CallBack( "AddDlg", "UPDATE | | |" );
ret = CallBack( "AddDlg", "MODIFY | | |" );
ret = CallBack( "AddDlg", "DELETE | | |" );
ret = CallBack( "AddDlg", "LIST | | |" );
ret = CallBack( "AddDlg", "COPY | | |" );
ret = CallBack( "AddDlg", "LOCK | | |" );
ret = CallBack( "AddDlg", "UNLOCK | | |" );

// -----
// Define table columns and BAPI acronyms
// (Fields of last modification do not have an acronym, they are defined by the constraint.)
// -----
// "Table column |Acronym |Index |Data type|Constraint |Comment"
// -----
ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
ret = CallBack( "AddElement", "gueltig_von |DATB | |date |KEY NOTNULL VDATB|valid from|" );
ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
ret = CallBack( "AddElement", "gueltig_von |DATB |Z |date |KEY NOTNULL VDATB ZIEL|Target valid from|" );
ret = CallBack( "AddElement", "gueltig_bis |DATE | |date |VDATE |valid to|" );
ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
ret = CallBack( "AddElement", "bemerkung |BEM | |char(100) | |Comment|" );
ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );
ret = CallBack( "AddElement", "p.person_name |PNAME | |char(40) |JOINED |First name of person|" );
ret = CallBack( "AddElement", "p.person_vorname |PVORNAME | |char(40) |JOINED |Last name of person|" );

return 0;
}

```

6.6.7 Additional BAPI for authorization

This version adds an authorization option to the data record. User and time of authorization are stored in the data record. A separate BAPI U_PNRRaum6.SIGN is created.

Database table

For this version, you must create a new table with the required data fields. The changes to the previous version are marked in gray.

SQL file (Oracle syntax)

```

create table u_pnr_raum6
(
    firma            char(4),
    person           integer,
    gueltig_von      date,
    gueltig_bis      date,
    geloescht        char(1),
    raumnummer       decimal(5,2),
    bemerkung        char(100),
    sign             char(1),
    sign_bearb       char(10),
    sign_date        date,
    sign_time        integer,
    bearb_date       date,
    bearb_time       integer,
    bearb            char(10)
);

```

Execution of the SQL file

```
/usr/mipl>hysql.out u_pnrraum.sql
```

BAPI definition

The BAPI of the previous section is extended. The dialog list is extended by the dialog SIGN. The functions *PerformSpecDlgCheck()* and *PerformSpecDlgAction()* are overwritten. See the rows marked in gray.

```

hydra basic;

// -----
//
// Tutorial
// Assignment room -- person
//
// Version 6: with additional bapis for "sign"
//
// -----

export ERRORTTEXT char(200);
export DLG_DATA char(30000);

// -----
long DeleteDB()
{
    ret          long;

    // Do not really delete data set, mark it only as deleted
    sqlxec( "update u_pnr_raum5 set " ||
            " geloescht = \"J\" " || ", " ||
            " bearb_date = " || bv( today() ) || ", " ||
            " bearb_time = " || bv( now() ) || ", " ||
            " bearb = " || BV( get_bapi_val( DLG_DATA, "BEARB" ) ) ||
            " where ( geloescht <> \"J\" or geloescht is null ) " ||
            " and person = " || BV( CallBack( "GetField", "person" ) ) ||
            " and firma = " || BV( CallBack( "GetField", "firma" ) ) ||
            " and gueltig_von = " || BV( CallBack( "GetField", "gueltig_von" ) ) ||
            "; " );

    ret = sqlcode();

    dprint( sqlnumrows() using "<<<<&" || " dat set(s) marke das deleted. Code " || ret using "<<<<<&" );

    if( sqlnumrows() != 1 )
    {
        ERRORTTEXT = sqlnumrows() using "<<<<&" || " dat set(s) marke das deleted. Code " || ret using "<<<<<&";
        ret = 100;
    }

    // Update validity of ohther data sets of same key
    if( ret = 0 )
    {
        ret = CallBack( "SetValidFromToDel", "" );
    }

    return ret;
}

// -----
long CheckKeys()
{
    ret          long;
    dialog       char(80);

    ret = CallBack( "Inherited", "" );

    if( ret = 0 )
    {
        dialog = get_Bapi_Val( DLG_DATA, "DLG" );

        if( pos( ".INSERT", dialog ) != 0 )
        {
            sqlxec( "select person name from personalstamm " ||
                    " where personalnummer = " || BV( CallBack( "GetField", "person" ) ) ||
                    " and firmen_nummer = " || BV( CallBack( "GetField", "firma" ) ) || "; " );

            if( sqlcode() != 0 )
            {
                ERRORTTEXT = "Person does not exist";
            }
        }
    }

    return ret;
}

// -----
long performSpecDlgCheck()
{
    ret          long;

    ret = 102; // Unknown dialog

    if( pos( ".SIGN", get_Bapi_Val( DLG_DATA, "DLG" ) ) != 0 )
    {
        // currently no plausibility check defined
        ret = 0; // OK
    }

    return ret;
}

```

```

}

// -----
long performSpecDlgAction()
{
    ret = long;

    if( pos( ".SIGN", get_Bapi_Val( DLG_DATA, "DLG" ) ) != 0 )
    {
        // Mark data set as signed
        sqlexec( "update u_pnr_raum6 set " ||
            " sign = \"J\" " || ", " ||
            " sign_date = \" || bv( today() ) || ", " ||
            " sign_time = \" || bv( now() ) || ", " ||
            " sign_bearb = \" || BV( get_bapi_val( DLG_DATA, \"BEARB\" ) ) ||
            " where ( geloescht <> \"J\" or geloescht is null ) " ||
            " and person = \" || BV( CallBack( \"GetField\", \"person\" ) ) ||
            " and firma = \" || BV( CallBack( \"GetField\", \"firma\" ) ) ||
            " and gueltig_von = \" || BV( CallBack( \"GetField\", \"gueltig_von\" ) ) ||
            ";";

        ret = sqlcode();

        dprint( sqlnumrows() using "<<<<&\" || \" ds signed. Code \" || ret using "<<<<&\" );

        if( sqlnumrows() != 1 )
        {
            ERRORTXT = sqlnumrows() using "<<<<&\" || \" ds signed. Code \" || ret using "<<<<&";
        }
        ret = 100;
    }

    return ret;
}

// -----
long main()
{
    ret char(20);

    // -----
    // Define table name
    // -----
    ret = CallBack( "SetTabName", "u_pnr_raum5" );

    ret = CallBack( "SetJoinTables", "outer personalstamm p" );
    ret = CallBack( "SetJoinClause", " and bt1.person = p.personnummer (+) " ||
        " and bt1.firma = p.firmennummer (+) " );

    ret = CallBack( "SetOrderByClause", "order by bt1.raumnummer" );

    ret = CallBack( "SetAdditionalWhereClause",
        " and (bt1.geloescht <> \"J\" or bt1.geloescht is null )" );

    // -----
    // Define dialogs
    // -----
    // "Dialog |Options |Comment"
    // -----
    ret = CallBack( "AddDlg", "SELECT | | |" );
    ret = CallBack( "AddDlg", "NEW | | |" );
    ret = CallBack( "AddDlg", "INSERT | | |" );
    ret = CallBack( "AddDlg", "UPDATE | | |" );
    ret = CallBack( "AddDlg", "MODIFY | | |" );
    ret = CallBack( "AddDlg", "DELETE | | |" );
    ret = CallBack( "AddDlg", "LIST | | |" );
    ret = CallBack( "AddDlg", "COPY | | |" );
    ret = CallBack( "AddDlg", "LOCK | | |" );
    ret = CallBack( "AddDlg", "UNLOCK | | |" );
    ret = CallBack( "AddDlg", "SIGN |SPECIAL CHECKLOCK | |" );

    // -----
    // Define table columns and BAPI acronyms
    // (Fields of last modification do not have an acronym, they are defined by the constraint.)
    // -----
    // "Table column |Acronym |Index |Data type|Constraint |Comment"
    // -----
    ret = CallBack( "AddElement", "person |PNR | |long |KEY NOTNULL |Person|" );
    ret = CallBack( "AddElement", "firma |FIR | |char(4) |KEY NOTNULL |Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB | |date |KEY NOTNULL VDATB|valid from|" );
    ret = CallBack( "AddElement", "person |PNR |Z |long |KEY NOTNULL ZIEL|Target person|" );
    ret = CallBack( "AddElement", "firma |FIR |Z |char(4) |KEY NOTNULL ZIEL|Target Company|" );
    ret = CallBack( "AddElement", "gueltig_von |DATB |Z |date |KEY NOTNULL VDATB ZIEL|Target valid from|" );
    ret = CallBack( "AddElement", "gueltig_bis |DATE | |date |VDATE |valid to|" );
    ret = CallBack( "AddElement", "raumnummer |RAUM | |double | |Room number|" );
    ret = CallBack( "AddElement", "bemerkung |BEM | |char(100)| |Comment|" );
    ret = CallBack( "AddElement", "bearb_date | | |date |DAT |Last edit date|" );
    ret = CallBack( "AddElement", "bearb_time | | |long |ZEI |Last edit time|" );
    ret = CallBack( "AddElement", "bearb | | |char(10) |BEARB |Last editor|" );
    ret = CallBack( "AddElement", "p.person_name |PNAME | |char(40) |JOINED |First name of person|" );
    ret = CallBack( "AddElement", "p.person_vorname |PVORNAME | |char(40) |JOINED |Last name of person|" );

    return 0;
}

```

7 Creating PDM lists using HYDRA script

7.1 Overview

The system provides the option to define so-called PDM lists using HYDRA script. The complete definition is then made in the HYDRA script.

PDM lists are lists that you can read from the system using the dialog "DLG=LIST;...". The system creates a list file on the server. The external application then loads and evaluates this list file.



The results of the lists are written in files on the server. The client passes the file name with a relative path to the server. The server creates the file. The client then loads the file and can process it.

The file should be created in the spool directory.

Note: The file name must be unique per client. Only then, the server will not overwrite files of another request. If unique file names are not guaranteed, processes can be blocked on the server because these processes must access the same file.

You can use the following methods to assign unique file names:

- Integrate a unique number per client in the file name (e.g. with AIP use the user number = terminal number + 2000)
- Integrate the current time stamp in the file name.

Examples:

- With user number: FILE=../spool/myfile**2043**.dat|
- With time stamp (format: MonDDhhmmssMMM with milliseconds):
FILE=../spool/myfile**Dec3123595999**.dat|

Example of a standard list from the PDM documentation:

PZE access authorizations are requested via list "27":

```
"DLG=LIST;27|DATEI={file name}|DAT=...|ZEI=...|USR=...|TNR=...|..."
```

If it is not list ID "27", you must assign a respective ID to the user defined lists. See examples below.
The file {file name} on the server then has e.g. the following content:

```

KNR=Kartennummer|PNR=Person|FIR=Firma|KTODAT=Kontendatum|DGBE=Dienstgangberechtigung|
1004|1004|BSP|N|
1009|1009|BSP|J|
2010|2132|01|J|
2011|1012|01|N|

```

The first line is a header. It contains an acronym and a name for every column, separated by a "=" sign (e.g. "|KNR=Kartennummer|").

The other lines are data rows. The columns described in the header are separated by a pipe character "|". The column contents are always restricted to the required length, i.e. strings have no trailing blanks and numbers have no leading zeros. The column width can therefore be different for each data record.



With standard lists, the column order is not specified! It can be changed by MPDV at any time. The external PDM application must therefore use the acronyms in the header to assign the columns!

There are two implementation stages with the definition of such lists via HYDRA script:

- **Creating a basic structure**

The basic structure provides an easy option to export contents from a database into a PDM list because the structure specifies the required table(s) and columns.

- **Extended options**

With the use of SQL statements and the complete control in the HYDRA script, any complex calculation and interim processing is possible when the lists are created.

In the sections that follow, the possible options are described using an example. In the example, the persons of the HR master data are read and output as list.



Note: the definitions for namespaces, scopes and other requirements, which are described in the general documentation of the server scripting and the HYDRA database, must be respected!



PDM dialogs are a proven technology. But you should only use PDM dialogs for clients that cannot call services of the WSP (Web Service Provider), e.g. the client AIP. If possible, you should create services because this technology is more forward-thinking.

7.2 Using the Server Scripting

You use the Server Scripting to define a BAPI. The script file is named after the list, e.g. "u_l_personen1".

- List: LIST;U_L_PERSONEN1|...
- Script file: u_l_personen1_<customer id>@local.hsc

Script file names are usually in lower case letters. The list is written in upper case letters when it is called.

A separate documentation describes the general server scripting details.

The following functions and import/export parameters are available. It is recommended to familiarize yourself with the functions. Read the examples in the following sections and use the descriptions as a reference.

7.2.1 Import and export variables

Import variables

Parameters	Type	Contents
DLG_DATA	C30000 (max.)	This variable contains the dialog string. Individual fields can be read from this dialog string using the function get_Bapi_Val(DLG_DATA, "<acronym>") . You can use these fields if you specify a Where clause to select the data to be displayed of a list.

Export variables

Parameters	Type	Contents
ERRORTXT	C200	You can assign a free error message text to this export variable. The error number in the return value is automatically set to a fixed code. The error message text is transferred to the client. The error message text is transferred in the long text (LT) of the error code (restricted to a max. length) and in full length, with the "ERR.TXT" identification, in the return string.
RET_DATA	C30000 (max.)	The content of this variable is attached to the string that is returned to the client. The content is used to transfer additional information, which is evaluated by the client. The string must contain information in BAPI format with acronyms and values, separated by pipes.

7.2.2 Script function long main()

Parameters

None

Return value

The return value of the main() function is returned as dialog result to create a list. It must be a long value:

0 : OK

Otherwise: Error code, e.g. SQL error code. For further information, refer to the section "Error handling".

Explanatory notes

The `main()` function controls the complete processing of list creation. No further functions are required in the script. (If required, however, further functions can be defined for use within the script.)

7.2.3 Callback function long "SetTables"

Parameters

SQL fragment with table name(s). If several tables are specified, they should have an alias. If you use "inner joins" or "outer joins" with ANSI syntax, also specify the conditions for the joins in the function "SetTables".

Return value

The return value is not important, the function always returns 0.

Explanatory notes

This function is used for the variant using the basic structure. This function specifies the name of the table(s) to be listed and sets an internal buffer with the table name(s). These table name(s) are later added to an SQL statement.

Example

```
ret = Callback( "SetTables", "hr_masterdata p, outer costcenter cst" );
```

7.2.4 Callback function long "AddColumn"

Parameters

This function includes three parameters separated by the pipe "|" character:

- 1) SQL fragment including the column to be selected from the table. Here, you can use table aliases and define column aliases.
- 2) Acronym for the header of the list. The acronym can be left empty here, then the column name or column alias specified in 1) is used as acronym.
- 3) Designation for the header. Whether the designation is required, depends on the external PDM application.



If you specify the ID "DCD=N|", you can define that only the acronyms and no designations are output in the header when the list is created.

Return value

The return value is not important, the function always returns 0.

Explanatory notes

This function is used for the variant using the basic structure. This function extends an internal buffer, which is used to create the header and the selected columns in an SQL statement.

Example

```
// -----  
// Define columns  
//           "Database column"      |Acronym |Designation"  
// -----  
ret = CallBack( "AddColumn", "personalnummer      |PNR      |Person no" );  
  
or:  
ret = CallBack( "AddColumn", "p.personalnummer      pnr|PNR      |Person no" );
```

7.2.5 Callback function long "SetClauses"

Parameters

SQL fragment with optional clauses:

- optional Where clause
- optional Group By clause
- optional Order By clause

You can use the optional Where clause to select specific data records or for join conditions with obsolete syntax.

Return value

The return value is not important, the function always returns 0.

Explanatory notes

This function is used for the variant using the basic structure. You use this function to create an SQL fragment, which is attached to the end of an SQL Select statement.

Example

```
ret = CallBack( "SetClauses",
    " where p.kostenstelle = kst.kostenstelle (+) " ||
    " and p.firm_number = kst.firm (+) " ||
    " and p.firmen_number like " || BV( get_bapi_val( DLG_DATA, "FIR" ) ) ||
    " order by 3, 1" );
```

7.2.6 Callback function long "MakeList"

Parameters

None.

Return value

The return value of this function should be returned in the main() script function. The return value has the same meaning as the one described above for the main() script function.

Explanatory notes

This function is used for variants with the basic structure and it eventually creates the list.

The system uses the specified acronyms and designations to create a header for the lists and writes these in the file passed.

An SQL select statement is composed from the columns, tables and clauses specified with the other CallBack functions and the selected data records are written to the list file.

The list includes a header which contains the acronyms and designations of the columns. The rows that follow include the data.



If you specify the ID "DCD=N|", you can define that only the acronyms and no designations are output in the header when the list is created.

Example

```
long main()
{
    ...

    //-----
    // Generate list
    //-----
    ret = CallBack( "MakeList", "" );

    return ret;
}
```

7.2.7 Callback function long "WriteLn"

Parameters

Row to be output in the list file.

Return value

The return value is not important, the function always returns 0.

Explanatory notes

This function writes the row specified as parameter into the list file.

This function is used for the variant with the extended options. It is not useful in combination with the basic structure.

Example

```
long main()
{
    ...

    ret = CallBack( "WriteLn", "FIR=Company|PNR=PersonId|NAME=Name|BER=Area|KST=Costcenter|" );

    ...

    line = "";
    line = add_bapi_val( line, "", fir );
    line = add_bapi_val( line, "", pnr );
        line = add_bapi_val( line, "", name );
    line = add_bapi_val( line, "", ber );
    line = add_bapi_val( line, "", kst );
        ret = CallBack( "WriteLn", line clipped );

    return ret;
}
```

7.2.8 Error handling

7.2.8.1 Error handling when using the basic structure

If the basic structure is used, the error handling is largely automatic.

Possible error causes when using the basic structure

Syntax error in the script

The return string identifies the error:

"RET=3024|KT=Script syntax error Script|LT=Syntax error in script file|".

Check the log files, as described in the section "Logging of SQL or system errors".

Correct the script, check if the syntax is correct.

Runtime error in the script

The return string identifies the error

"RET=3025|KT=Script runtime error |LT=Runtime error in the script file|".

Check the log files, as described in the section "Logging of SQL or system errors". Possible error causes are for example: wrong spelling of Callback functions or division by zero.

Correct the script.

The specified file cannot be opened

A file name must be entered in the dialog which requests the list. If this name is invalid, the return string identifies the following error:

"RET=410|KT=Invalid file name|LT=Error on opening/writing the file.|".

Check the log files, as described in the section "Logging of SQL or system errors". The incorrect file name is identified there.

Correct the file name in the request dialog. The file must be stored in the spool directory of the server.

Error in the "MakeList" Callback function

When the list is created, SQL errors may occur. The return value of the "MakeList" Callback function should be returned as return value of the main() function, so that the external PDM application identifies occurred errors.

Check the log files, as described in the section "Logging of SQL or system errors". Possible error causes are for example: wrong spelling of table or column names or SQL syntax errors. If an SQL syntax error occurs with the *Oracle* database system, the return string specifies e.g. the following error:

"RET=-933|KT=|LT=|"

The log file then contains the incorrect SQL statement:

```
04.02.2005 08:57:54.071 0 SQL=declare list_curs cursor for select p.personalnummer pnr, p.firmen_nummer fir, p.person_name ||
' || p.person_vorname, bereich, p.kostenstelle, kst.bezeichnung from personalstamm p, kostenstellen kst nowhere p.kostenstelle =
kst.kostenstelle (+) and p.firmen_nummer = kst.firma (+) and p.firmen_nummer like :ca order by 3, 1;|0|BSP|
```

The meaning of the error codes are explained in the documentation of your database system.

Correct the script, especially the parameters of the "SetTables", "AddColumn" and "SetClauses" CallBack functions.

If other tasks are executed in the script that are not part of the basic structure CallBack functions, then the error handling must be performed using the return value of the main() function, as it is the case with the "extended options" (see next section).

7.2.8.2 Error handling when using the extended options

The following errors, which were described in the previous section, are also handled automatically when the extended options are used:

- Syntax error in the script
- Runtime error in the script
- The specified file cannot be opened

All other errors, which might occur during execution of the main() function or of any user defined functions called from main(), must be handled there. If an error is identified, processing should be stopped and the main() function should be exited with a return value that does not equal zero. To improve traceability, a debug output using the function *eprint* and/or *pprint* is recommended (see section below: "Debug outputs"). The return string passes the return value to the external PDM application, where it can be evaluated.

Example:

```
long main()
{
    ...

    // error occured
    eprint( "Error X occured. Person ID " || personalnummer || " does not exist" );
    ret = 424;

    return ret;
}
```

issues the following return string:

```
"RET=424|KT=|LT=|"
```

7.2.8.3 Logging of SQL or system errors

SQL and system errors are automatically logged in log files in the "err" directory on the server. The log file has the name "hymwb.<UserNo>.err", "hymw.<UserNo>.err", "hybapi.<UserNo>.err" or also "hyddi.<UserNo>.err". The name depends on the program called. The err directory is in the sub directory with the system number (example: d:\mip2\2\err\hymwb.1109.err).

7.2.8.4 Debug outputs

You use the functions *pprint* and *eprint* to output in log files. See also the description of the HYDRA script language. The *eprint* function writes in the same log files as described in the previous section, "Logging of SQL or system errors".

If the system is in debug mode or if logging is activated, output with the command *dprint* is written to the log files in the server error directory or is displayed on the screen.

7.3 Creating the list

The list is created using the following dialog:

```
„DLG=LIST;U_L_PERSONEN1|DATEI={Filename}|DAT=...|ZEI=...|USR=...|PAR=...“
```

The file *{Filename}* should be stored in the "spool" directory of the server, e.g. "|DATEI=spool/u_le_personen.{UserNo}|". It must be specified in lower case letters and in Unix notation (slash instead of backslash).

The parameters (*PAR=*) in italics depend on the respective list.



See the note on unique file names in section "Overview".



If you specify the ID "DCD=N|", you can define that only the acronyms and no designations are output in the header when the list is created.

7.4 Introduction: Basic structure

The sections in the following illustrate how to create PDM lists using examples and describing the new functions in detail.

7.4.1 Example 1: Simple list of persons

In this example, all persons of the HR master data are listed. It is not necessary to specify a selection of persons.

Definition of the PDM list U_L_PERSONEN1:

The basic structure is sufficient for the definition of the list:

```

hydra basic;

// -----
//
// Tutorial
// PDM-Liste of persons
//
// Version 1: Simple version
//
// -----
/*-----*/
long main()
{
    ret          long;

    // -----
    // define database table(s)
    // -----
    ret = CallBack( "SetTables", "hr_masterdata" );

    // -----
    // Define columns
    // -----
    // „Database column      |Acronym |Designation"
    // -----
    ret = CallBack( "AddColumn", "personalnummer", |PNR |Person no" );
    ret = CallBack( "AddColumn", "firmen_nummer",   |FIR |Company" );
    ret = CallBack( "AddColumn", "person_name",     |NAME|Name" );
    ret = CallBack( "AddColumn", "bereich",         |BER |Area" );
    ret = CallBack( "AddColumn", "kostenstelle",    |KST |cost center" );

    // -----
    // Define optional clauses
    // An optional where clause, an optional group by clause and an optional
    // order by clause can be defined
    // -----

    // Not applicable in this example

    // -----
    // Generate list
    // -----
    ret = CallBack( "MakeList", "" );

    return ret;
}

/*-----*/

```

Result:

After creating the list with the dialog

„DLG=LIST;U_L_PERSONEN1|DATEI={Filename}|DAT=...|ZEI=...|USR=...|“

the content of the file {filename} is as follows:

```

PNR=Person no|FIR=Company|NAME=Name|BER=Area|KST=cost center|
999998|BSP|Meier|123|105|
999999|BSP|Schulz|123|105|
906075|BSP|Erhard|077|105|
1004|BSP|Hirsch|077|105|
1009|BSP|Mustermann|077|105|
400000|BSP|Kron|123|105|

```

Explanation

In the script created, the **main()** function is always called. In this function, the Callback function **"SetTables"** is used to specify the table "personalstamm".

Then the Callback function **"AddColumn"** is called to specify the required columns.

The Callback function **"MakeList"** is finally called to instruct the system to create the defined list and to write it to the file.

7.4.2 Example 2: List of persons with additional info and selection

This example is an extension of the previous version. All employees of the HR master data are listed. When the list is created, a company is specified as selection criterion. The list then only includes the persons of the company.

And for the cost centers, the name of the cost centers are added from the database table "kostenstellen".

Definition of the PDM list U_L_PERSONEN2:

The basic structure is sufficient for the definition of the list:

```
hydra basic;

// -----
//
// Tutorial
// PDM-List of persons
//
// Version 2: Simple version with join to another table and selection Parameters
//
// Test command line example:
// hymw -d -u2100 -c"DLG=LIST;U_L_PERSONEN2|FIR=BSP|DATEI=./spool/u_l_personen2.lst|USR=2100|"
//
// -----
import DLG_DATA      char(30000);

/*-----*/
long main()
{
    ret          long;

    // -----
    // define database table(s)
    // -----
    ret = Callback( "SetTables", "personalstamm p "||
        " left outer join kostenstellen kst "||
        " on p.kostenstelle = kst.kostenstelle "||
        " and p.firmen_nummer = kst.firma " );

    // -----
    // Define columns
    // -----
    // Database column      |Acronym |Designation"
    // -----
    ret = Callback( "AddColumn", "p.personalnummer      pnr|PNR      |Person no" );
    ret = Callback( "AddColumn", "p.firmen_nummer        fir|      |Company" );
    ret = Callback( "AddColumn", "p.person_name          \\\|\\\| \\", \" \\\|\\\| p.person_firstname " ||
        " |name          |Name" );
    ret = Callback( "AddColumn", "p.bereich              |BER      |Area" );
    ret = Callback( "AddColumn", "p.kostenstelle         |KST      |cost center" );
    ret = Callback( "AddColumn", "kst.bezeichnung        |BEZ_KST  |Cost center designation" );

    // -----
    // Define optional clauses
    // An optional where clause, an optional group by clause and an optional
    // order by clause can be defined
    // -----
    ret = Callback( "SetClauses",
        " where p.firmen_nummer like " || BV( get_bapi_val( DLG_DATA, "FIR" ) ) ||
        " order by 3, 1" );

    // -----
    // Generate list
    // -----
    ret = Callback( "MakeList", "" );

    return ret;
}

/*-----*/
```

Result:

After creating the list with the dialog

„DLG=LIST;U_L_PERSONEN2|FIR=BSP|DATEI={file name}|DAT=...|ZEI=...|USR=...|“

the content of the file {filename} is as follows:

```
PNR=Person no|FIR=Company|NAME=Name|BER=Area|KST=cost center|BEZ_KST=Designation cost center |
PNR=Personalnummer|FIR=Firma|NAME=Name|BER=Bereich|KST=Kostenstelle|BEZ_KST=Bezeichnung Kostenstelle|
906075|BSP|Erhard, Anton|077|105|Verwaltung|
1004|BSP|Hirsch, Harry|077|105|Verwaltung|
999998|BSP|Meier, Hans|123|105|Verwaltung|
1009|BSP|Mustermann, |077|105|Fertigung (ERF)|
999999|BSP|Schulz, Werner|123|105|Verwaltung|
```

Explanation

When the Callback function "**SetTables**" is called, several tables can be specified and have a table alias assigned. If you use "inner joins" or "outer joins", also specify the conditions for the joins in the function "SetTables".

If you call the Callback function "**AddColumn**", you can find examples for the work with table and column aliases. You can also see that the acronym can be left out where the appropriate column name or column alias is used. The definition of the name column demonstrates how character fields are connected using the SQL operator "||". Note: The pipe characters must be masked by backslash characters, otherwise the pipe character will be interpreted as a separator for the function parameters. The backslash character and the quotation mark (quotes) must be masked in the strings with another backslash, so that a more complex expression results:

```
" p.person_name \\|\\| '\\', '\\|\\|\\| p.person_vorname "
```

is converted into the SQL command

```
p.person_name || '|', ' ' || p.person_vorname
```

If you call the Callback function "**SetClauses**", a Where clause is specified with the condition to make a selection of persons according to the company. An Order-By clause for sorting by name and personnel number is also specified. For "outer joins" with obsolete syntax, you can also specify join conditions using the function "SetClauses".

The Callback function "**MakeList**" is finally called to instruct the system to create the defined list and to write it to the file.

7.5 Introduction: Extended options

7.5.1 Example 3: List of persons

This example is an extension of the previous version. Last name and first name are joined with a comma as separator, but only if a first name is available in the HR master data. If no first name is available, only the last name is displayed without comma.

The list is extended by the dynamically created column "OPT_COSTC_QUERY=Query cost center Y/N". The column must output Y/N. Y is output if the cost center name includes the key "(Q)". Otherwise, N must be output.

Definition of the PDM list U_L_PERSONEN3:

This list is implemented using the "extended options":

```

hydra basic;

// -----
//
// Tutorial
// PDM-Liste of persons
//
// Version 3: Extended features
// Test command line example:
// hymw -d -u2100 -c"DLG=LIST;U_L_PERSONEN3|FIR=BSP|DATEI=./spool/u_l_personen3.lst|USR=2100|"
//
// -----

import DLG_DATA      char(30000);

long main()
{
    ret      long;
    line char(8000);
    kst_bez  char(100);
    fistname char(80);
    surname  char(80);
    name     char(170);
    op_kst_input char(10);

    ret = CallBack( "WriteLn", "FIR=Company|PNR=PersonId|NAME=Name|BER=Area|KST=Costcenter|KST_BEZ=Costcenter
designatation|OPT_COSTC_QUERY=Query cost center Y/N|" );

/* AAAA */
// -----
// declare cursor
sqlxec( "declare list_curs cursor for " ||
        " select p.firmen_nummer, " ||
            " p.personalnummer, " ||
            " p.person_name, " ||
            " p.person_vorname, " ||
            " p.bereich, " ||
            " p.kostenstelle, " ||
            " kst.area " ||
        " from personalstamm p " ||
            " left outer join kostenstellen kst " ||
            " on p.kostenstelle = kst.kostenstelle " ||
            " and p.firmen_nummer = kst.firma " ||
        " where p.firmen_nummer like " || BV( get_bapi_val( DLG_DATA, "FIR" ) ) ||
        " order by 3, 1" );
if( sqlcode() = 0 )
{
/* BBBB */
// -----
// open cursor
sqlxec( "open list_curs;" );
if( sqlcode() != 0 )
{
    dprint( "SQL error" || sqlcode() || " pos" || sqleroffset() );
    eprint( "u_l_personen3: Error opening cursor" );
    ret = 1731;
}

// -----
// Loop over results
while( sqlcode() = 0 )
{
/* CCCC */
    sqlxec( "fetch list_curs;" );
    if( sqlcode() = 0 )
    {
        nachname = SqlColumn( 3 );
        vorname  = SqlColumn( 4 );
        kst_bez  = SqlColumn( 7 );

/* DDDD */
        // concatenate last and first name if first name is available
        if( firstname is not null )
        {
            name = nachname clipped || ", " || vorname clipped;
        }
        else
        {
            name = surname;
        }

/* EEEE */
        if( pos( "(Q)", kst_bez ) > 0 )
        {
            op_kst_erfass = "Y";
        }
        else
        {
            op_kst_erfass = "N";
        }

/* FFFF */

```

```
// format result set and output
line = "";
    line = add_bapi_val( line, "", SqlColumn(1) );
    line = add_bapi_val( line, "", SqlColumn(2) );
    line = add_bapi_val( line, "", name );
    line = add_bapi_val( line, "", SqlColumn(5) );
    line = add_bapi_val( line, "", SqlColumn(6) );
    line = add_bapi_val( line, "", kst_bez );
    line = add_bapi_val( line, "", op_kst_input );
    ret = CallBack( "WriteLn", line clipped );
}

/* GGGG */
// -----
// close cursor
sqlxec( "close list_curs;" );
}
else
{
    eprint( "u_l_personen3: error declaring cursor" );
    ret = 1731;
}

return ret;
}
```

Result:

After creating the list with the dialog

```
"DLG=LIST;U_L_PERSONEN3|FIR=BSP|DATEI={filename}|DAT=...|ZEI=...|USR=..."
```

the content of the file {filename} is as follows:

```
FIR=Company|PNR=PersonId|NAME=Name|BER=Area|KST=Costcenter|KST_BEZ=Costcenter designation|OPT_COSTC_QUERY=Query cost center Y/N|
BSP|906075|Erhard, Anton|077|105|Lathing 105|N|
BSP|1004|Hirsch, Harry|077|105|Lathing 105|N|
BSP|999998|Meier, Hans|123|106|Lathing 106 (Q)|Y|
BSP|1009|Mustermann|077|105|Lathing 105|N|
BSP|999999|Schulz, Werner|123|105|Lathing 105|N|
```

Explanation

With this script, the script entirely controls the data collection and the generation of list files.

The following procedure is required for the data collection:

- Declaration of an SQL cursor with a Select statement (marker /* AAAA */).
- Opening the SQL cursor with an Open statement (marker /* BBBB */).
- Reading the data records via Fetch statement in a loop, as long as data records are available. The data records read are output in the list file within the loop (markers /* CCCC */ to /* FFFF */).
- Closing the SQL cursor with a Close statement. (marker /* GGGG */).

Two steps are required to create the list file:

- Output of header with acronyms and designations (before marker /* AAAA */).
- Output of formatted data rows (marker /* FFFF */) in the loop when the data records are read.

The script functions *sqlexec()*, *sqlcode()*, *SqlColumn()* and *add_bapi_val()* are an integral part of the HYDRA script language. These script functions are described in a separate documentation.